



THE UNIVERSITY
of ADELAIDE

Enhancing State-of-the-Art Techniques Generating Low-Level Code for Cryptographic Arithmetic

JOEL KUEPPER

A thesis submitted for the degree of
DOCTOR OF PHILOSOPHY
at the
School of Computer and Mathematical Sciences
in the
Faculty of Sciences, Engineering, and Technology
of
The University of Adelaide

Supervised by
Dr. Yuval Yarom,
Dr. Chitchanok Chuengsatiansup, and
Dr. Markus Wagner.

September 2023

Contents

Contents	iii
List of Figures	ix
List of Tables	xi
List of Listings	xiii
Abstract	xv
Declaration of Authorship	xvii
Acknowledgements	xix
Papers and Presentations	xxi
1 Introduction	1
1.1 Motivation	1
1.2 Contributions	3
1.3 Integration	4
1.4 Optimisation	5
1.5 Summary of Contributions	6
1.6 Thesis Structure	7
2 Background	11
2.1 Low-level Programming and Hardware	11
2.1.1 Assembly Language and Instruction Sets	11
2.1.2 Registers and Instructions	12
Instruction Set Extensions	12
Instruction Intricacies	12
2.1.3 Processors	13
2.1.4 Code Performance Measurements	14
2.2 Compiler	15
2.2.1 Front End	15

2.2.2	Back End	17
2.2.3	Optimising Compilers	18
2.2.4	Domain-specific Compilers	20
2.3	Cryptography	20
2.3.1	Asymmetric Cryptography	21
2.3.2	Elliptic-Curve Cryptography	21
2.3.3	Constant Time	23
2.3.4	Cryptographic Implementations	24
2.3.5	Fiat Cryptography	26
2.4	Evolutionary Computing	27
2.4.1	Genetic Improvement	28
2.4.2	Random Local Search	29
2.4.3	Bet-and-Run	29
3	CRYPTOPT Components and Input Language	31
3.1	Components	31
3.2	Input Language	32
3.2.1	Function Signature	33
3.2.2	Function Body	34
3.2.3	JSON	34
4	Code Generation	39
4.1	Generating Assembly	40
4.2	Example	44
4.3	Order the Operations	44
4.3.1	Data Flow	45
4.3.2	Ordering Procedure	45
4.3.3	Example	48
4.4	Generate Assembly for the Example Function	50
4.4.1	Initialise Virtual CPU	51
4.4.2	Step Through the Operations	52
4.4.3	Code Decisions Remarks	55
4.4.4	Spilling	56
4.4.5	Finalise Function Pro- and Epilogue	57
5	Code Mutations	61
5.1	Scheduling Mutation	62
5.1.1	Bias	62
5.1.2	Bias Calculation	64

5.2	Decision Mutation	65
5.2.1	Example	65
5.2.2	Mutation	65
5.2.3	Types of Decisions	66
5.3	Pseudorandomness	67
6	Fitness Function	69
6.1	Time Counter	70
6.2	Variability of Program Execution	71
6.3	Measurement Procedure	72
6.3.1	Measurement Approach	72
6.3.2	Measurement Implementation	73
6.4	Robustness of the Measure Function	75
6.5	Parametrising the Batches	76
6.5.1	Scaling of the Batch Size	78
6.5.2	Number of Batches	79
6.6	Noise Reduction	79
6.7	MeasureSuite Module	80
7	Optimisation	81
8	Performance	85
8.1	Setup	86
8.1.1	Functions Under Test	86
8.1.2	Code Characteristics	87
8.1.3	Machines	88
8.2	Design Space Exploration	89
8.2.1	Alternative Bias	90
8.2.2	Bet-and-Run in CRYPTOPT	92
	Parametrising Bets	92
	Parametrising Bet Ratio	94
8.2.3	Alternative Optimisation Heuristic	94
8.2.4	Alternative Fitness Functions	97
8.2.5	Alternative Code Generation	101
	Spilling To Vector Registers	101
	Code Alignment	102
	Register Allocation Heuristic	104
8.3	Evaluation Method	106
8.3.1	Software Setup	108

8.3.2	Cross-Architecture Comparison	108
8.4	Optimisation Over Time	109
8.4.1	General Observations	111
8.4.2	One Function on One Machine	111
8.4.3	Same Function on Different Machines	114
8.4.4	Different Functions on the Same Machine	116
8.5	Length of Functions	118
8.6	Specificity of Implementations	120
8.6.1	Student's t -Test	120
8.6.2	Rank Implementations	120
8.6.3	Cross Validation	121
8.7	CRYPTOPT vs. Off-the-Shelf Compilers	123
8.8	State-of-the-Art Comparison	124
8.8.1	SUPERCOP	124
8.8.2	Implementations	125
8.8.3	Results	127
8.9	Conclusions	128
9	Real World Integration	129
9.1	BoringSSL	129
9.2	secp256k1	130
9.3	LLVM	130
9.4	BLS Signatures	131
9.5	Jasmin	132
9.6	Static Optimisations	134
9.6.1	Eliminate <code>static_casts</code>	134
9.6.2	Combine Operations	135
9.6.3	Resolve Nested Structures	135
9.6.4	Inline <code>argN[n]</code> s Into the Expression	136
9.6.5	Partial Evaluation	136
9.6.6	Common Subexpression Elimination	136
9.6.7	Dead Code Elimination	137
10	Conclusions	139
A	Machines	143
B	Field Arithmetic Performance Reports	145
B.1	Optimisation Over Time	145
B.2	Cross Validation	163

B.3 Scalar Multiplication	176
Acronyms	179
Bibliography	181

List of Figures

1.1	Overview of current implementations for cryptographic code and CRYPTOPT.	2
1.2	Integration design.	5
1.3	Iterative optimisation approach.	6
2.1	Compiler toolchain overview.	16
3.1	CRYPTOPT component and data-flow diagram.	32
4.1	Data-flow graph of the example function.	45
5.1	Scheduling interval of the operations of the example function.	63
5.2	Histogram of the biased sampling.	65
6.1	Violin plots show the robustness and sensitivity of the measure function.	77
7.1	Iterative optimisation approach.	82
7.2	Example CRYPTOPT optimisation run.	83
8.1	Curve25519, P-256, and P-521—Multiply with biased and uniform sampling for scheduling mutation.	91
8.2	Bet parameter comparison on Multiply for Curve25519, P-256, and P-521.	93
8.3	Bet-ratio parameter comparison on Multiply for Curve25519, P-256, and P-521.	95
8.4	Simulated Annealing acceptance likelihoods.	96
8.5	Optimisation runs with different levels of accept thresholds for worse candidates.	98
8.6	Different fitness functions used during optimisation.	100
8.7	Different modes of spilling to vector registers.	103
8.8	Different code alignments.	105
8.9	Different register allocation heuristic for choosing <code>rdx</code>	107
8.10	CRYPTOPT optimisation runs on three different machines.	110

8.11	Four optimisation runs for P-384-Square on i7 11G.	113
8.12	Four optimisation runs for P-384-Square on different machines.	115
8.13	Four optimisation runs for on two 7950X and i7 6G.	117
8.14	Violin plots of functions' lengths.	119
B.1	Seven optimisation runs for P-384-Square on i7 11G.	147
B.2	One optimisation run for P-384-Square on ten machines.	150
B.3	One optimisation run for each of the 22 functions on i7 6G.	156
B.4	One optimisation run for each of the 22 functions on 7950X.	162

List of Tables

2.1	Intricacies of x86-64 instructions to implement addition.	13
8.1	Overview of the fields, their size and their Fiat Cryptography implementation strategy.	88
8.2	Overview of machines.	89
8.3	Code statistics of the alternative bias.	90
8.4	Code statistics of the bet comparison.	92
8.5	Code statistics of the bet ratio comparison.	94
8.6	Code statistics of the Simulated Annealing comparison.	97
8.7	Code statistics of the alternative spilling.	102
8.8	Code statistics of the <code>nop</code> -code aligning.	104
8.9	Code statistics for <code>rdx</code> -loading heuristic.	106
8.10	Instruction count (average over all ten platforms).	118
8.11	Optimisation matrix for Curve25519-Solinas Multiply.	122
8.12	Geometric means of CRYPTOPT vs. off-the-shelf compilers over ten architectures.	124
8.13	Scalar multiplication cycle counts; geometric mean over ten machines.	126
A.1	Extended machine overview.	143
A.2	Motherboard details.	144
A.3	SSD details.	144
A.4	RAM details.	144
B.1	Optimisation matrix for Curve25519.	164
B.2	Optimisation matrix for Curve25519-Solinas.	165
B.3	Optimisation matrix for P-224.	166
B.4	Optimisation matrix for P-256.	167
B.5	Optimisation matrix for P-384.	168
B.6	Optimisation matrix for SIKEp434.	169
B.7	Optimisation matrix for Curve448.	170
B.8	Optimisation matrix for P-521.	171
B.9	Optimisation matrix for Poly1305.	172

B.10 Optimisation matrix for secp256k1-Dettman.	173
B.11 Optimisation matrix for secp256k1.	174
B.12 Optimisation matrix for Bitcoin Core (LLVM).	175
B.13 SUPERCOP compiler settings.	176
B.14 Scalar Multiplication cost (in cycles) for different implementations on different machines.	177

List of Listings

1	Input-language syntax.	33
2	C-like function signature of the CRYPTOPT input.	33
3	Example expression <code>(arg1[0] << 0x1F) arg2[0]</code> implemented in an SSA form in C.	35
4	Example function <code>out1[0] = (arg1[0] << 0x1F) arg2[0]</code> implemented in JSON (CRYPTOPT IR).	36
5	Nested body in JSON format.	37
6	Increment a number twice in x86-64 assembly.	40
7	Increment a number twice in x86-64 assembly, with context awareness.	41
8	Increment a number twice with spilling in x86-64 assembly. . . .	41
9	Load a spilled flag from a register while clearing <code>OF</code>	42
10	Load a spilled flag from a register while preserving <code>OF</code>	42
11	Add-with-carry in x86-64 assembly.	43
12	Example function in C.	44
13	Code to calculate <code>x2</code> in x86-64 assembly.	53
14	Code to calculate <code>x1</code> in x86-64 assembly.	53
15	Code to calculate <code>x4</code> in x86-64 assembly.	53
16	Code to calculate <code>x10</code> in x86-64 assembly.	54
17	Code to calculate <code>x3</code> in x86-64 assembly.	54
18	Code to calculate <code>x11</code> in x86-64 assembly.	55
19	Example function in x86-64 assembly.	55
20	Function pro- and epilogue in x86-64 assembly.	59
21	x86-64 assembly for the example function.	75
22	x86-64 assembly for the example function, but (unnecessarily) using the stack.	76

University of Adelaide

Abstract

Enhancing State-of-the-Art Techniques Generating Low-Level Code for Cryptographic Arithmetic

by Joel Kuepper

Fast cryptography is a crucial enabler for modern communication over the Internet. It provides authenticity, confidentiality, integrity, and non-repudiation in the vast majority of web traffic to date. Its ubiquitous deployment makes correctness and high speed of paramount importance.

The status quo for achieving high speed in cryptography is that implementations are written manually in assembly languages and hand-optimised by domain experts. While achieving high speeds, this process is tedious and error-prone. Prior work has shown how those hand-written implementations can be formally verified, and other work has shown how to optimise existing implementations. However, both approaches require significant manual effort.

This thesis presents CRYPTOPT, an optimising code generator that uses Genetic Improvement to automatically generate those high-speed assembly implementations. While doing so, the emitted code preserves enough structure to enable mechanised correctness validation.

CRYPTOPT uses a random local search heuristic. It starts by implementing the target function correctly. It then randomly mutates this implementation to generate a different implementation that is semantically equivalent. CRYPTOPT then runs both implementations on the target hardware and keeps the more performant one. By repeating those steps iteratively, the performance of the code is gradually improved. The main benefit of this technique is that the microarchitecture can be treated as an opaque unit, as opposed to the need for manual microarchitecture modelling in other approaches.

CRYPTOPT seamlessly integrates into Fiat Cryptography, a framework that follows a correct-by-construction approach for the automatic generation of cryptographic code. Before this thesis, Fiat Cryptography bottomed out at higher-level languages like C or Rust. Now, the formal correctness guarantees are extended to the assembly level.

The resulting optimised implementations are up to 2.96 times faster than what off-the-shelf compilers produce. For Curve25519 (part of the TLS standard) and the Bitcoin elliptic curve, the newly generated code is as fast as hand-written implementations on average, and can outperform on latest microarchitectures. The CRYPTOPT-generated code for Curve25519 is integrated into BoringSSL, the cryptographic library that powers Google Chrome and Microsoft Edge.

Declaration of Authorship

I certify that this work contains no material which has been accepted for the award of any other degree or diploma in my name, in any university or other tertiary institution and, to the best of my knowledge and belief, contains no material previously published or written by another person, except where due reference has been made in the text. In addition, I certify that no part of this work will, in the future, be used in a submission in my name, for any other degree or diploma in any university or other tertiary institution without the prior approval of the University of Adelaide and where applicable, any partner institution responsible for the joint award of this degree.

I give permission for the digital version of my thesis to be made available on the web, via the University's digital research repository, the Library Search and also through web search engines, unless permission has been granted by the University to restrict access for a period of time.

A handwritten signature in black ink, appearing to read 'Joel Kuepper', with a stylized, sweeping flourish extending from the end of the name.

Joel Kuepper

September 2023

Acknowledgements

I would like to sincerely thank my three supervisors Yuval, Chitchanok, and Markus. All of you are experts in your fields, and the combination of all your expertise and wisdom has shaped not only the project, the paper, and this thesis but also significantly shaped me as a researcher. Thank you for believing in the project and me. I wish you only the best for your future endeavours. Greatest thanks for the continuous energy you have put into supporting me. I was certainly lucky to have received this much support for the last three-plus years. On this note, also greatest thanks to my collaborators of the leading paper: Adam, Andres, Daniel, David, Jason, Livia, Owen, and Samuel.

Thank you Johannes Buchmann. Your introductory course during my master's degree made cryptography approachable to me and sparked my initial interest. Thank you, Matthias, for telling me "Yes, a PhD is fun, you'll like it!", which was essentially the kick that I needed to attend the Summer School 2019 in Croatia, where I met Peter. Thank you, Peter, for connecting me to Yuval thus facilitating my research career so far. I believe that if I had not had this chat with you, I would not have pursued my PhD further.

Besides my main goals to *have an impact* and *do my contribution towards a better future in the research world*, I also strove to explore a new place. My starting point in Australia was hectic, with my plane essentially being one of the last planes entering the country before the borders were closed for many months to come. However, in the coming months, I was lucky enough to meet so many great people who have become excellent friends. Among those old friends from back home and new friends are: Adel, Alice, Aline, Alysha, Amy, Anas, Andrew, Angus, Anne, Ash, Babsi, Bailey, Bao, Ben, Beth K, Beth W, Brad, Brian, Brice, Cecilia, Charlotte, Chris, Chrystal, Crystal, Damo, Daniel, Darcy, David, Deepak, Dominic, Doug, Ella, Emily, Erik, Fabien, Farzad, Gen, Grace, Haider, Hanna, Harley, Henry, Indie, Jake, James, Jannik, Jarrod, Jayne, Joel C, Joel M, Jonas, Jug, Julius, Katharina, Katie M, Katie N, Katie T, Khya, Kiet, Kristen, Kym, Leon, Lisa, Luke H, MC, Madura, Mady, Manuel, Marius, Matt, Mikayla, Mingtian, Mona, Money, Moritz, Nicky, Nicole K, Nicole M, Olivia, Paige, Patrick, Perle, Peter, Phil, Pia, Rachel, Ricco, Ricko, Ricky D, Ricky J, Riley, Robbie, Ronja, Roozbeh, RuiQi, Ruoxi, Saadia, Sab, Sacha, Sam, Samuel, Sara, Sarah E, Sarah S, Sarah T, Sasha, Scott, Seb, Sebastian, Shane, Simon, Sioli, Sophie, Steph, Sunny, Supun, Susie, TK, Tamara, Tara, Thierry, Thomas, Tim R, Tim W, Timo, Tobi, Tom H, Tom K, Toni, Tony, Top, Travis, Tristan, Viet, Vlad, William, Yusi, Zach, Zander, Zhiyuan. And I have certainly forgotten many – I apologise.

However, be assured, all of you above, everyone on other continents – if I have met you in the past months, you were part of what I am now, for which I am undoubtedly grateful. There is at least one good story that I share with every one of you and that I will hopefully never forget because I regret none of them. I wish you all the best; stay the way you are!

I wish to acknowledge the support that I have received from the University of Adelaide. In addition, I thank Google Research for supporting my travels to the PLDI conference.

Lastly, my words go out to my mother, to my father, to my brother and his wife, and to my brother and his husband. I should have called more often, but here and now for the record and eternity, I love you all wholeheartedly! Words cannot describe the gratitude that I feel to be able to call you my family. You raised me and shaped me into what I am today. I am proud of what you have achieved, and I hope to make you proud.

Papers and Presentations

CRYPTOPT has appeared in the press several times. This is an overview in the order of relevance:

The main paper [Kue+23b; Kue+23c] in the research-paper track of the *44th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI 2023)* as part of *FCRC'23* in Orlando. The paper received the *Distinguished Paper* award. The full link to the entry is: <https://pldi23.sigplan.org/details/pldi-2023-pldi/53/CryptOpt-Verified-Compilation-with-Randomized-Program-Search-for-Cryptographic-Primi>. The talk is recorded and available here: <https://www.youtube.com/live/1L1eKWwa6fE?si=AMxFIYHohAEEbQph>. The submission contains an artifact that bears the DOI: [10.5281/zenodo.7710435](https://doi.org/10.5281/zenodo.7710435).

Most recently, the work received the *Human-Competitive (HUMIES) GOLD Award*. The competition is part of *The Genetic and Evolutionary Computation Conference (GECCO)*. The SIGEVO website shows all the winners of GECCO 2023 in Lisbon: <https://gecco-2023.sigevo.org/Humies-Awards>. SIGEVO published an article about the HUMIES awards here: <https://evolution.sigevo.org/issues/HTML/sigevolution-16-3/>

The *Real World Crypto Symposium (RWC)* 2023 was held in Tokyo. CRYPTOPT's first public appearance was at RWC'23 – the entry can be found here: <https://rwc.iacr.org/2023/program.php>, and the talk is here: https://youtu.be/1ah03de3k_0?t=2790.

The CRYPTOPT project made it into the bibliography of genetic programming, maintained by W. Langdon [Lan23a]. This is significant because it can be seen as a one-stop shop for genetic programming or evolutionary computation in general.

The engineering of the optimiser is described in a paper [Kue+23a] published in the DEMO track of the *45th International Conference on Software Engineering (ICSE)* 2023 in Melbourne. The full link to the entry is <https://conf.researchr.org/details/icse-2023/icse-2023-demonstrations/3/CryptOpt-Automatic-Optimization-of-Straightline-Code>.

Monash University regularly holds *Software Systems and Cybersecurity (SSC) seminars*. CRYPTOPT was the topic of the SSC seminar held on the 31st of January 2023. The entry is here: <https://www.monash.edu/it/ssc/seminars/2023/cryptopt-verified-compilation-with-random-program-sea>

[rch-for-cryptographic-primitives](#), and the recording is available here:
<https://youtu.be/DZ0kaXNMtoA>.

In Cyber Today's EDITION 2, 2023 one article presents CRYPTOPT. The article is on page 27, here: [https://www.aisa.org.au/common/Uploaded files/Cyber Today/Edition 2 2023.pdf](https://www.aisa.org.au/common/Uploaded%20files/Cyber%20Today/Edition%202023.pdf)

Chapter 1

Introduction

Society has been depending on the Internet as a ubiquitous technology for many decades now and will continue to do so for the foreseeable future. However, as we know, with great power comes great responsibility; alongside all the great opportunities that the Internet gives come serious threats, too. In fact, the Internet is a notoriously dangerous place and nobody should be trusted by default – yet critical, confidential, and personal information is always being shared. Fast cryptography is a central enabler for this, ensuring privacy, integrity, and confidentiality.

The security of cryptographic systems is typically based on hard mathematical problems that cannot be practically solved in a reasonable amount of time. Many tools exist to validate that those cryptographic systems semantically indeed provide the proposed security goal, such as confidentiality or integrity. However, correctly implementing cryptography turns out to be extremely difficult. This can be seen in implementation failures such as Heartbleed [MIT13] in the widely-used open-source library OpenSSL or the goto-fail bug [MIT14] in Apple’s proprietary software.

When human users are involved in real-world applications, high speed is a non-functional requirement of increasing importance because users get impatient with increasing response time. As such, the high speed of cryptography becomes of paramount importance when convincing users to secure real-world applications with cryptographic systems.

1.1 Motivation

Cryptography can be implemented on different levels. Some levels provide more control than others to increase performance. Human effort is required to reuse those lower levels of implementations because they need to be re-optimised for additional microarchitectures. Figure 1.1 shows different levels of implementations on a graph that plots the performance of the cryptographic

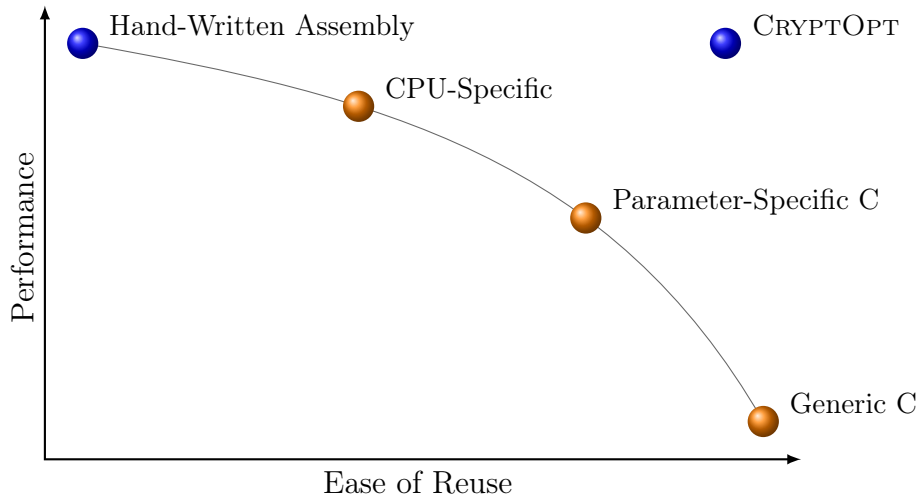


FIGURE 1.1. Overview of current implementations for cryptographic code and CRYPTOPT.

system versus the ease of reuse. On the lower right are generic C implementations that use compilers to generate the machine code. Those implementations are easy to adapt and reuse, but the code does not achieve the highest performance. A huge limiting factor of general-purpose compilers is that they resort to heuristics for their optimisation passes, which mostly rely on architectural assumptions of the underlying hardware. However, contemporary hardware has become incredibly complex, rendering those assumptions and the heuristics inherently inaccurate. As such, the compiler can – by design – not exploit the hardware to the fullest.

The generic C code can be adjusted to the parameters of a specific cryptographic system and even be optimised for specific CPUs. The more specific the code gets, the higher the achieved performance. However, this comes at the cost of making it harder to reuse those implementations because adjusting them to new architectures requires human effort and specialised knowledge. The highest performance can be achieved by directly hand-writing the assembly. This, however, is very costly and still error-prone.

The first three approaches not only do not produce the desired performance, but they also need compilers as part of the trusted computing base. The orange dots in the figure indicate this. As indicated in blue, hand-optimised code does not have this large trusted computing base.

The state-of-the-art to implement cryptography is to use a combination of high-level languages like C for the majority of the implementation and low-level languages like assembly for the core functions, where performance matters most.

This thesis looks at the problem of generating fast assembly for the core functions from an optimisation point of view. The main research question of this thesis is:

Can low-level code be automatically generated that performs at least as well as expert-written code while enabling easy correctness validation?

This thesis presents CRYPTOPT, an automatic optimising code generator. As indicated in [Figure 1.1](#), CRYPTOPT sits at the intersection of high speed and high ease of reuse and does not require a compiler toolchain in its trusted computing base. CRYPTOPT preserves enough input structure such that an external proven-correct translation validator can automatically prove equivalence to an already proven input. This renders the output of CRYPTOPT provably correct. This answers the research question with **yes**.

The novelty of CRYPTOPT is that instead of statically compiling into fast code, it uses techniques from the area of evolutionary computing to *find* fast implementations. Once a fast implementation is found, it can be automatically proven correct.

1.2 Contributions

This thesis presents CRYPTOPT, an optimising code generator designed explicitly for cryptographic code. As such, it seamlessly integrates into the **Fiat Cryptography** project, which generates provably correct code for cryptographic arithmetic. This integration enables automatic correctness validation by feeding the optimised assembly back into **Fiat Cryptography**.

CRYPTOPT avoids the problems that off-the-shelf compilers have by not relying on heuristics at all. Instead, it runs multiple semantically equivalent code versions on the hardware itself and measures its performance. In particular, CRYPTOPT follows the approach of Genetic Improvement (GI). The idea of GI is to start with some implementation and then to improve its quality gradually. GI is a relatively young research area, but despite its youth, it has been able to solve problems where traditional approaches are insufficient. CRYPTOPT uses it as follows: The first step is to create an assembly implementation for the required arithmetic function. This implementation is run on the hardware while observing its performance. CRYPTOPT then randomly mutates that implementation, which results in a slightly different but semantically equivalent implementation. This mutated implementation is run on the hardware, too, while also observing its

performance. The mutation is kept if it does not deteriorate the performance; otherwise, it is reverted. The mutate–measure cycle repeats with this more performant implementation as the new baseline.

Where typically off-the-shelf compilers use heuristics based on partial prior knowledge of the hardware to improve performance, the CRYPTOPT approach optimises code without any such heuristics.

The two main challenges to this approach are NP-hard problems that occur during code generation and precise performance measurements of short functions on contemporary hardware.

CRYPTOPT solves those code-generation problems essentially by using random starting points. If the random start is sub-optimal, the optimisation process finds better alternatives by mutating the code. In particular, the mutations change the order of operations and the choice of used instructions.

CRYPTOPT uses repeated, randomised measurement techniques to acquire robust performance measurements and utilises an in-memory assembler. Those two key techniques provide the means to measure small stretches of code accurately enough to guide the optimiser towards high-performing code.

In the current evaluation, CRYPTOPT generates code for x86-64 that significantly outperforms off-the-shelf compilers such as GCC and Clang, and achieves similar performance to what experts write by hand. The generated code is production-ready. As such, code for the high-speed de-facto standard Curve25519 [Ber06] has been integrated into BoringSSL – the cryptographic library that powers browsers such as Google Chrome and Microsoft Edge. Besides being formally assured to be correct, this code achieves an increased throughput of around 25%.

The following two sections describe the high-level concepts of how CRYPTOPT is integrated into existing tooling and how the optimisation works.

1.3 Integration

The overall aim of the CRYPTOPT project is to automatically generate fast and provably correct assembly programs for cryptographic arithmetic. This task is split between *generating* the program, *finding* a fast x86-64 assembly implementation, and subsequently *validating* it correct. Figure 1.2 shows the integration design. The automatic verification is enabled by connecting CRYPTOPT to Fiat Cryptography via its intermediate representation called Fiat IR. Fiat Cryptography generates the program from its internal collection of functional implementation strategies, which are instantiated with desired parameters into a specific program

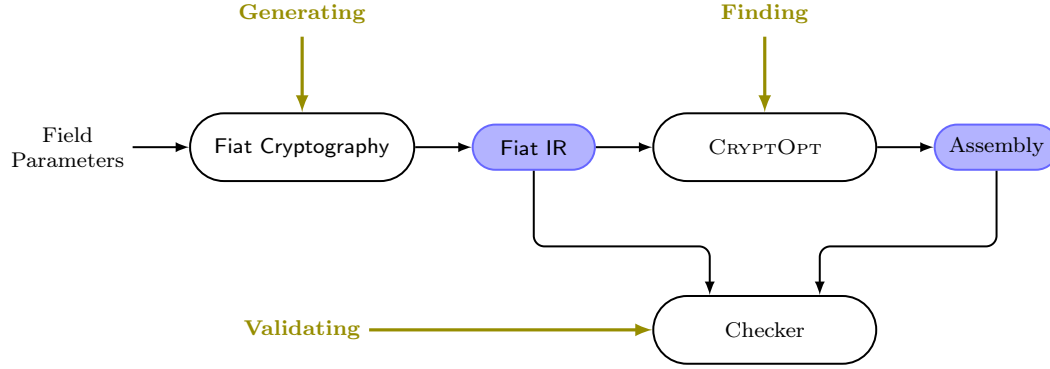


FIGURE 1.2. Integration design. White boxes represent individual components, arrows represent data flow, and filled boxes represent files. *CRYPTOPT* and *Checker* are the results of the CRYPTOPT project (i.e. [Kue+23b]). *Fiat Cryptography* and *Checker* are formally verified. Only *CRYPTOPT* is the subject of this thesis.

at run time. The resulting Fiat IR is then fed into the CRYPTOPT component, which solves the task of finding a fast program. It involves the various tasks of a code generator, such as instruction selection, instruction scheduling, and register allocation. CRYPTOPT optimises the generated x86-64 assembly code for a microarchitecture without the need to model it. It does this by running the code it generates on the actual hardware, which gives a realistic performance measure. As such, this thesis only focusses on Intel’s x86-64 Instruction Set Architecture (ISA).

The generated assembly, together with the Fiat IR is then the input to the Checker component, which validates equivalence between the assembly and the already proven-correct Fiat IR. This strategy of proving the equivalence of a translation (i.e. the x86-64 assembly) to a proven-correct baseline (i.e. the Fiat IR) is known as translation validation [PSS98; SMK13; TL08]. CRYPTOPT preserves enough structure of the Fiat IR input in the generated x86-64 assembly such that the equivalence checker can automatically prove its equivalence to the Fiat IR without further hints. In practice, the checker component is integrated into the Fiat Cryptography project, and as such not part of this thesis. Erbsen [Erb23b] and Kuepper et al. [Kue+23b] describe the equivalence checker in more detail. This thesis focuses on the optimising code generator CRYPTOPT.

1.4 Optimisation

A unique and novel property of CRYPTOPT is that instead of relying on classic compiler optimisation techniques, it draws on techniques from evolutionary

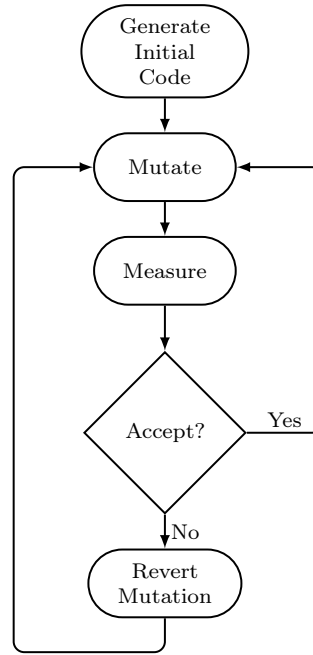


FIGURE 1.3. Iterative optimisation approach.

computation. Specifically, as illustrated in [Figure 1.3](#), the CRYPTOPT optimiser first randomly generates assembly code that implements the input function correctly. It then repeatedly mutates the code and measures the execution time of both the original and the mutated code, accepting a faster candidate. The process continues until a predefined computational budget is used up.

1.5 Summary of Contributions

This work identifies performance-critical aspects of low-level cryptographic code. In particular, different orders of operations and selections of instructions directly relate to performance. This thesis shows that simple search heuristics can exploit this property and generate code automatically that is on par with the hand-optimised state-of-the-art. Further, to facilitate such an approach, this thesis identifies and proposes solutions to the challenges that arise when measuring the performance of said code.

The CRYPTOPT project contributes several software components. The first contribution is CRYPTOPT itself. It is available as open-source software on GitHub, making it free to use and extend for anyone. Second, with the integration of **Fiat Cryptography**, it produces production-ready assembly code for cryptographic field arithmetic used in Elliptic Curve Cryptography. This code is formally assured to be correct, timing side-channel secure by following the constant-time programming paradigm, and performance-wise on par with

hand-optimised code. Last, the performance-measurement logic is encapsulated in a standalone module, available on GitHub, that can be used even outside of this project.

1.6 Thesis Structure

The thesis is structured as follows:

Chapter 2: Background

The thesis combines cryptography with optimisation and code generation. The background chapter briefly introduces the terminology and concepts of those three domains. As such, it starts by outlining low-level programming, the basics of processors and their dynamic frequency scaling, and challenges in measuring code performance. Further, the chapter explains how compilers generate code from higher-level languages to such sequences of instructions that processors can execute. Moreover, the background chapter provides information on cryptography, its different forms of implementation, and some caveats that need to be respected, such as the constant-time programming paradigm. The background chapter concludes with an introduction to specific terms and properties of evolutionary computing and optimisation strategies like Random Local Search and Bet-and-Run.

Chapter 3: CRYPTOPT Components and Input Language

This chapter explains the three main internal components: Code Generator, Optimiser, and Measurement. It also describes how those components communicate and the input language that is fed into CRYPTOPT.

Chapter 4: Code Generation

The first part of CRYPTOPT is the code generator. It generates x86-64 assembly code from the description of a desired input function. The chapter describes how CRYPTOPT solves the steps of ordering the operations, instruction selection, instruction scheduling, register allocation and spilling.

Chapter 5: Code Mutations

The second part of CRYPTOPT is the optimiser. This chapter describes how the intermediate code is changed to generate a different but semantically identical

implementation. In particular, it describes the mutations in operation ordering and choice of instructions.

Chapter 6: Fitness Function

The third part of CRYPTOPT is the performance measurement. The information on the performance of the implementation is crucial, as it guides the optimiser towards high-performing implementations. The chapter elaborates on the means of measuring time and how repeated, randomised measurements are conducted. Further, it lists the steps taken and not taken to reduce environmental noise during the measurements. It concludes with a description of the module that encapsulates the measurement logic.

Chapter 7: Optimisation

This short chapter describes the optimisation process. It describes the simple Random Local Search heuristic and how it is supercharged with the Bet-and-Run heuristic in CRYPTOPT.

Chapter 8: Performance

This chapter discusses alternatives to the current state of CRYPTOPT. Specifically, it discusses an alternative bias for the scheduling mutation, the parametrisation of the Bet-and-Run, alternative fitness heuristics, and fitness functions such as function length or performance predictions. Further, the chapter discusses possible code generation heuristics such as code alignment, spilling to vector registers, and register allocation preferences. Then, CRYPTOPT's capabilities are evaluated on 22 cryptographic functions across ten different hardware platforms. The chapter describes the functions and measurements and compares the results across the platforms.

Chapter 9: Real World Integration

Parts of the generated code are already used in real-world projects. This chapter describes how CRYPTOPT is used and where the limitations towards real-world problems lie. In particular, the chapter highlights the use of code for Curve25519 in BoringSSL, as well as work conducted to get CRYPTOPT's code into Bitcoin's core library. Integration to LLVM is a proof of concept with the C code from Bitcoin Core. CRYPTOPT generates code for the BLS signature scheme and makes efforts to consume the Jasmin programming language, but those results are not yet production-ready. The chapter also describes the preprocessing,

which performs static optimisations on the input that do not require running the code.

Chapter 10: Conclusions

This chapter summarises and concludes with possible steps for future work.

Chapter 2

Background

This background chapter briefly introduces the terminology and concepts needed for the remainder of this thesis. [Section 2.1](#) introduces assembly language and instruction sets, how those instructions are executed on processors and the challenges in measuring code performance. Further, [Section 2.2](#) explains the main concepts of how compilers generate code from higher-level languages to assembly. [Section 2.3](#) provides information on cryptography, its different forms of implementation, and some caveats that need to be respected, such as the constant-time programming paradigm. The background chapter concludes with [Section 2.4](#), an introduction to specific terms and properties of evolutionary computing used throughout the remainder of the thesis.

2.1 Low-level Programming and Hardware

This section first describes assembly language and instructions and then switches focus to how processors execute machine code. It concludes by describing the challenges that arise when measuring small stretches of code on contemporary hardware.

2.1.1 Assembly Language and Instruction Sets

Assembly is a representation of machine code that is text-based and, as such, human-readable. An *assembler* converts programs written in assembly language into machine code that can be executed on a processor, or Central Processing Unit (CPU). Different assembly languages exist for different architectures, such as ARM or x86-64. Assembly programs essentially are a long list of instructions. An *Instruction Set Architecture (ISA)* defines the instructions that can be executed on the CPU and how they need to be used.

2.1.2 Registers and Instructions

Internally, a CPU stores and modifies data in small storage units called *registers*. On 64-bit architectures, those registers can store 64 bits. During operation, the CPU reads *instructions* from the main memory and executes them. In Intel’s x86-64 ISA [Int20a], instructions consist of an *opcode* and optional *arguments*. The opcode defines the operation, and arguments can be immediate values, registers, and memory references. Symbolic names refer to registers. In Intel’s x86-64 ISA, those names are, for example, `rax`, `rbx`, or `rcx`. Memory is referred to by addresses that are read from registers. Registers can also be aliased to only refer to a subset of bits. For example, in Intel’s x86-64 ISA, the lowest 8 bits of the `rcx` register are written and read using the `cl` reference.

A special register of Intel’s x86-64 ISA is the `FLAGS` register. It holds several bits, indicating properties of the most recent instruction execution that updated the `FLAGS` register. For example, if the result of the last instruction was zero, one particular bit in the `FLAGS` register, the *zero flag* (ZF), is set. Similarly, if the last operation resulted in a signed overflow, the *overflow flag* (OF) is set, and an unsigned overflow sets the *carry flag* (CF). To save a carry bit into a register, the 8-bit variant of a register must be used: `setc cl` saves the CF into the `cl` register (without resetting the higher bits, i.e. bits 8 to 63).

Instruction Set Extensions

ISAs are extended in so-called *Instruction Set Extensions* (ISEs). For example, Intel’s BMI2 (added with Haswell in 2013) and ADX (Broadwell, 2014) extensions add support for new instructions such as `mulx`, `shlx`, `shrx`, `bzhi`, as well as `adcx` and `adox`. `mulx` allows for 128-bit $\leftarrow 64\text{-bit} \times 64\text{-bit}$ multiplication; `shrx` and `shlx` allow for shifts; `bzhi` allows for zeroing upper bits in a register; all without affecting the `FLAGS` register. `adcx` allows for addition chains that only using the CF, and `adox` only uses the OF. Those two last instructions always read and write this and only this flag. This enables the developer to implement two addition chains that interleave each other.

Instruction Intricacies

ISAs often provide multiple ways of implementing the same operation. For example, Table 2.1 shows five instructions that can be used to implement an addition: `add`, `adc`, `adcx`, `adox`, and `lea`. They can all add at least two source operands together, but the details differ in many ways. First, they differ in where the operands can reside, whether they can be in a register (R), in memory

TABLE 2.1. Intricacies of x86-64 instructions to implement addition.

Instruction	CF	OF	DEST	SRC 1	SRC 2	Req'd ISE
<code>add</code>	W	W		R/M	R/M/I	—
<code>adc</code>	R/W	W		R/M	R/M/I	—
<code>adcx</code>	R/W	—		R	R/M	ADX
<code>adox</code>	—	R/W		R	R/M	ADX
<code>lea</code>	—	—	R	R	R/(I)	—

(M), or if they can be immediate (I). `lea` is the only instruction that can read from two different registers and store the result in a third register, all the others *accumulate* one source with another source. The result is then stored in one source operand. Second, `lea` is the only instruction that does not affect any flags; all the others in the table at least read (R) or write (W) one flag according to the result. In particular, `add` and `adc` always write the carry flag CF and the overflow flag OF, whereas `adcx` and `adox` always read and write only one flag. Third, they differ in which ISE is required for the instruction to be available. `adcx` and `adox` need the ADX extension, whereas the others are available on all x86-64 processors. As inferred by this description, using an instruction that cannot read immediate values to implement an operation with an immediate value would require an additional `mov` instruction and a free register. In this case, a different instruction that can accept an immediate value is more suitable. Similarly, if the `FLAGS` register hold precious values, using an instruction that does not overwrite those values would save instructions to save the values of the `FLAGS` register. There are more intricacies in the instructions that are described in the Intel Software Manual [Int20a], but the key point is that depending on the requirements, one instruction may be more suitable than another. This range of instructions makes it difficult to know a priori which one will be the best performing one to use, as their performance depends on the context in which they execute. In addition to the context, the run time of the same instruction is also highly dependent on the underlying microarchitecture [AR19; AR20; AR22].

2.1.3 Processors

A CPU executes binary machine code. CPUs have come a long way since the introduction of the von Neumann model in 1945 [Von45]. They have now become incredibly complex pieces of hardware with many different types of caches, prediction mechanisms, and decoding engines [LL00; RCA99; RPH04; SL06; Smi82; Smi89; Sri+01; Vil+20; Wik23; Zha+23a; Zha+23b]. Modern

processors are super-scalar, meaning multiple instructions are executed in parallel, which improves throughput. To further improve hardware utilisation, a feature known as *out-of-order execution* or *dynamic instruction scheduling* [Smi89] is commonly implemented in modern CPUs. It enables the execution of decoded instructions *when possible* instead of *in the order in which they appear in the program* [Kuk15]. Those features influence the execution time of a binary and are different between families and generations of CPUs.

The inner workings of those CPUs are usually trade secrets of the manufacturer and, as such, are not publicly known. Those inner workings are referred to as the *microarchitecture* of the CPU. It has become an entire research area to reverse engineer and model the mechanics of CPUs. This is done for adversarial purposes on the one hand and for pure performance micro-benchmarking [AR19] on the other hand.

Dynamic Frequency Scaling

Dynamic frequency scaling is one feature that trades off energy consumption and high performance. In particular, the CPU scales down the frequency to reduce power consumption. The frequency is also scaled down to avoid overheating, known as *thermal throttling*. Alternatively, the frequency can be increased above its nominal frequency if the workload is high and as long as the temperature allows it. This is known as *boosting*. Intel calls this technology Intel Turbo Boost, and AMD calls it AMD Turbo Core or AMD Core Performance Boost.

A user can typically set the CPU's *governor* to two preferences: *performance* or *power save*. Those settings signal to the Operating System (OS) to either run at high frequencies to achieve higher performance or to conserve energy where possible. However, this is just a preference of the OS; but the CPU may overwrite that, for example, for thermal throttling purposes.

2.1.4 Code Performance Measurements

To be able to make informed decisions on how to optimise code performance, a prerequisite is to be able to measure code performance. To do that, either the code is run on the actual hardware, and the performance is measured, or the performance is predicted based on models of the microarchitecture.

The idea of measuring code performance on the actual hardware to improve the performance has been introduced previously [AR20; BSS15; Li+22; Pee+22; WO18]. However, it is still far from trivial, as the execution time can be affected by the state of memory layout and OS environment [CB13; Myt+09; Pan+19],

instruction order [Bos+20; GW01], process and thread context switches [AW03; PGB12], data and instruction caches [AW03; HE13; Myt+09; Ren+21], or more.

In addition, the execution time is affected by the CPU’s load because of its super-scalar nature. Thus, the same binary can have different execution times on the same CPU. Trivially, the performance of the same binary is different on different microarchitectures as they advance over time.

The other way to gain performance insights is to predict the performance of a given piece of code. Abel and Reineke [AR19; AR22] publish their tool *uiCA*, which predicts the performance of instructions for Intel microarchitectures. So far, it is limited to Intel architectures and needs constant updating of its model for every upcoming architecture. In particular, at the time of writing, it only supports microarchitectures up to Rocket Lake (Intels 11th Generation from 2021).

2.2 Compiler

At a high level, *compilers* (or *compiler toolchains*) are collections of tools that translate high-level programming languages such as C to assembly such as x86-64 assembly and then machine code, which a machine can run natively [ASU86]. As many different high-level programming languages and architectures exist, creating a compiler for the cross-product of every programming language and architecture would require much work and be very expensive. To avoid that, compilers are typically divided into a *front end* and a *back end*, which are connected via an Intermediate Representation (IR) such as the LLVM IR language in the LLVM project.

The front end reads one programming language into the IR, and the back end generates code for one architecture. That way, the front ends of various languages can be matched with the back ends for various architectures. An optional optimiser can work on this IR to increase performance. By optimising on the IR level, the optimisation logic does not need to be duplicated for other programming languages or other targets (cf. Section 2.2.3). Figure 2.1 shows a schematic of a typical compiler toolchain.

2.2.1 Front End

All compiler components dedicated to consuming the high-level language are typically referred to as the compiler front end. The front end scans the characters of the high-level language, such as C, Rust, or Go, into tokens. Those are then

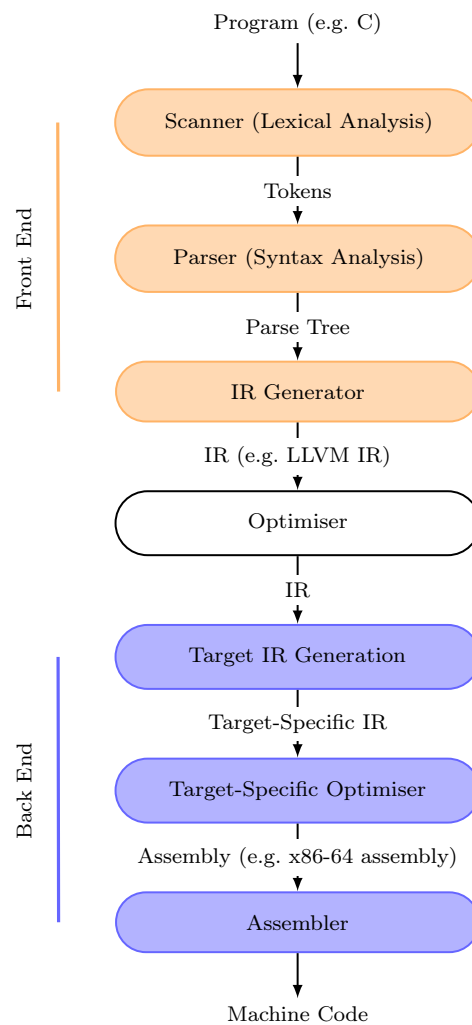


FIGURE 2.1. Compiler toolchain overview (Based on Figure 1.5 and Figure 1.6 in Aho, Sethi and Ullman [ASU86]). Front-end components are marked in orange; back-end components are marked in blue.

passed to the parser, which then syntactically checks them and emits a parse tree. The last component in the front-end chain is the IR generator, which generates the common format, such as LLVM IR.

2.2.2 Back End

The tools that generate machine code from the IR are called the compiler back end, sometimes called the *code generator*. Dedicated tooling exists for every supported architecture, such as x86-64 assembly, ARM, or MIPS. The first step in the back end is to generate a target-specific IR, which is semantically very close to the final machine code. While many optimisations benefit a wide array of targets, some are very specific to certain generations and families of CPUs. For example, instruction alignment in memory and choice of instructions are specific to one particular microarchitecture. This target-specific optimiser then emits the assembly for one specific architecture, which an assembler converts into machine code (i.e. binary executable).

Code Generation

During the code generation, three problems need to be solved [ASU86]:

- *Instruction scheduling*, which describes the process of deciding when to compute independent operations;
- *instruction selection*, which chooses one particular from potentially multiple hardware instructions for any given operation in the IR; and
- *register allocation*, which specifies the assignment of logical intermediate values to registers.

Solutions to those problems may restrict each other because one particular instruction may require a special register to be allocated. Hence, they cannot be effectively solved sequentially but should be solved simultaneously. On top of that, a solution for those three problems, i.e. a final assembly program that also achieves high performance, is highly dependent on the microarchitecture of the target CPU. In practice, the code generator needs to trade off compilation time vs. performance of the generated code. Register allocation is an NP-hard problem [Bou+06; Cha82]. Hence, code generators rely on heuristics, which are supposed to do reasonably well for the average program in a reasonable amount of compiler run time.

Spilling

Register allocation typically also describes *spilling* mechanics. Spilling happens whenever more live logical intermediate values exist than can be held live in the available registers. In that case, the code generator needs to decide which value(s) to temporarily offload, i.e. spill, to memory, and to fetch them again into registers as required.

The *stack* is a memory region private to a function, that can be used to spill values to. The special register `rsp` always points to the top of the stack.

In the x86-64 assembly, the `push` and `pop` instructions can be used to access the stack. `push rax` saves a value from the register `rax` to the top of the stack and `pop rax` reverses this operation by loading the value from the top of the stack back into the register `rax`. Because the stack is a memory region, a value in register `rax` can also simply be *moved* with `mov rax, [rsp + off]` to the stack at offset `off`. The benefit of using the `push` and `pop` instructions is that they implicitly increment and decrement the stack pointer `rsp`, but that comes at the cost that the program can only read values in reverse spilling order.

The *red zone* [Lu+18] is an area below the current stack pointer that a leaf function can use to store data. On Linux and macOS, a red zone is available and has a size of 128 bytes; Microsoft Windows does not support the concept of the red zone. If the red zone can be used, values can be written and read to and from the red zone with `mov rax, [rsp - off]`, with $0x8 \leq \text{off} \leq 0x80$. The benefit of using the red zone is, that the stack pointer does not need to be adjusted at the start and end of the function if the stack usage is 128 bytes or less.

2.2.3 Optimising Compilers

Ubiquitous off-the-shelf compilers like the GCC [GCC23] or Clang [Cla23] are so-called *optimising compilers*. That means they employ optimisation techniques on the IR intending to generate better (faster or smaller, depending on the setting) machine code in the back end. The white centre element of Figure 2.1 indicates this optimisation step. Typically, those optimisations are encapsulated in single *optimisation passes*, which focus on one task [ASU86].

For example, an optimisation pass exists to *eliminate dead code* (code that the intended control flow cannot reach). A second optimisation pass, Common Subexpression Elimination (CSE), eliminates expressions that do not need to be calculated multiple times. This second pass may result in dead code again. This small example shows that the order in which optimisation passes are applied

matters. In practice, a selection of optimisation passes is bundled into so-called *optimisation levels*. With that, a developer can choose to run a compiler, e.g. with the `-O1` option (optimise with level one) to get some optimisation passes or may select to run the compiler with `-O3` (optimise with level three) to get many more or different optimisation passes. Additionally, the compiler can be advised to performance-tune the generated code for specific microarchitectures with the `-mtune=` switch. Further, the microarchitecture can be specified to enable a compiler to use ISEs. The `-march=skylake` specifies that the compiler can generate code that uses instructions from ISEs that require a generation of at least Intel’s “Skylake”. Both the `-march` and the `-mtune` switches support the option `native`, which determines the processor’s microarchitecture, which the compiler currently runs and optimises for it.

Sometimes, those predefined levels and selections do not satisfy the requirements of some applications. Single optimisation passes can be enabled, too, but it requires significant work to manually tune those on a per-application basis. This results in a considerable amount of prior work that attempts to find sequences of compiler passes automatically, which improves code performance [BSS15; CSS99; Li+22; Pee+22; Ste+03]. Those compiler passes are so complex that there even exists work that attempts to merge machine learning techniques into compiler optimisation [BA06; Cal+97; PB10; WO18].

Superoptimiser

Massalin [Mas87] describes a *Superoptimiser* as a tool that exhaustively enumerates all possible programs that implement a given function and then chooses the one candidate that is the most fit for a given cost. In other words, it uses brute force to find the perfect candidate. At the time, it could generate provably shortest programs of 12 instructions after several hours of computing. However, CPUs nowadays are much more complex, and the shortest program is no longer necessarily the fastest.

The research community has since seen many more such superoptimisers with different scopes: Sasnauskas et al. [Sas+18] present SOUPER, an optimisation synthesising tool working on the LLVM IR; Joshi, Nelson and Randall [JNR02] present Denali, a superoptimiser, which uses CPU microarchitecture modeling and automatic solvers. Schkufza, Sharma and Aiken [SSA13] present their project STOKe, a synthesising and optimising tool. It can either create entirely new programs or optimise existing ones. It combines a correctness indicator with a performance indicator into the fitness function. As such, it can guide the search

towards correct and fast programs. Several extensions to this project [Sha+13; Sha+15; SSA14] attempt to solve special cases.

2.2.4 Domain-specific Compilers

A Domain-Specific Language (DSL) is a language that is typically only used in very specific domains, as opposed to C or Java, which are so-called multi-purpose languages. Two such DSLs that target cryptographic code are Jasmin [Alm+17; Alm+20a] and QHASM [Ber05a; Ber07]. Those languages come with their own compiler toolchain and may lack sophisticated optimisation techniques in widely used mainstream compilers. As such, their optimisation capabilities may fall behind, leaving much of the optimisation work to the developer.

2.3 Cryptography

Cryptography is believed to date back to 1500 BCE in the Mesopotamian region [Kah66] and was used to provide confidentiality in a message. With the rise of the Internet in the 1960s, new applications for cryptography were found. In the realm of the Internet, cryptography nowadays is used to accomplish the following four main goals: confidentiality, authenticity, integrity, and non-repudiation.

The first goal, confidentiality, is achieved using symmetric cryptography, which uses the same key to encrypt and decrypt a message. The most widely used Advanced Encryption Standard (AES) [DR98; Dwo+01] was standardised in 2001. According to a report by F5-Labs in 2021 [WV21], more than 96% of all TLS connections use AES, the next-most commonly used is CHACHA20 [NL18] with 2.3%.

Depending on the used protocol, the authenticity and integrity goals can be achieved using, e.g., Message Authentication Codes (MACs) or cryptographic hash functions. The HTTPS protocol uses *digital certificates* to achieve the goal of authenticity and to establish a shared key between the communicating parties. The security of digital certificates relies on the security of the attached *digital signatures*. *Asymmetric cryptography*, also known as *public-key cryptography*, is needed to agree on a shared key for symmetric encryption or to create digital signatures [MOV96].

2.3.1 Asymmetric Cryptography

While symmetric cryptography uses the same key for encryption and decryption, asymmetric cryptography uses distinct keys for encryption and decryption. Asymmetric cryptography can be used to generate a shared secret using protocols such as the Diffie-Hellman¹ key exchange [DH76] or to create a digital signature using the Digital Signature Algorithm (DSA) [NIS94]. The security of asymmetric cryptography relies on hard mathematical problems. For example, when using the well-known RSA [RSA78] protocols, the security relies on the hard mathematical problem of finding prime factors of large composite numbers.

This thesis focuses on a particular form of public-key cryptography called *elliptic-curve cryptography* [Kob87; Mil85].

2.3.2 Elliptic-Curve Cryptography

$$E : y^2 + a_1xy + a_3y = x^3 + a_2x^2 + a_4x + a_6 \quad (2.1)$$

An elliptic curve E over a field $\mathbb{K}_p$ is the set of points in $\mathbb{K}^2$ that satisfy Equation 2.1 for some coefficients $a_1, a_2, a_3, a_4, a_6 \in \mathbb{K}$. This thesis focusses on elliptic curves used for cryptography. These are commonly defined over fields of large prime orders. Two common examples are Curve25519 [Ber06] and *secp256k1* [Cer00].

Curve25519 is commonly used as the de-facto high-speed curve for TLS traffic. Curve25519 specifies $E : y^2 = x^3 + 486662x^2 + x$ and the prime field with $p = 2^{255} - 19$. Further, the curve *secp256k1* is used for the block signatures in the cryptocurrency “Bitcoin” [Nak08] and specifies its elliptic curve as $E : y^2 = x^3 + 7$ over $\mathbb{K}_p$ with $p = 2^{256} - 2^{32} - 2^9 - 2^8 - 2^7 - 2^6 - 2^4 - 1$.

The security of elliptic-curve-based cryptography relies on the assumption that the elliptic-curve discrete-logarithm problem (ECDLP) [GG16] is hard to solve in practice. The best-known algorithm with a classical computer to solve ECDLP for an elliptic curve defined over $\mathbb{K}_p$ costs approximately $\mathcal{O}(\sqrt{p})$. This means that the security is roughly the square root of the field size [HM11].

Scalar Multiplication

The set of points over the elliptic curve, together with a *point at infinity* $\mathcal{O}$, form a group. The point at infinity serves as the identity element. A point $P(x_P, y_P)$

¹According to Hellman [Hel02], it should be called the Diffie-Hellman-Merkle key exchange protocol.

is negated by negating the y -coordinate. Hence, $-P = (x_P, -y_P)$. Two distinct points $P(x_P, y_P)$ and $Q(x_Q, y_Q)$ are added to point $R(x_R, y_R)$ by calculating

$$\begin{aligned} s &= \frac{y_Q - y_P}{x_Q - x_P} \\ x_R &= s^2 - x_P - x_Q \\ y_R &= s(x_P - x_R) - y_P \end{aligned}$$

A point can be added to itself. This is called *point doubling*. To double a point $P(x_P, y_P)$, the slope s is calculated as

$$s = \frac{3x_P^2 + a_4}{2y_P}$$

The core operation in Elliptic Curve Cryptography (ECC) is the *scalar multiplication*, which computes

$$nP \leftarrow P + P + \cdots + P$$

or adding n copies of P where n is a scalar (integer) and P is a point on the elliptic curve. nP is again a point on the curve. This operation relates to the security of elliptic-curve-based schemes, which rely on the hardness of the ECDLP, i.e. it is difficult to recover n given nP .

Scalar multiplication is the most time-consuming operation in ECC. Therefore, any efficiency improvements in scalar multiplication typically directly translate to improved performance of the entire cryptographic scheme. The scalar-multiplication operation itself can be decomposed into a series of point-arithmetic operations, which further decompose into field-arithmetic operations. The costly field operations are field multiplication and field squaring, where the latter is a special case of the former where both arguments are identical.

Elliptic-Curve Diffie-Hellman

To create an Elliptic-Curve Diffie-Hellman Key Exchange, Alice and Bob agree on the public curve parameters E , p , and a public generator point G , also called *base point*. Alice and Bob each choose a private scalar a and b and calculate their respective key share aG and bG using scalar multiplication. After sending their key shares to each other, both compute abG and baG , which results in the same point, because point arithmetic is commutative. Now, both derive the shared secret by, for example, taking the x coordinate of the point abG .

Note that only the points aG and bG are transmitted, not the secrets a or b itself. An attacker cannot calculate the secret a from aG , because that would require solving the ECDLP, which is assumed to be a hard problem.

Elliptic-Curve Digital Signature

The protocol for elliptic-curve digital signatures [JMV01; Kob87] is moderately more complicated: The public parameters are a large prime p to specify the finite field $\mathbb{K}_p$, the elliptic curve E , and a base point G whose order n must be prime.

The key generation works as follows: The signing party selects a cryptographically secure random integer d as the private key from the range $[1, n - 1]$, where n is the order of G . The public key Q corresponding to the private key d is $Q = dG$.

To sign the hash $h(M)$ of a message M , which must be at most as long as the bit length of the group order n , the signing party computes another cryptographically secure random integer k (different for every signature) to calculate the point $K = kG$. Next, calculate $r = x_K \bmod n$ and $s = k^{-1}(h(M) + dr) \bmod n$. If $s = 0$ or $r = 0$, another integer k must be chosen; otherwise the pair (r, s) is the signature.

Given the signature (r, s) , the public parameters, the public key Q , and the hash $h(M)$, the verification works as follows: Check that Q is not the identity element $\mathcal{O}$, Q lies on the curve E and $nQ = \mathcal{O}$. Then, verify that r and s are integers in $[1, n - 1]$, compute $u_1 = h(M)s^{-1} \bmod n$ and $u_2 = rs^{-1} \bmod n$, and finally compute curve point $V = u_1G + u_2Q$. If $V \neq \mathcal{O}$ and $r \equiv x_V \bmod n$, the signature is valid.

Similar to the Diffie-Hellman Key Exchange, the scalars d and k are also secret in the elliptic-curve digital signature algorithm (ECDSA), and scalar multiplication must be implemented in constant time. The performance of ECDSA also boils down to fast scalar multiplication, which in turn leads to fast field arithmetic.

2.3.3 Constant Time

For secure cryptographic implementations, the commonly accepted method is to follow the *constant-time* programming paradigm. Essentially, it states that the run time of a cryptographic scheme cannot depend on secrets because if there is any correlation, observing the timing behaviour would leak secret information. Exploiting this behaviour is called performing *side-channel attacks* and has

been shown in various works [Ge+18; Jan+22; Koc+19; Lip+18]. In particular, constant-time programming [Ge+18; Jan+22] means that implementations cannot feature

- secret-dependent control flow,
- secret-dependent memory accesses, or
- use instructions whose timing depends on the value of their arguments if those values are secret [Zha+23a]. The constant-timeness of instructions themselves depends on the microarchitecture, too [DGF22].

The first two, secret-dependent control flow or memory accesses, change the microarchitectural state (e.g., the cache) of the CPU, which can cause changes in timing. The third point of dynamic-timing instructions (e.g., `div`) inherently influences the timing behaviour of the program.

Because the scalar is secret in ECC, scalar multiplication must be implemented in constant time to avoid timing side-channel vulnerabilities [Alm+20b; DGF22; Ge+18; Jan+22; Koc+19; Lan10; RBV17].

2.3.4 Cryptographic Implementations

For general compilation, compilers can be used to automatically generate well-performing code from high-level languages. However, extra caution must be taken when implementing cryptographic algorithms [Ass04; Hol06; MOV96]. In particular cryptographic implementations

- *must* be correct: i.e. semantically the same as in the specification,
- *must* be secure: i.e. not be vulnerable to side-channel attacks,
- *should* be fast, to be practical.

Historically, achieving those three goals simultaneously is very expensive because typical compilers cannot guarantee correctness or security. On top of that, cryptographic code, especially field arithmetic, is very different from typical code: it is heavy on arithmetic rather than control flow. The lack of control flow has two benefits: First, it helps with security, because if there is less control flow, then there is less control flow that depends on secrets. Second, less control flow can speed up programs [Ang+22; Kac09; LL19; Pat+00], because there are fewer branches that can be mispredicted, and larger sequences of instructions can be executed out of order. Thus, *cryptographic functions result in abnormally straight-line code* [Lan10].

Big Number Representations

The arithmetic in ECC needs to handle field elements that are larger than the size of a register. For example, in Curve25519 and secp256k1, the field elements are 256 bits long, which is much larger than what a 64-bit register can store. Hence, one field element is split into multiple limbs that are at most 64 bits in length and can be kept in registers.

However, it is not trivial to decide how many limbs an element should be split into. A *saturated* representation uses four 64-bit limbs for a 256-bit number, whereas an *unsaturated* representation uses more, e.g. five 52-bit limbs. The benefit of using unsaturated representations is that multiple limbs can be added, and the carry propagation into the next limbs can be delayed, which can increase performance on some CPUs. However, it comes at the cost of calculating the arithmetic with more limbs.

Typical general-purpose programming languages like C do not have semantics for unsaturated representations (i.e., there is no 52-bit integer), so the developer has no easy way to convey the implementation semantics to a compiler.

In addition to the representation of the individual field elements, the field arithmetic over a specific prime number can be implemented using different strategies, where some strategies typically suit some prime (shape) better than others. For example, Solinas primes (i.e. generalised Mersenne prime) [Sol99] are much more efficiently implemented using the (un)saturated Solinas strategy. In contrast, many other primes can be implemented efficiently using techniques described by Gueron [Gue12]. The latter is referred to as the word-by-word Montgomery implementation strategy.

Correctness vs. Performance

Unlike “general” code, in cryptography, minor errors that are unlikely to occur with benign input may result in a breach of security when exploited maliciously. This is why correctness in cryptography is more important than for other code.

However, because faster field arithmetic directly relates to an overall faster cryptographic scheme, many implementations tend to implement the field arithmetic for ECC by hand in assembly [BCL14; Ber+14; Ber06; BS12; Cho15; FH15], thereby e.g. relying on unsaturated representations. The widely deployed library OpenSSL [Ope23a] includes multiple hand-optimised assembly implementations for Curve25519, choosing which one to use at runtime.

Most of those implementations are not proven correct directly. Verification tools like CryptoLine [Che+14; Fu+19; Hwa+22; Pol+18; TWY17] can help achieve provable security by proving existing implementations correct.

Some projects follow a correct-by-construction approach, such as Fiat Cryptography, published by Erbsen et al. [Erb+19] (cf. Section 2.3.5). Fiat Cryptography starts with proven-correct functional programs that can be instantiated into high-level languages such as C or Go. Although slightly slower than the hand-tuned assembly, this C code has been integrated into BoringSSL [Bor23], Google’s fork of OpenSSL. This shows that some libraries prioritise formal assurance over the highest performance.

Security

Implementations can leak information through *side channels*. A side channel can be exploited by observing the timing, power consumption, or electromagnetic radiation [Alm+20b; DGF22; Ge+18; Jan+22; Koc+19; Lan10; RBV17].

This thesis only considers timing-side channels in scope, which can be avoided by obeying the constant-time programming paradigm. Some tools can aid developers to ensure that the code is constant-time goals are met [DGF22; Lan10; RBV17; RLT15], but according to a comprehensive user survey [Jan+22] they seem to be too impractical to be widely used.

Additionally, implementing an algorithm in a high-level language and using an off-the-shelf compiler like Clang may strip constant-time constructs [LLV23].

2.3.5 Fiat Cryptography

Fiat Cryptography [Erb+19] is a framework to generate provably-correct field arithmetic for various languages like C, Zig, Go, and Rust. It generates timing side-channel secure code because all array accesses are constant offsets, and Φ -nodes are implemented with dedicated conditional-move functions. However, as compilers are still involved, the correctness and security are not guaranteed per se. In fact, compilers are known to have bugs [Erb+19] and may optimise away side-channel protections [LLV23].

The Fiat Cryptography framework ships with proven-correct implementations for the different implementation strategies that fit the required finite field. When the tool is used, it instantiates those implementations with parameters like limb width (e.g. 64-bit) and prime into its Fiat IR and, from there, into actual code in various languages. In particular, in its own evaluations [Erb+19],

Fiat Cryptography generates code for the prime fields underlying the following cryptographic schemes:

- The NIST P curves [NIS00]
 - Curve P-224,
 - Curve P-256,
 - Curve P-384 and
 - Curve P-521,
- the field of the popular “Bitcoin” curve secp256k1 [Cer00; lib23],
- the high-speed de-facto standard Curve25519 [Ber06],
- a high-security Curve448 [Ham15],
- the (now rendered insecure [CD23; NIS22]) post-quantum scheme SIKEp434 [Aza+19], and
- the Poly1305 message-authentication scheme [Ber05b].

It is worth noting that Erbsen et al. [Erb+19] reported that their generated C code was roughly the best-performing available for all elliptic curves, up to the usual vagaries of C compiler optimisers fluctuating in behaviour across versions. At the time of publishing, this C code has already been incorporated into popular SSL libraries like BoringSSL [Bor23] and Mozilla NSS [Moz23], which, among other projects, power Chromium-based browsers like Google Chrome and Microsoft Edge as well as Mozilla Firefox.

2.4 Evolutionary Computing

Eiben and Smith [ES15] describe *evolutionary computing* as a research area that is inspired by the process of natural evolution. In particular, the view of natural evolution is offered by molecular genetics: The outside view of an individual is manifested in its genetic code and a change in the genetic code will typically manifest itself in the individual’s properties that can be seen from the outside.

The key idea of optimisation algorithms in evolutionary computing is to maintain a population of individuals that improves over time. This population is randomly initialised. Iteratively, each individual’s quality is assessed, and a subset of the population is selected (typically the highest-quality individuals) to survive and produce the next generation. The generation steps from one

generation into the following, consisting of unary *mutations* of individuals and *recombination* of pairs or sets of individuals.

2.4.1 Genetic Improvement

Genetic Improvement (GI) is an area that overlaps with the field of evolutionary computing. GI is often used for automatic software improvement [Lan15]. Typical improvements can be the automatic removal of bugs in software [Wei+10], improved memory resource usage [Wu+15], or to run faster [LH15; LVO17; Pee+22]. To solve real-world problems with GI algorithms, at least the following three components must be available. First, a representation of the individuals must exist in the algorithm. Second, a function must exist that assigns a quality measure to an individual. Last, at least one variation operator must exist that can be applied to an individual to generate another one.

Representation

The *search space* is the space of all potential solutions; typically, it is too large to enumerate exhaustively. Solutions, or simply points of this space, are called the *genotypes*. Genotypes encode the real-world solution, which is called the *phenotype*. An analogy from the world of construction: The blueprint of the building is the genotype, and the constructed building is the phenotype. There must exist means to transform the genotype into the phenotype.

Fitness Function

To guide the optimiser, a *fitness function* (also known as an *objective function* or *evaluation function*) assigns a quality measure to genotypes [ES15]. Typically, the fitness function creates the phenotype from the genotype and measures its quality in the phenotype space. The result of the fitness function generally is a floating point value. During the optimisation, the search space is searched for an individual that maximises or minimises this function.

Mutation

A mutation is the unary variation operator applied to the genotype. It delivers a (slightly) modified *mutant*, also known as a *child* or *offspring* [ES15]. This operator is called a recombination (or crossover) operator when multiple genotypes are combined into one or several offspring.

2.4.2 Random Local Search

Random Local Search (RLS) [AD11; DN19] is a simple optimisation strategy often efficient and effective in finding *local optima* in the search space. A local optimum is a point in the search space with only lower-quality neighbours. Related is the idea of a plateau, which consists of multiple solutions that are of equal quality.

A run of RLS typically starts from a random genotype. It then applies a random mutation to the current genotype, generating another solution within the search space. If the mutation improves (or does not deteriorate) the phenotype, the mutated genotype is kept, replacing the current one. Otherwise, the mutant is discarded. This mutation-and-evaluation cycle repeats until some predefined termination condition is satisfied.

2.4.3 Bet-and-Run

RLS is often highly sensitive to the initial conditions, i.e., the genotype from which it starts. To address this, the simple Bet-and-Run heuristic [FM14; WWW19] turns the sensitivity to the initial conditions into an advantage by employing multiple runs. A typical use of Bet-and-Run starts with numerous independent runs of RLS, each optimising for a predefined number of mutations. After this initialisation step, the algorithm selects the best run and lets this run continue optimising from that step, stopping when the overall budget of mutations is exhausted.

Chapter 3

CRYPTOPT Components and Input Language

This chapter introduces the components that are used in the CryptOpt project, shows how those components fit together and with which data structures the components communicate.

3.1 Components

Multiple components are involved in CRYPTOPT. [Figure 3.1](#) shows those components inside of CRYPTOPT and how data flows between them. CRYPTOPT’s input is essentially a stringified Fiat IR. Input from other tools must process their representations accordingly.

The internal representation that CRYPTOPT uses is called CRYPTOPT IR. It consists of the operations, their order, and implementation details. As such, it serves as the genotype for genetic optimisation. Small preprocessing steps translate this input into CRYPTOPT’s internal CRYPTOPT IR. This preprocessing handles trivial optimisations, such as dead code elimination, [Section 9.6](#) elaborates. The mutation to generate another genotype is applied on this CRYPTOPT IR. Applying the mutation to the CRYPTOPT IR evades the need for multi-level optimisation on the assembly itself and, as such, also reduces technical debt.

The *Code Generation* process generates the actual assembly (the phenotype) from the CRYPTOPT IR (the genotype). Recall that every genotype needs to be assigned a quality measure during genetic optimisation. This quality measure is assigned to each genotype by creating the phenotype and feeding it into the measure component *MeasureSuite*. *MeasureSuite* executes both assembly programs on the hardware and assigns a robust fitness value to both programs. This fitness value guides the optimiser in the search space. *Guiding* in this context means marking the slower mutant to discard it.

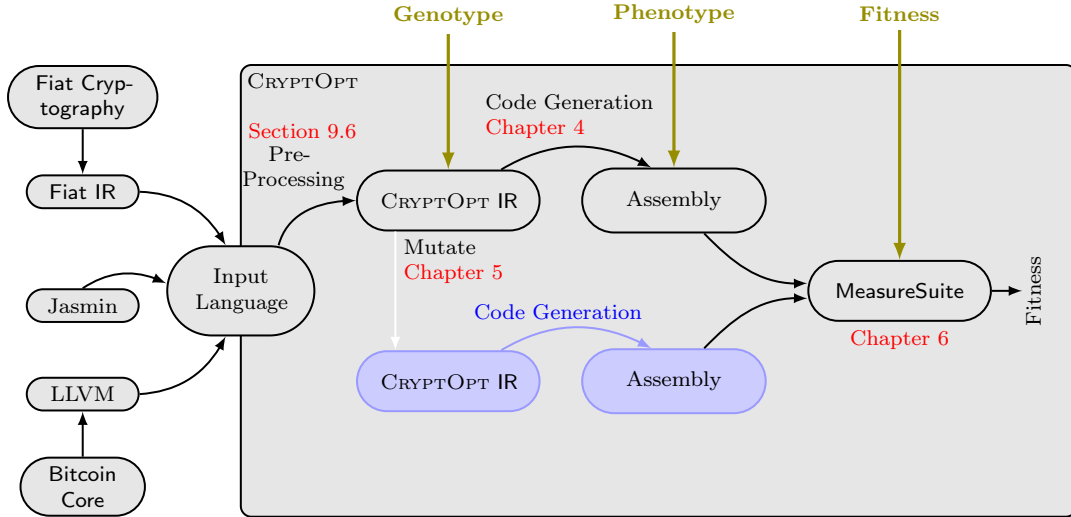


FIGURE 3.1. CRYPTOPT component and data-flow diagram.

3.2 Input Language

The input language supports the operations that are needed in cryptographic implementations. In particular, those implementations for the arithmetic do not require complex control flow like loops or exceptions but only logic and arithmetic operations. [Listing 1](#) shows the complete grammar of the input language. It consists of the typical unary and binary operations for logic and arithmetic, including shifts and rotates. In addition to that, it features add-with-carry (`addcarryx`) and subtract-with-borrow (`subborrowx`), which can read and write carry bits explicitly into intermediate variables. Lastly, the grammar allows the implementation of Φ -nodes with a conditional-move operation. One requirement is that the input must be in Static Single Assignment (SSA) form, i.e. every symbol can only be assigned a value once.

The remainder of this section describes a C-like function signature ([Section 3.2.1](#)) and a C-like function body ([Section 3.2.2](#)). [Section 3.2.3](#) concludes with the JSON specification of the function that is actually fed into CRYPTOPT. For brevity, the remainder of this thesis uses the C-like functions to convey the arithmetic of the functions that are used as the input.

Intermediate Variable	xN
Read-Only Variable	$\text{argN}[n]$
Write-Only Variable	$\text{outN}[n]$
Immediate Integer	i
Operand	$e ::= xN \mid i \mid \text{argN}[n]$
Unary Operator	$uo ::= ! \mid \sim \mid = \mid \text{static_cast}$
Binary Operator	$bo ::= \& \mid \mid ^ \mid * \mid + \mid - \mid \text{mulx}$
Tertiary Operator	$to ::= \text{addcarryx} \mid \text{subborrowx}$
Shift Operator	$so ::= << \mid >> \mid \text{ror} \mid \text{rol}$
Conditional Move Operator	$\text{cmo} ::= \text{cmovznz}$
Expression	$E ::= xN \leftarrow uo(e)$ $\quad \mid \text{outN}[n] \leftarrow =(e)$ $\quad \mid (xN, xN) \leftarrow bo(e, e)$ $\quad \mid (xN, xN) \leftarrow to(e, e, e)$ $\quad \mid (xN, xN) \leftarrow so(e, i)$ $\quad \mid (xN) \leftarrow \text{cmo}(xN, e, e)$

LISTING 1. Input-language syntax.

3.2.1 Function Signature

The cryptographic arithmetic is specified to CRYPTOPT as a function with input and output arrays. Listing 2 depicts a C-like function signature of this input.

```

1 void function(
2     uint64_t out1[n], ..., uint64_t outN[n],
3     uint64_t arg1[n], ..., uint64_t argN[n]
4 );
```

LISTING 2. C-like function signature of the CRYPTOPT input.

The input is read from the $\text{argN}[n]$ arrays and the output is written to the $\text{outN}[n]$ arrays. Those variables are explicit elements of the grammar as listed in Listing 1. The maximum number of function parameters is six, and the array length n is configurable. There must be at least one output and one input. The output arrays must come first, and the input arrays after that.

Pointer Aliasing

Pointer aliasing describes the situation in which memory regions can overlap. This can be a problem if the generated code assumes the regions to be distinct and overwrites input data. Pointer aliasing is currently supported on one of three levels.

On the first level, CRYPTOPT ensures that all values are read from the input arrays before writing any values to any output arrays. With this setting, the memory regions can partly overlap, and the pointer can be equal. This allows for calls such as `function(&x, &x + 1)`, with the first parameter being the output and the second parameter being the input.

The second level allows pointers to be equal, but the data cannot partially overlap. In this mode, CRYPTOPT ensures that `arg1[n]` is read before `out1[n]` is written, for all array slots $0 \dots n-1$ individually. This allows for function usage, which calculates results in place, e.g. `function(&x, &x)`.

The last level assumes that the input is entirely distinct from the output. In this mode, CRYPTOPT can emit code that reads values from any `arg1[n]` at any point in the program, especially after `out1[n]` has been written with the same `n`. This restricts function calls to `function(&x, &y)` where the pointers are distinct and the memory regions do not overlap, i.e. $x \neq y$, $x + n < y$, and $y + n < x$.

The way to handle the pointer aliasing can be set to CRYPTOPT at runtime via the `--memory-constraints` switch.

3.2.2 Function Body

Inside the function body, the elements of the input array can be combined with arithmetic and logical operations. Intermediate results must be written to logical variables (or *symbols*), and the final output must be written to the output arrays. The expression shown in Equation 3.1 is implemented by the example function shown in Listing 3. This showcases the semantics of the function body.

$$(\text{arg1}[0] \ll 0x1F) \mid \text{arg2}[0] \tag{3.1}$$

Precisely, to calculate the intermediate result of the shift operation (`<<`) in parenthesis, the input is read from the `arg1[0]` parameter and the shifted result is written to the intermediate variable `x1`. The second operation calculates the logical or operation (`|`) and writes the result to `x2`. The final operation is the assign operation (`=`) that writes the intermediate result `x2` to the output.

3.2.3 JSON

CRYPTOPT reads the specification for the input function in JSON syntax. Listing 4 shows the JSON specification for the example function from above (Listing 3).

```

1 void shortexample(
2     uint64_t out1[1],
3     uint64_t arg1[1], uint64_t arg2[1]
4 ){
5     uint64_t x1 = arg1[0] << 0x1F;
6     uint64_t x2 = x1 | arg2[0];
7     out[1]      = x2;
8 };

```

LISTING 3. Example expression (`arg1[0] << 0x1F`) `| arg2[0]` implemented in an SSA form in C.

The function signature consists of the property `operation`, which specifies the name of the function, and the properties `arguments` and `returns`, which specify the input and output arguments to the function. All objects that specify function arguments in the arrays specify the `name` of the function’s argument, the length of the elements per array in `datatype`, and the lower and upper bound of the elements in the arrays.

The operations of the function’s body are specified as an array of operation objects in the `body` property of the top-level object. Every such operation typically has a `datatype` of `u64` and specifies the name of the symbol in which the operation writes the intermediate result (`x1`, `x2`). The datatype `u128` is partially supported: CRYPTOPT will pre-process those to two separate `u64` variables. The logic or arithmetic operation is specified in the `operation` property of the operation object. Those are the operators defined in the grammar (Listing 1), such as `<<`, `|`, or `=`. The operands to the operation are specified in the `arguments` property. Those operands can either be intermediate values (`x1`), function arguments (`arg1[0]`) or immediate values such as `0x1F`. An operand can also be a nested, anonymous operation object. Listing 5 shows this. It essentially inlines the operation object that calculates `x1` into the operation object that calculates `x2`. In that case, the `name` property can be empty, i.e. its value is ignored.

```

{
  "operation": "shortexample",
  "arguments": [
    {
      "name": "arg1",
      "datatype": "u64[1]",
      "lbound": [ "0x0" ],
      "ubound": [ "0x1ffffffff" ]
    }, {
      "name": "arg2",
      "datatype": "u64[1]",
      "lbound": [ "0x0" ],
      "ubound": [ "0xffffffffffffffff" ]
    }
  ],
  "returns": [
    {
      "name": "out1",
      "datatype": "u64[1]",
      "lbound": [ "0x0" ],
      "ubound": [ "0xffffffffffffffff" ]
    }
  ],
  "body": [
    {
      "datatype": "u64",
      "name": [ "x1" ],
      "operation": "<<",
      "arguments": [ "arg1[0]", "0x1F" ]
    }, {
      "datatype": "u64",
      "name": [ "x2" ],
      "operation": "|",
      "arguments": [ "x1", "arg2[0]" ]
    }, {
      "datatype": "u64",
      "name": [ "out1[0]" ],
      "operation": "=",
      "arguments": [ "x2" ]
    }
  ]
}

```

LISTING 4. Example function implemented in JSON (CRYPTOPT IR).

```
{
  "body": [
    {
      "datatype": "u64",
      "name": [ "x2" ],
      "operation": "|",
      "arguments": [
        {
          "datatype": "u64",
          "name": [],
          "operation": "<<",
          "arguments": [ "arg1[0]", "0x1F" ]
        },
        "arg2[0]"
      ]
    }, {
      "datatype": "u64",
      "name": [ "out1[0]" ],
      "operation": "=",
      "arguments": [ "x2" ]
    }
  ]
}
```

LISTING 5. Nested body in JSON format. The function's signature from Listing 4 is omitted here for brevity.

Chapter 4

Code Generation

The code generation phase transforms CRYPTOPT IR to x86-64 assembly code. This includes the three steps of instruction scheduling, instruction selection, and register allocation. While those problems have been solved for general programs [ASU86], CRYPTOPT focusses on a special subset of programs. By restricting CRYPTOPT's input to straightline arithmetic and its output to assembly for x86-64, the problems become simpler, and alternative approaches become viable, which are not viable for the general case. Further, CRYPTOPT is designed to allow mutation of instruction selection and order of operations. This chapter describes how CRYPTOPT solves the problems of code generation. The high-level idea to generate the code with CRYPTOPT is to step through the operations in a valid order and emit code while maintaining a virtual CPU state to keep track of values in registers and flags.

The order of operations influences the order of instructions and the pressure on registers because intermediate values are calculated by operations and stored in registers. A core feature of CRYPTOPT is that the operations are initially ordered randomly. The optimiser later changes this order to improve performance.

[Section 4.1](#) describes conceptually how the code is generated. To illustrate the code generation, [Section 4.2](#) introduces a running example, used throughout the remainder of the chapter. [Section 4.3](#) describes how CRYPTOPT initially orders the operations. Recall from [Section 2.1](#) that arithmetic operations can often be implemented using different assembly instructions. This includes which instructions to use and how to merge the new instructions into the rest of the program. [Section 4.4](#) illustrates this with one possible implementation of the example function.

Aligning code in memory can make a difference in performance [Myt+09; Zhu+15]. CRYPTOPT generates the code using `AssemblyLine` [0xA23a] aligned to the start of a memory page. CRYPTOPT does not directly explore other alignments; it only indirectly explores them by choosing different instructions

and orders. [Section 8.2.5](#) explores alternative code alignment approaches during the code generation phase.

4.1 Generating Assembly

Code generation maps the operations of the CRYPTOPT IR to x86-64 assembly. A naïve way to create this x86-64 assembly is to define a state in the CPU that fits all operations. For example, a state where each variable is stored in a pre-allocated location in the stack frame while the contents of the registers and flags are ignored. With this pre-defined state, all operations can be mapped to blocks of instructions that use the pre-defined state to do the operation and ensure that the pre-defined state is prepared for the next operation. However, this is a costly approach, because it involves many unnecessary memory-read and memory-store instructions. [Listing 6](#) shows x86-64 assembly code that increments a number from the stack twice. Notice, in particular, the unnecessary write-then-read operations in lines 3+5. The benefit of this approach is, that every block of instructions is self-contained, and can be moved to other locations in the program. The two blocks in this example are visually separated by the empty line 4 in [Listing 6](#).

```
1  mov rax, [ rsp ]      ; load value from the stack
2  inc rax               ; increment
3  mov [ rsp ], rax      ; save value to the stack
4
5  mov rax, [ rsp ]      ; load value from the stack (again)
6  inc rax               ; increment (again)
7  mov [ rsp ], rax      ; save value to the stack (again)
```

LISTING 6. Increment a number twice in x86-64 assembly.

CRYPTOPT’s approach aims to reduce the number of (unnecessary) instructions while choosing the most suitable instruction(s) for an operation. In particular, CRYPTOPT uses a virtual CPU that tracks the state of registers and memory before and after emitting instructions for any operation. That way, intermediate values can live purely in registers if a future operation consumes them. [Listing 7](#) shows how the second block uses of the intermediate value already in register `rax`. This approach does not require the two `mov` instructions from [Listing 6](#). However, those two blocks depend on each other, i.e. cannot be moved freely across the program, as the second block requires the value to be in `rax` and only ensures that the result is on the stack.

```
1 mov rax, [ rsp ]      ; load value from the stack
2 inc rax               ; increment
3
4 inc rax               ; increment (again)
5 mov [ rsp ], rax      ; save value to the stack
```

LISTING 7. Increment a number twice in x86-64 assembly, with context awareness.

With this approach, the cross-product of CPU states and potential operations is too large to enumerate. Hence, CRYPTOPT uses *recipes* to break down the preconditions of an instruction into smaller, reusable parts. For example, the first block in Listing 7 is divided into the recipe “load a value from memory into a general-purpose register” and the recipe “increment a value in a general-purpose register”. When the code generation reaches the second block, it simply forgoes this step of loading the value, as it is aware that the value is already in a general-purpose register.

Those recipes can cascade to ensure further preconditions. For example, the aforementioned “load a value from memory into a general-purpose register” requires at least one general-purpose register to be free to use. If not, the code generation cascades into the recipe for “free a register by spilling a value to the stack”. Once this recipe is completed, i.e., the instructions are emitted, and the code generation continues with the prior recipe of moving that value to a register (that it previously freed). This situation is shown in Listing 8, where line 1 spills `rax` to a slot on the stack, in this case to slot position 1 (`[ rsp + 0x08 ]`). The rest of this listing then continues as before. Section 4.4.4 elaborates on CRYPTOPT’s spilling mechanics.

```
1 mov [ rsp + 0x08 ], rax; spill rax to a free stack slot
2
3 mov rax, [ rsp ]      ; load value from the stack
4 inc rax               ; increment
5 inc rax               ; increment (again)
6 mov [ rsp ], rax      ; save value to the stack (again)
```

LISTING 8. Increment a number twice with spilling in x86-64 assembly.

Similar to the mentioned recipes, recipes exist for situations to load a 1-bit value from a register into a flag (CF or OF). The code for two alternative recipes is shown in Listings 9 and 10. Listing 9 loads a flag from the 8-bit `al` register into the carry flag CF using the `add` instruction. This has two side effects: It

changes the value of `al` and it clears the overflow flag `OF`. If the flag's value is still needed for another operation an optional `setc al` after the `add` instruction can set the value to `al` again. To load the value into `OF` while clearing `CF`, `0x7f` can be used instead of `0xff`.

```
1 add al, 0xff          ; load flag from al to CF, zeroes OF
2 setc al              ; optionally setting the value back to al
```

LISTING 9. Load a spilled flag from a register while clearing `OF`.

Clearing `OF` while loading to `CF` is unwanted if the overflow flag contains a value that needs to be preserved. In that case, the `adcx` instruction can be used, which leaves the value in `OF` unchanged but requires a spare 64-bit register that contains the value -1 .¹ Listing 10 shows the x86-64 assembly for this scenario.

```
1 clc                  ; clear CF
2 movzx rax, al        ; zero-extend al
3 mov rbx, -1          ; mov -1 to rbx
4 adcx rax, rbx        ; load flag to CF, OF unchanged
```

LISTING 10. Load a spilled flag from a register while preserving `OF`.

For the flag-preserving case (using `adcx`), there is no equivalent to `clc` that solely clears `OF`. But there do exist the `inc` and `dec` instructions, which increment or decrement a value, respectively, while clearing `OF` and not affecting `CF` [Int20a]. Recall that `adox` and `adcx` read and write the flags in their addition. Hence, for addition operations where the carry-in is zero, only one flag needs to be cleared. To achieve this, CRYPTOPT emits blocks to decrement a 0 with `dec`, or to increment a -1 using `inc`.

As shown in Listing 10, a -1 can be helpful to have in a register to load a flag. At the same time, a 0 can be useful, too, e.g. if a flag (`OF`) needs to be added to a value: `adox rax, rbx` with `rbx` being 0.

A more complex example that showcases the flag loading with a 0 and the cascading nature of CRYPTOPT's code generation is a 64-bit addition of two operands that are in general-purpose registers (`r8` and `r9`), the carry-in is in an 8-bit register (`c1`), the carry flag `CF` contains some value that cannot be overwritten, and the overflow flag `OF` does not contain any value required for the

¹Additions in x86-64 are unsigned, while the `FLAGS` register indicates signed overflow. The trick of adding -1 to the carry-in is that if the carry-in is zero, the carry-out (i.e. overflow flag when using `adox`) is zero. If the carry-in is one, the addition will overflow and, in turn, set the carry-out flag `OF`.

computation (i.e. can be overwritten). Listing 11 shows potential instructions as CRYPTOPT emits them. The last line, line 6, is the actual add operation, where the previous instructions (lines 1-4) set up the precondition that the carry-in is in the overflow flag OF. The cascade step is to load the carry-in into the OF. To do that, CRYPTOPT issues the `adox` in line 4 with a cleared OF, one register holding the carry-in from the memory and the other register holding the value -1 . Those preconditions are met by cascading further. The `adox` instruction cannot read from 8-bit registers, so CRYPTOPT issues the `movzx` instruction in line 3. This zero extends the value in the 8-bit `cl` to the 64-bit `rcx`. To clear the OF and to create a register with the value -1 , CRYPTOPT cascades into further recipes. In particular, it issues the `dec` instruction (decrement) in line 2 in combination with the `mov` instruction in line 1, assuming `rax` is free to use and no other (unused) register already contains zero.

```

1  mov    rax, 0x0                ; mov 0 to rax
2  dec    rax                    ; clears OF; rax has value -1
3  movzx  rcx, cl                ; load flag to 64-bit register
4  adox   rcx, rax               ; load flag
5
6  adox   r8, r9                 ; final addition, with the OF

```

LISTING 11. Add-with-carry in x86-64 assembly.

Note that in line 4 of Listing 11, the value of `rcx` is overwritten, leaving `rax` with the value -1 . Alternatively, the operands could be swapped, overwriting `rax` and preserving the flag in the register `rcx`.

What can be seen from those examples is that the recipes contain decisions. For example, the decision to load a flag with `add` or `adc` (cf. Listings 9 and 10). Another example is deciding whether to clear the OF by moving -1 to a register and using `inc`, or moving 0 and `dec`.

Those decisions can be grouped into *straightforward* and *not-straightforward* decisions depending on the context, i.e. the state of the virtual CPU. If OF contains an unknown value, the former decision is a straightforward decision – use `add` because it defines the state of OF to 0 and uses fewer instructions. Similarly, if OF contains a value that needs to be preserved, it is straightforward to use `adc`. For the latter decision, whether to use -1 or 0, it is not straightforward which one is superior, as can be seen in Listings 10 and 11; resulting in -1 and 0 can both equally likely save instructions in future operations.

At the time the code generator implements one operation and encounters those not-straightforward decisions, CRYPTOPT chooses randomly which alternative to use, and lets the optimising component change this decision later, [Section 4.4.3](#) elaborates.

4.2 Example

[Listing 12](#) shows a C version of an example function that is used to explain the code generation and the optimisation. It consists of six operations, four multiplications, and two 128-bit additions. The first four multiplication operations calculate 128-bit products from the 64-bit arguments. After that, the intermediate 128-bit results are added together with the addition operations. One possible implementation is described in the following two sections. [Section 4.3](#) defines the order and [Section 4.4](#) describes the code emission.

```

1  #include <stdint.h>
2
3  typedef unsigned __int128 u128;
4
5  void example(
6      uint64_t out1[3],
7      uint64_t arg1[3], uint64_t arg2[3]
8  ) {
9      u128 x1 = (u128)(arg1[2]) * arg2[0];
10     u128 x2 = (u128)(arg1[1]) * arg2[0];
11     u128 x3 = (u128)(arg1[1]) * arg2[1];
12     u128 x4 = (u128)(arg1[0]) * arg2[2];
13     u128 x10 = x1 + x2;
14     u128 x11 = x3 + x4;
15     // Further code that writes to output is omitted for brevity.
16 }
```

LISTING 12. Example function in C.

4.3 Order the Operations

The input function to CRYPTOPT is specified as the set of operations. It is specified in a Static Single Assignment (SSA) form, i.e. any symbol can only be assigned a value once (cf. [Section 3.2](#)). This operation-ordering step aims to order those operations such that the next step can emit the assembly code by simply visiting the operations in this particular order. This order is only

generated from scratch *once per optimisation run*. The optimising steps may later change this order (Section 5.1).

The remainder of this section explains how the set of operations is represented as a data-flow graph (Section 4.3.1) and how the ordering procedure works at a high level (Section 4.3.2). Section 4.3.3 applies the ordering procedure to the example function.

4.3.1 Data Flow

Because the input is in SSA form, the operations implicitly create a data-flow graph, as shown in Figure 4.1. In this graph, the nodes represent the operations, and the arrows represent the data flow. The operation that calculates the intermediate variable `x1` is called `X1`, analogously for the other variables. This data-flow graph clearly shows the dependencies between the operations. `x1` and `x2` must have been calculated in order to calculate `x10`; similarly for `x11`. Calculating `x10` and `x11` are independent of each other and can be calculated in any order, i.e. first `x10`, then `x11` or first `x11` and then `x10`. Similarly, the multiplications can be calculated in any order without changing the semantics of the code.

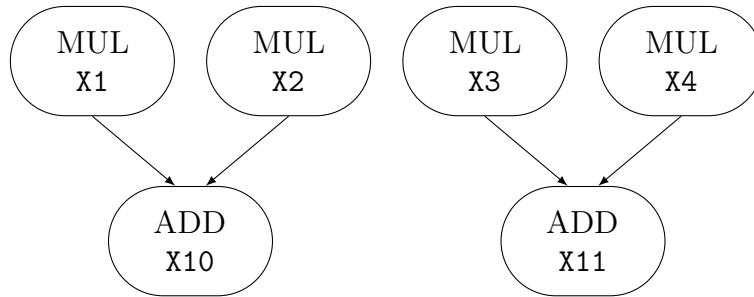


FIGURE 4.1. Data-flow graph of the example function.

Any order of the operations that obeys the data flow is a valid implementation order. Hence, the ordering procedure aims to find a topological order of the nodes in the data-flow graph.

4.3.2 Ordering Procedure

As the order of operations influences the order of instructions and the pressure on registers, CRYPTOPT initially orders the operations randomly. The optimiser may change this order later. At a high level, CRYPTOPT's ordering procedure is based on Kahn's algorithm [Kah62]. Kahn's algorithm iteratively creates a topological order of *events* based on *predecessor constraints* between those events.

Kahn's algorithm maintains lists of the events and their constraints between each other that are updated whenever an event is put into the resulting list. An event e can be scheduled if and only if the number of events constrained to be a predecessor of event e is zero. In this resulting list, any event e is only positioned if all predecessor events are positioned before event e , creating a topological order. A topological order can be created for any directed acyclic graph, and Kahn's algorithm works for any instance of a directed acyclic graph. CRYPTOPT's *events* are the operations, and the *predecessor constraints* are implicitly created by the data flow of intermediate values between the operations.

Algorithm 1 depicts CRYPTOPT's implementation. It exposes the function `toposort`, which is called with the set of operations as an input and returns a valid order to implement those operations. The input to the function `toposort`

Algorithm 1: `toposort`

```

input  : all the set of all operations
output : out a topological order of operations

1 function toposort(all)
2 begin
3   out  $\leftarrow$  []
4   available  $\leftarrow$  []
5   repeat
6     schedulable  $\leftarrow$  []
7     foreach op  $\in$  all do
8       if isSchedulable(op, available) then
9          $\lfloor$  add op to schedulable
10        /* choose one schedulable operation uniformly at random */
11        chosen  $\xleftarrow{\$}$  schedulable
12        append chosen at the end of out
13        add chosen's computed symbols to available
14        remove chosen from all
15  until all is empty;
16  return out

```

is the set `all`, which contains all operations of the input function. The output list `out` is an ordered list of operations that is a valid topological order of the input. The list `out` is initialised empty (line 3), and filled with operations during the execution of `toposort` (line 11). CRYPTOPT analyses the data flow by maintaining a set `available` (line 4+12) of all the intermediate symbols that are computed by the operations that are already in the list `out`.

To fill the output list `out`, the algorithm performs the following: Scan the set of `all` (line 7) operations for schedulable operations (line 8+9). An operation is

Algorithm 2: isSchedulable

```

input : op operation to test
         available set of available symbols
output: boolean, whether op can be scheduled

1 function isSchedulable(op, available)
2 begin
3   args ← op's arguments
4   foreach arg ∈ args do
5     if arg is a function argument then
6       /* argN[n] are always 'available' */
7       /* (memory constraint logic omitted for brevity) */
8       continue
9     if arg is constant then
10      /* 0x... are always available */
11      continue
12    /* must be an intermediate variable */
13    if arg ∉ available then
14      /* and if arg is not available, op cannot be scheduled */
15      return false
16  return true

```

schedulable if all arguments are either values from the function parameters (i.e. `argN[n]`), constants, or symbols contained in the `available` set. CRYPTOPT selects one schedulable operation uniformly at random (line 10) and appends it to the end of the output list `out` (line 11), adds its computed symbols to the `available` set (line 12), and removes it from the `all` set (line 13). At the start of the algorithm, the set of `schedulable` are the operations that only consume the function's arguments and constants. Once they are scheduled, their symbols are added to `available` rendering other operations schedulable. For a valid input function, it is guaranteed that there is always at least one operation schedulable because the data-flow graph of the input function is a directed acyclic graph. CRYPTOPT iteratively repeats this main loop until the set of `all` is empty (line 14). At the point where `all` is empty, all operations are in the output list `out` if the input is a valid directed acyclic graph. This output list is returned and contains a valid order of operations that can be stepped through to generate the assembly.

To check that an operation is schedulable, CRYPTOPT uses [Algorithm 2](#), which exposes the function `isSchedulable`. In general, an operation `op` is schedulable if its arguments are available. Hence, first, the algorithm extracts the arguments (line 3) and checks their availability. Arguments to the function, i.e.

arguments of the form `argN[n]`, are always considered available (line 5+6). Constants such as `0x0` or `0x007fffffffffffffff` are also always available (line 7+8). If an operation's argument is not an argument to the function and not a constant, it must be an intermediate value that another operation calculates. In this case, line 9 checks whether the requested argument `arg` is in the `available` set, i.e. is calculated by an operation that is already in the `out`-list. If `arg` is not in this list, `op` cannot be scheduled (line 10) and `isSchedulable` returns *false*. Alternatively, `isSchedulable` returns *true* to indicate that the operation `op` can currently be scheduled.

Recalling from the pointer aliasing section in [Section 3.2.1](#), memory regions of `argN[n]` and `outN[n]` can overlap. If they overlap and a value `arg1[0]` is read by an operation after a previous operation wrote data to `out1[0]`, it results in a dirty read, because they refer to the same region in memory.

The previous description assumes that any `argN[n]` is always available, so the function must ensure that no operation overwrites this data. The logic to handle memory constraints is omitted in [Algorithm 2](#) but consists of an extra check for operations that write to `outN[n]`. This additional check allows for those operations to be scheduled,

- either always (when no constraints are present) or
- if no respective `argN[n]` needs to be read by other operations that are not in the output list already (if pointers can be equal, but data cannot partially overlap) or
- if any `argN[n]` needs to be read by other operations that are not in the output list already (the most restrictive setting that allows for calls of the function with any memory configuration).

4.3.3 Example

This section applies the ordering procedure to the example function to illustrate its workings. The `toposort` function is called with the set of all operations from the example function in [Listing 12](#). After the initialisation and before the loop starts in line 5, the three sets are:

```
all   : [ X1, X2, X3, X4, X10, X11 ]
available : [ ]
out   : [ ]
```

The inner loop (line 7) iterates over `all` and determines, which values are schedulable. At line 10, the array `schedulable` contains all values that only read from function arguments, because `available` is currently empty.

```
schedulable : [ X1, X2, X3, X4 ]
```

Line 10 of the algorithm chooses randomly from `schedulable`, e.g. `X2`. It adds `X2` to the end of `out` (line 11), adds its computed values `x2` to `available` (line 12) and removes `X2` from `all` (line 13). At line 13, the sets are:

```
all  : [ X1, X3, X4, X10, X11 ]
available : [ x2 ]
out  : [ X2 ]
```

The next iteration is very similar because no other values become schedulable by calculating `x2`. This means that `schedulable` contains:

```
schedulable : [ X1, X3, X4 ]
```

The randomness may choose `X1` in the next step in line 10. It adds `X1` to the end of `out`, adds its computed values `x1` to `available`, and removes `X1` from `all`. After this, line 13, the sets are:

```
all  : [ X3, X4, X10, X11 ]
available : [ x2, x1 ]
out  : [ X2, X1 ]
```

In iteration three, after calculating the array `schedulable` at line 10, `schedulable` now contains all operations that only read from function arguments (`X3, X4`) and `X10` because `x1` and `x2` are available to calculate it.

```
schedulable : [ X3, X4, X10 ]
```

The randomness decides which one of those to pick, e.g. `X4`. It adds `X4` to the end of `out`, adds its computed values `x4` to `available`, and removes `X4` from `all`. After this, line 13, the sets are:

```
all  : [ X3, X10, X11 ]
available : [ x2, x1, x4 ]
out  : [ X2, X1, X4 ]
```

In the next iteration of the algorithm, `schedulable` at line 10, will contain [`X3`, `X10`], out of which a random one is selected, e.g. `X10`. After this, line 13, the sets are:

```

all      : [ X10, X11 ]
available : [ x2, x1, x4, x10 ]
out      : [ X2, X1, X4, X10 ]

```

In the second to last iteration `schedulable` contains the operation `X3`, which is the only one to be selected. In the last iteration `schedulable` only contains the remaining [`X11`], which finally is schedulable, because all required arguments are in `available`. At the end of the algorithm, the sets contain:

```

all      : [ ]
available : [ x2, x1, x4, x10, x3, x11 ]
out      : [ X2, X1, X4, X10, X3, X11 ]

```

This terminates the loop because the set `all` is empty. The function returns the array `out` with contents [`X2`, `X1`, `X4`, `X10`, `X3`, `X11`].

4.4 Generate Assembly for the Example Function

The code-generation phase emits the x86-64 assembly code from the list of ordered operations, as illustrated in [Section 4.3.3](#). This phase includes the steps, that are typically handled by code generators: instruction scheduling, instruction selection, and register allocation. CRYPTOPT addresses those steps simultaneously. The high-level procedure is: Initialise a virtual CPU that tracks the contents of registers and flags as well as the stack. Then, step through the operations, while emitting assembly code and updating the virtual CPU with the new values that are moved and written by the emitted instructions. The final step is to generate function pro- and epilogues to make the generated code compliant with calling conventions.

[Section 4.4.1](#) describes the details of the virtual CPU and how it is initialised in the general case as well as based on the example. [Section 4.4.2](#) illustrates the general case of emitting code, and [Section 4.4.3](#) elaborates on special cases. [Section 4.4.4](#) handles the case that the number of intermediate results the program needs exceeds the number of available registers (i.e. spilling). [Section 4.4.5](#) describes how CRYPTOPT generates the function pro- and epilogue.

4.4.1 Initialise Virtual CPU

The idea in CRYPTOPT’s code generator is to maintain a virtual CPU that tracks the data manipulated during the code generation. During the code generation, the virtual CPU acts as a single point of truth about which data is stored where. This information is critical for choosing valid instructions for the operations.

The virtual CPU in CRYPTOPT contains a virtual stack that is assumed to be infinite in size, a list of general-purpose registers, and parts of the **FLAGS** register. In CRYPTOPT’s current use case, only the carry flag **CF** and the overflow flag **OF** from the **FLAGS** register are maintained as they are used in addition chains. Other flags are not needed in the current workload and, as such, are omitted from the implementation.

The stack is implemented as an array whose elements are symbols of the operations and their position in the array corresponds to the position on the virtual stack. All stack elements are 64-bit wide. Longer elements may be split across multiple slots (e.g. 128-bit value **x3** is split into **x3_lo** and **x3_hi**, which are stored independently), and shorter elements such as bytes or bits occupy only the lower bits of the 64-bit slot. For example, the element “**x2**” in position 4 in the stack array describes the existence of **x2**’s value at `[ rsp + 4 * 0x08 ]`. The array is initialised empty. [Section 4.4.4](#) elaborates.

The flags **CF** and **OF** are initialised with undefined values, and the general-purpose registers are initialised as follows: First, the callee-saved registers **rbx**, **rbp**, **r12**, **r13**, **r14**, **r15** hold unknown values that need to be restored before return if those registers are used in the function. Secondly, according to the specification of arguments in the input function and the System-V calling convention [[Lu+18](#)], the registers **rdi**, **rsi**, **rdx**, **rcx**, **r8**, **r9** get initialised with **outNs** and **argNs**.

In the example function from [Listing 12](#), the input function has one output array and two input arrays. Accordingly, the registers in the virtual CPU will be assigned as follows.

```

rdi : out1
rsi : arg1
rdx : arg2
rbx : precious-rbx
rbp : precious-rbp
r12 : precious-r12
r13 : precious-r13
r14 : precious-r14
r15 : precious-r15

```

Every register that is not filled with anything is considered free to use. In the example case, registers `rax`, `rcx`, `r8`, `r9`, `r10`, and `r11` are free. Those registers are used without any preference. Even the register `rdx`, which has special semantics within the `mulx` instruction, is used like any other general-purpose register. An alternative to this bias-free selection of free registers is to preferably load values into `rdx` if multiple `mulx` instructions would use the same value. [Section 8.2.5](#) evaluates this approach.

4.4.2 Step Through the Operations

[Section 4.3](#) describes how CRYPTOPT orders the operations in the first step; this section describes the second step in the code generation phase, which emits the assembly while stepping through the order.

At a high level, different code patterns are emitted for each operation. This section first explains the general code generation with the example function from [Listing 12](#) and elaborates on it in [Section 4.4.3](#). [Section 4.4.4](#) describes how CRYPTOPT spills values to the stack, if necessary.

Recall that the order of operations from the last step results in the following order: [`X2`, `X1`, `X4`, `X10`, `X3`, `X11`]. With the virtual CPU initialised and this order fixed, CRYPTOPT emits the assembly for every operation in order. The first operation is `X2` and calculates `u128 x2 = (u128)(arg1[1]) * arg2[0]`. CRYPTOPT uses the `mulx` instruction for multiplications that result in 128-bit numbers. `mulx` requires one factor to be in the special register `rdx`. The second factor is encoded as its last argument. `mulx` stores the high and low results in the registers that are specified as the first two arguments. The emitted assembly for this multiplication is in [Listing 13](#). Note that `rdx`, as per the initialisation of the virtual CPU, contains the value `arg2`. Because `rdx` is required for `mulx`, CRYPTOPT emits a `mov` to save the pointer `arg2` to a free register, e.g. `rax`.

Line 1 in Listing 13 shows this copy of `arg2`, line 2 shows the loading of the first factor into `rdx`, and line 3 shows the multiplication.

```

1  mov  rax, rdx                ; save arg2-pointer
2  mov  rdx, [ rax ]            ; load arg2[0]
3  mulx r10, r11, [ rsi + 0x08 ] ; calculate x2

```

LISTING 13. Code to calculate `x2` in x86-64 assembly.

This concludes the code emission for `X2`. The virtual CPU is updated to `rdi=out1`, `rsi=arg1`, `rax=arg2`, `rdx=arg2[0]`, `r10=x2_hi`, `r11=x2_lo` (omitting the caller-save registers for brevity).

After this update, CRYPTOPT emits code for the next operation, `X1`, which calculates `u128 x1 = (u128)(arg1[2]) * arg2[0]`. This results in the code shown in Listing 14.

```

1                ; arg2[0] already in rdx
2  mulx r8, r9, [ rsi + 0x10 ] ; calculate x1

```

LISTING 14. Code to calculate `x1` in x86-64 assembly.

Noteworthy here is that the instruction to load `arg2[0]` can be omitted because it is already present in the register `rdx`. The only required instruction is the `mulx` instruction shown in line 2 of Listing 14. After this, the virtual CPU gets updated to `rdi=out1`, `rsi=arg1`, `rax=arg2`, `rdx=arg2[0]`, `r8=x1_hi`, `r9=x1_lo`, `r10=x2_hi`, `r11=x2_lo`.

The next operation is `X4`, calculating `u128 x4 = (u128)(arg1[0]) * arg2[2]`, the resulting x86-64 assembly is shown in Listing 15. Line 1 loads one factor into `rdx`. Note that `rcx` is the last free register. However, `mulx` first reads the value from `rdx`, then performs the calculation, and then writes the values to the output registers. Hence, `rdx` can be reused. Line 2 shows how the high result of `x4` is saved in `rdx`. After this, the virtual CPU gets updated to `rdi=out1`, `rsi=arg1`, `rax=arg2`, `rdx=x4_hi`, `rcx=x4_lo`, `r8=x1_hi`, `r9=x1_lo`, `r10=x2_hi`, `r11=x2_lo`.

```

1  mov  rdx, [ rax + 0x10 ]      ; load arg2[2]
2  mulx rdx, rcx, [ rsi ]        ; calculate x4

```

LISTING 15. Code to calculate `x4` in x86-64 assembly.

The next operation is `X10`, calculating `u128 x10 = x1 + x2`. This operation is a 128-bit addition. The x86-64 ISA does not have a single instruction for

this; instead, two 64-bit additions are required to add the low limbs and high limbs separately. In general, CRYPTOPT supports those carry chains by using the `adcx` and `adox` instructions, which use the CF or OF flags, as described in [Section 2.1.2](#). The resulting x86-64 assembly is shown in [Listing 16](#). Line 1 shows how both flags are cleared with the `test al, al` instruction. Line 2 shows the addition instruction using the OF for the low limb. This will save the carry bit of the low-limb addition in OF. Line 3 calculates the high limb while reading the carry bit from the low-limb addition.

```

1 test al, al                ; clear both flags
2 adox r9, r11              ; calculate x10_lo using OF
3 adox r8, r10              ; calculate x10_hi using OF

```

LISTING 16. Code to calculate x10 in x86-64 assembly.

After this, the state is updated to `rdi=out1`, `rsi=arg1`, `rax=arg2`, `rdx=x4_hi`, `rcx=x4_lo`, `r8=x10_hi`, `r9=x10_lo`, `r10=x2_hi`, `r11=x2_lo`, `CF=0`. Note how the state keeps track of the cleared carry flag CF.

The next operation X3 calculates `u128 x3 = (u128)(arg1[1]) * arg2[1]`. Note that the state still tracks `x2_lo` and `x2_hi` from the previous operation. Additionally, note that there are no free registers to calculate the current operation and that the special register `rdx` contains a live value. In this situation, CRYPTOPT scans the current state of registers to find registers that contain values that are no longer required. In this case, values from `r10` and `r11` are no longer needed, so CRYPTOPT allocates those for the new results `x3`. The resulting x86-64 assembly is shown in [Listing 17](#). Line 1 shows how `rdx` is saved, line 2 loads the factor, and line 3 calculates the product. After this, the state is updated to `rdi=out1`, `rsi=arg1`, `rax=arg2`, `rdx=x3_lo`, `rcx=x4_lo`, `r8=x10_hi`, `r9=x10_lo`, `r10=x3_hi`, `r11=x4_hi`, `CF=0`.

```

1 mov r11, rdx              ; save x4_hi
2 mov rdx, [ rax + 0x08 ]   ; load arg2[1]
3 mulx r10, rdx, [ rsi + 0x08 ] ; calculate x3

```

LISTING 17. Code to calculate x3 in x86-64 assembly.

The last operation is X11, calculating `u128 x11 = x3 + x4`. The resulting x86-64 assembly is shown in [Listing 18](#). The instructions are similar to those shown when calculating the other addition (X10 in [Listing 16](#)), but note that there is no instruction to clear the flags. This is because the virtual CPU keeps track of

the flags. At this point in the code, the addition can be facilitated using the `adcx` instruction and CF.

```
1  adcx rdx, rcx ; calculate x11_lo using CF
2  adcx r10, r11 ; calculate x11_hi using CF
```

LISTING 18. Code to calculate x11 in x86-64 assembly.

The full x86-64 assembly code for the code in Listing 12 is shown in Listing 19. Note how the operations in the C code seem similar, while the corresponding code blocks for each of those operations on the assembly level are very different. This is because CRYPTOPT emits instruction blocks for each of those operations depending on the state of the virtual CPU.

```
1  mov rax, rdx ; save arg2-pointer
2  mov rdx, [ rax ] ; load arg2[0]
3  mulx r10, r11, [ rsi + 0x08 ] ; calculate x2
4
5  ; arg2[0] already in rdx
6  mulx r8, r9, [ rsi + 0x10 ] ; calculate x1
7
8  mov rdx, [ rax + 0x10 ] ; load arg2[2]
9  mulx rdx, rcx, [ rsi ] ; calculate x4
10
11 test al, al ; clear both flags
12 adox r9, r11 ; calculate x10_lo using OF
13 adox r8, r10 ; calculate x10_hi using OF
14
15 mov r11, rdx ; save x4_hi
16 mov rdx, [ rax + 0x08 ] ; load arg2[1]
17 mulx r10, rdx, [ rsi + 0x08 ] ; calculate x3
18
19 adcx rdx, rcx ; calculate x11_lo using CF
20 adcx r10, r11 ; calculate x11_hi using CF
```

LISTING 19. Example function in x86-64 assembly.

4.4.3 Code Decisions Remarks

The previous sections describe how to order the operations and how to generate x86-64 assembly code for them. This section elaborates on the *decisions* CRYPTOPT needs to make while generating this code. CRYPTOPT could optimise over all those decisions, which would cause the search space to grow significantly. The larger the search space, the longer optimisation naturally takes. Hence, the

decisions are categorised into *straightforward* decisions and *not-straightforward* decisions (cf. [Section 4.1](#)). Recall that straightforward decisions are decisions that result in sensible short code, whereas, for not-straightforward decisions, it is not clear from the prior context which code to emit. Only those not-straightforward decisions are directly subject to change by the optimiser during the optimisation. The example has several straightforward and not-straightforward decisions: First, to calculate `x2` (line 2 in [Listing 19](#)), CRYPTOPT decides to load the value of `arg2[0]` into `rdx`, which is not straightforward, because the other factor could have been loaded, too. This has the effect that for the calculation of `x1` (line 6), the value in `rdx` can be reused, which thus omits the `mov` instruction to load one factor and makes the decision of which value to load into `rdx` for calculating `x1` straightforward (i.e. do not emit any instruction).

It is also not straightforward to load the value of `arg2[2]` into `rdx` when calculating `x4` (line 9) instead of loading the other factor `arg1[0]`. Another not-straightforward decision is to clear the flags with `test al, al` and to use the overflow flag instead of the carry flag when calculating `x10` (lines 11-13). Choosing which value to load to `rdx` when calculating `x3` is again not straightforward. Finally, it is straightforward to use `adc` for the last addition (line 19 and line 20) instead of clearing `OF` using the overflow flag because `CF` is zero and `OF` is not.

Those decisions are not straightforward because, in the current state of the virtual CPU and the current operation, it is unclear which decision will result in the best code. Randomly decides on those decisions (as portrayed in the example) and lets the optimising component change those decisions later. The initial random outcome of those not-straightforward decisions is attached to any particular operation in the CRYPTOPT IR. In the example, after the implementation above, operation `X2` bears the decision to “load `arg2[0]` into `rdx`” instead of `arg1[1]`. Similarly, operation `X10` bears “clear the flags with `test al, al` and use `OF` for the addition”.

On the contrary, straightforward decisions are not attached to the operations. In particular, operation `X11` does not contain any information on using `CF` because, in this implementation, it results from the virtual CPU’s context.

4.4.4 Spilling

Every operation that does not write an intermediate value to the output array writes values to registers. When an operation needs a new register to write a value to but no free register exists, one register must be freed. This typically

happens by spilling its value into memory (cf. [Section 2.2.2](#)). Alternatively, the value could be offloaded into unused vector registers. This approach is evaluated in [Section 8.2.5](#). CRYPTOPT spills into memory using the `mov` instruction and keeps track of the next free stack slot in its virtual CPU. A value in register `rax` is spilled to the stack at offset `off` with `mov rax, [rsp + off]`. Subsequently, CRYPTOPT increments the offset `off` by 8 (bytes) for the subsequent spill. If the red zone can be used, `off` is initialised to -16×8 for the first spill to be `mov rax, [rsp - 0x80]`. If the red zone cannot be used, `off` is initialised to 0 to start the first spill at `mov rax, [rsp + 0]`. The stack pointer is adjusted in the function pro- and epilogue if needed (cf. [Section 4.4.5](#)).

What to Spill

The example is short enough that no spilling is required. However, for longer functions, spilling values is typically unavoidable, and it is a hard decision to make which value to spill.

Naturally, spills need instructions and memory bandwidth, and as such, decrease performance in general. Hence, CRYPTOPT tries to reduce the number of spills needed to increase performance. The spilling heuristic prefers to spill registers with the `precious-` prefix (i.e. callee-saved registers). Once they have been spilled, intermediate values of the input function must be spilled. At any given point while stepping through the operations, CRYPTOPT can infer when any value will be used again by simply looking ahead in the current order of future operations. Hence, CRYPTOPT chooses the register to spill that contains the value that will be used latest in the upcoming operations to reduce the number of spills.

For example, if the registers contain values `x1`, `x2`, `x3`, `x4`, and they are used by subsequent operations in the order `x3`, `x2`, `x1`, `x4`, CRYPTOPT chooses to spill the register that contains `x4`. This avoids spilling, for example, `x3` and un-spilling it in the very next operation. Hence, this heuristic overall reduces the amount of spills because values are kept spilled longer.

4.4.5 Finalise Function Pro- and Epilogue

During the assembly code generation, as described above, instructions are emitted to implement the operations while the stack is assumed to be infinite in size. This section now describes how CRYPTOPT generates the function pro- and epilogue.

The first step in the function epilogue is to restore the callee-saved (i.e. caller-safe) registers. Recall from [Section 4.4.1](#) that the callee-saved registers hold precious values that need to be restored. Specifically, all callee-saved registers are assigned values such as `precious-rbx` for the `rbx` register when initialising the virtual CPU. This is now used by scanning the stack for values with this `precious-` prefix. Those values are moved from the stack position back into the respective register with respective `mov rbx, [ rsp + off ]` instructions.

The second step in the function epilogue is to restore the stack pointer to point to the previous stack frame. This step is implemented hand-in-hand with generating the function prologue, which sets up the stack for a specific size. Recall from [Section 4.4.1](#), that the stack is assumed infinite in size. This assumption is made to not need to pre-calculate any stack usage. Instead, after all the instructions have been emitted, the size of the used stack is known. With this value `STACKSIZE` (in bytes), the function prologue is the instruction `sub rsp, STACKSIZE`. Similarly, an `add rsp, STACKSIZE` is emitted to the function epilogue.

The last instruction at the end of the generated x86-64 assembly is the return instruction `ret`, which is emitted at the end. [Listing 20](#) illustrates an example, where two stack slots are used. Line 8 spills `rbx` at some point in the program to the stack, and at some later point, `r12` is spilled (line 12). Lines 19 and 20 restore those values at the start of the function epilogue. The total size of the stack is two slots, with each 8 bytes. Hence, the prologue reserves the stack size (line 2), and the epilogue restores the stack pointer (line 21). The final instruction is the `ret` instruction in line 23.

```
1  ; Prologue start
2  sub rsp, 16
3
4  ; Function operations start
5
6  ; some instructions that implement an operation
7
8  mov [ rsp ], rbx          ; spill rbx to slot 0
9
10 ; some more instructions ...
11
12 mov [ rsp + 0x08 ], r12    ; spill r12 to slot 1
13
14 ; even more instructions ...
15
16
17 ; Function epilogue starts
18
19 mov rbx, [ rsp ]          ; restore rbx from slot 0
20 mov r12, [ rsp + 0x08 ]    ; restore r12 from slot 1
21 add rsp, 16
22
23 ret
```

LISTING 20. Function pro- and epilogue in x86-64 assembly.

Chapter 5

Code Mutations

CRYPTOPT is an optimising code generator that produces x86-64 assembly code. CRYPTOPT relies on techniques from the domain of evolutionary computation. As such, as illustrated in [Figure 1.3](#), CRYPTOPT first randomly generates one x86-64 assembly program that implements the input function as described in [Chapter 4](#). In particular, [Section 4.3](#) describes the initial ordering, or *scheduling*, of operations, and [Section 4.4.3](#) elaborates on not-straightforward decisions that may be changed by the optimiser later. Those two are the main aspects that CRYPTOPT changes during the optimisation. After the code is generated, CRYPTOPT repeatedly mutates the code and measures the execution time of both the original and the mutated code. The mutation is only kept if it does not deteriorate the code’s performance. This technique is called Random Local Search (RLS). A similar method is called *Simulated Annealing*, in which a worse-performing alternative is accepted with a certain likelihood. This alternative is evaluated in [Section 8.2.3](#).

This chapter now describes the mutation details. CRYPTOPT supports two types of mutations:

- (i) Change the scheduling of the operations and
- (ii) change the decisions attached to an operation.

Importantly, those two types of mutations can be applied without changing the program’s semantics. For every mutation in the entire optimisation process (cf. [Figure 1.3](#)), CRYPTOPT selects uniformly at random to change either, the scheduling or the decisions.

[Section 5.1](#) explains how the scheduling of the operations is mutated. [Section 5.2](#) explains how the decisions attached to the operations in the CRYPTOPT IR are mutated. Finally, [Section 5.3](#) outlines how the pseudorandomness is generated in CRYPTOPT.

5.1 Scheduling Mutation

The scheduling of the operations influences the register allocation and the lifetime of intermediate variables. The influence on the lifetime also influences the spilling. A good operation scheduling can significantly reduce the number of spills.

The workflow for CRYPTOPT to mutate the scheduling of the operations is as follows: CRYPTOPT selects one operation $\mathcal{O}$ in the current operation order and selects a new position for it while obeying the constraints of the data flow. The new position of operation $\mathcal{O}$ can be anywhere between the last operation that computes the value of a symbol that $\mathcal{O}$ reads and the first operation that reads a symbol that $\mathcal{O}$ computes. This creates an interval $\mathcal{I}$ in which $\mathcal{O}$ can be scheduled. The interval $\mathcal{I}$ is recomputed with every scheduling mutation because it changes with every prior scheduling mutation.

Example

The example function from Listing 12 is used here to illustrate the above algorithm. Figure 5.1 shows the scheduling of the operations and the intervals in which an operation can be positioned.

The scheduling mutation picks a random operation, e.g. X10. The interval is indicated by the grey line in Figure 5.1. It can be scheduled only one position before its current position because it reads a value (x1) computed two operations prior. This results in a lower bound of -1 . It can also be scheduled later up to after the last operation because no other operation reads its computed value x10 (recall that writing to out1 is omitted in this example). This results in an upper bound being $+2$; the resulting interval is $\mathcal{I} = [-1, 2]$.

5.1.1 Bias

A priori, the best position of any operation $\mathcal{O}$ in its interval $\mathcal{I}_{\mathcal{O}}$ is unknown. The naïve approach is to uniformly at random select a new position from this interval. However, relocating an operation towards the interval's limits could hypothetically improve the code's performance because the lifetime of intermediate values and, thereby, spilling is significantly changed. For example, if an operation is relocated to a position where its arguments have just been computed or its computed values are consumed within the upcoming few operations, the amount of spills is reduced because the lifetime of values is shorter. Hence, the scheduling mutation biases relocations towards the interval's limits to improve the optimisation.

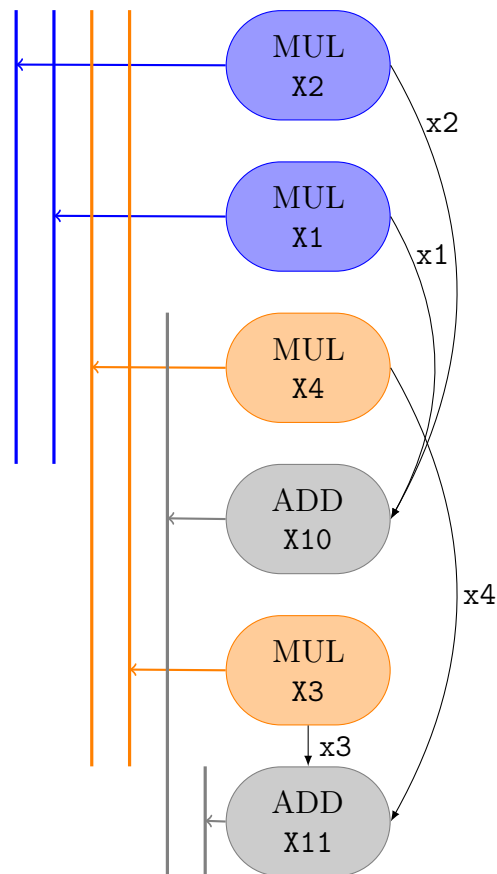


FIGURE 5.1. Scheduling interval of the operations of the example function. Rounded rectangles represent operations, grey arrows represent data dependencies, and vertical lines represent the intervals in which the operations can be scheduled.

5.1.2 Bias Calculation

To bias-select an integer i from the interval $\mathcal{I} = [i_{min}, i_{max}]$, CRYPTOPT calculates the following: Calculate the size of the interval $\Delta = |\mathcal{I}| = i_{max} - i_{min}$, generate a standard normal distribution D using Box-Muller transformations [BM58; PW34],¹ and sample a value d from D . Next, scale the sampled d to the interval's range by calculating $d \leftarrow \frac{d}{8.49} \times \frac{\Delta}{2}$ and round it towards the nearest integer. The factor 8.49 is chosen as it is the practical maximum value that can be sampled using Box-Muller and CRYPTOPT's runtime (Node.js).² This also avoids the need for re-sampling. After this scaling, d is likely to be close to 0 and within the interval $[-\frac{\Delta}{2}, \frac{\Delta}{2}]$. In the final step, the offset i for the new position is calculated with Equation 5.1. This calculated offset i is used to shift the operation $\mathcal{O}$ from its current position $i_{\mathcal{O}}$ to $i_{\mathcal{O}} + i$.

$$i = \begin{cases} i_{min} + |d| & \text{if } d < 0, \\ i_{max} - d & \text{else.} \end{cases} \quad (5.1)$$

This method creates a bath-tub-shaped histogram of the relative relocations. Figure 5.2 shows an example histogram of the values i of the interval $\mathcal{I} = [-5, 11]$. Values from the interval's limits are likely to be sampled from that distribution. Section 8.2.1 compares this bias to the naïve method of choosing uniformly at random.

Example

The following example illustrates the math: Let an operation $\mathcal{O}$ be at position $i_{\mathcal{O}} = 50$, and its data-flow constraints allow for repositioning five positions before ($i_{min} = -5$) and eleven after its current position ($i_{max} = 11$). Hence, the interval $\mathcal{I}$ of relative repositioning is $\mathcal{I} = [-5, 11]$, and the absolute new position must be a value from the interval $[50 - 5, 50 + 11] = [45, 61]$. The size of this interval is $\Delta = 11 - -5 = 16$. Let the sampled value d be $d = -1.7$; scaling d results in $d \leftarrow \frac{d}{8.49} \times \frac{\Delta}{2} = \frac{-1.7 \times 16}{8.49 \times 2} = -\frac{1360}{849} \approx -1.60$. Rounding this value towards the nearest integer results in $d = -2$. With Equation 5.1, the final value for i is $i = i_{min} + |d| = -5 + |-2| = -3$. With this relative value $i = -3$ the new absolute position of $\mathcal{O}$ is $i_{\mathcal{O}} + i = 50 + -3 = 47$, and $47 \in [45, 61]$ holds.

¹Box-Muller is used because the Node.js runtime does not provide normal distributed random integers.

²Because numbers in Node.js are of finite precision, there is an inherent lower bound on how close they can be to 0. In particular, this results in the largest number to be sampled using Box-Muller to be $\max = \sqrt{-2 \times \ln(2 \times \text{Number.MAX_SAFE_INTEGER}^{-1})} \approx 8.49$

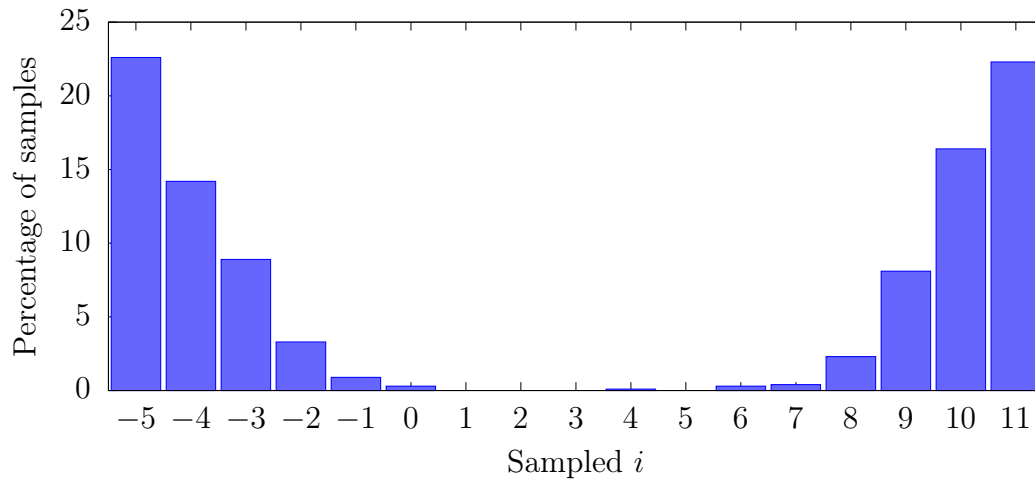


FIGURE 5.2. Histogram of the biased sampling. Sample size $n = 1\,000$, for the interval $\mathcal{I} = [-5, 11]$.

5.2 Decision Mutation

The code generation might result in decisions that result in suboptimal performance. This section now describes which decisions exist and how CRYPTOPT changes those decisions. [Section 5.2.1](#) explains the concept of a decision mutation based on the running example function. [Section 5.2.2](#) then explains the general case of how decisions are mutated. [Section 5.2.3](#) concludes with a list of which types of decisions are supported within CRYPTOPT.

5.2.1 Example

The example in the code generation section ([Section 4.4.3](#)) uses the overflow flag for the addition when implementing X10. The decision mutation can change this and other attached decisions. In the example, this would change the implementation for X10 from using OF to using CF. Additionally, because X10 now uses CF, the effect cascades down to the instructions for implementing X11. X11 can no longer use the CF because it is no longer zero. Instead, the virtual CPU will signal to the code generator that OF is zero, and subsequently, X11 uses OF. Note that this means that a potential decision on the operation X11 to use CF is disregarded in this context, i.e. state of the virtual CPU.

5.2.2 Mutation

For CRYPTOPT to mutate a decision, it first filters all operations that contain not-straightforward decisions that impacted the code generation in the previous iteration (would select, e.g. one of X2, X4, X10, X3).

From this operation subset, one operation is selected uniformly at random. Let CRYPTOPT randomly select, e.g. `X10`. From this selected operation, the decision is then changed to an alternative. One is chosen from all possible alternatives without bias but must be distinct from the current one. For example, if the decision at `X10` was to use `OF` for an addition, it is changed to use `CF` instead.

5.2.3 Types of Decisions

The idea of attaching the decision to the operation is to track the decisions that are made by the code generator. The different types of decisions are based on the mathematical operation that an operation object in the CRYPTOPT IR represents. Additionally, regardless of the operator, every operation has the *Choose Argument* decision, which keeps track of how to handle arguments. For example, in the case of the `mulx` operation, it keeps track of which argument is loaded into the `rdx` register. Similarly, for add operations, this decision keeps track of which value to load into a register explicitly, if both arguments happen to be in memory. Depending on the operation, additional types of decisions are supported:

- Strength reduction when multiplying with a constant,
- how to clear high bits of a value,
- how to clear both flags and which flag to use for an addition operation.

Strength Reduction

Some operations multiply a value by a fixed constant. The default instruction that works for any constant is the `imul` instruction. However, this invalidates the flags and there may be alternatives. If the constant allows for it, a simple form of strength reduction can be applied. In particular, if the constant is a power of 2, i.e. from the set $\{2, 4, 8, 16\}$, CRYPTOPT can choose using `shl`, `lea` or `shlx` in addition to the `imul` instruction. If the constant is in the set $\{2, 3, 4, 5, 8, 9, 10, 16, 19\}$, CRYPTOPT may also choose from using the `lea` instruction, or a pair thereof. For example, `x * 3` could be implemented with `lea rax, [ rax + 2 * rax ]`. Using `lea` has the benefit that it does not invalidate the flags, but the drawback is that it cannot read from memory. Hence, a particular decision here may have side effects of needing to load values from memory explicitly.

Clear High Bits

When operations from Fiat Cryptography need to clear the high bits, Fiat Cryptography generates an operation such as `uint64_t x2 = x1 & 0x000000007fffffff`. For logical `and` operations, CRYPTOPT checks whether the constant has a binary representation with only 1s led by 0s. If it does, CRYPTOPT may choose to implement this with the `bzhi` (e.g. *Zero High Bits Starting with Specified Bit Position*) instruction. If it does not, it may only use the default `and` instruction.

Add Operation

An addition operation requires two distinct decisions. First, to decide which flag to use for an *add-with-carry* as two alternatives are available: CF and OF. Second, how to clear flags in the case that the carry-in bit is 0.

CRYPTOPT supports two methods to clear both flags. Either, choose a spare register `rax` (spill if necessary) and use the `xor rax, rax` instruction to clear both flags and zero `rax`, or use the `test al, al` instruction. On the one hand, using `xor` has the benefit that it results in a register with a known value of 0, which may be helpful in adding a carry flag to another value, but it comes at the cost of requiring a register with an unused value. On the other hand, `test` does not require a register but also does not have the side effect of creating a 0.

The decision mutation will only change either of those, not both simultaneously. In particular, either change `xor` to `test` or change using CF to using OF.

5.3 Pseudorandomness

To ensure controlled starting conditions for the optimisation, all randomness in CRYPTOPT is deterministically generated using the SHA-1 hash algorithm. In particular, CRYPTOPT accepts a seed value at runtime that initialises the pseudorandom number generator. Whenever a random number is needed, the current value is hashed into a new one.

Practical Usage

In the initialisation steps, CRYPTOPT needs random values to order the operations. In the mutations steps, it needs to choose from alternatives (cf. decision mutation) or new positions (cf. scheduling mutation).

The SHA-1 algorithm produces 160 bits of output, but the values, that are needed in CRYPTOPT are always in a range from zero to some maximum value. This maximum value can be in the range of hundreds when initially scheduling available operations or much smaller in the range of tens to select new positions in an interval. CRYPTOPT scales the 160-bit number to the required range: First, the lower 52 bits are taken from the 160 available bits.³ Second, the resulting number is scaled down into a floating point number between 0 and 1 by dividing it by $2^{53} - 1$. Lastly, as CRYPTOPT needs an integer value between 0 and n , the random floating point number is subsequently multiplied with the maximum number n and rounded to the nearest integer.

Keccak [NIS15] or a linear-feedback shift register could be used as an alternative pseudorandom function. Using SHA-1 is an implementation detail, and investigating whether other randomness functions yield better results is left for future work.

³Only the lower 52 bits are used because of Node.js's limit for natural numbers, which is 2^{53} .

Chapter 6

Fitness Function

The previous chapters describe how the code generator creates phenotypes from the genotypes and how the optimiser mutates any current solution to generate another intermediate solution. The optimiser needs to evaluate the quality of any intermediate solution. This quality measure of a given piece of code could be its length, stack usage or performance. Because in CRYPTOPT, a high-quality solution is a highly performant one, CRYPTOPT runs the code on the target hardware while measuring the execution time. The smaller the execution time, the higher the performance and, thus, the higher the code's quality. The performance could also be predicted as an alternative to running the code. This and the other alternatives of quality measures are evaluated in [Section 8.2.4](#). Running the code allows CRYPTOPT to treat the CPU and its microarchitecture as a black box without the need to model it. While this is clearly a benefit, the challenge becomes to measure the execution time accurately.

The naïve approach would be to take the timestamp before and after the execution of the code and the difference between the two timestamps is the time that the implementation took to execute. However, this naïve approach is insufficient for two main reasons: the means of measuring time are inaccurate on contemporary hardware, and the execution time of a program is not constant.

The following sections give detailed information on the exact problems of measuring execution time and how they are resolved within the scope of CRYPTOPT. In particular, [Section 6.1](#) describes the clocks that are available and their inherent potential and drawbacks. [Section 6.2](#) elaborates on how the CPU states and environmental noise impact the execution time of a program. [Section 6.3](#) first describes the high-level steps that are taken to mitigate the inaccuracies inherent to measurements, then describes how to minimise the variability in execution time, and concludes with a description of CRYPTOPT's implementation. Short experiments with the proposed measure function are conducted in [Section 6.4](#). [Section 6.5](#) elaborates on parametrising the measurements. [Section 6.6](#) explains which noise-reduction steps help with the measurements during the optimisation

and which steps do not help. [Section 6.7](#) concludes with implementation details and simple correctness checks.

6.1 Time Counter

On x86-64 CPUs, two main counters can be used as high-precision clocks: the timestamp counter and the Performance Counters (PMC). This section describes both and how CRYPTOPT uses them.

Timestamp Counter

The timestamp counter is semantically a counter that increases with every clock cycle. However, there are different interpretations of what that means. In particular, different manufacturers may implement it differently. On Intel CPUs, the increment is based on the family of the CPU and may increase *with every internal processor clock cycle* or *increment at a constant rate* [[Int20a](#)].

Similarly, AMD states in their manual that *The processor [...] increments the counter every clock cycle* [[Adv05](#)]. However, this was not always observed on the machines used in the experiments of this thesis. For example, the AMD Ryzen 5800X processor increases the cycle counter by the multiplier value with every cycle of the Front-Side Bus (FSB) instead of incrementing with every CPU cycle. Further, the software manuals do not specify exactly how frequency changes (such as boosting or thermal throttling) affect this counter.

Performance Counter

The performance counters can count specific events when executing a program. Among others, one of those events is the hardware CPU clock cycles event. In Linux, this event is named `PERF_COUNT_HW_CPU_CYCLES`. Linux Kernel Manual [[pag32](#)] documents it as *Total cycles. Be wary of what happens during CPU frequency scaling*. Elevated privileges are required to set the counter up and read the value of this event.

Usage in CRYPTOPT

The aim of CRYPTOPT is to measure small performance differences caused by slight changes in the instruction sequence. The instruction sequences of interest, such as the cryptographic functions, are rather short. They are in the order of only hundreds or thousands of instructions.

Popular benchmarking frameworks for cryptographic implementations, such as SUPERCOP [BL22], prefer using the performance counters and fall back to the timestamp counter if needed. However, both those counter values are integers with a granularity of, at best, 1, but the average time required to execute a short instruction sequence can be rational. As such, those timings cannot be distinguished by an integer value.

Because the counter’s accuracy cannot be increased, the subject of the measurement, i.e., the measured instruction sequence, must be changed. In particular, the solution to the clock inaccuracies is to measure multiple executions of the same function instead of just one. In CRYPTOPT, multiple such executions of the same function are called a *batch*.

6.2 Variability of Program Execution

The mere execution time is affected by the CPU’s state and environmental noise [BAW20; Bok+19; KJ13]. Both of those are generally unpredictable and unavoidable.

CPU State

The state is unpredictable, and its effects and biases interfere with accurate measurements. One such effect is the *warm-up effect* [Eis+22; MI04; WP78a; WP78b], also known as *initialisation bias*. It essentially states that the first executions of a code sequence can have different timing behaviour than subsequent ones because the CPU changes from a cold state into a warm state.

Another factor that influences the execution time is the multitude of prediction mechanics for instructions, caches, and branches that are present in modern CPUs. This means that if the same code is (predictably) executed, the observed timings may not accurately represent the true required time. Similarly, if the same data is (predictably) loaded, it may influence the timings, too. Further, Mytkowicz et al. [Myt+09] state that the memory layout can significantly affect the performance of the software. Memory layout refers to the location of the executable code in memory because that correlates with caching and prediction mechanisms.

Environmental Noise

The execution time is also affected by uncontrollable environmental noise. For example, other processes running in parallel on the same system cause noise.

Noise can also be simple kernel activity, such as writing files to disk or handling network interrupts.

Those interrupting events effectively pause the execution of the measured implementation to run their interrupt routine. Those routines alter the microarchitectural state of the CPU and, as such, influence the timing. This noise can be reduced but not fully eliminated. Simply collecting multiple measurements of the same implementation and applying statistical functions can mitigate the effects of environmental noise.

6.3 Measurement Procedure

Bokhari, Alexander and Wagner [BAW20] investigated hardware measurements in the context of energy consumption, which is also susceptible to environmental noise and CPU state. They propose a measurement technique that executes tests in a rotated-round-robin fashion, which they name R3-VALIDATION. In essence, it repeats measurements tightly, and after each set of repetitions, the device is rebooted, and the configuration is changed. After the configuration change the measurements are repeated with the new configuration. The technique to randomise the execution can even out biases caused by processor states. In this case, the more unpredictability there is, the more accurate the measurements approximate the actual value.

The measurement technique in CRYPTOPT is based on this R3-VALIDATION. The remainder of this section introduces the measurement approach on an abstract level and then details the implementation.

6.3.1 Measurement Approach

Recall that CRYPTOPT does not necessarily need an absolute performance measure for a given implementation during optimisation. It only needs an indicator of whether one implementation is no worse than another to decide whether to keep a mutation or not.

In CRYPTOPT's measurement procedure, the two implementations are assembled into specific regions in memory to mitigate biases based on unknown memory layouts. The implementations are measured in tight loops to mitigate counter inaccuracies. Those tight repetitions are executed in a randomised fashion to even out prediction behaviour. Finally, the collection of multiple batches and using the median thereof as the performance measure mitigate potential warm-up biases as well as measurement noise.

6.3.2 Measurement Implementation

[Algorithm 3](#) shows the measurement implementation of CRYPTOPT. At a high level, the measurement logic exposes the function `measure`. This function is called with the two implementations given as an x86-64 assembly string. `AssemblyLine` [\[0xA23a\]](#) is used to assemble any implementation into the start of a memory page in an attempt to mitigate the bias of memory layout [\[Myt+09\]](#). The bias will still be there, but equally so for both implementations.

In addition, the baseline implementation (C code generated by Fiat Cryptography) is loaded as a shared object, and the pointer to that code is also given to the `measure` function as `C`, the pointer to the C-compiled code. Collecting batches for another (the C-compiled implementation) allows for even less predictability because three implementations are executed rather than only two.

The overall semantic is that the function randomly executes multiple batches per implementation while measuring the elapsed cycles per batch. A batch has a size (cf. batch size `bs`), which denotes the number of iterations an implementation is run in a tight loop (cf. innermost loop in [Algorithm 3](#)). For each implementation, the `measure` function collects multiple of those batches, specified as the number of batches `nob`, in the outer loops in [Algorithm 3](#). The order in which those batches are executed is randomised (cf. `shuffled`). The Fisher-Yates [\[FY53\]](#) algorithm produces unbiased permutations. CRYPTOPT uses it to shuffle the order in which it runs the implementations. The final performance measure, the quality of an implementation, is the median value of any batch (cf. lines 17–19 in [Algorithm 3](#)).

CRYPTOPT focuses on cryptographic code that can be used to compute secret data. One inherent property of that is that the code follows the constant-time programming paradigm [\[Alm+20b\]](#) to protect microarchitectural side-channel attacks [\[Ge+18\]](#). This property benefits CRYPTOPT in that different data cannot be responsible for the performance and possible variations thereof. Thus, the implementation can be called with random data as it does not affect the timings. In particular, there is no need to sample data with any special properties and then take any average over that. To find a majority of correctness bugs, the implementations are called with the same random input per batch, such that the output can be compared for equivalence, [Section 6.7](#) elaborates.

Algorithm 3: measure

```

input : a code A, (original) as x86-64 assembly
        b code B, (mutated) as x86-64 assembly
        C pointer to code C, as performance baseline
        bs size of a measurement batch
        nob number of batches to measure

output :  $P_A, P_B, P_C$  performance of code A, B and C in cycles

1 function measure(a, b, C, bs, nob)
2 begin
    /* initialize cycle arrays */
3   cyclesA ← []
4   cyclesB ← []
5   cyclesC ← []
    /* assemble x86-64 assembly to the start of a memory page */
6   A ← assemble(a)
7   B ← assemble(b)
8   for 1 to nob do
    /* shuffle function pointers */
9     shuffled ← shuffle([A, B, C])
10    foreach f ∈ shuffled do
        /* run bs times and get elapsed cycles */
11      start ← currentCycleCount()
12      for 1 to bs do
        /* execute implementation */
13        f(...)
14      end ← currentCycleCount()
        /* code for correctness checks omitted */
        /* add elapsed cycles to set A, B, C respectively */
15      elapsed ← end − start
16      append elapsed to cyclesf

17   PA ← median(cyclesA)
18   PB ← median(cyclesB)
19   PC ← median(cyclesC)
20 return PA, PB, PC

```

6.4 Robustness of the Measure Function

This section experimentally showcases the (i) robustness and (ii) sensitivity of the proposed measure function.

Robustness

To show robustness against e.g. warm-up effects (i), the same two code snippets are measured multiple times while restarting the measurement process. Naïvely, the median value is expected to be constant across the runs per function.

Sensitivity

To show the sensitivity (ii), two code snippets with the same arithmetic semantics, but with different implementations are tested. Listings 21 and 22 implement the example expression `(arg1[0] << 0x1F) | arg2[0]` from Listing 3. Listing 22 uses the stack to temporarily store pointers to the function’s arguments, and Listing 21 avoids utilising the stack. The longer code contains twelve instructions, whereas the shorter code only uses five. Naïvely, the longer code is expected to take longer than the shorter code.

```

1 mov rax, [ rsi ]
2 shl rax, 0x1f
3 or  rax, [ rdx ]
4 mov [ rdi ], rax
5 ret

```

LISTING 21. x86-64 assembly for the example function.

Execution

During measurement, both x86-64 assembly snippets are assembled with the in-memory assembler `AssemblyLine` into the start of two separate memory pages. The experiment contains ten individual runs with each 100 data points per code snippet (`nob=100`). Each data point relates to 100 tight runs of the snippet (i.e. batch size of `bs=100`).

Results

Figure 6.1 shows violin plots of the experiment. The wide spread of the violins indicates that the measurements are not consistent. As such, depending on which value two random measurements result in, a conclusion such as “long code

```

1  mov  [ rsp - 8 ], rdi
2  mov  [ rsp - 16 ], rsi
3  mov  [ rsp - 24 ], rdx
4  mov  rax, [ rsp - 16 ]
5  mov  rax, [ rax ]
6  shl  rax, 0x1f
7  mov  rcx, [ rsp - 24 ]
8  or   rax, [ rcx ]
9  mov  rcx, rax
10 mov  rax, [ rsp - 8 ]
11 mov  [ rax ], rcx
12 ret

```

LISTING 22. x86-64 assembly for the example function, but (unnecessarily) using the stack.

is faster than the short code” can be drawn. However, the opposite conclusion, “the short code is faster than the long”, can also be drawn and is equally valid.

Even the median of both functions fluctuates across the runs. As such, an absolute measure of either code does not give conclusive information on the performance of the code snippets. Looking at the long function, the median is between 2030 (run 4) and 2079 (run 8) ($\sigma = 16.78, \bar{x} = 2057.45$). For the short function, the median is between 2022 (run 4) and 2055 (run 6) ($\sigma = 11.36, \bar{x} = 2039.9$). Because of this behaviour of arbitrary conclusions of constant code is not useful to decide on the code’s performance, another metric is needed.

The median values of the shorter function in each run is pairwise lower than the median from the longer code. Hence, the one consistent single conclusion of this example can be drawn that the short code can be assumed to run faster than the long code. This means the proposed measure function reliably identifies the faster code in every run. An alternative approach to using the median value per run is to use the minimum value. Empirically, in experiments with CRYPTOPT, this also worked, but it is less reliable than the median. In the experiment, only run 1 shows that the minimum would indicate the longer code to be faster. This matches the empirically gathered data with CRYPTOPT.

6.5 Parametrising the Batches

The measurement of the batches is executed according to the specification of two parameters: the batch size **bs** and the number of batches **nob**. This section describes how the batches are parametrised for the optimisation in CRYPTOPT.

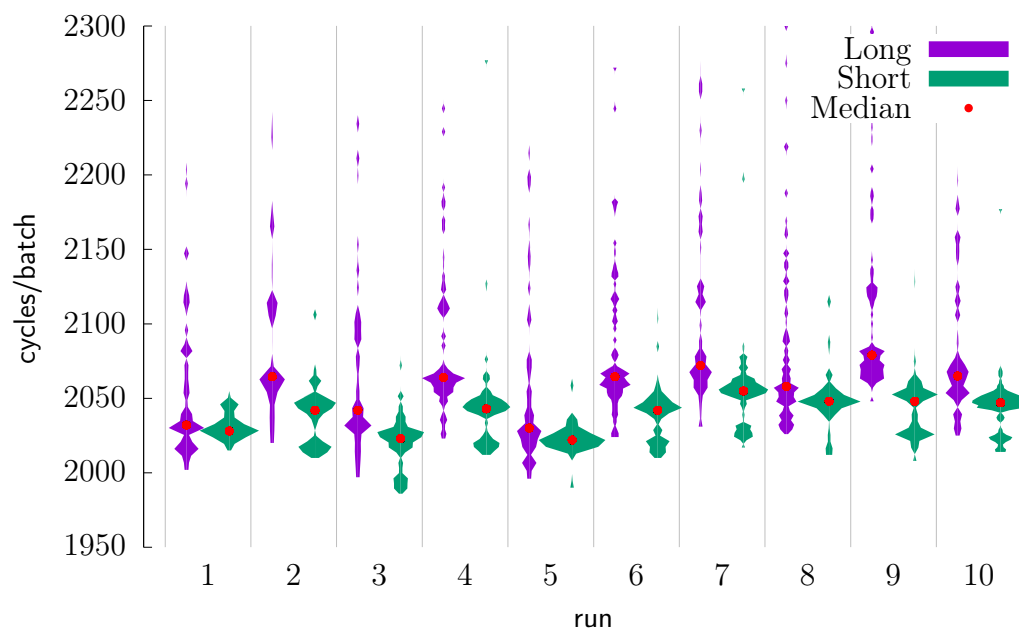


FIGURE 6.1. Violin plots show the robustness and sensitivity of the measure function. Ten runs of the measure function measuring both code snippets along the x-axis. The y-axis shows the elapsed cycles (performance counter) on an Intel i5 CPU for one batch with a size of `bs=100` – Dark-green coloured violin plots for the shorter code- purple for the longer code. The red dot marks the median in each run.

6.5.1 Scaling of the Batch Size

Recall that the aim here is to measure slight differences in code. If the batch size is too small (i.e. too few instructions are measured), slight changes in the code are effectively rounded to the next cycle counter value. Similarly, if the batch size is too big (i.e. too many instructions are measured), small changes in the code get lost in the statistical variance. An optimal value for the batch size **bs** depends on the size of the code under test: longer stretches of code require smaller batch sizes and vice versa.

Empirically, there is no silver-bullet value for the batch size **bs** that fits the range of all functions of **Fiat Cryptography**. To avoid having another set of magic numbers, **CRYPTOPT** adjusts **bs** dynamically based on the absolute measure of cycles. That is, the measured cycle count for one batch across any function should be in the order of *cyclegoal* = 10 000 measured cycles. The reason for this particular number is that this results in four to five digits of accuracy. **CRYPTOPT** scales the value for **bs** up or down by comparing the measured cycles of the C code against the *cyclegoal*. The scaling happens after any call to **measure**. It starts with a batch size of **bs** = 200, and after the call to **measure**, **bs** is adjusted using Equation 6.1.

$$\mathbf{bs} \leftarrow \left\lceil \frac{\text{cyclegoal}}{P_c} \times \mathbf{bs} \right\rceil \quad (6.1)$$

For example, if the cycle goal is set to *cyclegoal* = 10 000, and **measure** is called with **bs** = 200, the cycle result could be $P_c = 4\,500$. For the subsequent measurement, the batch size is adjusted to

$$\mathbf{bs} \leftarrow \left\lceil \frac{10\,000}{4\,500} \times 200 \right\rceil = 445.$$

Tiny functions, huge functions, or measurement errors can result in situations where **bs** is adjusted too highly. Hence, the equation needs some form of edge case handling. This is done by clipping **bs** into the interval [5, 10 000] with equation Equation 6.2.

$$\mathbf{bs} \leftarrow \text{argmin}(10\,000, \text{argmax}(5, \mathbf{bs})) \quad (6.2)$$

This described scaling technique also mitigates the problems of ununified definitions of cycles and dynamic frequency scaling because **CRYPTOPT** adjusts **bs** in each evaluation of any mutation (i.e., after each call to **measure**). Moreover, it does so on each platform independently. This makes the *accuracy* comparable

for any function across any platform, and this, in turn, enables CRYPTOPT to decide which mutations to keep.

6.5.2 Number of Batches

The trade-off in specifying the number of batches lies in measuring a small number of batches, which can decrease reliability but also decreases the run time of the optimiser, and measuring a large number of batches, which increases reliability in the measurements but also increases the run time of the optimiser. An optimal value is as small as possible while providing enough reliability. In the experiments, settling on `nob = 31` is robust enough. A higher value may be needed, if the standard deviation is more significant. Another option could be to dynamically adjust the number of batches based on the standard deviation, but this is left for future work.

6.6 Noise Reduction

Intuitively, reducing noise improves performance measurement and guides the optimiser better. In an attempt to reduce the noise, the following steps were done on the machines on which CRYPTOPT runs. First, run an OS without any unnecessary tasks to keep it as quiet as possible, and run the optimiser with the least amount of layers between the software and the hardware (i.e. no virtualisation). Second, avoid context switches by pinning the optimisation task to a logical CPU. Finally, if possible, good cooling in the case enclosure should be provided to prevent thermal throttling.

Contrary to typical measurement practices, where an absolute and precise measurement is required [Bal+12; BCS13; Beu+23; Bos+16; Chu+21; FLS14; Lin+23], disabling dynamic frequency scaling and disabling simultaneous multi-threading (SMT) (cf. Intel’s Hyper-threading) does not improve the optimisation itself. Moreover, fixing the CPU frequency to a specific (minimum) frequency and using the performance governor also does not improve the optimisation. Another way to reduce the noise is to isolate the cores from user programs to be scheduled on particular logical cores. This can be done using the `isolcpu` kernel parameter, but it also does not help with optimisation. Lastly, in order to increase the cycle counter accuracy itself (e.g. on the AMD Ryzen 5800X) the frequency multiplier can be decreased in the BIOS. This increases the counter accuracy to the frequency multiplier’s value, but it does not improve optimisations. It only takes much longer to finish the optimisation. Empirically, the

potential measurement stability those practices may provide does not justify their reduction in experiment evaluations per time unit. In other words, they may give slightly better performance measurements, but they slow down the system more, and thus, much fewer mutations can be evaluated in the same wall clock time.

6.7 MeasureSuite Module

The programming logic to measure the performance of implementations is encapsulated in a standalone module called **MeasureSuite**. **MeasureSuite** is available on GitHub [0xA23c] and as an npm (node package manager) package, on which CRYPTOPT depends. It is a C library with TypeScript bindings for easy use within CRYPTOPT and also provides a command line interface as tooling for the developer, even for use outside of this project. **MeasureSuite** is implemented in C to provide a high level of control to measure only the timing of the implementation and the positioning of the implementations themselves in memory.

As a standalone module, **MeasureSuite** also provides a simple form of correctness validation. In particular, the C-compiled version that is fed into the measure function is not only used as a performance baseline but also as a correctness ground truth. In general, the functions under test can be called with arbitrary data. With this, **MeasureSuite** performs a monkey testing approach to find most correctness bugs.

Monkey Testing

As the comment in **Algorithm 3** indicates, the simple correctness checks are omitted in the pseudo-code. To identify incorrect code and because the functions always have input and output arrays of data, **MeasureSuite** calls all functions with the same random data and checks if the results are equivalent. This random data is in the range as specified with the bounds (`lbound` and `ubound`) in the function signature in the input JSON. The data is sampled randomly with every new batch. Note that this sampling does not depend on the pseudorandomness described in **Section 5.3**. Instead, it is read from the Linux file `/dev/urandom`. This testing technique is known as *Monkey Testing* or *fuzz testing* [BLB10; SGA07] and only finds obvious implementation errors, which may happen during development. However, it does not provide formal correctness guarantees.

Chapter 7

Optimisation

CRYPTOPT uses the optimisation approach shown in [Figures 1.3](#) and [7.1](#). The CRYPTOPT optimiser first randomly generates assembly code that correctly implements the input function. It then repeatedly mutates the code and measures the execution time of both the original and the mutated code. Depending on an acceptance criterion, the new solution is accepted. The process continues until a predefined computational budget is used up.

The acceptance criterion depends on the optimisation heuristic used. Random Local Search (RLS) and Simulated Annealing (SA) are two possible optimisation heuristics. In RLS, the new solution is accepted if the performance is not deteriorating or if the new solution is strictly improving the performance. In Simulated Annealing (SA), worse solutions can be accepted, depending on how much worse the new solution is and how much of the budget has been used up. Typically, the likelihood of accepting a much worse solution gradually decreases during the optimisation, according to an *annealing schedule* or *cooling schedule* [[BT93](#); [VA87](#)]. The optimisation heuristics are evaluated in [Section 8.2.3](#).

RLS is often highly sensitive to the initial conditions, i.e., the candidate solution in the search space from which RLS starts. In other words, the number of mutation evaluations needed to improve a bad start is higher than for an initial solution with higher quality. To address such behaviour, CRYPTOPT uses the Bet-and-Run heuristic from Fischetti and Monaci [[FM14](#)]. This turns the initialisations with varying quality into CRYPTOPT's advantage. The core idea of Bet-and-Run is to explore multiple starting conditions for short runs and then bet on the most promising one, which is then optimised further. However, as Bet-and-Run is still dependent on the initial conditions, it does not guarantee that it will perform better than a single long run without exploring multiple starting points.

The initial starting conditions in CRYPTOPT are purely based on the seed that initialised the pseudorandom number generator. Hence, to employ Bet-and-Run, CRYPTOPT generates multiple such seed values and tries them sequentially

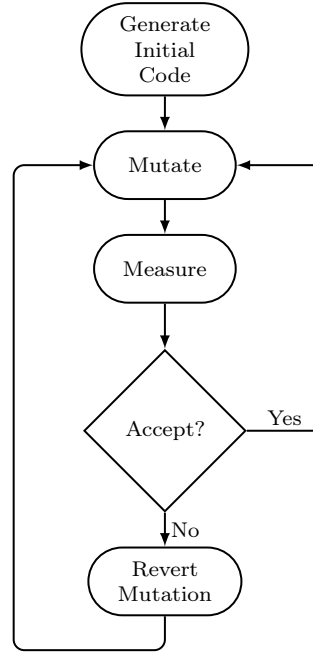


FIGURE 7.1. Iterative optimisation approach.

in new and independent optimisation processes for several of mutations each. The final optimisation run is subsequently conducted by reloading the internal CRYPTOPT state with the best performance at the end of the bets and optimise for the remaining budget.

Notation of Bet-and-Run in CRYPTOPT

The Bet-and-Run heuristic is used in CRYPTOPT with the following notation: A total budget of mutation evaluations e is divided into a bet part e_b and a run part e_r such that $e = e_b + e_r$. This is determined by the bet ratio $br \in [0, 1]$. For example, if 10% of the total budget should be used for the bets, then $br = 0.1 = 10\%$. The budget e_b is then divided evenly between all the bets. For example, if the optimisation run is allowed 5 000 mutation evaluations, with $br = 0.1$ on 5 bets, then each bet will optimise for $e_b = \frac{5\,000 \text{ evaluations} \times 0.1}{5 \text{ bets}} = 100 \frac{\text{evaluations}}{\text{bet}}$. After the bet stage, the remaining budget of $e_r = 5\,000 - 5 \times 100 = 4\,500$ is used on the most promising bet. [Section 8.2.2](#) discusses how to parameterise the amount of bets and the bet ratio for a typical CRYPTOPT workload.

Bet-and-Run in Practise

[Figure 7.2](#) visualises a typical CRYPTOPT optimisation run. The x-axis shows the elapsed time in evaluated mutations, and the y-axis shows the speedup against the off-the-shelf compiler Clang. The lines at the start correspond to the Bet-and-Run part of the optimisation. After the budget of e_b is used up,

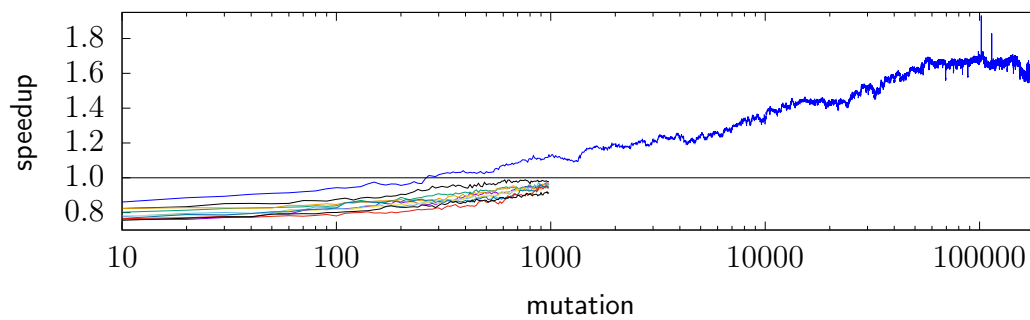


FIGURE 7.2. One example CRYPTOPT optimisation run of P-384-Square on i7 11G. Each line traces an optimisation run over time (x-axis, log-scale, one data point every ten mutations), showing the relative performance gain over Clang with options `-O3 -march=native -mtune=native`. Bet-and-Run used for 10 runs with a budget of 1 000 mutations each. The best-performing candidate continues for another 190 000 mutations.

CRYPTOPT chooses the best performing solutions and continues to optimise it. The blue line in the figure shows this.

Chapter 8

Performance

This chapter analyses the design decisions of CRYPTOPT by comparing them with alternative approaches. Then, this chapter evaluates CRYPTOPT with different workloads on different microarchitectures by answering the following questions:

- (I) How does optimisation progress over time?
- (II) How reliably does CRYPTOPT find the best-performing implementation?
- (III) How specific is the generated code to one microarchitecture?
- (IV) How well does the performance of the generated code compare against off-the-shelf compilers and hand-optimised state-of-the-art?

To answer those questions, the following methodology is applied: First, generate workload, i.e. the functions to be optimised with CRYPTOPT. Second, optimise this workload into x86-64 assembly on various machines while observing the performance progress over time. This answers question (I). Third, compare the time progressions of multiple runs on different machines and functions against each other to answer question (II). Fourth, find the best-performing implementation for each machine and run it on other machines in order to see how *specific* any implementation is. This answers question (III). Last, to answer question (IV), the methodology is to generate code for well-known functions and compare it against hand-tuned and compiler-generated code. In particular, the evaluation focuses on Curve25519, part of the TLS standard, and the curve used to sign blocks in the blockchain cryptocurrency “Bitcoin”. The compilers used are GCC [GCC23] in version 12.3.0 and Clang [Cla23] from the LLVM project in version 15.0.7.

The rest of this chapter is organised as follows: Section 8.1 introduces the setup for evaluation. In particular, it introduces the workload and the different machines that are used throughout this evaluation. Section 8.2 compares alternative design approaches, such as different approaches to spilling, code

alignment and register allocation. [Section 8.3](#) describes how the performance of the implementations is compared across the machines. The following section ([Section 8.4](#)) analyses and discusses the optimisation behaviour of CRYPTOPT over time. [Section 8.5](#) presents an overview of the resulting code lengths as another reliability metric. [Section 8.6](#) describes how one implementation is labelled the best one for each machine that is then compared against others from other machines in order to see how *specific* an implementation is to any microarchitecture. [Section 8.7](#) shows the performance data compared against off-the-shelf compilers across all machines. The comparison with state-of-the-art hand-tuned implementations is done using the SUPERCOP benchmarking framework in [Section 8.8](#). It comes with elaborate benchmarking methods and also ships with hand-optimised x86-64 assembly and C code. [Section 8.9](#) summarises the performance evaluation and answers the questions above.

8.1 Setup

This section describes the setup for evaluating CRYPTOPT. [Section 8.1.1](#) introduces the functions that will serve as the workload. [Section 8.1.2](#) elaborates on high-level characteristics regarding similarities and differences in the workload. [Section 8.1.3](#) gives an overview of the machines that are used in this evaluation.

8.1.1 Functions Under Test

CRYPTOPT is an optimising code generator for cryptographic arithmetic. It is designed as a compiler back end that works seamlessly with Fiat Cryptography. As such, Fiat Cryptography generates the majority of the workload. Fiat Cryptography generates code for the field arithmetic of eleven well-known finite fields (cf. [Section 2.3.5](#)). The multiply and square functions for those fields form the basis of the evaluation in this chapter. At the time of Fiat Cryptography’s publication [[Erb+19](#)], their code was the fastest-known C code for those particular fields. Additionally, Fiat Cryptography’s C code has been incorporated into the popular SSL libraries BoringSSL [[Bor23](#)] and Mozilla NSS [[Moz23](#)]. Hence, the C code serves as the baseline for performance comparison against off-the-shelf compilers.

Those eleven fields comprise of the nine fields that Fiat Cryptography reported in their publication (cf. [Section 2.3.5](#)). In particular, the fields underlying the following schemes are used: Curve25519, Curve P-224, Curve P-256, Curve P-384, SIKEp434, Curve448, Curve P-521, Poly1305, and secp256k1. In addition, as a

result of the collaboration efforts regarding the CRYPTOPT paper [Kue+23b], two additional implementation strategies (i.e. functional programs) are compared. Those are for Curve25519 and secp256k1.

For Curve25519, the saturated and the unsaturated implementations are compared, both based on the Solinas implementation strategy. The former results of the collaboration and uses four 64-bit limbs, referred to as *Curve25519-Solinas*. The latter is the existing one and uses five 52-bit limbs, called *Curve25519*. Historically, the unsaturated version of Curve25519 outperforms saturated implementations [Ope17], but this has changed in recent hardware, which is why saturated implementations are typically preferred nowadays [Ope23b].

For secp256k1, two different implementation strategies are compared: the word-by-word Montgomery implementation and the *Dettman* implementation. The former was Fiat Cryptography’s go-to implementation at the time of their publication, and the latter incorporates implementation tricks present in the Bitcoin Core library `libsecp256k1` [lib23].

The Bitcoin Core library includes a hand-tuned assembly implementation for the field arithmetic and a C fallback version for non-AMD64 platforms. To showcase CRYPTOPT’s flexibility, this C fallback is compiled into LLVM IR and transformed into CRYPTOPT IR using simple scripts (Section 9.3 elaborates). Those functions are referred to as *Bitcoin Core (LLVM)*. Table 8.1 summarises the fields, implementation strategies, and number of limbs per field element.

8.1.2 Code Characteristics

As shown in Table 8.1, there are two distinctive properties in the form and size of the different functions. The first difference is the choice of implementation strategy. Those range from saturated and unsaturated Solinas implementations over the saturated word-by-word Montgomery strategy to the Dettman implementation for secp256k1. The second difference is the size of the prime field, which roughly determines the number of limbs that need to be incorporated into the implementation. For example, Poly1305 needs only three 64-bit limbs, whereas P-521 needs nine 64-bit limbs.

Types of Operations

The implementation strategy controls the types of operations used. On the one hand, the unsaturated Solinas strategy (used for Curve25519, Curve448, P-521, and Poly1305) predominantly uses 128-bit integer additions, double-precision

TABLE 8.1. Overview of the fields, their size and their Fiat Cryptography implementation strategy.

Field Name	Fiat Cryptography Implementation Strategy	Field Element Width	# Limbs
Curve25519	Unsaturated Solinas	256 bits	5
Curve25519-Solinas	Saturated Solinas	256 bits	4
P-224	word-by-word Montgomery	256 bits	4
P-256	word-by-word Montgomery	256 bits	4
P-384	word-by-word Montgomery	384 bits	6
SIKEp434	word-by-word Montgomery	448 bits	7
Curve448	Unsaturated Solinas	448 bits	8
P-521	Unsaturated Solinas	521 bits	9
Poly1305	Unsaturated Solinas	130 bits	3
secp256k1-Dettman	Dettman	256 bits	5
secp256k1	word-by-word Montgomery	256 bits	4
Bitcoin Core (LLVM)	N/A	256 bits	5

shifts, and multiplication with small constants. On the other hand, the word-by-word Montgomery implementations (used for P-224, P-256, P-384, SIKEp434, and secp256k1) use multiplications with large constants and always end with subtraction and conditional moves. The Dettman implementation strategy contains 128-bit additions that are mixed with large-constant multiplication as well as double-precision shifts. For the saturated Solinas (Curve25519-Solinas) and the word-by-word Montgomery implementations, the 128-bit additions are carried out as individual 64-bit additions with separate symbols for the carries. In contrast, in the unsaturated Solinas strategy, the C code relies on the compiler’s support for 128-bit arithmetic.

Those differences also reflect how well off-the-shelf compilers perform: GCC tends to slightly outperform Clang for the unsaturated Solinas C code, whereas Clang significantly outperforms GCC for the word-by-word Montgomery implementations.

8.1.3 Machines

CRYPTOPT generates very microarchitecture-specific code. Hence, it is useful to see its abilities on different machines. To that end, CRYPTOPT generates code on the ten microarchitectures as per Table 8.2. Subject to this evaluation are four AMD machines ranging from Zen 1 (introduced in 2017) to Zen 4 (September 2022) microarchitectures. Additionally, there are six Intel machines ranging from the Intel Core sixth generation “Skylake” microarchitectures (August 2015) to

TABLE 8.2. Overview of machines used in the experiments: Name, CPU, and microarchitecture. Extended details of the important machine components in [Appendix A](#).

Name	CPU	Microarchitecture
1900X	Ryzen Threadripper 1900X	Zen 1
5800X	Ryzen 7 5800X	Zen 3
5950X	Ryzen 9 5950X	Zen 3
7950X	Ryzen 9 7950X	Zen 4
i7 6G	Core i7-6770HQ	Skylake-H
i7 10G	Core i7-10710U	Comet Lake-U
i9 10G	Core i9-10900K	Comet Lake-S
i7 11G	Core i7-11700KF	Rocket Lake-S
i9 12G	Core i9-12900KF	Alder Lake-S
i9 13G	Core i9-13900KF	Raptor Lake-S

Intel Core 13th generation “Raptor Lake” (October 2022). All desktop machines have been equipped with adequate cooling of the case enclosure to avoid thermal throttling. For the remainder of this thesis, those machines are referred to by the name listed in the table. For the Intel machines i9 12G and i9 13G, only the P-cores (performance cores) are used. Detailed information on the other components in the machines are listed in [Appendix A](#).

8.2 Design Space Exploration

This section evaluates alternative approaches. In particular, [Section 8.2.1](#) discusses the use of a uniform bias during the scheduling mutation; [Section 8.2.2](#) elaborates on the Bet-and-Run parameters; and [Section 8.2.3](#) discusses an alternative optimisation heuristic. In particular, it compares Simulated Annealing, in which a worse-performing alternative is accepted with a certain likelihood. Continuing with alternative optimisation parameters, [Section 8.2.4](#) identifies other fitness functions that can be used instead of the cycle count to assign a quality measure to the mutants. [Section 8.2.5](#) investigates different approaches when generating the code itself. In particular, the section discusses approaches to spilling, code alignment, and register allocation.

Those alternative approaches are briefly introduced with context, expectations and short experiments. The experiments are done using the same three methods. Specifically, the methods are the multiply methods generated by Fiat Cryptography of the fields Curve25519, P-256 and P-521. Those fields are chosen to represent the intended workload of CRYPTOPT. P-521 is considerably

TABLE 8.3. Number of instructions, stack size and speedup compared to Clang of the resulting code for Uniform and Biased scheduling mutations. Biased Scheduling impacts longer functions more than shorter functions, but generally improves optimisation.

Method Name	Mode	# Instructions	Stack Size	Speedup
Curve25519–Multiply	Biased	175	15	1.18
	Uniform	185	26	1.13
P-256–Multiply	Biased	199	18	1.45
	Uniform	211	19	1.40
P-521–Multiply	Biased	549	84	1.12
	Uniform	619	136	1.06

larger than the other two methods, and P-256 uses a different implementation algorithm than Curve25519 and P-256 (cf. [Section 8.1.1](#)). Unless otherwise noted, the same three functions are optimised for 10 000 mutations. The same seed value is used for all experiments, such that they all start from the same location in the search space.

8.2.1 Alternative Bias

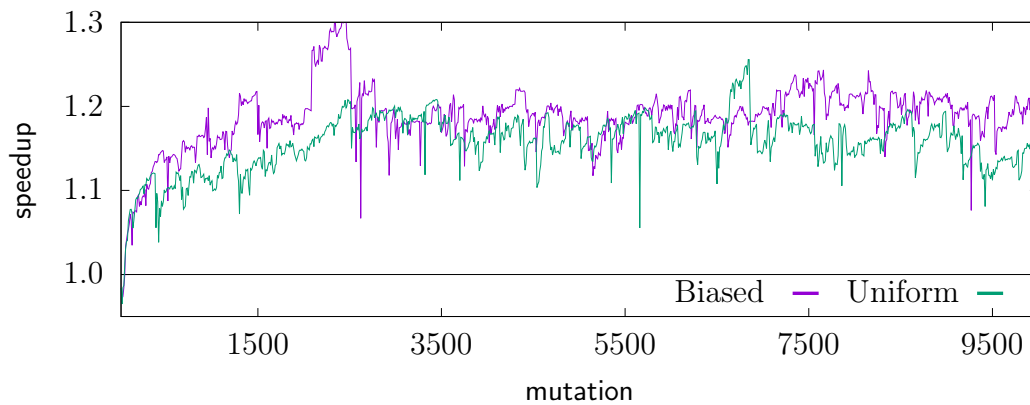
[Section 5.1](#) describes the biased selection in the scheduling mutation. The naïve alternative to biasing the scheduling mutation is to uniformly at random select from the interval $\mathcal{I}$.

Experimental Setup

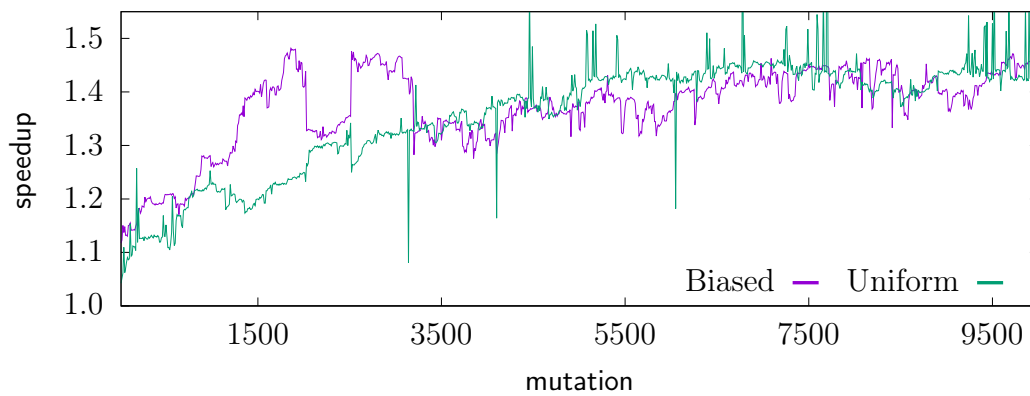
The experiment visualises the impact of the bias. The three functions are optimised with both alternative mutation preferences. The biased selection is expected to guide the optimiser faster and further (i.e., greater final speedup for a constant number of evaluated mutations), and the resulting code is expected to have fewer memory spills. This effect is expected to be more prominent for longer functions, such as P-521, because more live variables exist, and the intervals are larger.

Results

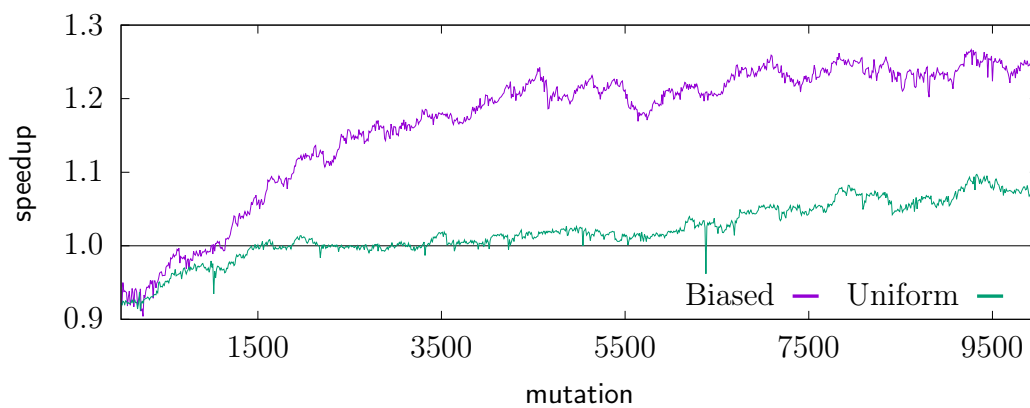
[Figure 8.1](#) shows the results of optimising the methods with uniform bias and with the proposed bias. The biased optimisation run finds higher-performing code more effectively in the larger function P-521. For the smaller functions Curve25519 and P-256, the bias has less impact but also improves effectiveness.



(A) Optimisation of Curve25519-Multiply.



(B) Optimisation of P-256-Multiply.



(C) Optimisation of P-521-Multiply.

FIGURE 8.1. Optimising Curve25519, P-256, and P-521-Multiply with biased and uniform sampling, comparing against Clang. The biased optimisation runs find higher performing code more effectively in the larger function P-521. The bias has less impact for the smaller functions Curve25519 and P-256.

TABLE 8.4. Speedup compared to Clang of the resulting code for different numbers of bets. Depending on the method, various settings of bets can yield higher performance and be more effective.

Method Name	Bet Setting				
	b=1%	b=5%	b=10%	b=20%	b=40%
Curve25519–Multiply	1.24	1.24	1.14	1.16	1.16
P-256–Multiply	1.38	1.41	1.40	1.39	1.40
P-521–Multiply	1.23	1.16	1.19	1.18	1.13

Table 8.3 shows that the biased optimisation run results in shorter functions with less stack usage and greater speedup for the same mutation budget.

8.2.2 Bet-and-Run in CRYPTOPT

Chapter 7 describes how random initialisation affects the quality of the starting points in the search space and how CRYPTOPT uses Bet-and-Run. Recall that the total budget of mutation evaluations e is divided into a bet part e_b and a run part e_r such that $e = e_b + e_r$. Further, recall that e_b is determined by the bet ratio $br \in [0, 1)$. Section 8.4 uses the parameters: $b = 10$, the budget $e = 200\,000$, and $br = 0.05\%$. The following two sections evaluate different parameterisations of the number of bets (b) and the bet ratio (br).

Parametrising Bets

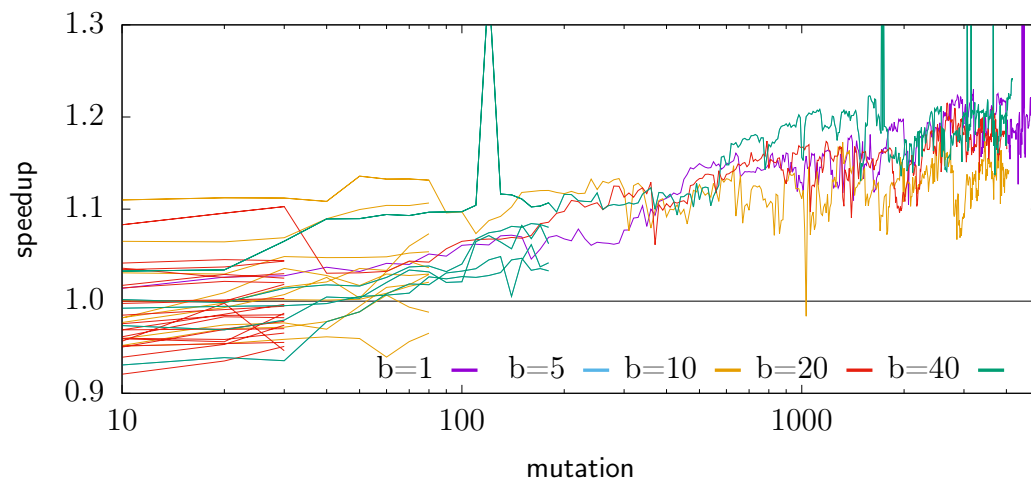
The most effective parametrisation of the number of bets depends on the method to optimise.

Experimental Setup

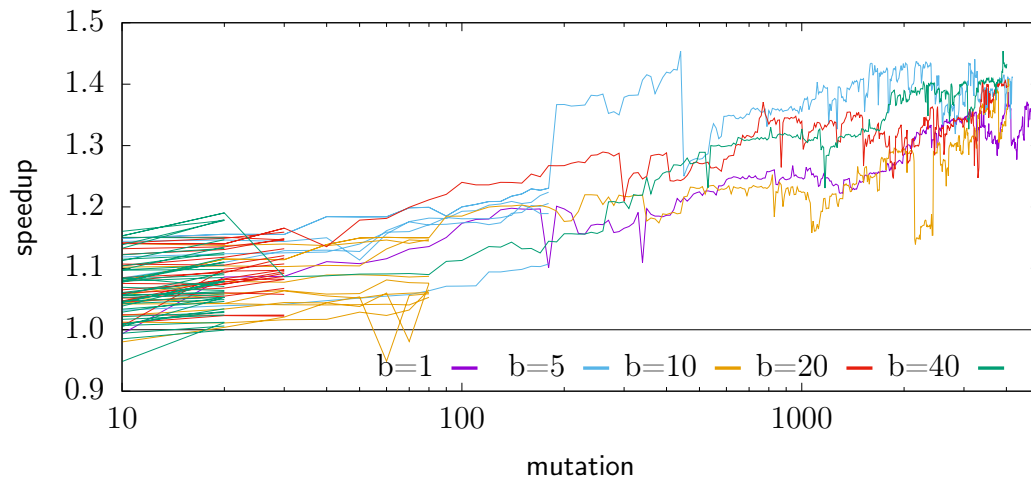
All functions are optimised with a total budget of $e = 5\,000$ mutation evaluations and bet settings ranging from a single bet ($b = 1$) to 40 bets ($b = 40$). Each of the optimisation runs uses $br = 0.2 = 20\%$ of the total budget on the bets.

Results

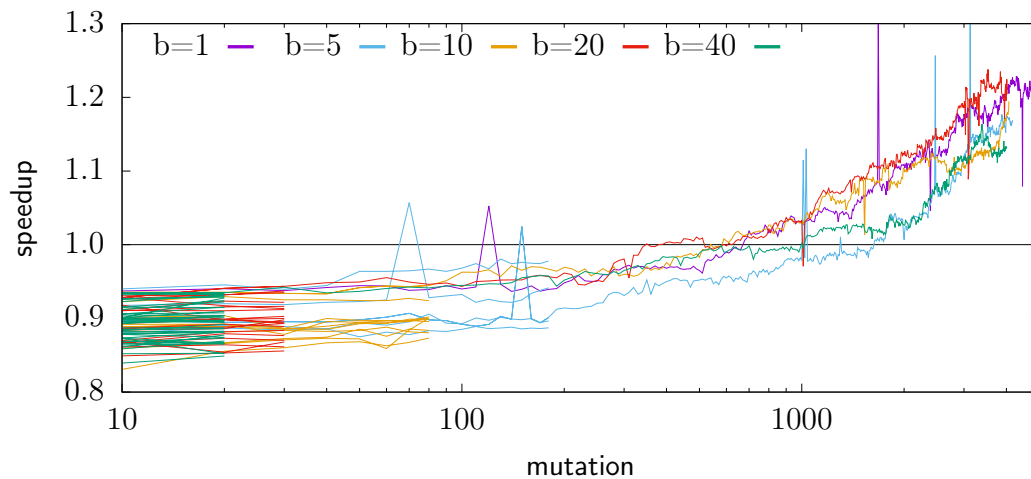
Figure 8.2 shows the experiment over time (i.e., mutations), and Table 8.4 shows a summary of the resulting assembly files. Both show that for Curve25519 and P-256, the setting of $b = 5$ had the best result, whereas for P-521 one single run was able to create the fastest code. However, the other settings for the number of bets result in codes of similar quality.



(A) Optimisation of Curve25519-Multiply.



(B) Optimisation of P-256-Multiply.



(C) Optimisation of P-521-Multiply.

FIGURE 8.2. Optimising the multiply method of the fields Curve25519, P-256, and P-521 with different amounts for the bet parameter; speedup comparing against Clang. The x-axis is logarithmic to show the starting conditions better.

TABLE 8.5. Speedup compared to Clang of the resulting code for different number of bet ratios. Depending on the method, different settings of the bet ratio can yield higher performance more effectively.

Method Name	Bet Ratio Setting				
	br=0%	br=1%	br=10%	br=20%	br=50%
Curve25519–Multiply	1.22	1.16	1.17	1.12	1.19
P-256–Multiply	1.38	1.35	1.41	1.39	1.41
P-521–Multiply	1.23	1.22	1.13	1.19	1.18

Parametrising Bet Ratio

As for the numbers of bets, the most effective parametrisation of the bet ratio also depends on the method to optimise.

Experimental Setup

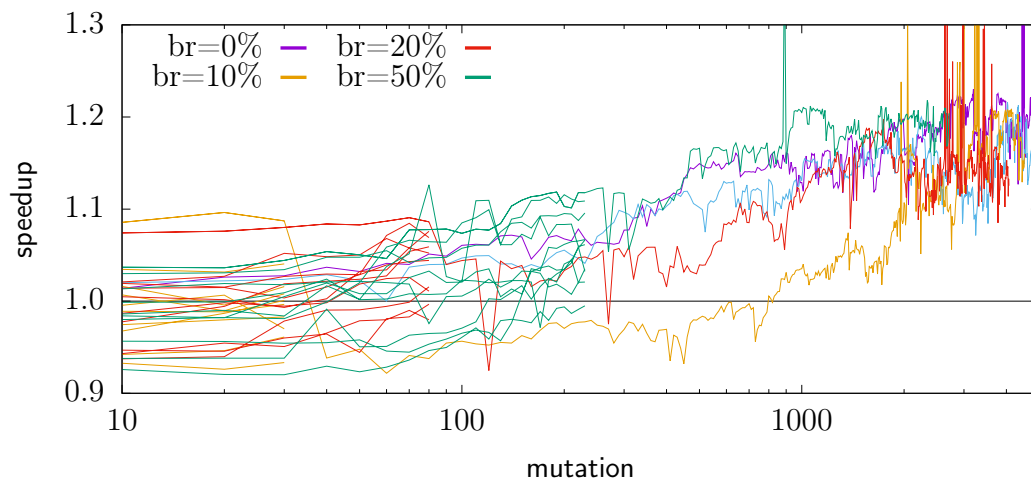
This experiment optimises the same methods with the same total budget of $e = 5\,000$ mutations. This time, the bet ratio changes from 1% ($br = 0.01$) to 50% ($br = 0.5$) and is compared against $br = 0$, i.e. a single run without bets. Each of the optimisation runs uses $b = 10$ bets.

Results

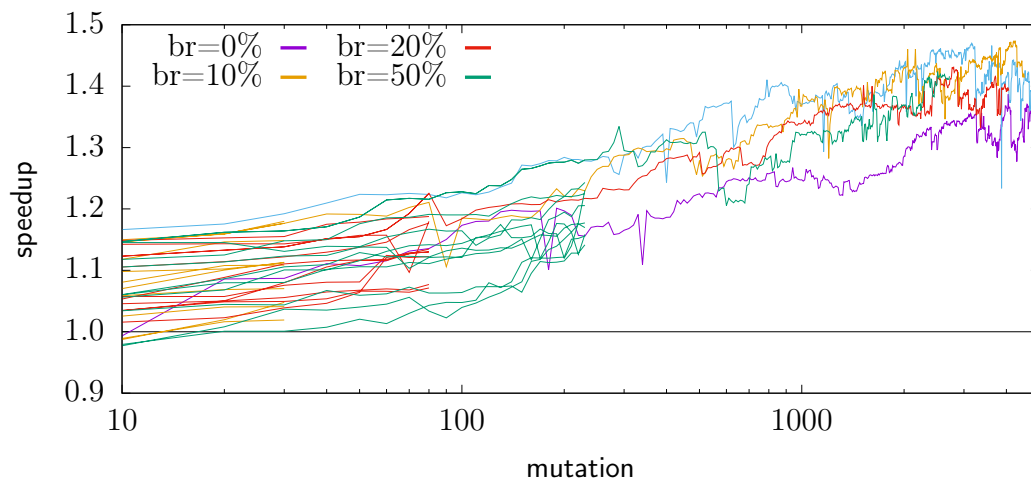
Figure 8.3 shows the experiment over time (i.e., mutations), and Table 8.5 summarises the resulting assembly files. The table shows that for Curve25519 and P-521, the single run is best. For P-256, $br = 0.1$ and $br = 0.5$ yield better results. However, the other settings for the bet ratio generate code of similar quality.

8.2.3 Alternative Optimisation Heuristic

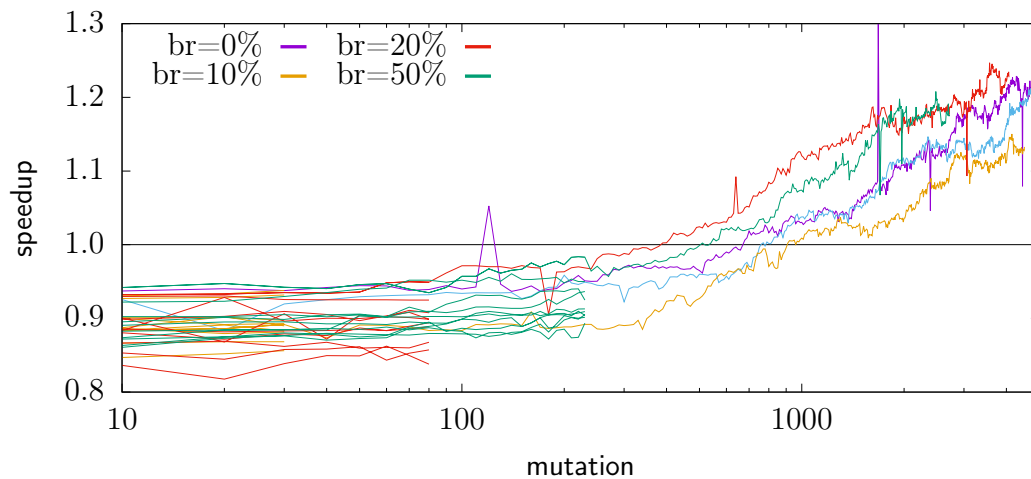
Recall the optimisation heuristic from Chapter 7, which needs an acceptance criterion. This section explores RLS and SA with different thresholds. In particular, the experiment features RLS with a strict-improvement policy (only accepting new candidates that are strictly improving the performance) and a more lenient approach where the new candidate only cannot deteriorate the current quality. In addition, a simplified SA is used with constant acceptance-thresholds (i.e., not implementing any annealing schedule) and variable thresholds that are based on the “badness” of the new candidate.



(A) Optimisation of Curve25519-Multiply.



(B) Optimisation of P-256-Multiply.



(C) Optimisation of P-521-Multiply.

FIGURE 8.3. Optimising the multiply method of the fields Curve25519, P-256, and P-521 with different amounts for the bet-ratio parameter; speedup comparing against Clang. The x-axis is logarithmic to show the starting conditions better.

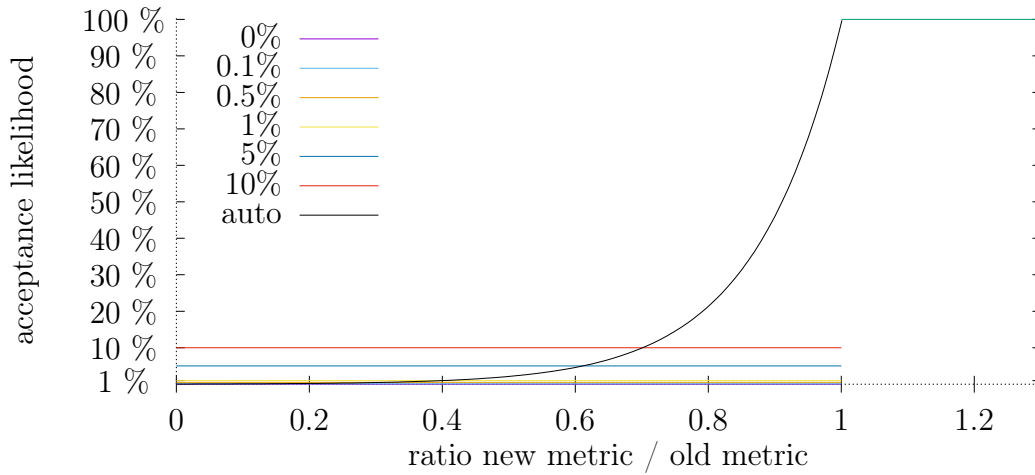


FIGURE 8.4. Simulated Annealing acceptance likelihoods. The x-axis shows the ratio of the old metric divided by the new metric. If the new metric is smaller than the old metric, that quotient is greater than one, and the likelihood of acceptance is 100%. The 'auto' setting accepts much worse mutations much less likely than only slightly worse candidates. The likelihood is independent of "how much worse" the new candidate is for the other settings. Those settings are between 0% and 10%.

Experimental Setup

The functions are optimised with different levels of accept-if-worse likelihoods. The default is RLS, which accepts equally good or better candidates, called 0%. The next variant is RLS, which only accepts strictly-improving candidates, called 0% str. Shifting to SA, the likelihood of accepting worse candidates increases from 0.1% to 10%, i.e. 1 out of 10 worse candidates are accepted, regardless of "how much worse" they are. The *auto* option is based on an exponential formula proposed in the literature [VA87]. Figure 8.4 shows the acceptance likelihoods for SA as a graph. Equation 8.1 shows the formula that likely accepts only slightly worse mutations, and the worse the mutations get, the more likely it is that it gets disregarded. The parameters are chosen such that

- there is a one hundred per cent likelihood that a better solution is accepted, and
- that new solutions that are 10 times worse are rejected 99% of the time.

Thus, the graph includes points (1, 1) and (0.1, 0.001), and the parameters in the equation are adjusted accordingly. The variables "new" and "old" in the

TABLE 8.6. Speedup compared to Clang of the resulting code for different threshold values for Simulated Annealing. Depending on the method, different thresholds can be beneficial.

Method Name	Worse Accept Threshold							
	0%	0% str	0.1%	0.5%	1%	5%	10%	auto
Curve25519–Multiply	1.19	0.96	0.95	0.89	0.95	1.06	1.07	0.96
P-256–Multiply	1.39	1.04	1.03	1.01	1.05	1.02	1.04	1.01
P-521–Multiply	1.33	0.92	0.92	0.92	0.91	0.99	1.03	1.07

equation are the metrics (i.e. elapsed cycles) of each function, respectively.

$$\text{acceptance likelihood} = \frac{1}{10} e^{\frac{10 \ln(10)}{3} \times \frac{\text{new}}{\text{old}}} \quad (8.1)$$

The expected behaviour is that the more worse mutations are accepted, the less performant the end result becomes. However, to a certain degree, accepting a worse solution can help escape a local optima.

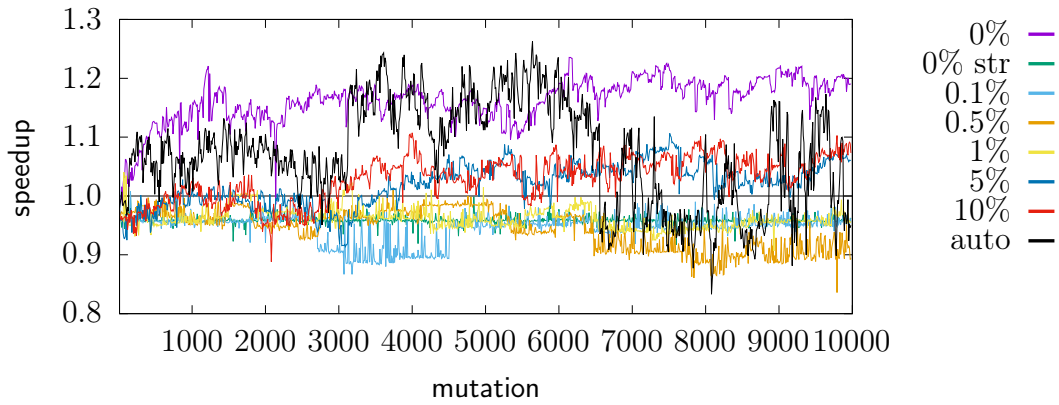
Results

Figure 8.5 shows the optimisation over time, and Table 8.6 shows the speed ups of the resulting x86-64 assembly files for the different thresholds. The “lenient” option stands out and results in the best results among the tested methods. The other variant of RLS and all SA variants are producing code with worse performance. For P-256–Multiply, those other variants are clustered slightly above the 0% speedup mark, where the other methods show a wider spread in quality of the results. Because accepting solutions of equal and better quality results in the highest-performing code, CRYPTOPT uses this heuristic as the default.

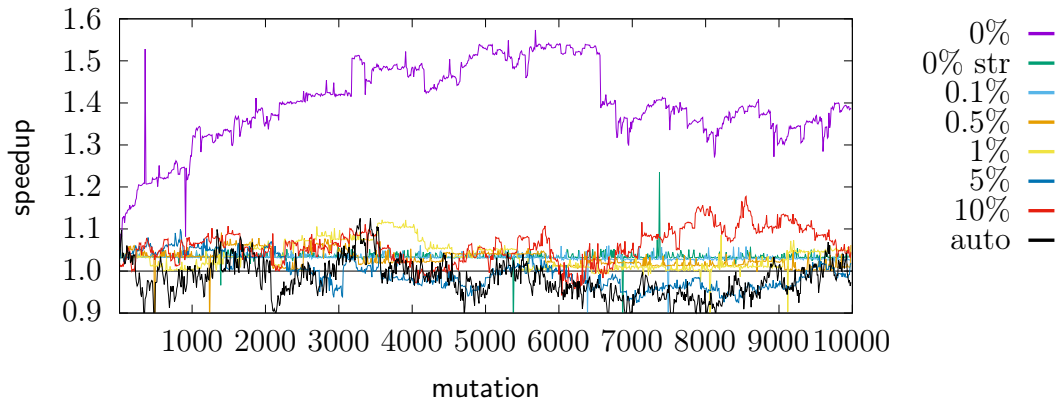
8.2.4 Alternative Fitness Functions

The thesis discusses running the functions on the hardware and using the actual elapsed cycles as the fitness function. Three other alternatives include

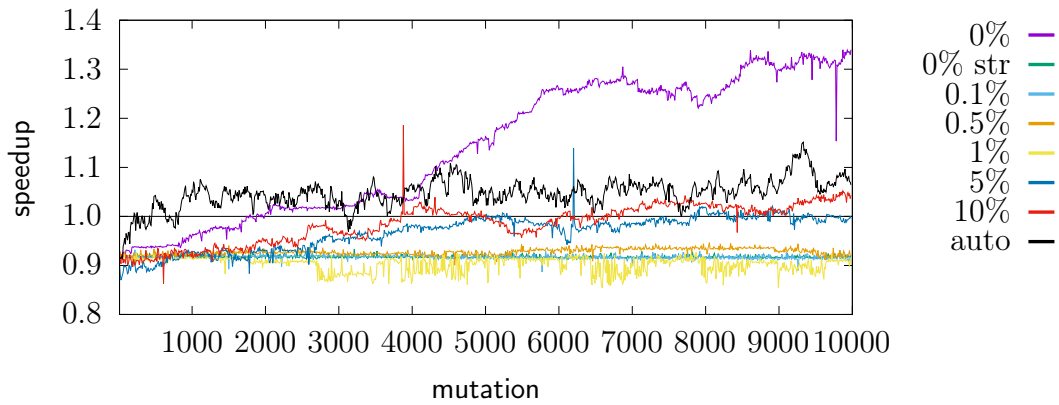
- (i) using the function’s length as the fitness function such that CRYPTOPT optimises for shorter code,
- (ii) using the function’s stack usage as the fitness function such that CRYPTOPT optimises for fewer spills to the stack,
- (iii) using an estimation tool to estimate the throughput of a given assembly program for a specific architecture.



(A) Optimisation of Curve25519-Multiply.



(B) Optimisation of P-256-Multiply.



(C) Optimisation of P-521-Multiply.

FIGURE 8.5. Optimising Curve25519-Multiply, P-256-Multiply, and P-521-Multiply with Simulated Annealing, comparing speedup against Clang.

One benefit of those approaches is that they do not require the hardware for which CRYPTOPT is optimising. Hence, the machine that generates the code does not matter for the resulting assembly programs.

Experimental Setup

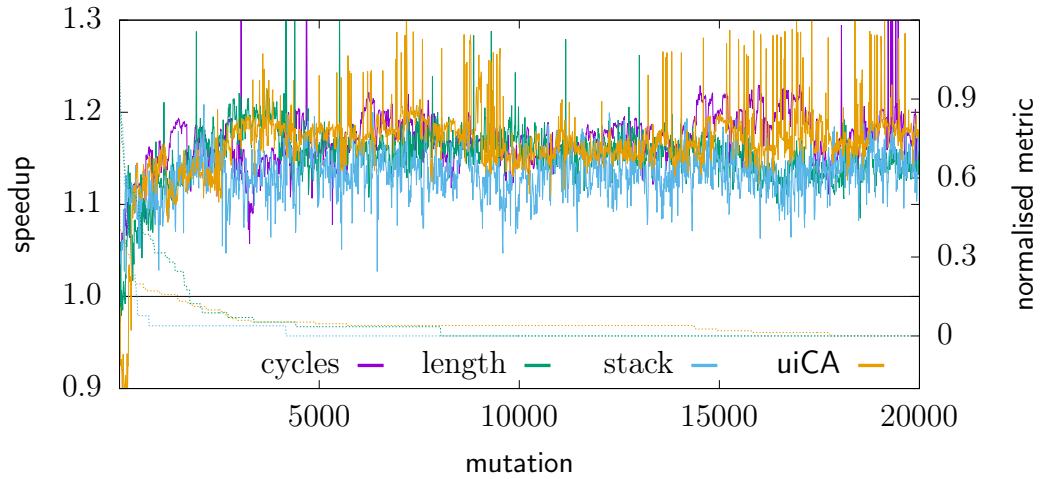
The same functions are used. The setup for alternatives (i) and (ii) is trivial: instead of running the code and determining whether to keep a mutation is not based on the elapsed cycle count but instead on the total number of instructions (alternative (i)) or the number of `mov [ rsp + off ], r12` instructions (alternative (ii)). For the alternative (iii), the optimiser relies on uiCA [AR22], a tool¹ to predict the throughput of programs for Intel Microarchitectures. It is worth noting that at the time of writing, it only supports microarchitectures up to Rocket Lake (Intel 11 G). At the same time, CRYPTOPT can also optimise for newer Intel architectures and AMD architectures. Another drawback is that it is very slow compared to measuring the code. Using this tool to evaluate the quality of the Curve25519 code, CRYPTOPT only evaluates ~ 2 mutations per second, as opposed to 150 per second when simply running and measuring the code.

In this test, CRYPTOPT generates the x86-64 assembly as usual and for alternatives (i), and (ii), a simple string analyser suffices to determine the phenotype’s quality by counting the relevant instructions. To use uiCA, CRYPTOPT writes the x86-64 assembly to a temporary file, uses `AssemblyLine` to assemble it into binary code and then calls the uiCA-Python script to analyse the binary and report the estimated throughput. The following command is used: `./uiCA.py -TPonly -arch SKL -raw BINARY.bin`, where SKL specifies the architecture “Skylake” and BINARY.bin file containing the binary code that `AssemblyLine` emits.

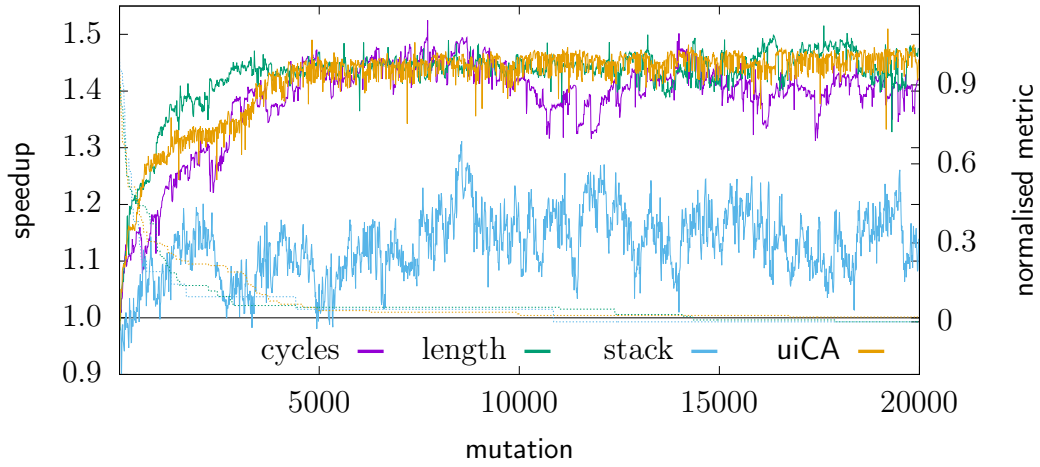
Results

Figure 8.6 shows that all approaches seem to work well and produce faster code. In addition to the speed up, the graphs also show the metrics as dotted lines. All runs optimise the function, but the instruction length, stack usage and uiCA-based runs cannot continue to guide the optimiser at some point, because that metric is not changing anymore. This is shown in the graph by the flat dotted lines. The optimiser then randomly accepts solutions that do not deteriorate the current solution. Importantly, the optimiser is stuck in the local optima in the search space from this point because no worse solution is accepted. This is less

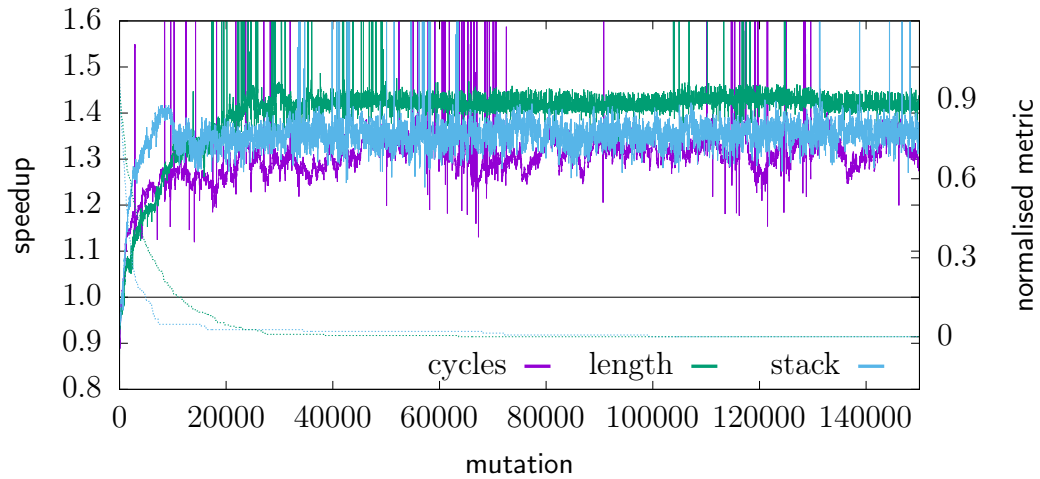
¹The tool is available at <https://github.com/andreas-abel/uiCA>, commit hash 5efa85d



(A) Optimisation of Curve25519-Multiply.



(B) Optimisation of P-256-Multiply.



(C) Optimisation of P-521-Multiply.

FIGURE 8.6.

Optimising with either cycles, number of instructions (length), stack size or throughput prediction (uiCA) as the fitness function. Solid lines graph the speedup over Clang while dotted lines show the fitness function criteria, i.e. # instructions, stack size, throughput respectively.

likely to happen with cycle-based optimisation because of the higher resolution of the metric and because of occasional wrong measurements, especially when the solutions have very similar performance. This turns the wrong measurements into CRYPTOPT’s advantage and approaches a Simulated-Annealing practice (cf. [Section 8.2.3](#)) [VA87].

Additionally, using stack size and instruction length is independent of the CPU that CRYPTOPT is running on. Hence, the optimiser is not able to exploit any intricacies of the microarchitecture. Using uiCA does guide on a per-microarchitecture basis, but it seems too slow to be viable. It is so slow, in fact, that optimising longer functions like P-521–Multiply would take multiple days to optimise. Hence, it is missing in the figure.

8.2.5 Alternative Code Generation

The previous chapters describe how the final CRYPTOPT prototype spills and allocates registers. This section explores alternative approaches to spilling, code alignment, and register allocation.

Spilling To Vector Registers

Modern CPUs provide a limited number of *vector units* to support processing data in parallel. Specialised operations that use those vector units are called *Single Instruction Multiple Data (SIMD)* operations. They allow to execute the same operation across multiple sets of values in parallel.

Instead of spilling values from general purpose registers into memory on the stack, the values can also be offloaded into those vector registers, which is expected to reduce the number of memory accesses and thus impact the performance. This section discusses spilling into those vector registers.

Experimental Setup

The experimental setup gives CRYPTOPT the functionality to spill into vector registers. This is compared against the default mode of not using any vector registers. Two modes either force CRYPTOPT to use the vector registers or to use the optimiser to decide whether or not to spill to vector registers or to resort to spilling into memory. In summary the three modes to compare are:

- (i) Baseline: spill to the next stack slot without using any vector units,
- (ii) use vector units but use the optimiser to potentially opt out and spill to memory and

TABLE 8.7. Number of instructions, vector spills, stack size and speedup (higher is better) compared to Clang of the resulting code for the three modes of spilling.

Method Name	Mode	# Instructions	Stack Size	# Vector Spills	Speedup
Curve25519–Multiply	Base	170	16	0	1.19
	Use	172	12	1	1.22
	Force	178	0	12	1.12
P-256–Multiply	Base	199	16	0	1.45
	Use	220	15	4	1.37
	Force	213	0	12	1.37
P-521–Multiply	Base	512	58	0	1.37
	Use	618	82	35	1.14
	Force	578	57	30	1.24

- (iii) spill to all available vector registers first, and only if no more vector registers are available, spill to memory.

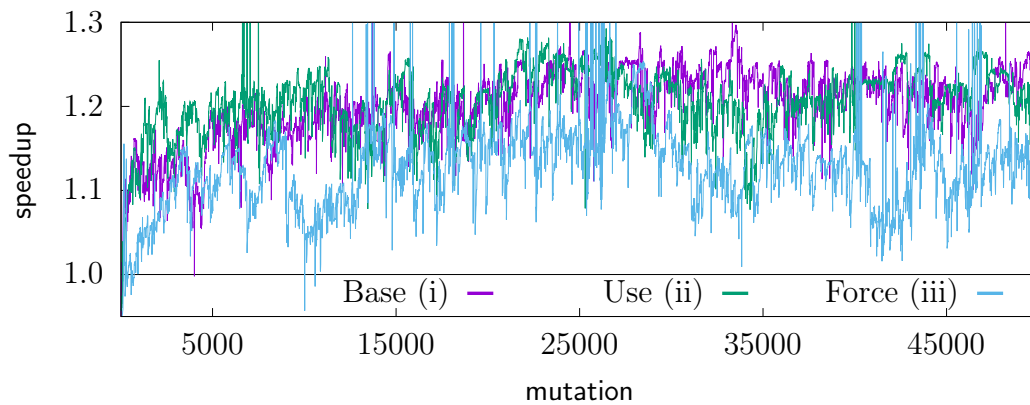
Results

Adding the spill decision to the optimiser increases the size of the search space. Hence, this time, the optimisation evaluated 50 000 mutations. [Figure 8.7](#) shows the graphs of optimising the functions, and [Table 8.7](#) shows the statistics of the resulting x86-64 assembly files. As expected, the stack utilisation is reduced when using the vector registers. However, optimising without vector registers seems to yield the best results. For Curve25519–Multiply, the optimiser optimises away most of the vector spills (cf. [Table 8.7](#)) while forcing the use of vector spills slows the program down. Similar for P-256–Multiply, although Use and Force result in the same speedup. Optimising P-521–Multiply with vector registers seems to slow down the program.

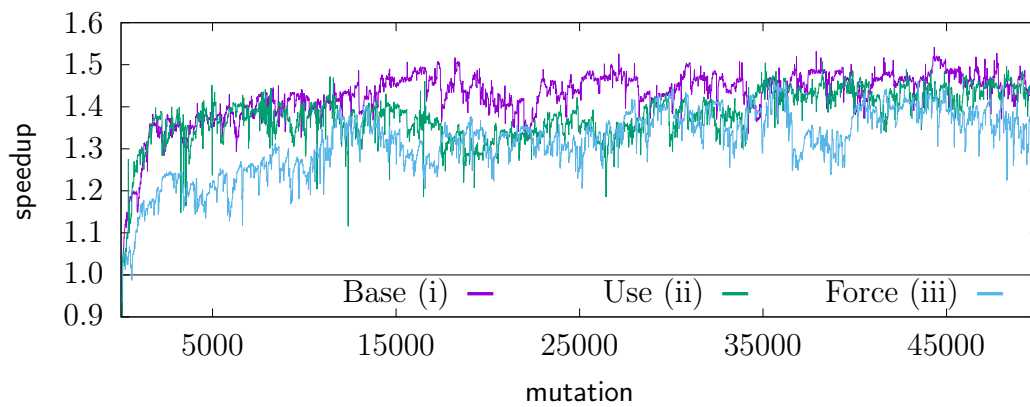
In general, using vector registers does not seem to improve program performance, which may be because of the warm-up phases of the vector registers. However, this effect might be reduced in future generations of processors.

Code Alignment

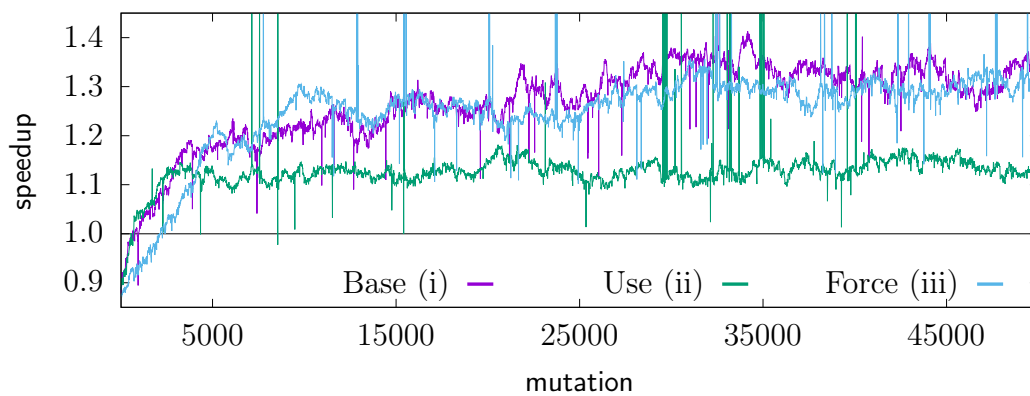
Aligning code in memory can make a difference in performance [[Myt+09](#); [Zhu+15](#)]. That is firstly because CPUs fetch the instructions in chunks and may need to recombine instructions that were only partly fetched and secondly because of the intricacies of the instruction cache [[ALS19](#); [Zhu+15](#)]. AssemblyLine has an



(A) Optimisation of Curve25519-Multiply.



(B) Optimisation of P-256-Multiply.



(C) Optimisation of P-521-Multiply.

FIGURE 8.7. Different modes of spilling when optimising Curve25519, P-256, and P-521-Multiply. All optimisation runs use a mutation budget of 50 000 mutations. Using vector registers does not seem to improve program performance, and for small programs is slows the execution down.

TABLE 8.8. Speedup of the result x86-64 assembly while aligning the code with `nop`-instructions to 8,16,32, or 64-byte boundaries.

Method Name	None	Alignment Boundary			
		8 bytes	16 bytes	32 bytes	64 bytes
Curve25519–Multiply	1.21	1.19	1.18	1.21	1.14
P-256–Multiply	1.42	1.34	1.41	1.43	1.41
P-521–Multiply	1.25	1.25	1.24	1.24	1.28

auto-aligning feature that inserts multi-length no-operation instructions [Int20b] into the code such that a given chunk boundary is not crossed [0xA23a]. Using this feature, the following experiment uses different chunk sizes to see if one is superior.

Experimental Setup

The experiment optimises the methods by aligning the code to 8, 16, 32, or 64 bytes. Additionally, an optimisation without any explicit alignment serves as the baseline.

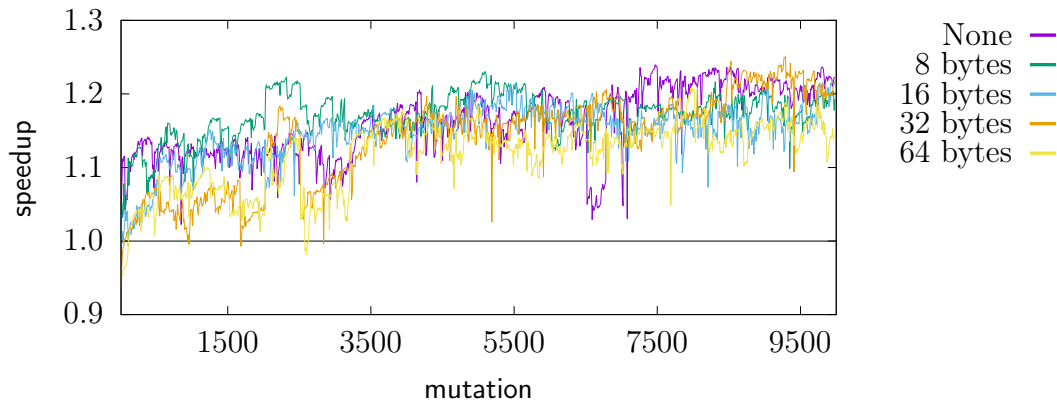
Results

Figure 8.8 shows the optimising runs. All modes optimise the methods effectively. Table 8.8 shows the statistics of the resulting x86-64 assembly files for each method-align combination. The alignments only increase the performance of the resulting code for P-256 at 32 bytes by 1% and P-521 at 64 bytes by 3%, compared to the non-aligned baseline. For the small function of Curve25519, alignments deteriorate the performance by up to 7%.

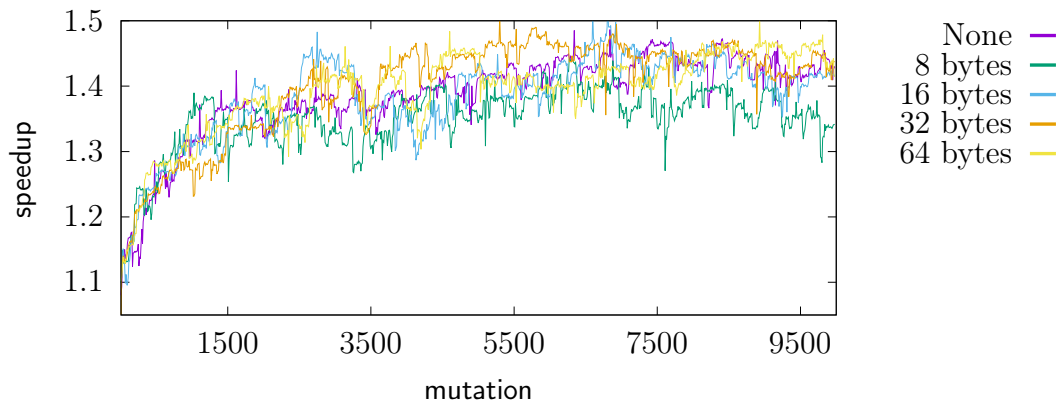
In general, the `nop`-insertion-based alignment does not positively impact the resulting code for one particular alignment.

Register Allocation Heuristic

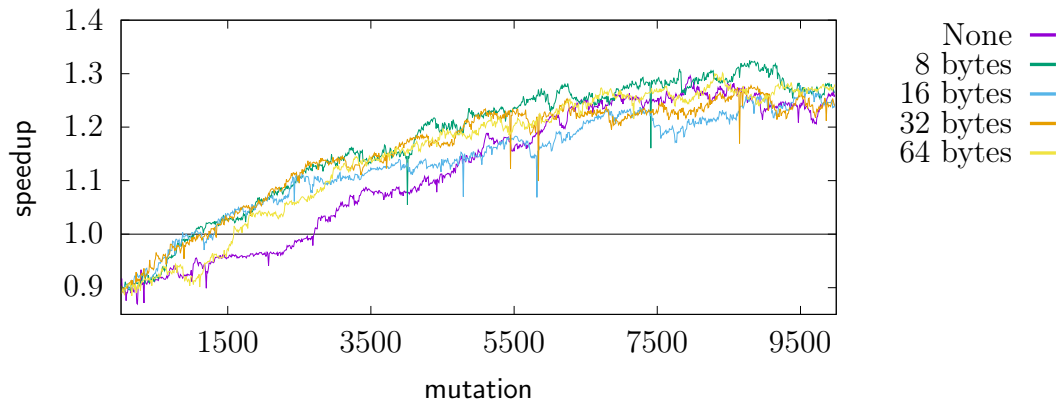
Register allocation is a crucial part that CRYPTOPT solves with its optimiser. Alternatively, allocation heuristics could be employed that deterministically choose values to be stored in particular ways. One such heuristic is to choose the value that is loaded into `rdx` for 128-bit multiplication using the `mulx` instruction when upcoming multiplication operations share a common factor. Specifically, if there are, e.g. three upcoming multiplications $a_0 \times b_0$, $a_0 \times b_1$, $a_0 \times b_2$, this heuristic would deterministically load the common factor a_0 into `rdx`, rather than choosing randomly and let the optimiser choose depending on the performance.



(A) Optimisation of Curve25519-Multiply.



(B) Optimisation of P-256-Multiply.



(C) Optimisation of P-521-Multiply.

FIGURE 8.8. Aligning the code with `nop`-instructions to 8, 16, 32, 64 bytes when optimising the multiply method of the fields Curve25519, P-256, and P-521. All modes optimise the method effectively after 10 000 mutations, compared to Clang.

TABLE 8.9. The table shows the code statistics of Curve25519, P-256, and P-521–Multiply after optimising with a deterministic register allocation heuristic for choosing `rdx`. The heuristic does not improve the code performance in those six runs.

Method Name	Mode	# Instructions	Stack Size	Speedup
Curve25519–Multiply	Random	182	22	1.16
	Heuristic	183	26	1.13
P-256–Multiply	Random	204	19	1.44
	Heuristic	211	19	1.40
P-521–Multiply	Random	531	74	1.29
	Heuristic	555	90	1.21

The expected effect is that this heuristic reduces the number of instructions because `mov` instructions are made redundant.

Experimental Setup

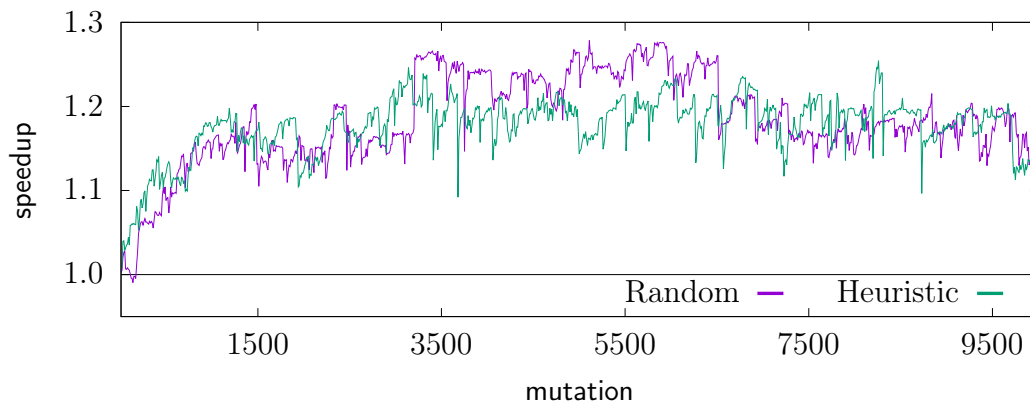
The experiment optimises the methods with the heuristic, cf. “Heuristic” and an additional optimisation without any explicit allocation serves as the baseline, cf. “Random”.

Results

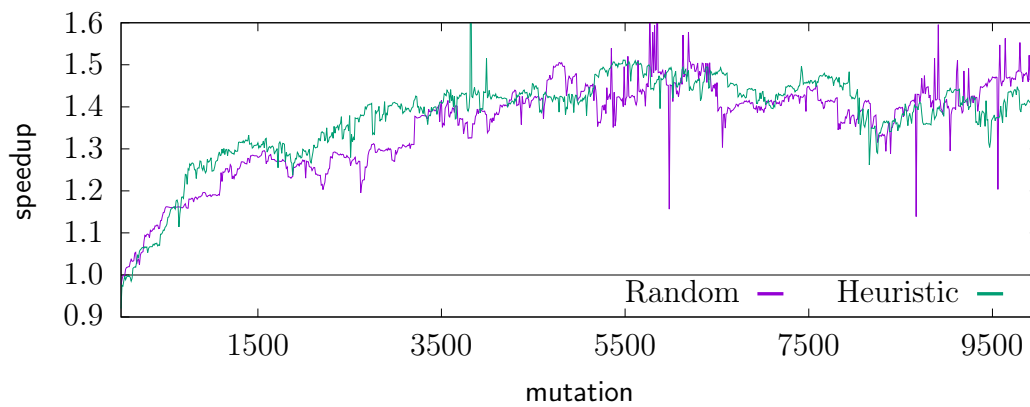
Figure 8.9 shows the optimising runs. Both modes effectively optimise the methods and achieve a similar end result. The heuristic-based code generation only has a slight impact on the optimisation. The close lines of the optimisations in the graphs show this behaviour. Table 8.9 shows the statistics of the resulting x86-64 assembly files for each method. The table shows that the end results are slightly worse (2–4%) with the heuristic. Additionally, the number of instructions is not reduced with the heuristic.

8.3 Evaluation Method

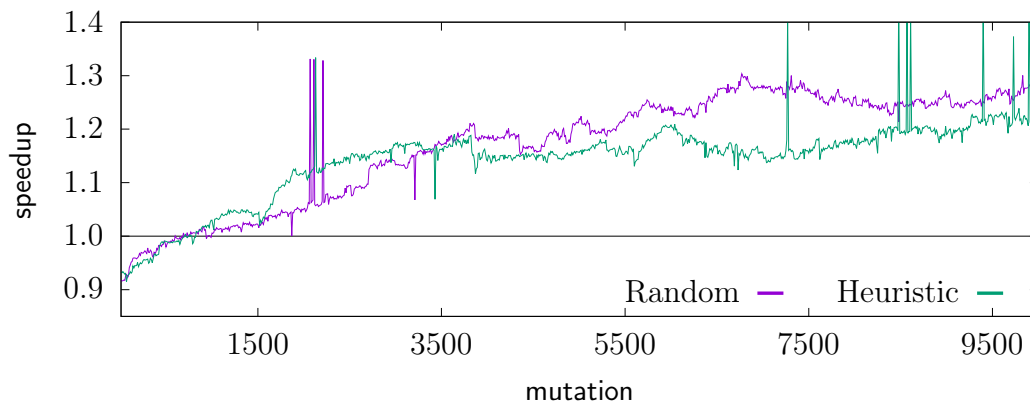
This section describes the evaluation methodology. The first part of the section describes the software setup of the machines and how CRYPTOPT is run. The second part describes how different implementations are compared across the different machines.



(A) Optimisation of Curve25519-Multiply.



(B) Optimisation of P-256-Multiply.



(C) Optimisation of P-521-Multiply.

FIGURE 8.9. Optimising three methods with a deterministic register allocation heuristic for choosing `rdx`. All modes optimise the method effectively after 10 000 mutations, compared to Clang. The heuristic does not seem to improve the code performance.

8.3.1 Software Setup

All machines run a fully up-to-date Ubuntu 22.04.3 LTS Server OS without any graphical user interface (GUI) or other unnecessary background tasks. CRYPTOPT is run bare metal, i.e. without any virtualisation or containerisation layer such as Docker.² The optimisation processes are pinned to one logical CPU core using `taskset`. One physical core is always left for general OS activity. All the other cores run optimisations. As such, the number of optimisations that are run in parallel depends on the number of physical cores in any hardware platform.

The CRYPTOPT paper [Kue+23b] states only using three runs on all machines to demonstrate CRYPTOPT within a fair comparison across multiple microarchitectures. This thesis additionally explores the stability of CRYPTOPT as per question (II). Hence, the setting in this thesis sets the scenario more realistic than the lab environment in the paper. As such, the experiments use the full potential of all machines by using all but one core. This results in some machines having fewer samples than others. For example, i7 6G has only four physical cores and, as such, three implementations per function, whereas 7950X has 16 physical cores and thus 15 assembly implementations per function.

8.3.2 Cross-Architecture Comparison

It is not trivial to compare the performance of different implementations of different functions across different microarchitectures because of the lack of a common baseline. One option is to compare to an off-the-shelf compiler to generate portable code without any optimisation enabled (i.e. `-O0` [Pee+22]). However, it seems unfair to compare CRYPTOPT-generated code with a non-optimised baseline because the simplest step to gain performance would be to use a well-known off-the-shelf compiler with optimisation enabled. That is, compile with `-O2` or `-O3` and compare whether `-O3` actually benefits the performance, as it does not always do [Myt+09]. However, that still restricts the compiler to emitting portable code, i.e., no use of ISEs. Once the compiler is used in a non-portable mode, i.e., enabling the usage of ISEs, and enabling tuning with it, it results in multiple different versions for different machines to compare against, out of which none would objectively “seem fair” as a common baseline.

To avoid this unfairness of compiler-generated implementations as a performance baseline, this thesis uses the CRYPTOPT-generated version for every

²CRYPTOPT seems to work successfully in a containerised environment (Docker). The artifact evaluation of the CRYPTOPT paper [Kue+23b] was done successfully using Docker.

microarchitecture as the baseline for each microarchitecture. Any other implementation, including compiler-generated versions, is compared against it by calculating the quotient of the cycles as per Equation 8.2.

$$\text{speedup} = \frac{\text{foreign implementation}}{\text{native CRYPTOPT generated}} \quad (8.2)$$

Accordingly, if a foreign implementation is slower (i.e. consumes more cycles, “good for CRYPTOPT”) than the native CRYPTOPT baseline, then the speedup factor is > 1 . If the foreign implementations are faster, the speedup factor is in the open interval $(0, 1)$.

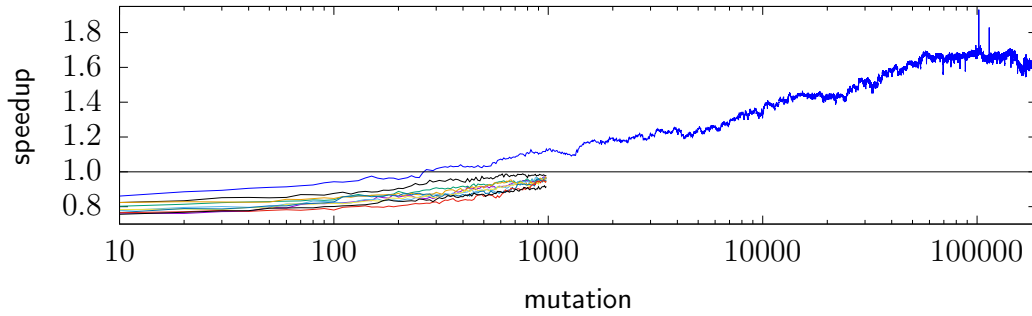
To compare a compiler against CRYPTOPT the following is respected: As CRYPTOPT aims to generate microarchitecture-specific high-performance code, off-the-shelf compilers are enabled to do the same. That is, with the highest optimisation options `-O3 -march=native -mtune=native`. This enables the compiler to use all available instructions and ISEs as well as tune the implementation as much as possible to the current microarchitecture.

8.4 Optimisation Over Time

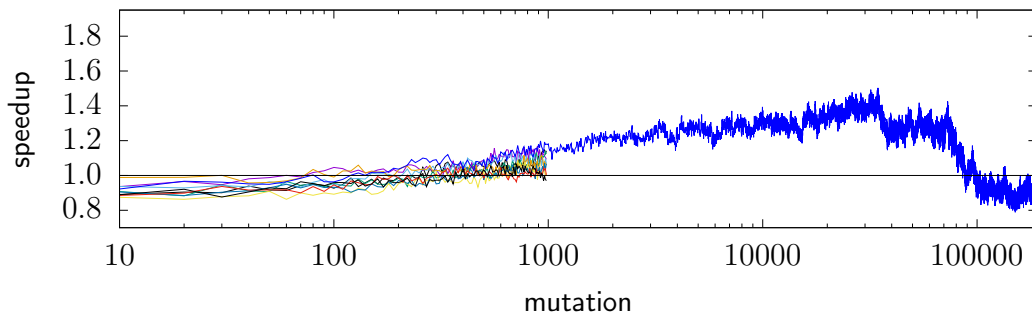
The previous sections describe the functions, the machines, and the alternative approaches. This section discusses the optimisation of those different functions on those different machines, how reliable CRYPTOPT is, and how sensitive the behaviour is.

CRYPTOPT starts the optimisation at a random point in the search space. From this starting point, the performance is gradually improved. For this evaluation, Bet-and-Run is used for $b = 10$ runs with a budget of 1 000 mutations each. The best-performing candidate continues for another $e_r = 190\,000$ mutations. Bet-and-Run is used because it is possible to start in a part of the search space that has worse performance than other parts and that prohibits the optimiser from getting out of. Using multiple starting points reduces the likelihood of this happening. Chapter 7 elaborates.

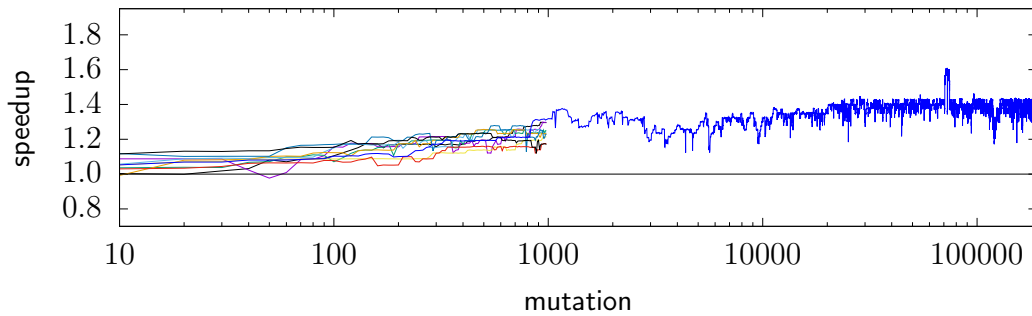
To optimise all 22 functions, it takes between 20 wall-clock hours on the fastest machine and 40 wall-clock hours on the slowest one. This optimisation behaviour depends heavily on the function and machine. In other words, no one function can be easily optimised on all machines, nor is one machine optimising all functions easily. To illustrate those points, Section 8.4.1 first summarises the general observation and Sections 8.4.2 to 8.4.4 subsequently compare various functions-machine combinations. The above hyperparameters



(A) P-384-Square on i7 11G



(B) P-384-Square on i9 13G



(C) Curve25519-Solinas-Multiply on i7 6G

FIGURE 8.10. Three CRYPTOPT optimisation runs. The first two of P-384-Square on i7 11G and i9 13G, the third one of Curve25519-Solinas-Multiply on i7 6G. Each line traces an optimisation run over time (x-axis, log-scale, one data point every ten mutations), showing the relative performance gain over Clang with options `-O3 -march=native -mtune=native`. Bet-and-Run used for 10 runs with a budget of 1 000 mutations each. The best-performing candidate continues for another 190 000 mutations. P-384-Square on i7 11G shows an ideal hill-climbing optimisation. P-384-Square on i9 13G shows that CRYPTOPT can drift at some point in the optimisation space. Curve25519-Solinas-Multiply on i7 6G shows an early saturation in a local optimum.

are chosen empirically to have one common baseline configuration across functions and machines. This allows to compare across this heterogeneous setup. As fine-tuning those hyperparameters is use-case dependent, it is left for future work.

8.4.1 General Observations

Figure 8.10 shows three optimisation runs with very different characteristics. In Figure 8.10a, CRYPTOPT optimised P-384-Square on i7 11G, and it shows a near-ideal hill-climbing optimisation (within rather noisy measurements). When CRYPTOPT optimised the same function on i9 13G, Figure 8.10b shows that CRYPTOPT can drift at some point in the search space, up to the point where it even diverts away from a local optimum. Figure 8.10c shows the optimisation of another function, Curve25519-Solinas-Multiply, on another microarchitecture: i7 6G. Here, an early saturation in local optimums can be observed that CRYPTOPT keeps (re-)discovering with subsequent mutations. It is unclear from that figure if those are the same optima (i.e., the exact same implementation) or if those are different implementations with the same fitness.

The following three sections discuss three different machine-function combinations to address the following three questions:

- (i) How stable are the results when the same settings are used?
- (ii) How sensitive is the behaviour to the choice of the function?
- (iii) How sensitive is the behaviour to the choice of the machine?

Section 8.4.2 answers question (i) by analysing four runs of the *same function* on the *same machine*, Section 8.4.3 answers question (ii) by analysing the *same function* on four *different machines*, and Section 8.4.4 answers question (iii) by analysing two *different functions* on two *different machines*.

8.4.2 One Function on One Machine

Figure 8.11 shows four optimisation runs of P-384-Square on i7 11G. There is no reason for this particular choice of machine and function combination because it is similar per machine across other combinations. Every optimisation run is independent of one another and initialised with a different seed, which causes the location in the search space to be different. This is shown with the different starting points of the runs in Figure 8.11.

The figure shows that one function is relatively stable on one machine (question (i)). In particular, all runs start at around 0.78x–0.85x speedup over the Clang compiled C code. After 190 000 mutations, the speedup is between 1.55x and 1.60x.

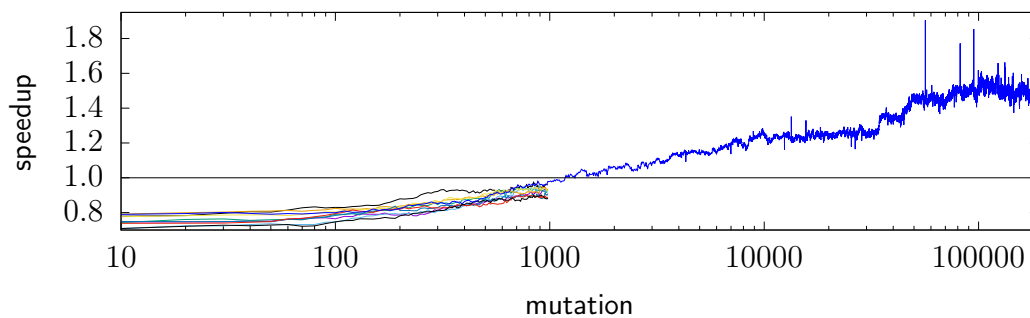
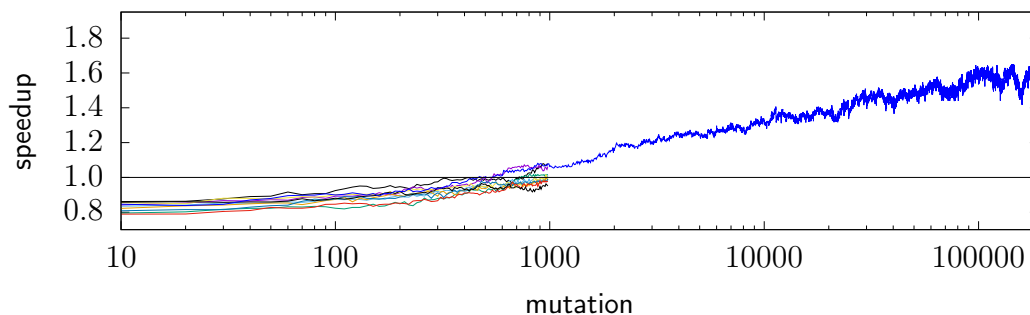
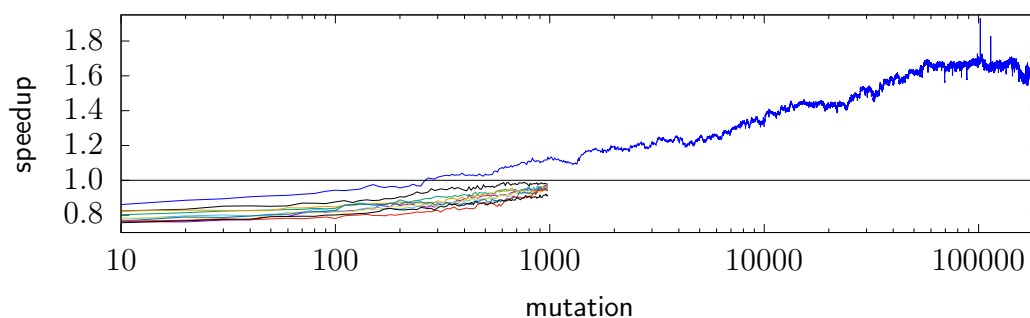
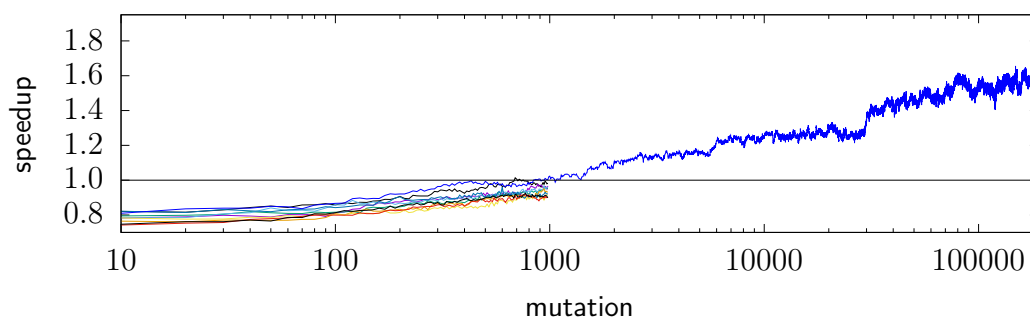
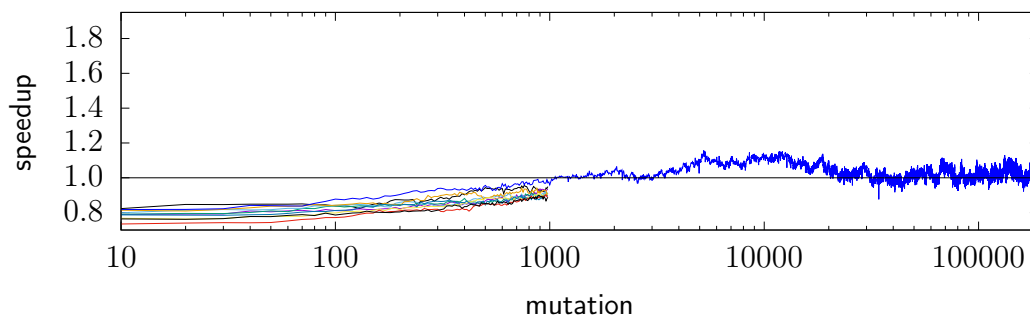
(A) 1st run of P-384-Square on i7 11G(B) 2nd run of P-384-Square on i7 11G(C) 3rd run of P-384-Square on i7 11G(D) 4th run of P-384-Square on i7 11G

FIGURE 8.11. Four CRYPTOPT optimisation runs for P-384-Square on i7 11G, with different seeds each, one data point is generated every ten mutations. All runs start at around 0.78x–0.85x speedup over the Clang compiled C code. After 190 000 mutations, the speedup is between 1.55x and 1.60x.

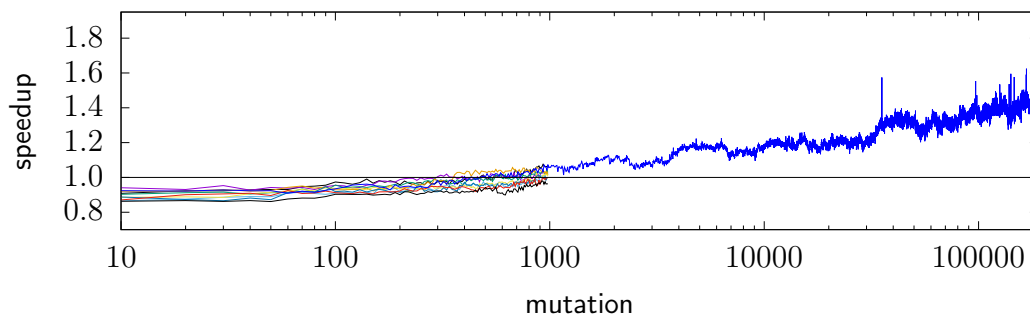
8.4.3 Same Function on Different Machines

To answer question (ii), this section analyses the same function P-384-Square being optimised on four more machines in addition to the two in [Figures 8.10a](#) and [8.10b](#). [Figure 8.12](#) shows vastly different optimisation behaviour: two optimisations on AMD platforms ([Figures 8.12a](#) and [8.12b](#)) and two on Intel ([Figures 8.12c](#) and [8.12d](#)) without any clear distinguishing characteristics.

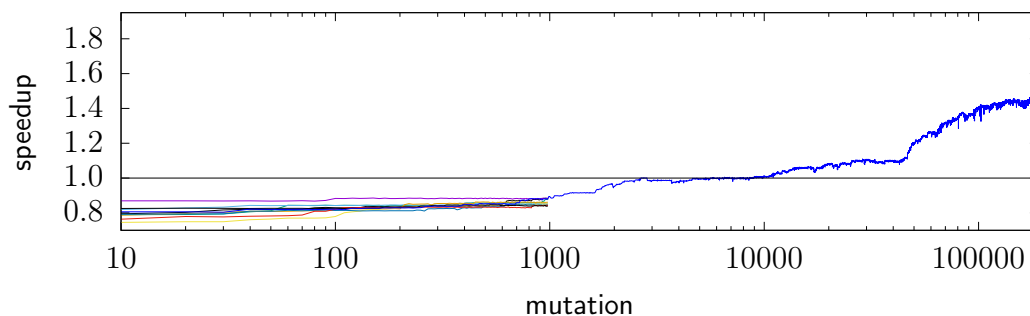
[Figure 8.12a](#) (1900X) shows a rather noisy optimisation that seems to saturate into an optimum rather early; [Figure 8.12b](#) (5950X) seems similarly noisy but does not seem to be in a local optimum at the end of the optimisation. [Figure 8.12c](#) (i7 6G) shows rather precise measurements and very clear, almost ideal hill-climbing behaviour. [Figure 8.12d](#) (i9 12G) is similar to [Figure 8.10b](#) (i9 13G). They show Intel's twelfth and thirteenth generations. Both show a significant performance drop near the end of the optimisation, after around 70 000 mutations.



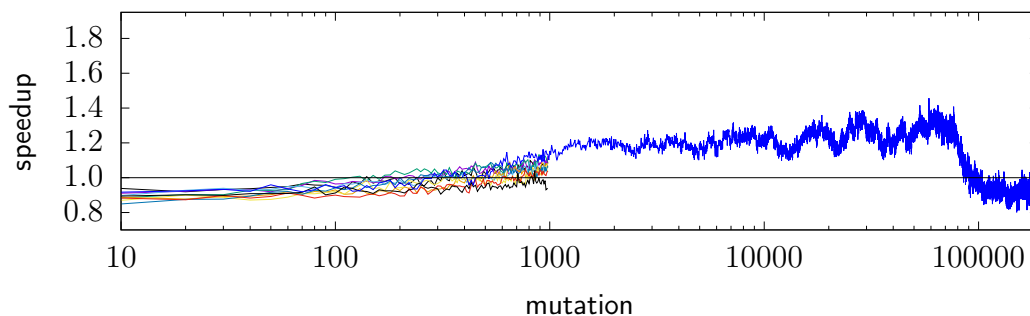
(A) P-384-Square on 1900X



(B) P-384-Square on 5950X



(C) P-384-Square on i7 6G

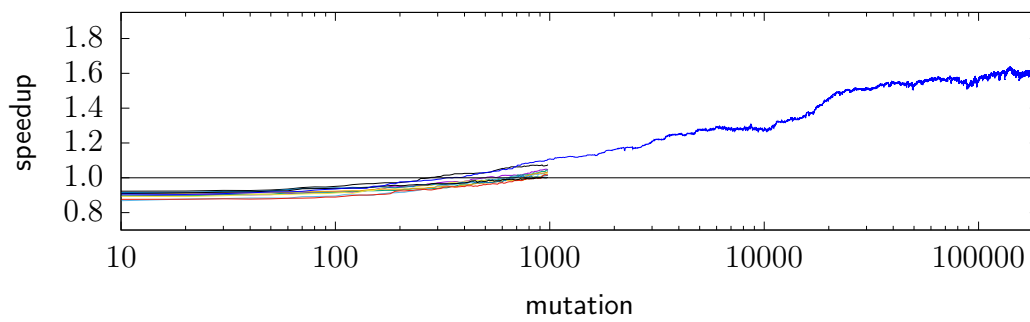


(D) P-384-Square on i9 12G

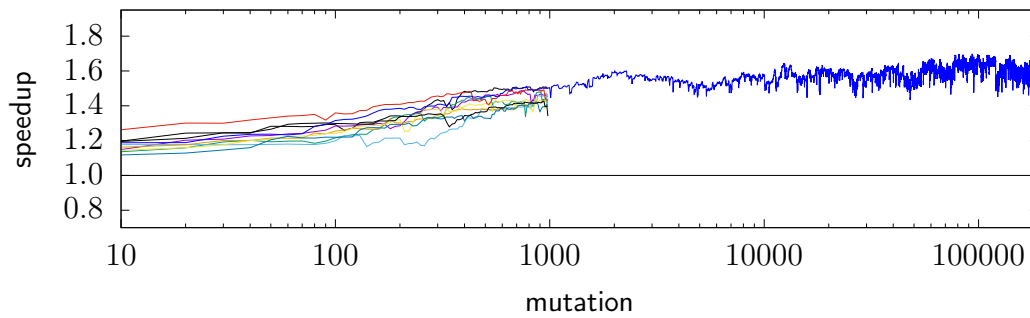
FIGURE 8.12. Four CRYPTOPT optimisation runs for P-384-Square on different hardware. One data point is generated every ten mutations. All four runs show very different optimisation behaviour, from near-ideal hill-climbing behaviour over early saturations to performance drops on newer Intel microarchitectures.

8.4.4 Different Functions on the Same Machine

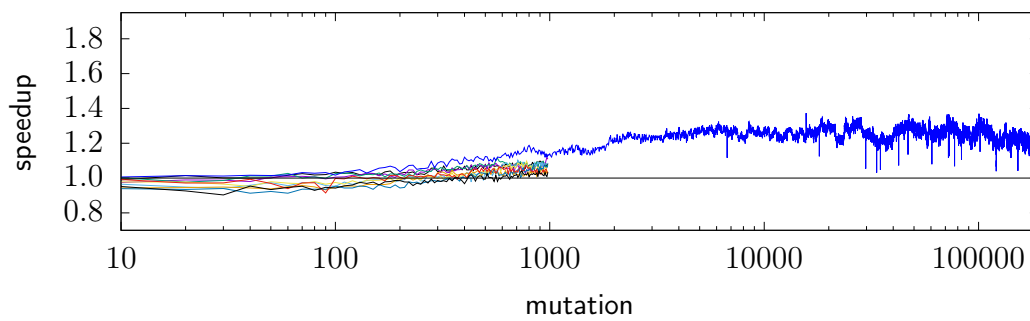
P-384-Square does not have any particular obvious properties that make it harder or easier to optimise on one particular hardware platform than another. This section answers question (iii) by analysing two scenarios of two different functions (SIKEp434 and secp256k1-Montgomery), each on two different machines (7950X and i7 6G). Figure 8.13 shows the four optimisation runs. It shows twice that the different functions, each one independently optimised on two microarchitectures, are very different. In particular, Figures 8.13a and 8.13c both show the square function of the field for SIKEp434. On the i7 6G, it again shows near-ideal hill-climbing behaviour, but it seems much more challenging on the 7950X, where only moderate optimisation can be observed. Similarly, on the flip side, Figures 8.13b and 8.13d show the square function of the secp256k1 field (Montgomery implementation), which, despite more apparent noise, seems to be optimised better on the AMD 7950X than on the Intel i7 6G.



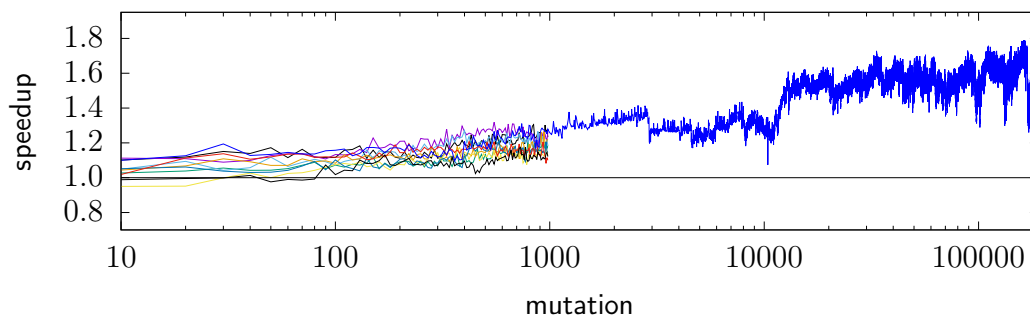
(A) SIKEp434-Square on i7 6G



(B) secp256k1-Square on i7 6G



(C) SIKEp434-Square on 7950X



(D) secp256k1-Square on 7950X

FIGURE 8.13. Four CRYPTOPT optimisation runs for SIKEp434-Square and secp256k1-Square on 7950X and i7 6G. One data point is generated every ten mutations. All four runs show very different optimisation behaviour: From near-ideal hill-climbing behaviour to early saturations.

TABLE 8.10. Instruction count (average over all ten platforms).

Primitive	Multiply	Square
Curve25519	185.686	128.679
Curve25519-Solinas	154.434	105.772
NIST P-224	265.903	263.948
NIST P-256	237.262	232.224
NIST P-384	717.124	712.575
SIKEp434	1100.017	1100.396
Curve448	575.365	375.993
NIST P-521	616.655	378.706
Poly1305	75.965	57.127
secp256k1-Dettman	208.972	157.475
secp256k1	259.586	254.924
Bitcoin Core (LLVM)	231.151	176.237

8.5 Length of Functions

Section 8.1.2 describes that the number of operations depends on the size of the field. It follows logically that the length of a function’s implementation is proportional to the number of operations. In general, functions with much fewer instructions take less time and are thereby more performant than much longer (around 1.5 times as long) functions. Hence, the combination of random start and gradual increase in performance results in the fact that a function’s length is higher at the start and decreases during the optimisation.

This section uses the function’s length as a reliability metric that is comparable across multiple CRYPTOPT runs and without any additional reference, such as a compiler baseline. Table 8.10 shows the average instruction counts that are needed to implement a function. Smaller fields such as Poly1305 need an order of magnitude less instructions than larger fields such as SIKEp434. Figure 8.14 shows the functions’ lengths in violin plots across all machines. In particular, it shows that most functions have small variances, especially those with smaller instruction counts. It also shows that the implementations for P-384 and SIKEp434 have a much larger variance. Longer functions have a larger search space. This could be a reason for the less reliable results.

In conclusion, CRYPTOPT reliably emits fast code with the current set of hyperparameters when targeting the smaller fields, regardless of initial starting conditions. For larger fields, the search space is larger, and as such, the resulting implementations are less reliable.

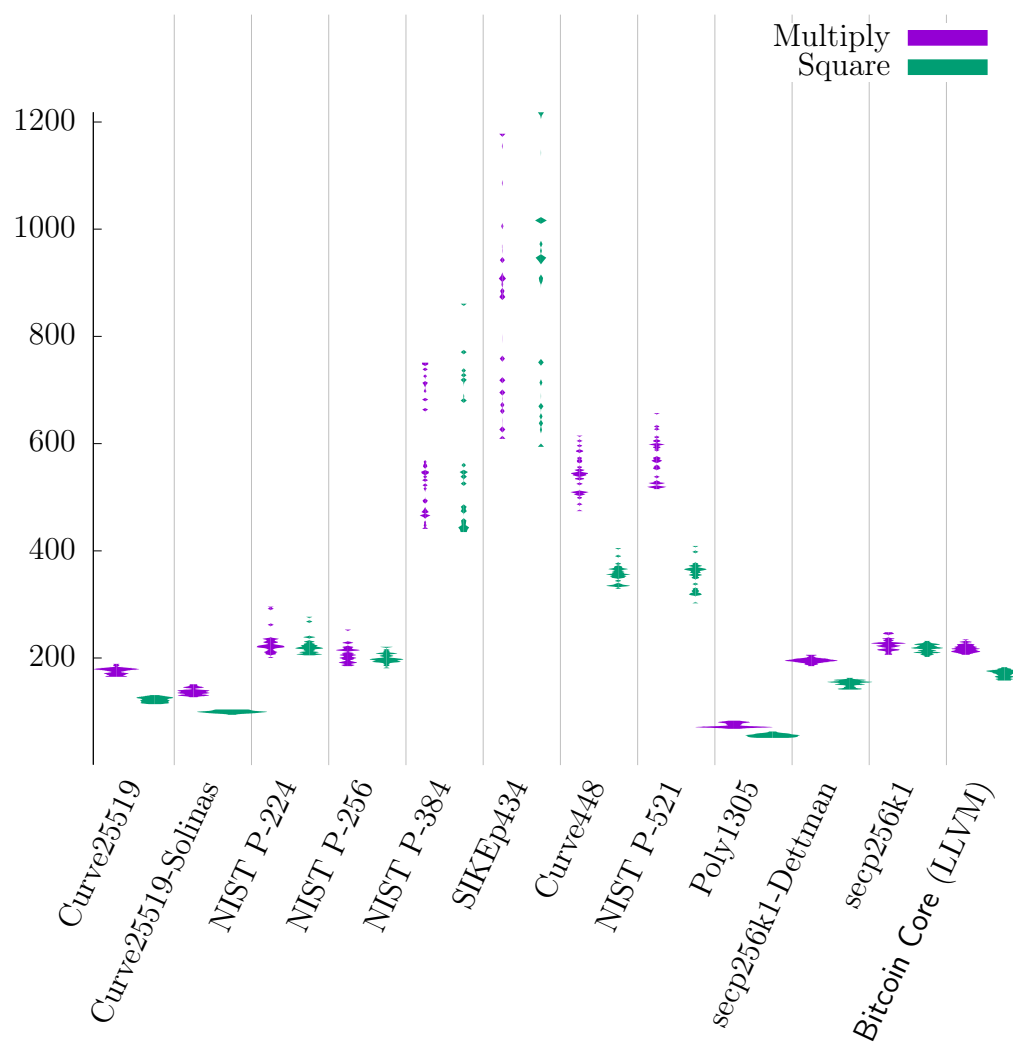


FIGURE 8.14. Violin plots showing the functions' length as a number of instructions across all machines.

8.6 Specificity of Implementations

This section describes how specific a CRYPTOPT-generated implementation is to one microarchitecture. The methodology for doing that is to generate implementations on one machine and run it on another. However, recalling that there are multiple implementations per machine, the first step is to find the best-performing one for each platform. [Section 8.6.1](#) gives background information on the required statistical methods, and [Section 8.6.2](#) describes how to rank and elect the best implementations per microarchitecture. This best implementation is then evaluated on other machines ([Section 8.6.3](#)).

8.6.1 Student's t -Test

Student's t -tests [[Gos08](#)] can be used to gain knowledge of a population by only observing samples of that population. In particular, a *Two-Sample t -Test* can be used to test whether differences between samples are statistically significant. If they are it is evidence that they are drawn from different distributions. If they are not, it is not evidence that they are from the same distribution. To use the test, the pooled variance S_p^2 of the population is estimated as described in [Equation 8.3](#).

$$S_p^2 = \frac{(n_1 - 1) \times S_1^2 + (n_2 - 1) \times S_2^2}{n_1 + n_2 - 2} \quad (8.3)$$

Where S_x^2 is the variance of the sample x , and n_x is the size of sample x . Then, a t -value is calculated using [Equation 8.4](#).

$$t = \frac{(\bar{X}_1 - \bar{X}_2)}{\sqrt{S_p^2 \times (\frac{1}{n_1} + \frac{1}{n_2})}} \quad (8.4)$$

Where $\bar{X}_x$ is the mean of the sample x . This t value is compared to a critical value p from a table, according to a given confidence level. If the calculated value is smaller than the critical value ($t < p$ is true), then the samples are considered to be from the same distribution.

8.6.2 Rank Implementations

Running multiple optimisations with different starting conditions, such as the seed, results in multiple implementations. On top of that, as seen in [Figure 8.10b](#), the final result may not be the overall best-observed implementation, and more or less evaluated mutations could possibly increase the probability of getting the best implementation at the end of the predefined budget. Because of measurement

errors such as the spikes in [Figure 8.10a](#), simply taking the implementation with the best-observed performance would not work. The task of finding optimal abort criteria is left for future work, but to work around this problem in the meantime is to let CRYPTOPT emit the current state and current implementation in regular intervals during the optimisation. This interval is currently configured to emit the implementation at every 10% of the optimisation run.

The basis for the ranking is the *measure* algorithm shown in [Algorithm 3](#), which gives one measurement per generated assembly file. In addition, the following actions are performed to get more precise, absolute quality measures for the assembly programs. First, the machines are set to be even quieter by disabling boosting behaviour, setting the frequency to the minimum (cf. [Table A.1](#)), and using the performance governor. Second, the *measure* function is only called with one assembly string and a C-compiled baseline. The execution order is still randomised. Third, instead of only using one single result of the *measure* function, a set of 20 measurements is collected. Those measurements of this set are randomly split into two sets of ten.³ Finally, those two sets are checked with the two-sample Student’s *t*-test (confidence level of 95%) to determine whether they are likely to come from different distributions. If they do not, the median of all measurements is the final measure for the assembly code. Otherwise, this procedure repeats with the collection of another 20 samples. In practice, this only happens in less than one per cent of the measurements.

8.6.3 Cross Validation

By design, CRYPTOPT optimises exactly for the microarchitecture it runs on, and the result may overfit the microarchitecture. This can be a double-edged sword, as it can provide better performance for one but may be slower on another microarchitecture. Thus, on the one hand, on server farms, where the microarchitectures are mostly homogeneous, a highly-fit solution may be desired. On the other hand, extra caution should be exercised to choose one of those highly-fit implementations for usage in a library that is deployed on a plethora of diverse CPUs. However, now, with CRYPTOPT, multiple such implementations could be generated cheaply, and all of them could be included in a library.

To visualise how *specific* an implementation is to the underlying microarchitecture, CRYPTOPT optimises on one machine, and the best result (as described

³So far, no statistical analysis has been performed on those magic numbers because they empirically worked well. It yielded similar rankings of the code under test on repeated measurements and only consumed moderate time for the measurement itself. de Winter [[de 13](#)] reports the viability of the *t*-test with sample sizes of $N \leq 5$, so 10 seems reasonable for this matter.

TABLE 8.11. Optimisation matrix for Curve25519-Solinas Multiply.

Curve25519-Solinas												
MULTIPLY	run on	1900X	5800X	5950X	7950X	i7 6G	i7 10G	i9 10G	i7 11G	i9 12G	i9 13G	G.M.
	opt on											
	1900X		1.15	1.15	1.15	1.09	1.11	1.14	1.18	1.10	1.13	1.12
	5800X	1.05		1.02	1.01	1.04	1.07	1.09	1.13	1.06	1.09	1.06
	5950X	1.03	0.99		1.05	1.04	1.07	1.09	1.09	1.10	1.12	1.06
	7950X	1.07	1.06	1.06		1.04	1.07	1.09	1.09	1.07	1.10	1.06
	i7 6G	1.06	1.14	1.15	1.17		1.02	1.05	1.04	1.02	1.04	1.07
	i7 10G	1.09	1.08	1.10	1.12	0.98		1.02	1.02	1.04	1.06	1.05
	i9 10G	1.07	1.10	1.11	1.10	0.95	0.98		1.02	1.02	1.04	1.04
	i7 11G	1.09	1.11	1.12	1.15	1.02	1.04	1.07		1.01	1.03	1.06
	i9 12G	1.06	1.13	1.14	1.13	1.00	1.02	1.05	1.05		1.02	1.06
	i9 13G	1.09	1.13	1.14	1.12	1.00	1.02	1.04	1.04	0.98		1.05
Clang	1.66	1.79	1.81	1.81	1.46	1.50	1.53	1.73	1.62	1.67	1.65	
GCC	2.36	2.35	2.38	2.37	2.70	2.76	2.82	2.70	2.77	2.84	2.60	
Final	1.66	1.81	1.81	1.81	1.53	1.53	1.53	1.73	1.65	1.67	1.67	

in [Section 8.6.2](#)) is run on all other machines. To that end, CRYPTOPT-generated code for the 24 functions (22 Fiat Cryptography-based plus two from the LLVM) on the ten microarchitectures, as per [Table 8.2](#). The visualisation is in table form, such as [Table 8.11](#), which shows the function *Curve25519-Solinas Multiply*. The full detailed overview for all functions is in [Appendix B.2](#).

The columns of the table show on which machine the implementation was run, and the rows show how the implementation was produced. That is, either by CRYPTOPT on a certain machine or from an off-the-shelf compiler on the highest optimisation settings. The cell itself specifies the speedup factor as described in [Section 8.3](#). The 10-by-10 matrix in the upper left shows how *hardware-specific* a CRYPTOPT-generated implementation is. For example, in [Table 8.11](#), cell *opt on=1900X run on=5800X* with the value 1.15, which means that when the function Curve25519-Solinas Multiply is optimised on the 1900X machine and run on machine 5800X, it takes 1.15 times as many cycles than the implementation that was natively optimised with CRYPTOPT on 5800X. This means that the natively optimised implementation outperforms the foreign implementation. Thus, the number is greater than 1, and the cell is marked in blue. Otherwise, it would be less than 1 and coloured in an orange tone. The

faster an implementation is, the more saturated the cell is (full saturation if the value x of a cell is $0.9 \geq x \geq 1.1$).

The last column, G.M., shows the geometric mean of the row, i.e. implementation. It gives an indication of *how good the implementation is on average*; the higher the number, the better.

The subsequent two rows (Clang/GCC) compare the optimised version versus the off-the-shelf compilers with options `-O3 -march=native -mtune=native`. Again, the higher the number the better is the CRYPTOPT-generated implementation. For example, the 1.66 means that on the 1900X, the version from Clang takes 1.66 times as many cycles as the CRYPTOPT-generated version.

Lastly, “Final” shows the time savings of the best-performing implementation (regardless of where it was optimised on) over the best-performing compiler-generated version.

Discussion

Table 8.11 shows that implementations fit similar processor families and microarchitectures. In particular, the smaller sub-matrices across machines of similar microarchitecture contain numbers very close to 1. For example, there seems to be only extremely subtle differences on Zen 3 microarchitectures because the 2-by-2 sub-matrix of 5800X and 5950X only contains values of the set $\{0.99, 1.02\}$. Extending this sub-matrix to the Zen 4 microarchitecture 7950X, the range of the numbers is relatively small, in the interval of $[0.99, 1.06]$. Similarly, the sub-matrix of essentially all the Intel machines is between 0.95 and 1.08.

When looking at cross-microarchitecture differences, Intel vs. AMD, that is, CRYPTOPT can find implementations that, when used natively, outperform all other microarchitectures. The bottom line seems to be the following: The closer the target microarchitecture is to what CRYPTOPT runs on, the better the final implementation fits the actual target. That being said, all implementations outperform off-the-shelf compilers significantly.

8.7 CRYPTOPT vs. Off-the-Shelf Compilers

Table 8.12 shows the geometric means over all the machines against Clang and GCC, respectively, for all functions. The results show that, on average, for most of the tested functions, CRYPTOPT-generated code significantly outperforms off-the-shelf-compiled code, up to a speedup of 2.96x. In the other cases, CRYPTOPT at least matches the performance of off-the-shelf compilers. The complete detailed overview of all functions against all compilers is in [Appendix B.2](#).

TABLE 8.12. Geometric means of CRYPTOPT vs. off-the-shelf compilers over ten architectures.

Curve	Multiply		Square	
	Clang	GCC	Clang	GCC
Curve25519	1.27	1.16	1.25	1.18
Curve25519-Solinas	1.65	2.60	1.45	2.80
P-224	1.62	2.66	1.45	2.63
P-256	1.75	2.69	1.69	2.64
P-384	1.51	2.58	1.41	2.61
SIKEp434	2.04	2.96	1.88	2.66
Curve448	1.21	1.00	1.09	1.05
P-521	1.35	1.03	1.39	1.09
Poly1305	1.15	1.22	1.12	1.26
secp256k1-Dettman	1.11	1.26	1.07	1.16
secp256k1	1.88	2.71	1.81	2.67
Bitcoin Core (LLVM)	1.08	1.21	1.05	1.28

8.8 State-of-the-Art Comparison

So far, the evaluation has only considered the field arithmetic in isolation. Recall from [Section 2.3.2](#) that the performance-dominating function for signatures and key exchanges in ECC is the scalar multiplication, which itself is dominated by the field arithmetic. This section discusses how much time can be saved in this scalar multiplication by using the CRYPTOPT-optimised field arithmetic and how this compares to hand-optimised implementations. The SUPERCOP [\[BL22\]](#) framework is used to benchmark this scalar multiplication.

[Section 8.8.1](#) introduces SUPERCOP and what it benchmarks, and [Section 8.8.2](#) describes the implementations that are compared. [Section 8.8.3](#) then discusses the performance results.

8.8.1 SUPERCOP

SUPERCOP tries nine different compiler option combinations (cf. [Table B.13](#)) and reports the smallest cycle count for four scalar multiplications. Those are split up into two fixed point multiplications and two variable point multiplications. In its benchmarking, it calculates the four scalar multiplications as follows: G is the base point, and a and b are random scalars. With [Equations \(8.5a\)](#) and [\(8.5b\)](#) two random points A and B are calculated. With [Equations \(8.5c\)](#) and [\(8.5d\)](#) those new points are then multiplied with the respective other scalar, resulting in C and D . As scalar multiplication is commutative, [Equation 8.5e](#) holds, and

C and D must be equivalent to each other and abG . SUPERCOP also checks memory constraints, i.e. the implementations do not incorrectly access memory, while those scalar multiplications are performed. Importantly, it also checks whether C and D are equivalent at the end of the four scalar multiplications and errors out in case they differ.

$$a \times G = A \tag{8.5a}$$

$$b \times G = B \tag{8.5b}$$

$$a \times B = C \tag{8.5c}$$

$$b \times A = D \tag{8.5d}$$

$$C = aB = abG \wedge D = bA = baG = abG \implies C = D = abG \tag{8.5e}$$

8.8.2 Implementations

SUPERCOP packages many implementations for scalar multiplications already. This section describes those implementations on the basis of the results table. [Table 8.13](#) shows the average results over all microarchitectures for various implementations, [Table B.14](#) shows the detailed results on individual machines. The table shows two sections, one for Curve25519 and one for secp256k1. The first five implementations are shipped with SUPERCOP itself. “sandy2x” is an implementation from Chou [[Cho15](#)], mostly hand-written in QHASM and compiled into vectorised x86-64 assembly. “amd64-{64,51}” are implementations from Chen et al. [[Che+14](#)], which are hand-written in scalar x86-64 assembly. The suffix indicates a saturated or unsaturated limb representation. “donna{,-c64}” are implementations from Langley [[Lan23b](#)], which are hand-written in either vectorised x86-64 assembly (“donna”) or portable C (“donna-c64”).

In all of the following implementations in the first part of the table, the implementation for the scalar multiplication is based on the same implementation from OpenSSL [[Ope23a](#)]. The distinguishing factor is the implementation of the field arithmetic. The implementations that have a suffix attached signify the saturation. The first implementation, “fe-51 off-the-shelf”, is written in C using unsaturated limbs. It is used as a fallback in the library when no assembly version exists for the current architecture. Alternating, the table lists “off-the-shelf” and “+**CRYPTOPT**”, all of which have their field arithmetic written in x86-64 assembly. The off-the-shelf ones are based on the “amd64” implementations (by [[Che+14](#)]). The “+**CRYPTOPT**” suffix denotes that the field arithmetic has been replaced by Fiat Cryptography-generated and CRYPTOPT-optimised

TABLE 8.13. Cycle counts for four scalar multiplication, geometric mean over ten machines. Column “Language” denotes the language in which the field arithmetic is implemented.

Field	Implementation	Language	Geometric Mean	
Curve25519	sandy2x	asm-v	496k (1.07x)	
	amd64-64	asm	553k (1.19x)	
	amd64-51	asm	558k (1.20x)	
	donna	asm-v	994k (2.14x)	
	donna-c64	C	585k (1.26x)	
	OpenSSL	fe-51 off-the-shelf	C	531k (1.14x)
		fe-51 off-the-shelf	asm	531k (1.14x)
		fe-51+ CRYPTOPT	asm	527k (1.13x)
		fe-64 off-the-shelf	asm	466k (1.00x)
		fe-64+ CRYPTOPT	asm	465k (1.00x)
		fe-51+ Best	asm	530k (1.14x)
		fe-64+ Best	asm	475k (1.02x)
secp256k1	libsecp256k1	off-the-shelf	asm	559k (1.10x)
		off-the-shelf	C	541k (1.06x)
		+ CRYPTOPT (Opt)	asm	533k (1.05x)
		+ CRYPTOPT (Fiat)	asm	510k (1.00x)
		+ Best (Opt)	asm	543k (1.06x)
		+ Best (Fiat)	asm	511k (1.00x)

implementations. This replacement happens locally, meaning that *every machine runs its own locally optimised implementation*. On the contrary, for the implementations with “+**Best**” annotations, *one single implementation* is run on all machines. This “+**Best**”-row shows, how portable the x86-64 assembly is, if only one implementation would be shipped.

For the secp256k1 section, all the implementations for the scalar multiplication are based on the implementations from the **Bitcoin Core** library `libsecp256k1` [lib23]. The first two, with only “off-the-shelf”-suffix, are the plain versions from the library, one in hand-written x86-64 assembly, the other one in portable C code.⁴ The x86-64 assembly version is preferred by default if the library is built for a compatible architecture.

The field arithmetic is replaced in the four subsequent implementations as follows: Like in the Curve25519 block, the “+**CRYPTOPT**” suffix denotes that the locally optimised version is used and the “+**Best**” means that only *one* fast implementation is run on all machines. Different from the Curve25519 block are the (Fiat) or (Opt) suffixes. With the (Fiat) suffix, the input to **CRYPTOPT** is based on **Fiat Cryptography**, which implicitly also features formal correctness guarantees. The (Opt) suffix denotes that the input to **CRYPTOPT** is based on the LLVM that is generated from the library’s C code (cf. [Section 9.3](#)). This implementation then does not have formal correctness verification because there is no truth against which to check equivalence.

8.8.3 Results

The fastest implementation per section is marked in **bold** and **(1.00x)**. All other implementations are then compared against this fastest in the form of the ratio, which is written in parenthesis. For example, the 1.07x in the “sandy2x” row means that on average (Geometric Mean), “sandy2x” takes 1.07 times as many cycles as the fastest, which is the “fe-64+**CRYPTOPT**”.

The final result is that the code that **CRYPTOPT** generates automatically, on average, matches the performance of hand-optimised code for both the saturated and unsaturated implementations of Curve25519. [Table B.14](#) shows that on newer microarchitectures (11th – 12th generation Intel CPUs), the generated code outperforms the existing hand-optimised code by a factor of about 1.06x. The connection to **Fiat Cryptography** provides formal correctness guarantees for all the generated implementations for Curve25519.

⁴The C code features a slight optimisation (PR #810) to avoid overly wide multiplications. This has not been ported to the x86-64 assembly. This costs around 3 % ($559k/541k \approx 1.03$) of performance, as seen in the first two rows of the secp256k1 part of [Table 8.13](#).

For the Bitcoin field “secp256k1”, on average, the Fiat Cryptography +**CRYPTOPT** generated code is 1.10 times faster than libsecp256k1’s assembly and 1.06 times faster than what the off-the-shelf compilers can produce from their C code. In addition, the Fiat Cryptography +**CRYPTOPT**-generated code has formal correctness guarantees. For the LLVM-based approach, **CRYPTOPT**-generated code is roughly on par with off-the-shelf compiler (1.06x vs. 1.05x).

8.9 Conclusions

This section concludes the performance evaluation. The first two questions were about how the optimisation progresses over time and how reliable **CRYPTOPT** finds best-performing implementation (questions (I) and (II)). The answer is that, in general, **CRYPTOPT** starts with lower performance and reliably finds better-performing implementations. With the given set of hyperparameters, **CRYPTOPT** performs better on smaller fields that require shorter implementations. Longer functions can still be optimised but less reliably.

The third question, how specific the generated code is to one microarchitecture (question (III)) is answered as follows: **CRYPTOPT** can in general, despite inaccuracies in the measurements on some platforms, optimise precisely for one microarchitecture. In particular, **CRYPTOPT** finds implementations that run with the best performance on one particular microarchitecture. This one implementation is then sometimes outperformed by other **CRYPTOPT**-generated solutions for other platforms. The closer any microarchitecture is to the target microarchitecture, the closer **CRYPTOPT** finds a fast implementation.

The final question is how well **CRYPTOPT**-generated code compares to the state-of-the-art (question (IV)). **CRYPTOPT** generates code that significantly outperforms up-to-date off-the-shelf compilers for most functions. Compared to the hand-optimised state-of-the-art, **CRYPTOPT** finds solutions that yield similar performance on average, and can outperform slightly on the latest Intel architectures.

Chapter 9

Real World Integration

After being showcased at several conferences, CRYPTOPT has met the interest of various research groups around the globe. This section notes how the capabilities of CRYPTOPT are already in use, and where the current limitations and open problems lie in integrating further. First, CRYPTOPT-created code is now in BoringSSL (Section 9.1). Second, CRYPTOPT-generated code is considered to be incorporated into the Bitcoin Core library (Section 9.2). Third, a simple LLVM-to-CRYPTOPT converter is used to create code based on Bitcoin Core-C code into CRYPTOPT IR. Section 9.3 describes these efforts. Fourth, CRYPTOPT created code for another signature scheme (Section 9.4). Fifth, the attempts and limitations in regards to Jasmin (Section 9.5). Finally, Section 9.6 describes static optimisations that are used by all the integrations.

9.1 BoringSSL

BoringSSL [Bor23] is a cryptographic library maintained by Google. It originated as a fork of OpenSSL [Ope23a] but strives for more simplicity to avoid bugs. A primary difference to OpenSSL is the implementation of Curve25519. It only features one single C implementation, whereas OpenSSL has at least two hand-optimised assembly implementations (one saturated and one unsaturated) and a backup unsaturated implementation in C.¹

When Fiat Cryptography was published, BoringSSL incorporated Fiat Cryptography’s C code for Curve25519 and P-256, because of its higher assurance, and comparable performance. This C code for Curve25519 is the unsaturated implementation, and as seen in the performance analysis (Chapter 8), newer microarchitectures tend to excel on saturated implementations. The Fiat Cryptography team implemented the saturated implementation strategy for Curve25519 as a result of the collaborative effort in regard to the CRYPTOPT

¹OpenSSL’s implementations of Curve25519 and its runtime dispatch can be found in the file `crypto/ec/curve25519.c` of OpenSSL’s code base.

paper [Kue+23b]. This new implementation, in combination with CRYPTOPT, extends the level of assurance down to the assembly level and also provides significant performance benefits over C compilers. This was enough to convince the maintainers of BoringSSL to use CRYPTOPT-generated code for Curve25519; it has been part of the library since 30.05.2023. Commit 9d4f833e [Erb23a] in the BoringSSL code base reports that the throughput of signatures has increased by 25.5% on Zen 2 architectures.

9.2 secp256k1

The popular cryptocurrency Bitcoin [Nak08] uses blockchain technology for its distributed ledger. Every block is signed with an ECC signature over the `secp256k1` curve. The C library `libsecp256k1` is an Open Source project hosted on GitHub. It advertises itself as an *Optimized C library for EC operations on curve secp256k1* [lib23]. During the CRYPTOPT development, that library contained a hand-optimised implementation for the field arithmetic.

The Fiat Cryptography team followed the library’s specific implementation strategy to replicate another functional program in Fiat Cryptography that uses the same implementation tricks specific to this field. With this functional program, Fiat Cryptography can generate C code and JSON for CRYPTOPT. CRYPTOPT generated code on all the test machines, and the best (on average) implementation is currently being merged into the library. The pull request has the number #1329. Preliminary results show performance improvements of around 12 % compared to the existing assembly code.² In addition to the performance improvements, the assembly is also formally ensured to be correct.

9.3 LLVM

The performance evaluation chapter, Chapter 8, lists the Bitcoin Core evaluation from two perspectives. The Fiat Cryptography-based approach and the LLVM-based approach. This section gives details on how the connection from Bitcoin Core via LLVM to CRYPTOPT is established.

The core of this connection is a collection of simple pattern-matching scripts, i.e. the Unix tool `sed` is used in the experiments of this thesis. The inputs to those scripts are preprocessed and pre-compiled C files.

²The 12 % performance improvement is based on the scalar multiplication with the library’s benchmarking suite. In particular, as the pull request shows, the existing code requires 15.7 ms the CRYPTOPT-generated code only requires 13.97 ms.

In particular, the C files are compiled with `clang` and the options `-S -emit-llvm -O1`. This output is then fed into `opt`, which is the optimiser tool from the LLVM project. `opt` is called with options `-S -enable-new-pm-passes='default<01>' -print-after-all`. This output is then in a form that is semantically very similar to the CRYPTOPT IR. Most importantly, it is in SSA form. The pattern matching scripts translate this intermediate LLVM format into a JSON format that is compatible with CRYPTOPT. Inside CRYPTOPT, another small set of translation steps is required that cannot be handled via simple context-free replacement rules. The majority of those internal scripts are simple checks and translation steps, such as *looking up an identifier and replacing a symbol with it* or *checking if an immediate value is less than 64 bits and converting it into hexadecimal form*.

CRYPTOPT does not (by far) implement enough logic to be a general-purpose back end for the LLVM toolchain. In this thesis, only the bare minimum is implemented to support another input besides Fiat Cryptography. In particular, this minimum is whatever is needed to translate the field arithmetic from the Bitcoin Core library.

The performance of the field arithmetic of this input chain is marked with “Bitcoin Core (LLVM)” in Tables 8.12 and B.12. The scalar multiplication performance is marked with the “(Opt)” suffix in Tables 8.13 and B.14.

9.4 BLS Signatures

The initial development of CRYPTOPT only experiments with the 18 functions from the Fiat Cryptography paper [Erb+19]. Boneh, Lynn and Shacham [BLS01] publish the BLS (Boneh–Lynn–Shacham) signature scheme. Its unique property is that multiple signatures can be aggregated into a single short signature. Among other blockchains, the Ethereum blockchain [But14] uses those BLS signatures³ because it solves major scalability problems in the verification steps.

CRYPTOPT generated the x86-64 assembly code for the field arithmetic of the prime fields needed in the BLS12-381 curve. This generated code partly outperforms the hand-optimised x86-64 assembly from the relic-toolkit, “a modern research-oriented cryptographic meta-toolkit with emphasis on efficiency and flexibility.” The implementations are here: <https://github.com/relic-toolkit/relic/tree/main/src/low/x64-asm-61>; commit hash: c65ae21.

³The use of BLS signatures is described here: <https://github.com/ethereum/consensus-specs/blob/c5c7233e49e86d4ee386cc4735ebe3f81807ea73/specs/phase0/beacon-chain.md#bls-signatures>.

The CRYPTOPT-generated code is only in the CRYPTOPT GitHub repository [0xA23b] and has not been integrated into the relic-toolkit for the following reason: The performance of the CRYPTOPT-generated code is not objectively higher. In particular, the field multiply and square may be faster than the hand-optimised code in isolation, but when used in scalar multiplication, the square function slows the scalar multiplication down. Interestingly, using the field multiply for multiplying and squaring increases the throughput overall. This raises the question of whether a function could be optimised based on performance measurements in another context. In particular, the idea is to assemble the square function into the scalar multiplication but then measure the performance of the entire scalar multiplication rather than just the field function. Then, based on the observed performance a mutation can be kept or discarded. This is left for future work.

9.5 Jasmin

Jasmin [Alm+17] is a very low-level programming language tailored for cryptographic code. It is very close to assembly. It is so close, in fact, that tasks of register allocation and instruction scheduling are mainly handled by the developer. In particular, the developer needs to specify when variables are in registers and when they reside on the stack. The Jasmin compiler generates assembly for x86-64 and ARM-M4. In addition, the Jasmin compiler can generate EasyCrypt proof files [Alm+19; Alm+20b] to verify the implementation. This enables developers to write low-level code that can automatically be verified correct.

Connection to CRYPTOPT

CRYPTOPT features a preliminary module to parse Jasmin-IR into CRYPTOPT IR. This parser discards all information of stack and register information and leaves this to the optimiser. CRYPTOPT is able to generate working x86-64 assembly, but the correctness of the generated code has not yet been proven.

Preliminary Results

Current experiments with the scalar multiplication of Curve25519 generate code that is much longer than what has been optimised using Fiat Cryptography. In particular, the number of instructions is around 2 500, which is 2 – 10 times larger than prior (cf. Section 8.5).

The code that CRYPTOPT generates is slower than that generated by the Jasmin compiler. The optimiser starts with a code performance that is $\approx 35\%$ as fast as the Jasmin compiler, i.e. CRYPTOPT's code is about three times slower. The final code performance at the end of the optimisation is still slower than that of the Jasmin compiler. In particular, that performance is only $\approx 80\%$ as fast; In other words, CRYPTOPT's generated code is still $\approx 20\%$ slower. It seems that at some point, the optimiser oscillates unproductively in the search space. It is quite susceptible to changes in hyperparameters. Maybe a vastly increased budget in the number of mutation evaluations is needed. A tenfold increase takes around a week to finish and did not yield better results. A hyperparameter search for the number of batches, batch sizes, or cycle counts could help, but this is left for future work.

Analysis

Note that this function is structurally different from the field arithmetic from *Fiat Cryptography*. In particular, Jasmin inlines all calls to the field arithmetic, when compiling the scalar multiplication. This is intentional because only then would CRYPTOPT be able to interleave those function calls, and that could strip register spills significantly. However, this also means that the search space is significantly larger. In particular, when only optimising one field function, it sufficed for CRYPTOPT to find *one* optimisation mutation. This one optimisation mutation is then replicated (simply by function calls in production) to all instances (i.e. calls) of that function. In the Jasmin case, CRYPTOPT needs to find this *one* optimisation multiple times (as many times as the field function is used, which is in the order of tens of times). However, this is also intentional because one optimisation at one position may not *necessarily* be beneficial at all others. This claim needs to be verified though.

Future Work

A mechanism to propagate changes across similar, potentially far-spread sub-stretches could help to find beneficial mutations more efficiently. Another option could be to change the optimisation method. Either to have a population-based approach, which in the crossover method could merge differently-shaped field arithmetic. Another change could be to adjust the mutation heuristic, to have multiple mutations at a time, to speed up the optimisation on the one hand, but also get out of local minima on the other hand.

9.6 Static Optimisations

The transformation from the input language into CRYPTOPT IR is referred to as preprocessing in [Figure 3.1](#). There are multiple preprocessing steps that parse the input and apply static transformations. Those steps are performed without measuring performance. The result of the entire preprocessing steps is then called the CRYPTOPT IR. The aim of the preprocessing is to simplify the code by, for example, inlining or removing dead code. This also reduces the size of the search space. This section explains the preprocessing steps with abstract examples in C-like code snippets. The preprocessing steps apply simple transformations in the following order:

- Eliminate `static_casts`,
- combine operations,
- resolve nested structures,
- inline `argN[n]`s,
- partial evaluation,
- Common Subexpression Elimination (CSE), and
- dead code elimination.

9.6.1 Eliminate `static_casts`

Fiat Cryptography emits `static_casts` to explicitly cast a value to a certain-sized type to consciously ignore potential overflows. A C equivalent would be

```
uint64_t x1 = (uint64_t) ( (uint64_t) arg2[0] * 0x13 );
```

The multiplication `arg2[0] * 0x13` overflows, and the C semantics do not provide means to capture that. However, Fiat Cryptography internally keeps track of the bit width of the limbs, and especially with unsaturated implementations, this overflow does not jeopardise the correctness. Hence, Fiat Cryptography emits those casts to signal to the compiler that it is valid to truncate the result. CRYPTOPT does not need this explicit cast. Instead, CRYPTOPT assumes that a value is safe to be stored in a 64-bit type (in this case, the `uint64_t`) just as specified. This preprocessing step rewrites the above to

```
uint64_t x1 = arg2[0] * 0x13;
```

9.6.2 Combine Operations

The input syntax allows for patterns where multiple operations can be merged and subsequently implemented with one single dedicated instruction. For example, the following code

```
uint64_t x1 = arg1[0];
uint64_t x2 = arg1[1];
uint64_t x3 = ( (x1 >> 51) | (x2 << 13) ) & 0xffffffffffffffff;
```

semantically left-shifts `x2` by 13 and fills up with bits from `x1`, or equivalently right-shifts `x1` by 51 and fills up with bits from `x2`. This construct can be implemented with the `shld` or `shrd` in x86-64 assembly [Int20a]. This preprocessing step rewrites this particular pattern to an `shrd` operation, instead of using two shifts and an `or` operation.

9.6.3 Resolve Nested Structures

The input syntax allows operands to be anonymously nested, as shown in Listing 5 and explained in the description of the input language in Section 3.2. A C version of this nesting is shown in the following code snippet:

```
uint64_t x1 = arg1[0] * ( arg1[1] * ( arg1[2] * 0x13 ) );
```

The parenthesis contains intermediate values that are not associated with a logical symbol. This makes the data-flow analysis more complex and reduces the ability to schedule those inner expressions. Hence, those nested structures are resolved to independent operations. In particular, this preprocessing step rewrites the above snippet to

```
uint64_t x100 = arg1[2] * 0x13;
uint64_t x101 = arg1[1] * x100;
uint64_t x1   = arg1[0] * x101;
```

New intermediate symbols are created as needed. The new symbols in this case are `x100` and `x101`. The advantage is that the calculation of the new intermediates can be scheduled at different positions. The disadvantage is that for commutative expressions like the three multiplications above, neither `arg1[0] * 0x13` nor `arg1[0] * arg1[1]` can be calculated. CRYPTOPT does not have the ability to change the expression itself via the distribution rule. This is left for future work.

9.6.4 Inline argN[n]s Into the Expression

The input syntax allows for simple value aliasing, which Fiat Cryptography uses, especially when reading input values. For example:

```
uint64_t x1 = arg1[0];  
uint64_t x2 = arg2[0];  
uint64_t x3 = x1 + x2;  
uint64_t x4 = x1 * 0x13;
```

CRYPTOPT inlines those single assignments and rewrites the above as

```
uint64_t x1 = arg1[0];  
uint64_t x2 = arg2[0];  
uint64_t x3 = arg1[0] + arg2[0];  
uint64_t x4 = arg1[0] * 0x13;
```

Note, that this does not necessarily mean, that `arg1[0]` is read from memory multiple times. This value may be cached in a register if the optimiser decides to do so.

9.6.5 Partial Evaluation

The input syntax allows for multiplication with immediate values. In particular, it also allows for chaining such multiplications, where multiple constant factors are known at compile time. For example:

```
uint64_t x1 = arg1[0] * 0x2;  
uint64_t x2 = x1 * 0x3;
```

The numbers `0x2` and `0x3` are the immediate values in this example. This preprocessing step combines those into one operation that partially evaluates the expression 2×3 and rewrites the above to

```
uint64_t x1 = arg1[0] * 0x2;  
uint64_t x2 = arg1[0] * 0x6;
```

9.6.6 Common Subexpression Elimination

The input syntax allows for creating the statically known same expression multiple times. For example:

```
uint64_t x1 = arg1[0];
uint64_t x2 = arg1[0];
uint64_t x3 = x1 * 0x13;
uint64_t x4 = x2 * 0x13;
uint64_t x5 = x4 + 0x1;
uint64_t x6 = x3 + 0x2;
```

Note that `x1` and `x2` are equal and, as such, the values of `x3` and `x4`. The inlining step earlier will have inlined `arg1[0]` into both expressions, but it would still compute the multiplication `arg1[0] * 0x13` twice. To avoid this, CRYPTOPT analyses all operations to find duplicates by operator and arguments. If two operations have the same operator and the same arguments, only the first occurrence is kept, i.e. computing `uint64_t x3 = arg1[0] * 0x13`. For all subsequent operations, e.g. the one computing `x4`, the output symbol(s) (i.e. `x4`) is replaced by the one being kept, i.e. `x3`. In other words, the above is rewritten to

```
uint64_t x1 = arg1[0];
uint64_t x2 = arg1[0];
uint64_t x3 = arg1[0] * 0x13;
uint64_t x4 = arg1[0] * 0x13;
uint64_t x5 = x3 + 0x1;
uint64_t x6 = x3 + 0x2;
```

9.6.7 Dead Code Elimination

The input syntax, especially after the Common Subexpression Elimination (CSE), inlining, and other rewrite preprocessing steps can result in operations whose result is never used. Those are *dead* operations, which would result in *dead code*. Any dead code that is found in the CRYPTOPT IR at this point in the preprocessing, is removed from the input. For example, expressions such as `uint64_t x1 = arg1[1]` with `x1` never being read (for example, because it has been inlined in a previous step) are simply removed from the input.

Chapter 10

Conclusions

The traditional approach to achieving high performance for cryptographic routines is writing and optimising them by hand. This thesis explores the automatic optimisation of straightline cryptographic code using a black-box approach.

The main research question in this thesis is whether it is possible to generate code automatically that is faster than typical compilers and comparable to hand-optimised approaches while still enabling automatic correctness checking. Luckily, this thesis answers this question affirmatively and presents CRYPTOPT. CRYPTOPT uses a new approach that, instead of statically compiling into fast code, uses methods from search-based optimisation to *search* for fast implementations. In particular, CRYPTOPT starts with random implementations and gradually improves the performance. Once a fast implementation is found, it is automatically proven correct by a novel, external equivalence checker. This checker was created in a collaborative effort with the team of Fiat Cryptography in regard to the CRYPTOPT paper [Kue+23b]. This checker is integrated into the Fiat Cryptography tooling.

A major challenge in this project using search-based optimisation techniques boils down to being able to accurately measure the performance of small stretches of code. In this regard, the standalone module **MeasureSuite** is developed. It can measure the performance accurately enough to guide the optimiser. **MeasureSuite** can be of use even outside of this project to benchmark small code snippets.

The generated results outperform off-the-shelf compilers, and their performance is comparable to that of hand-written code. In addition, the connection to Fiat Cryptography enables end-to-end correctness proofs, which closes another major gap in state-of-the-art approaches. The generated code for Curve25519 is included in BoringSSL, the cryptographic library powering the Chromium project. Chromium is the foundation of many browsers, such as Microsoft Edge, Google Chrome, Vivaldi, Opera, and Brave. As such, the CRYPTOPT-generated code now powers the Curve25519 TLS Handshake for 78.4 per cent of the worldwide

desktop browsers (Chrome 63.52%, Edge 11.15%, and Opera 3.73%, according to <https://gs.statcounter.com/> at the end of September 2023).

Further, the generated code for secp256k1 is on the way into the Bitcoin Core library. Those two cases show that the generated code is beneficial to the public and that CRYPTOPT has the potential to generate more code for more input functions and more architectures.

Outlook and Future Work

This thesis evaluates CRYPTOPT in connection with Fiat Cryptography, which generates code for finite-field arithmetic usable in Elliptic Curve Cryptography. However, the core of CRYPTOPT is flexible and may, as such, be used within different environments.

Optimisation Criteria

Currently, CRYPTOPT runs the code on its own hardware, but this is not necessarily required. In particular, CRYPTOPT only needs feedback from *some entity* on whether or not to keep a mutation. This entity does not necessarily need to measure performance but could measure stack-memory usage, cache utilisation, energy consumption, or latency. CRYPTOPT could optimise in regard to any of those. Further, as described in the section on the BLS signatures (Section 9.4), if this measure entity could provide performance information of the functions in different contexts, it could guide the optimiser into different optima. In particular, those optima could contain solutions that are sub-optimal when measured in isolation but provide better performance in different contexts. For example, the optimiser is tasked to optimise the implementation of field arithmetic, but the performance data that guides the optimiser is taken from the context of scalar multiplication (that uses the field arithmetic that is currently optimised).

Optimisation Strategy

Another candidate for improvement is the optimisation strategy. This thesis only explores the simple Random Local Search in combination with Bet-and-Run. While generally suitable, the results vary across functions and microarchitectures. A possible avenue to investigate is to use population-based approaches. Those

could potentially exploit recombination mechanics, or more sophisticated mutation operations, which could help with overcoming local optima or inaccurate measure results.

Different Hardware

CRYPTOPT could extend its hardware support to non-x86-64-based processors by emitting assembly instructions for other architectures. ARM-based desktop processors, phones, and embedded microcontrollers that are not able to run CRYPTOPT natively may need different means of measurement. In theory, this should not impose a huge barrier, as CRYPTOPT can run anywhere as long as the feedback mechanism is provided. This could even be extended to virtual hardware, or virtual machines, i.e. optimise code to run in virtual machines, potentially on a cloud platform.

Stopping Criteria

Another potential limitation is the use of a pre-determined budget for each optimisation run. The evaluation shows that there are rare cases where the optimiser deteriorates the code's performance at the end. It remains to be researched if dynamic stopping criteria can be established that reliably emit the code with the best-measured performance.

Hyperparameter Search

Tying into the stopping criteria, there might be potential to do extensive hyperparameter searches for parameters like evaluations, cycle goals, and number of batches. Empirically, the current set of parameters seems to be working well on average for the majority of the Fiat Cryptography-generated workload, but other workloads or other machines may need tweaking of those parameters.

Different Input

This thesis explores the optimisation of the straightline arithmetic from Fiat Cryptography. This input of only straightline code can be extended into different algorithms or languages. The first steps for this have already been taken with the Jasmin connection, but as presented in [Section 9.5](#), it needs more investigation to generate code with competitive performance.

Different Code Generation Heuristics

A major design choice is the spilling mechanics. Currently, CRYPTOPT chooses to spill the value from the registers that is used last in the upcoming operations. This may be improvement potential. For example, the ability to read the value from memory again could influence the spilling. In particular, an intermediate value that must be loaded into a register to be used in upcoming operations should be less likely to be spilled than a value that is used in an operation that can encode a memory operand.

Another spilling heuristic is to spill the value which is read second to last, to allow the out-of-order execution to prefetch that data, or the spilling decisions could be non-deterministic and be a part of the optimiser.

Appendix A

Machines

This section shows a more detailed configuration of the machines used in the experiments.

[Table A.1](#) shows more details of the CPUs used and the frequency they have been set to during cross-validation, [Table A.2](#) shows the motherboard models, [Table A.3](#) shows the hard drive (SSD) configurations, [Table A.4](#) shows the memory (RAM) configurations.

TABLE A.1. Extended overview of machines used in the experiments, CPU, # of physical cores, and low frequency used.

Name	CPU	# Physical Cores	Low Frequency
1900X	Ryzen Theadripper 1900X	8	3.8 GHz
5800X	Ryzen 7 5800X	8	3.8 GHz
5950X	Ryzen 9 5950X	16	3.4 GHz
7950X	Ryzen 9 7950X	16	4.5 GHz
i7 6G	Core i7-6770HQ	4	2.6 GHz
i7 10G	Core i7-10710U	6	1.1 GHz
i9 10G	Core i9-10900K	10	3.7 GHz
i7 11G	Core i7-11700KF	8	3.6 GHz
i9 12G	Core i9-12900KF	16 (8P+ 8E)	3.2 GHz
i9 13G	Core i9-13900KF	24 (8P+16E)	3.0 GHz

TABLE A.2. Motherboard details.

Name	Manufacturer	Model
1900X	ASUS	ROG STRIX X399-E Gaming
5800X	Gigabyte	B550 AORUS ELITE V2
5950X	Gigabyte	X570 GAMING X
7950X	Gigabyte	X670 GAMING X AX
i7 6G	Intel	NUC6i7KYB
i7 10G	Intel	NUC10i7FNB
i9 10G	Gigabyte	H470 HD3
i7 11G	ASRock	Z590 Pro4
i9 12G	Micro-Star	PRO Z690-A Wifi (MS-7D25)
i9 13G	ASRock	Z790 Pro RS

TABLE A.3. SSD details.

Name	Capacity	Manufacturer	Model
1900X	120GB	KINGSTON	SA400S3
5800X	500GB	Crucial	CT500P1SSD8
5950X	1TB	Samsung	SSD 980 PRO
7950X	480GB	Western Digital	WDC WDS480G2G0C-00AJM0
i7 6G	250GB	Samsung	SSD 850
i7 10G	250GB	Western Digital	WDS250G3X0C-00SJG0
i9 10G	250GB	Western Digital	WDC WDS250G2B0C-00PXH0
i7 11G	500GB	KINGSTON	SNVS500G
i9 12G	500GB	Samsung	SSD 980 PRO
i9 13G	480GB	Western Digital	WDC WDS480G2G0C-00AJM0

TABLE A.4. RAM details.

Name	Slots	Manufacturer	Model
1900X	2x8GB	Kingston	KHX2400C15/8G
5800X	2x16GB	Corsair	CMK32GX4M2B3200C16
5950X	2x8GB	Crucial	BL8G32C16U4B.M8FE1
7950X	2x8GB	Kingston	KF552C40-8
i7 6G	2x16GB	Kingston	9905663-007.A00G
i7 10G	2x16GB	Kingston	9905700-047.A00G
i9 10G	2x16GB	Corsair	CMK32GX4M2B3200C16
i7 11G	4x16GB	Kingston	KHX2666C16D4/16GX
i9 12G	2x16GB	Kingston	9905783-014.A00G
i9 13G	2x16GB	Corsair	CMH32GX5M2B5600C36

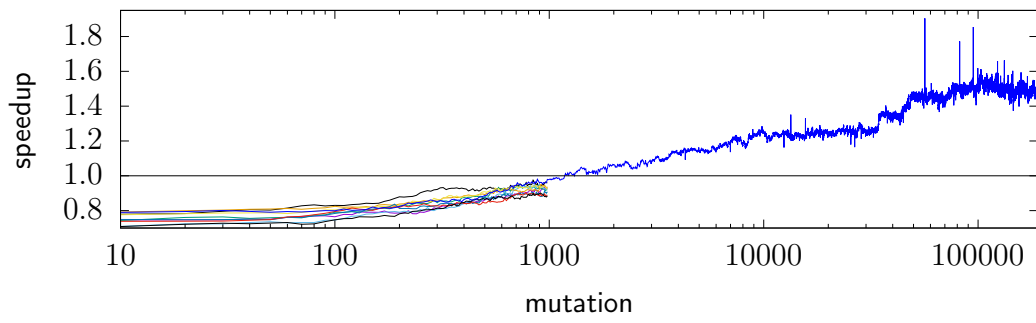
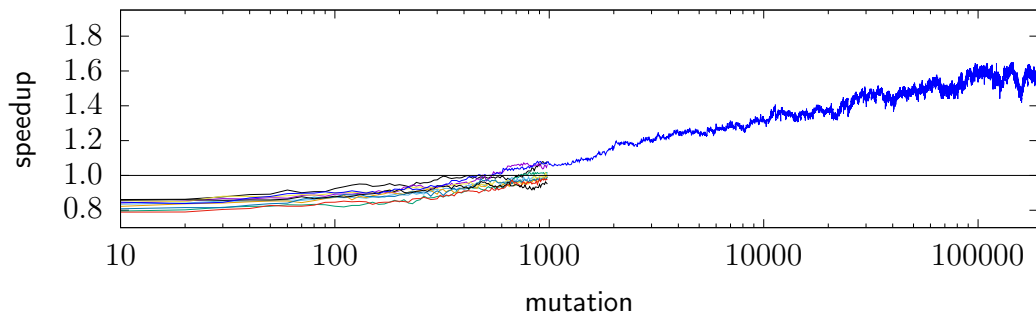
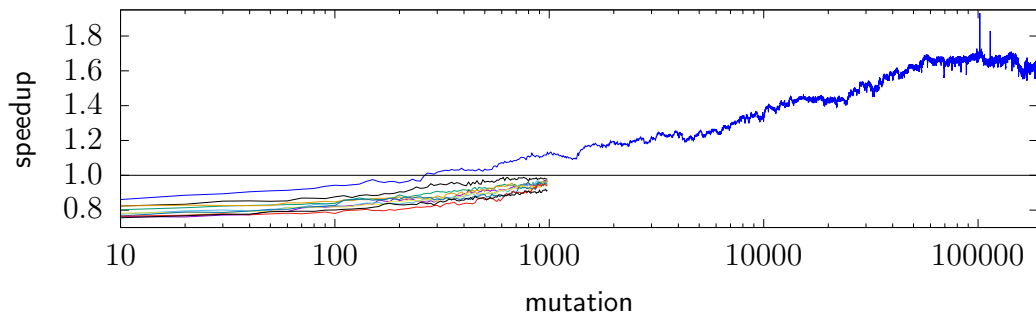
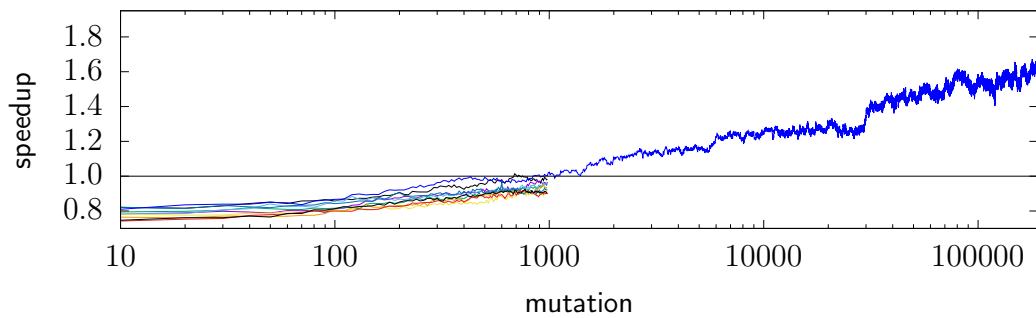
Appendix B

Field Arithmetic Performance Reports

This chapter elaborates with detailed information on the optimisations of the field arithmetic. First, how the optimisation behaves over time (cf. [Appendix B.1](#)), then how specific the resulting x86-64 assembly is in [Appendix B.2](#). [Appendix B.3](#) details the performance of the scalar multiplication when using the optimised field arithmetic.

B.1 Optimisation Over Time

CRYPTOPT cannot cleanly optimise all functions on all targets equally well. First, to show that for one function on one machine, the optimisation is relatively consistent, [Figure B.1](#) shows seven optimisation runs of P-384-Square on the machine i7 11G, each with a different seed and pinned to a different physical core. All runs start at around 0.78x–0.85x speedup, and after 190 000 mutations, the speedup is between 1.49x and 1.66x over the Clang compiled C code. The progress over time is roughly logarithmic in the number of mutations.

(A) 1st run of P-384-Square on i7 11G(B) 2nd run of P-384-Square on i7 11G(C) 3rd run of P-384-Square on i7 11G(D) 4th run of P-384-Square on i7 11G

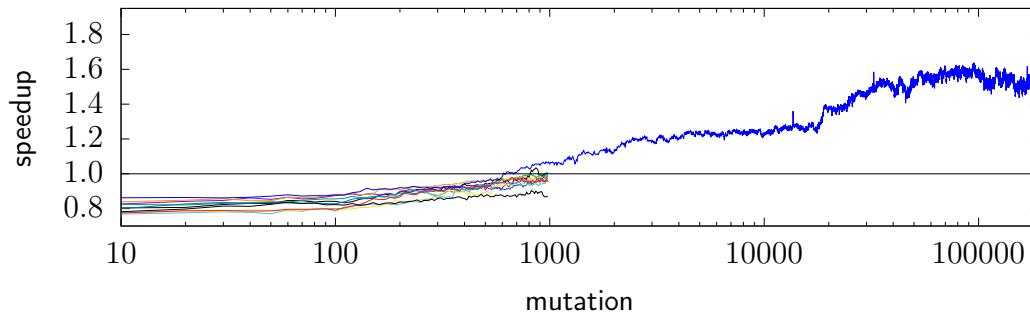
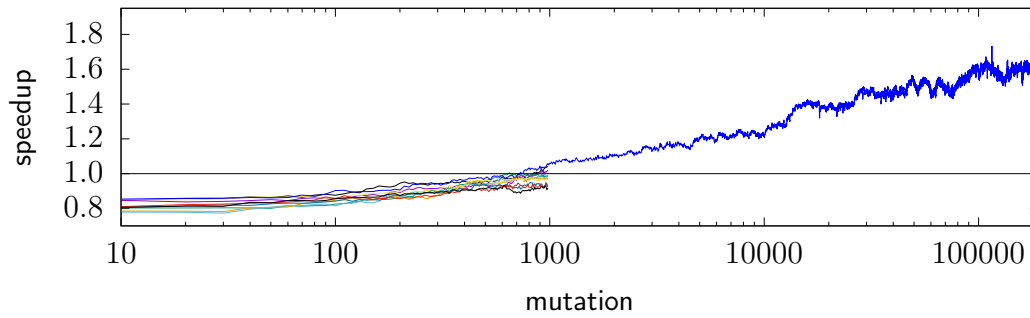
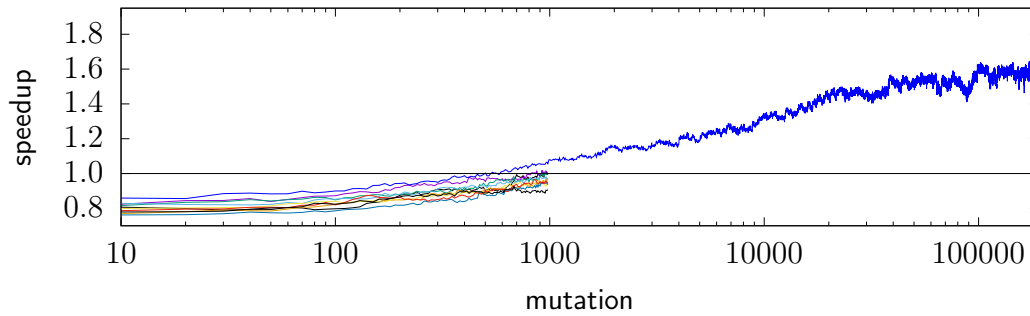
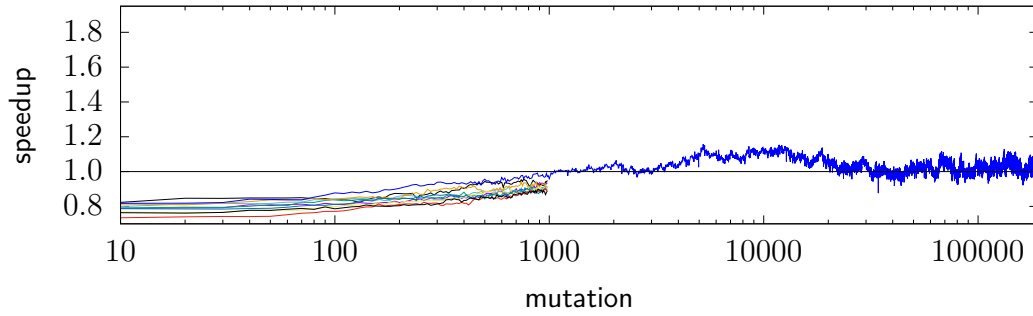
(E) 5th run of P-384-Square on i7 11G(F) 6th run of P-384-Square on i7 11G(G) 7th run of P-384-Square on i7 11G

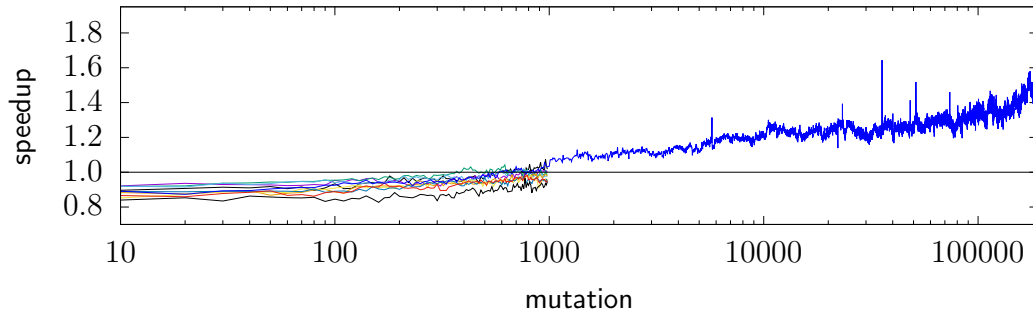
FIGURE B.1. Seven CRYPTOPT optimisation runs for P-384-Square on i7 11G, with different seeds each, one data point is generated every ten mutations. All runs start at around 0.78x–0.85x speedup, and after 190 000 mutations, the speedup is between 1.49x and 1.66x over the Clang compiled C code.

However, that does neither mean that P-384-Square is easy to optimise generally nor that one microarchitecture (e.g. i7 11G) is particularly well suited for optimising. [Figure B.2](#) shows ten graphs for the same function P-384-Square, but on all ten machines. Some optimisations (i.e. Intel i7 6G, i7 10G, i9 10G, i7 11G) show a relatively clear hill-climbing curve (modulo the expected occasional measure variances). Other optimisations are much noisier (i.e. on the AMD machines) but can still be optimised until the end. The measurements on the latest generation Intel CPUs (i9 12G, i9 13G) are very noisy and even show

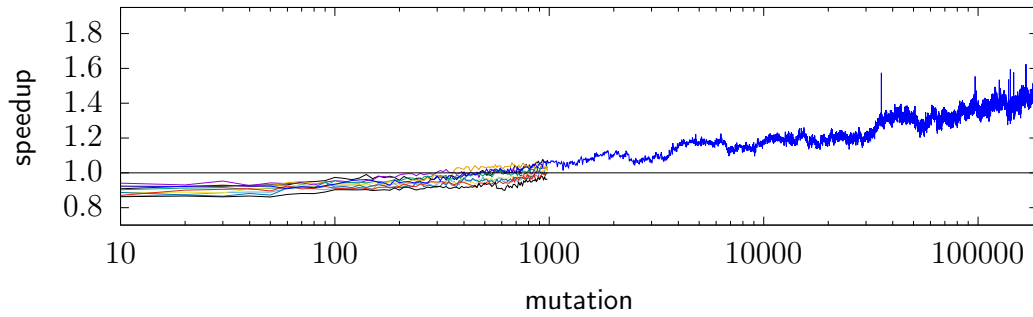
a decline in relative performance speedup after 70 000 and 30 000 mutations, respectively.



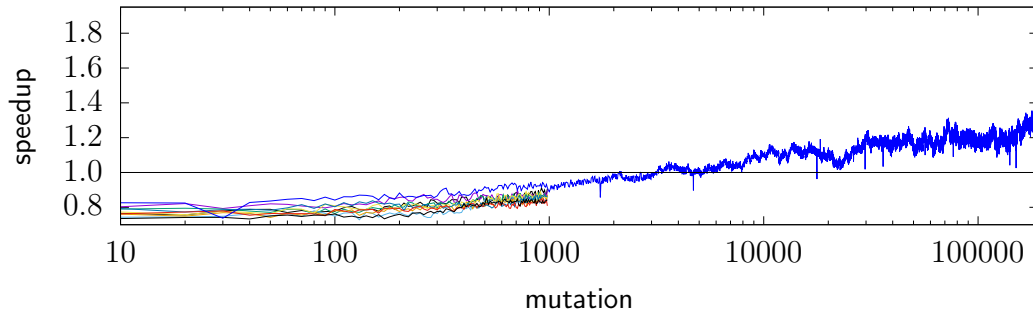
(A) P-384-Square on 1900X



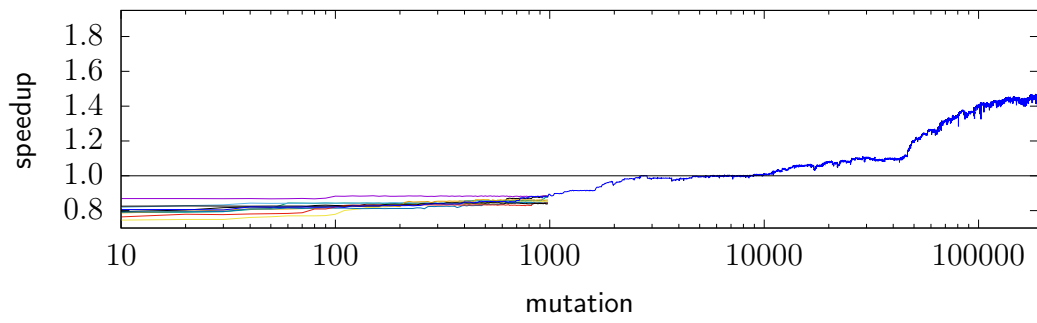
(B) P-384-Square on 5800X



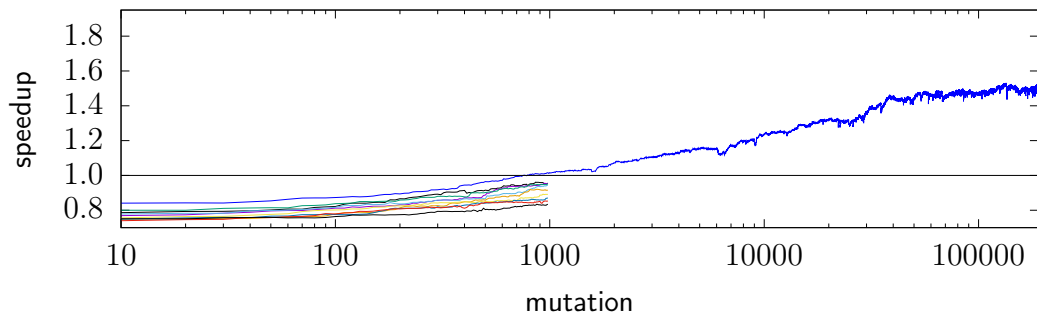
(C) P-384-Square on 5950X



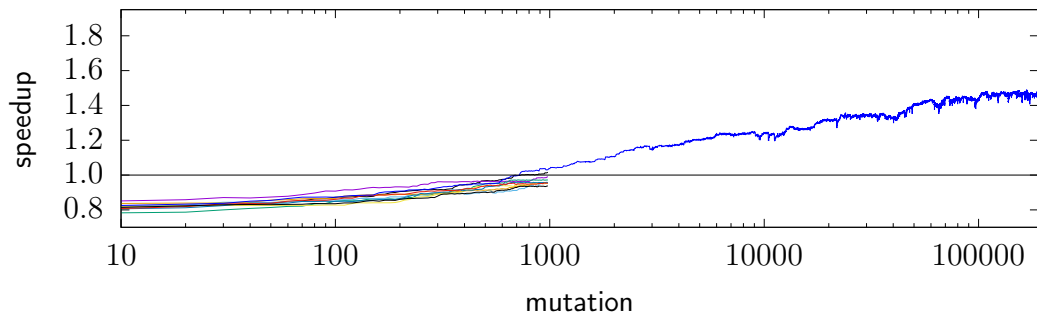
(D) P-384-Square on 7950X



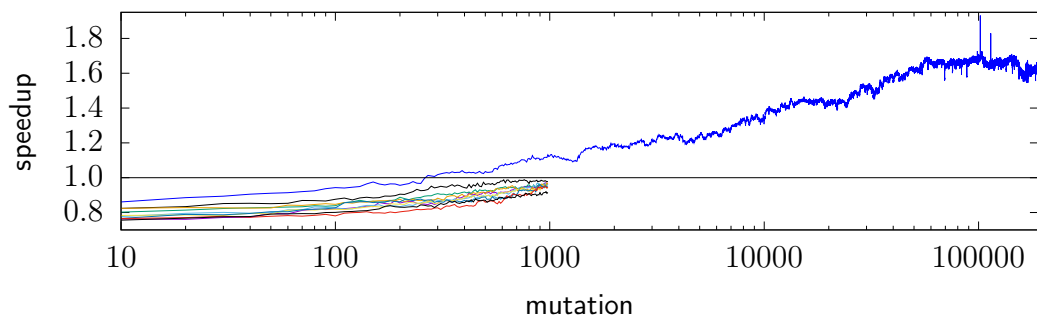
(E) P-384-Square on i7 6G



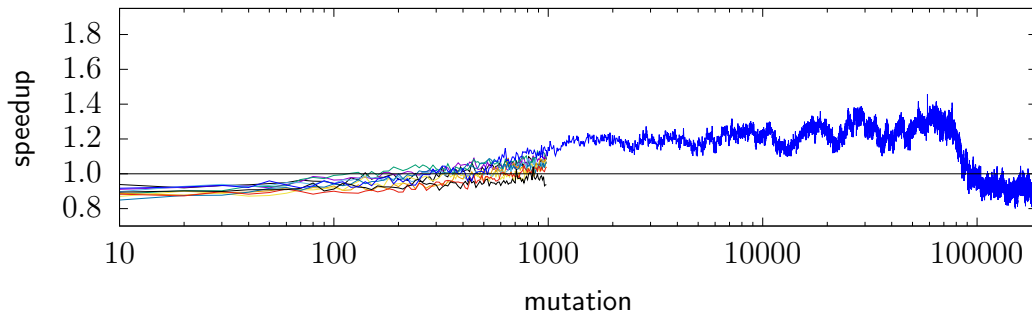
(F) P-384-Square on i7 10G



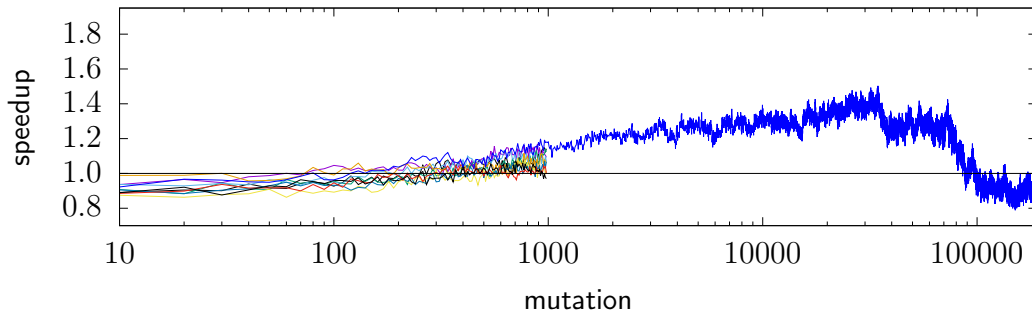
(G) P-384-Square on i9 10G



(H) P-384-Square on i7 11G



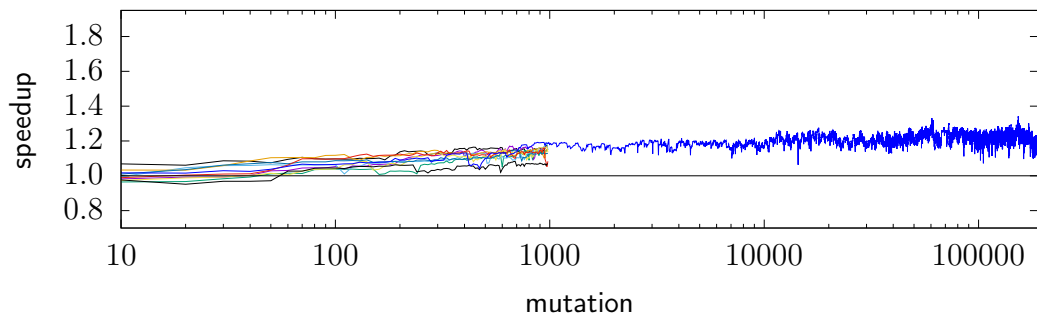
(I) P-384-Square on i9 12G



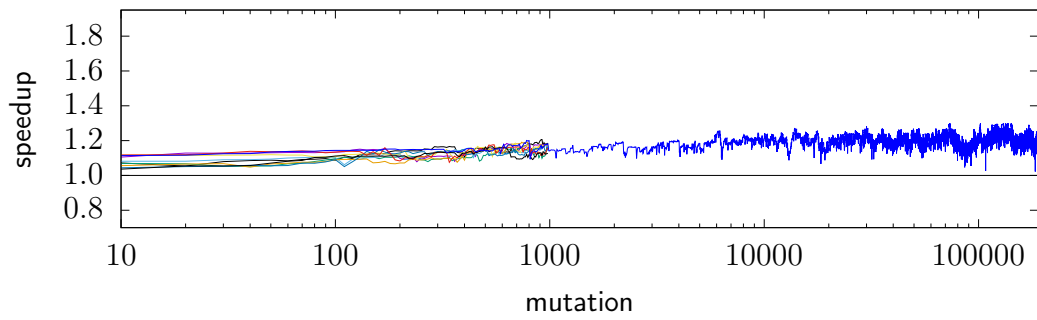
(J) P-384-Square on i9 13G

FIGURE B.2. CRYPTOPT optimisation runs of P-384-Square on all microarchitectures; one data point on the x-axis is generated every ten mutations. The y-axis shows the relative speedup against Clang with the microarchitectures' respective `-O3 -march=native -mtune=native`. Some optimisations show a relatively clear hill-climbing curve (modulo the expected occasional measure variances), while other optimisations are very noisy, and some even fail to optimise until the end.

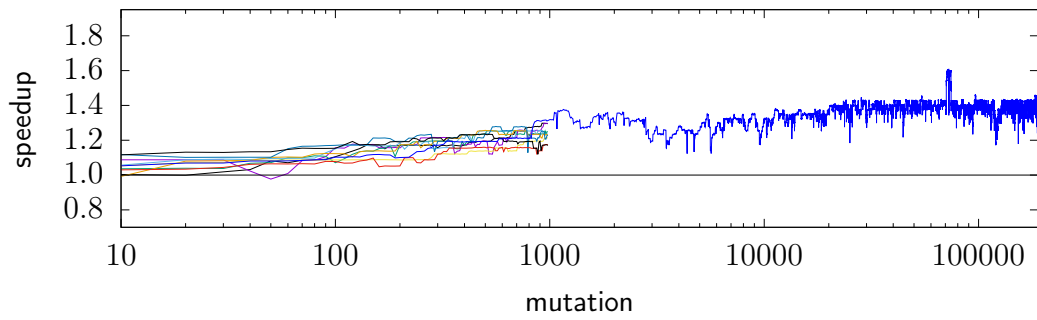
There is still no machine that can optimise all curves well either. The following two figures show the 22 different functions: 9 primes fields plus two extra implementation strategies for Curve25519 and secp256k1, with multiply and square functions each. [Figure B.3](#) shows all functions on i7 6G, and [Figure B.4](#) for AMD's 7950X. Generally, the measurements on AMD machines seem to be a bit noisier compared to the Intel machines, but that does not render optimisation impossible, as the figure shows.



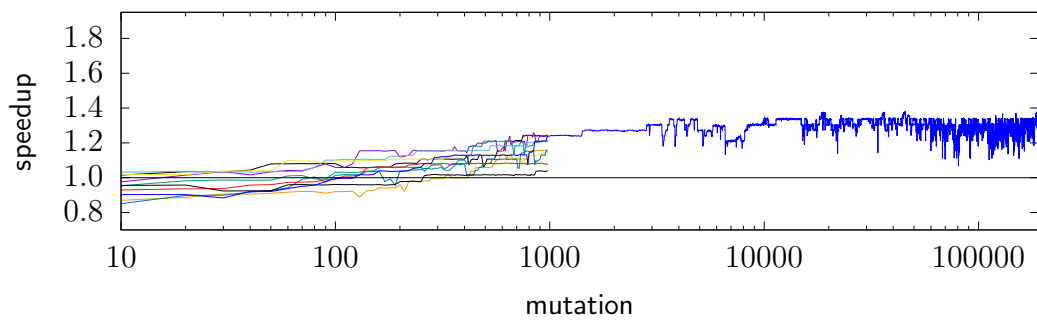
(A) Curve25519-Multiply on i7 6G



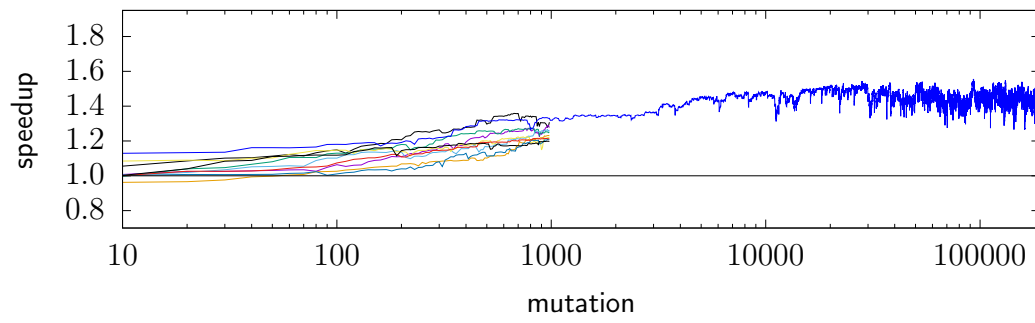
(B) Curve25519-Square on i7 6G



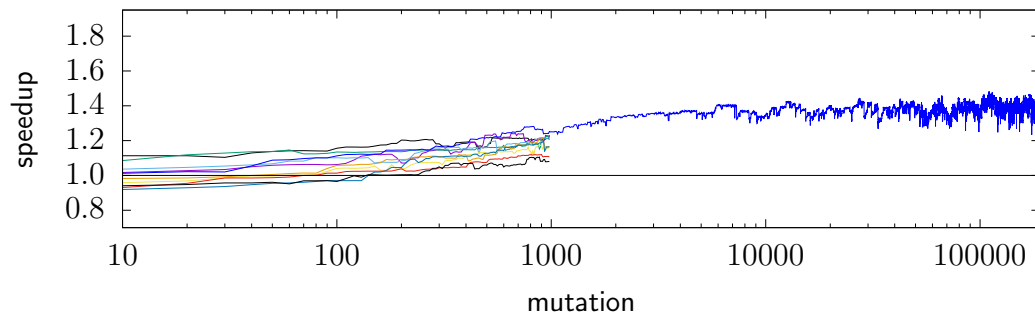
(C) Curve25519-Solinas-Multiply on i7 6G



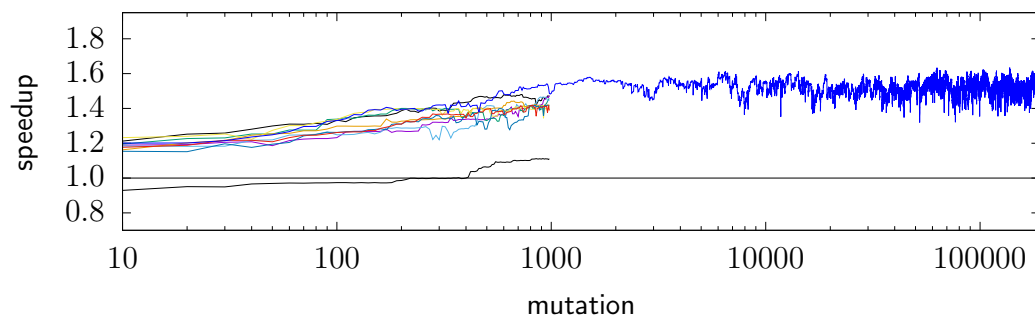
(D) Curve25519-Solinas-Square on i7 6G



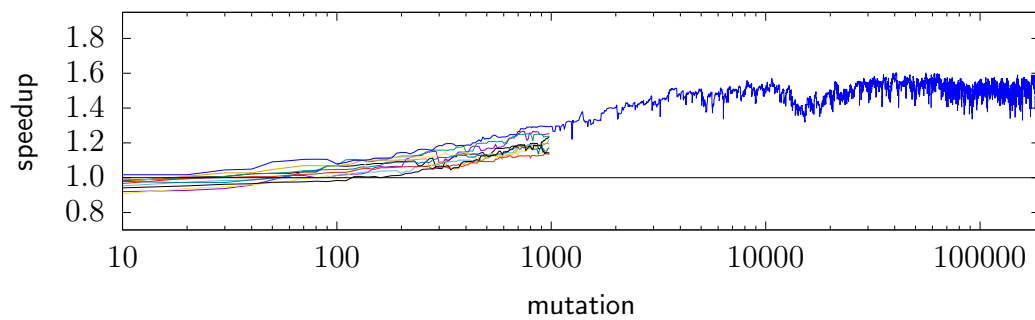
(E) P-224-Multiply on i7 6G



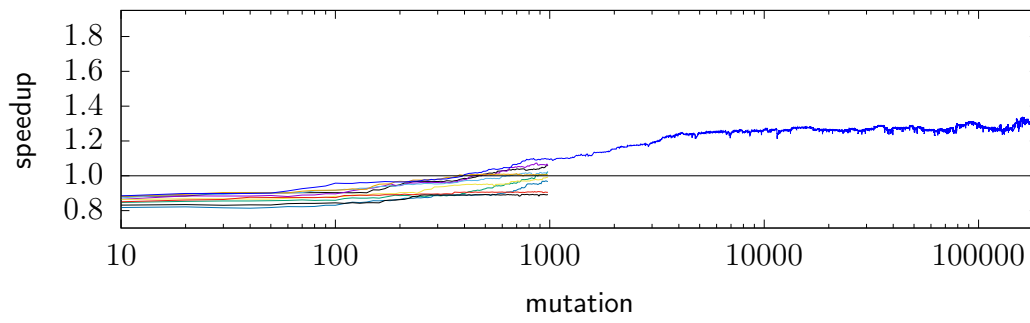
(F) P-224-Square on i7 6G



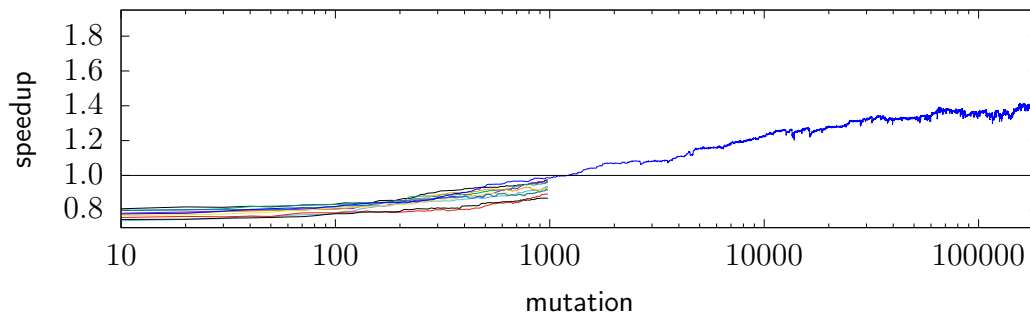
(G) P-256-Multiply on i7 6G



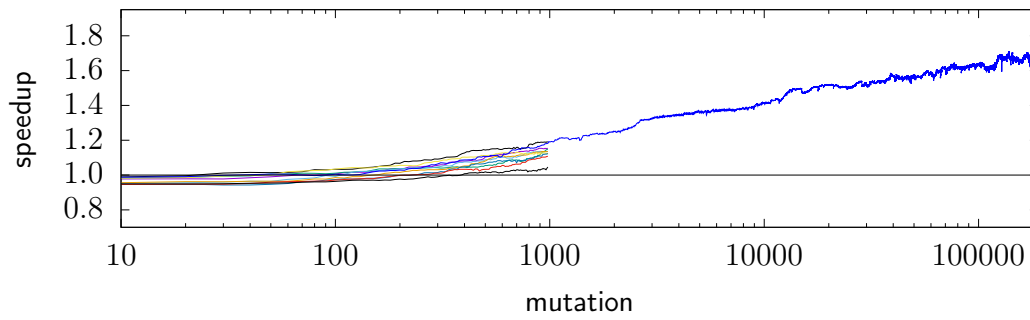
(H) P-256-Square on i7 6G



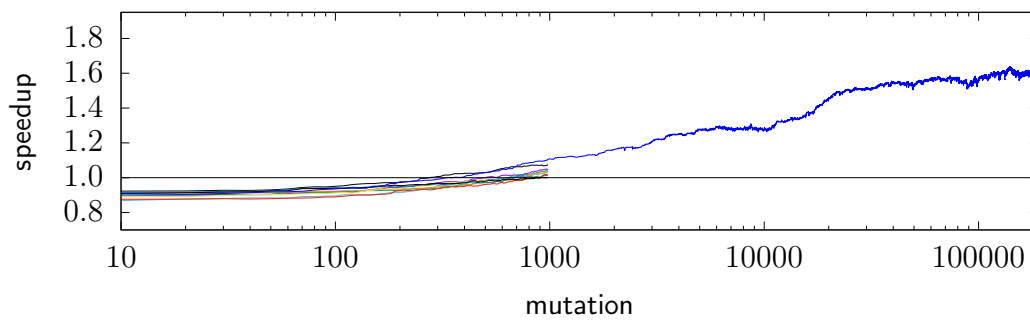
(I) P-384-Multiply on i7 6G



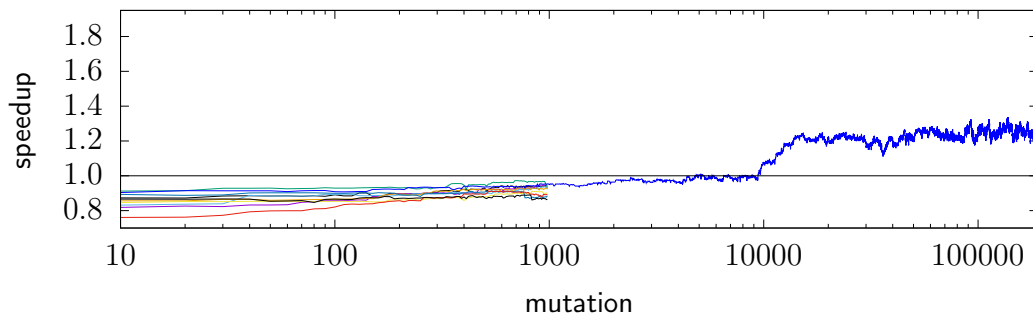
(J) P-384-Square on i7 6G



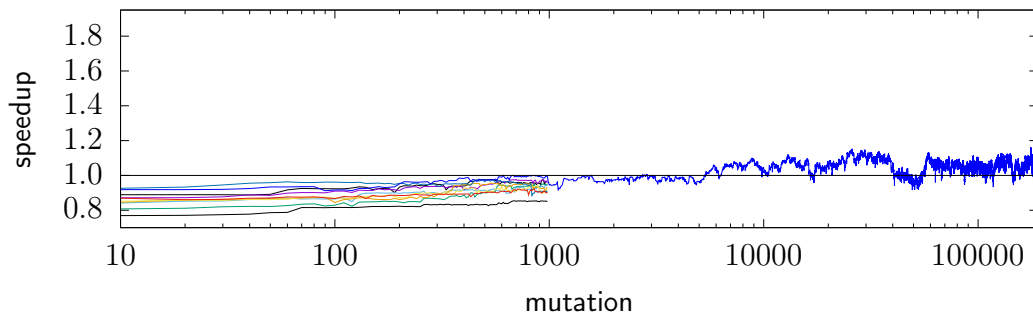
(K) SIKEp434-Multiply on i7 6G



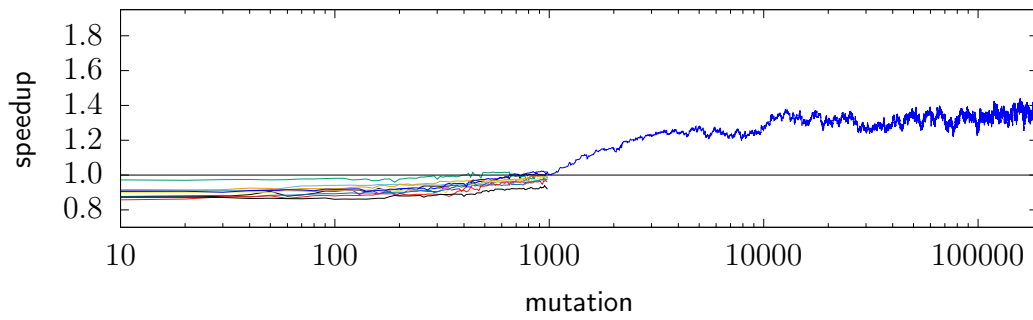
(L) SIKEp434-Square on i7 6G



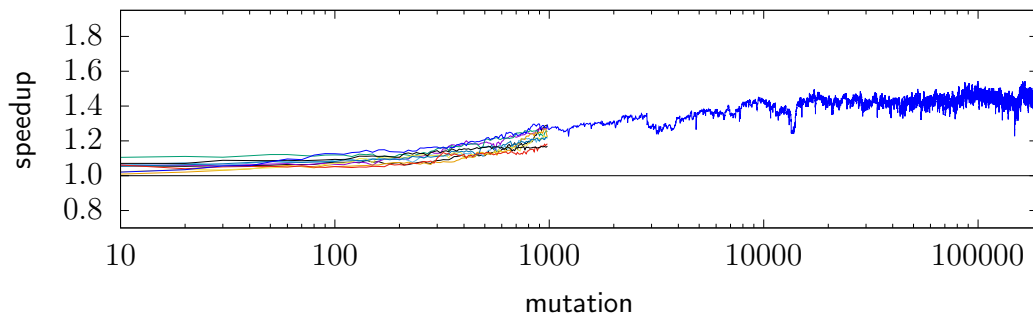
(M) Curve448-Multiply on i7 6G



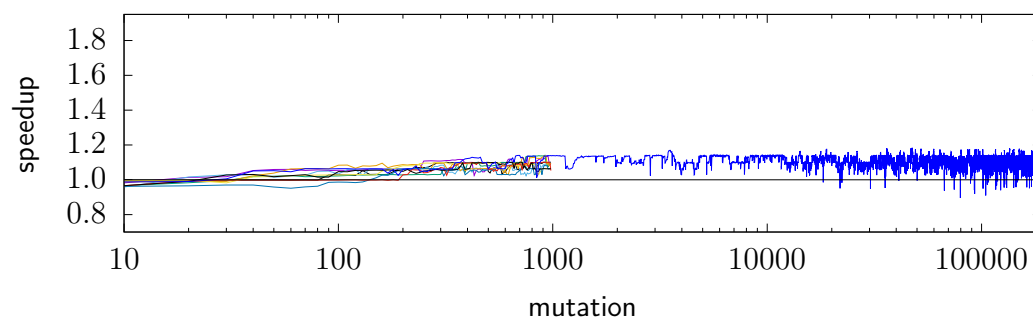
(N) Curve448-Square on i7 6G



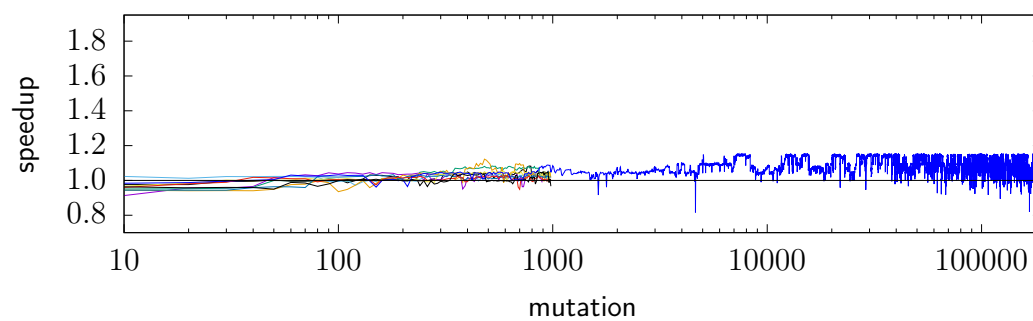
(O) P-521-Multiply on i7 6G



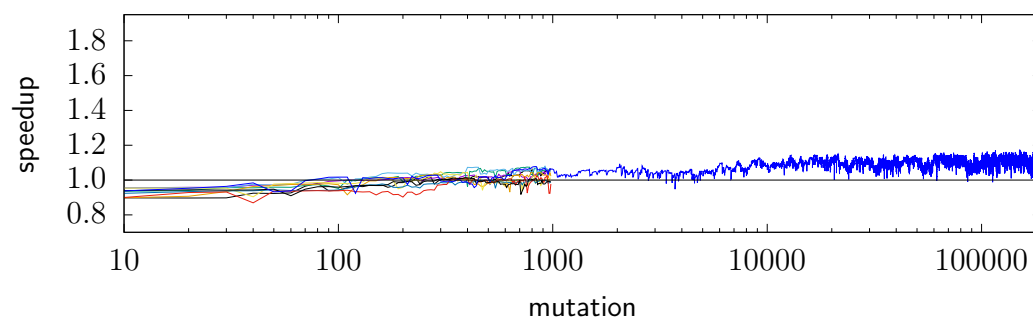
(P) P-521-Square on i7 6G



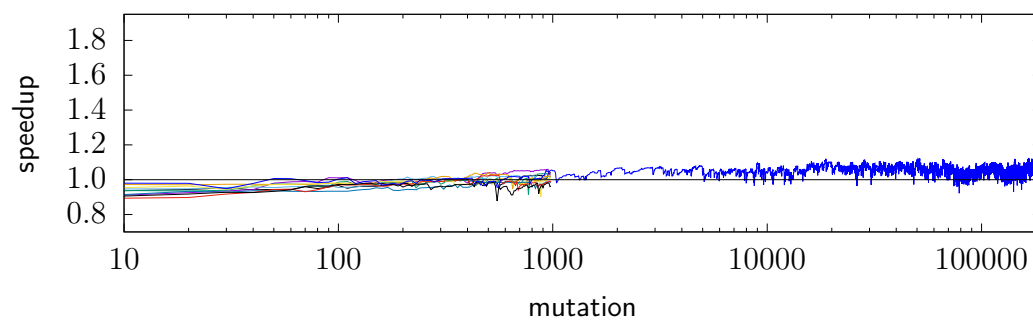
(Q) Poly1305-Multiply on i7 6G



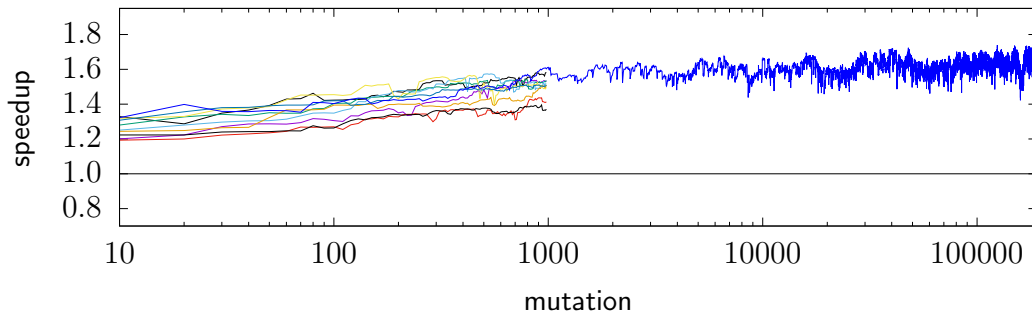
(R) Poly1305-Square on i7 6G



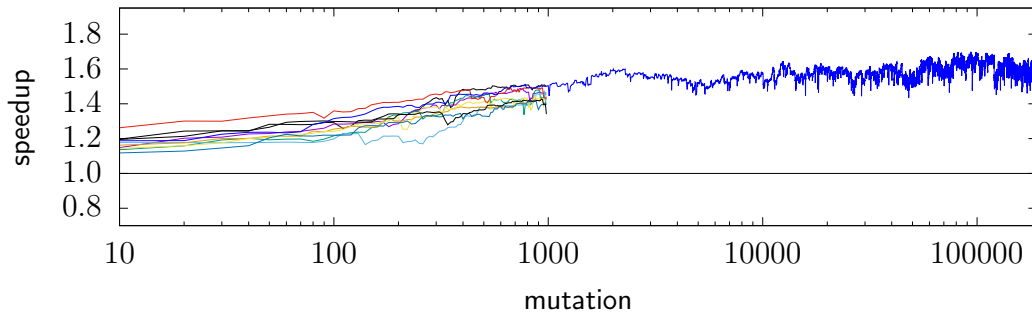
(S) secp256k1-Dettman-Multiply on i7 6G



(T) secp256k1-Dettman-Square on i7 6G

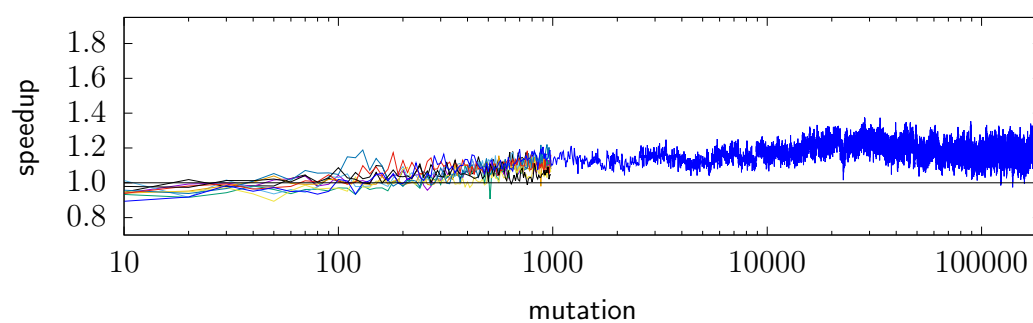


(u) secp256k1-Multiply on i7 6G

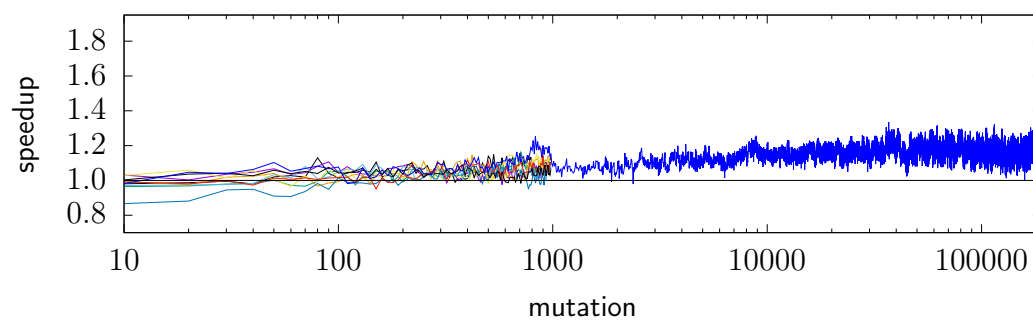


(v) secp256k1-Square on i7 6G

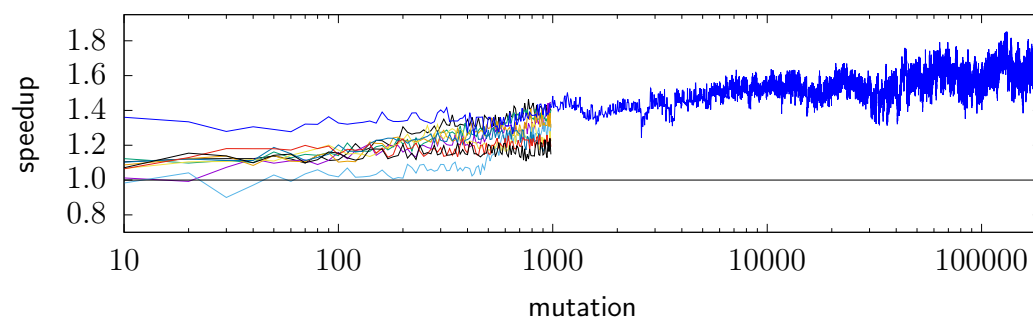
FIGURE B.3. All 22 functions are optimised on the i7 6G microarchitecture, and one data point on the x-axis is generated every ten mutations. The y-axis shows the relative speedup against Clang with `-O3 -march=native -mtune=native`. Some optimisation runs show a relatively clear hill-climbing curve (modulo the expected occasional measure variances); other optimisations are rather noisy and plateau early.



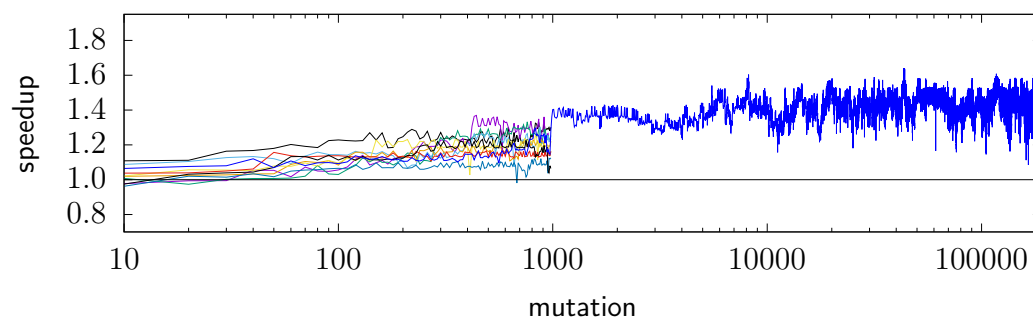
(A) Curve25519-Multiply on 7950X



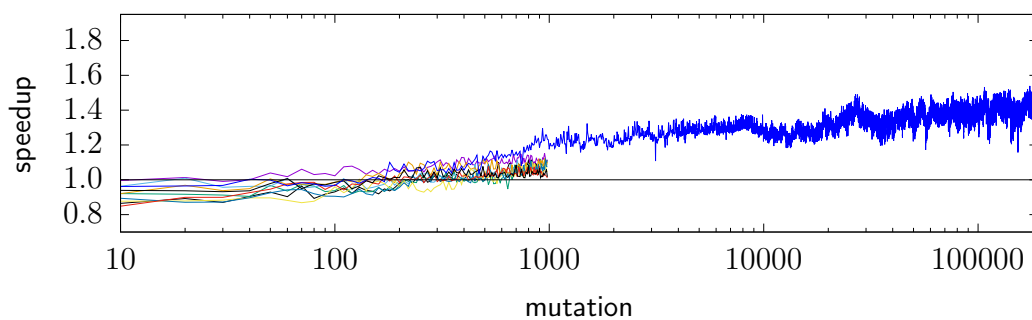
(B) Curve25519-Square on 7950X



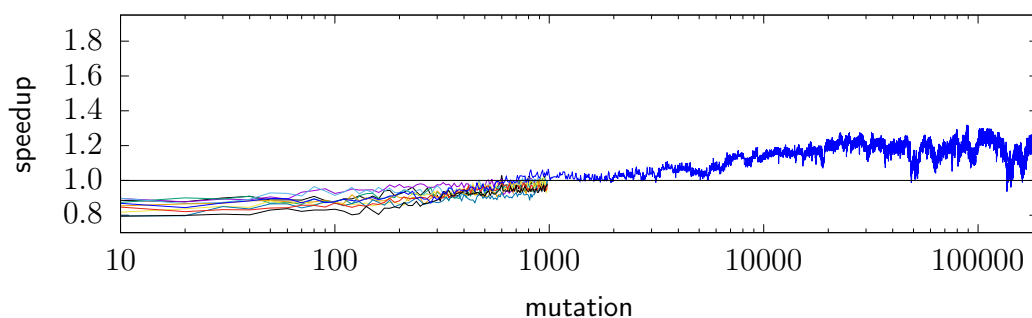
(C) Curve25519-Solinas-Multiply on 7950X



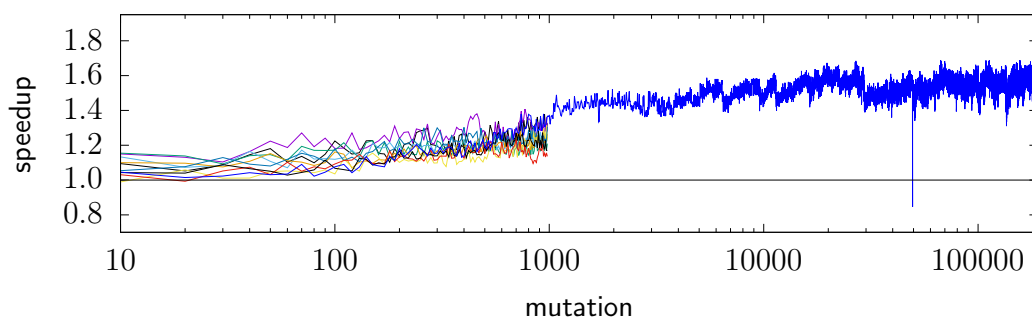
(D) Curve25519-Solinas-Square on 7950X



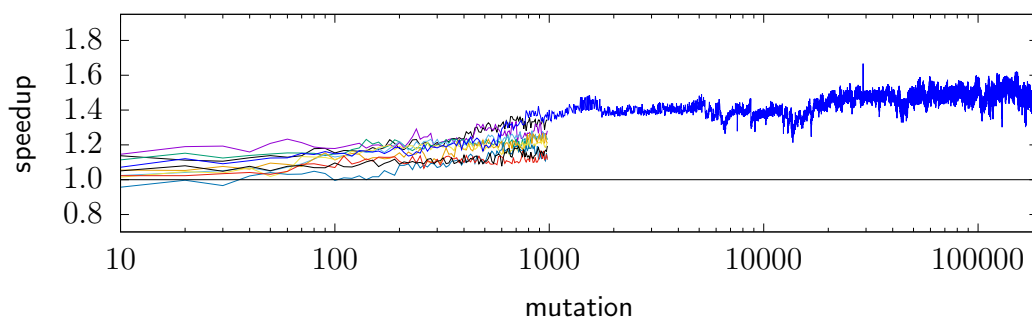
(E) P-224-Multiply on 7950X



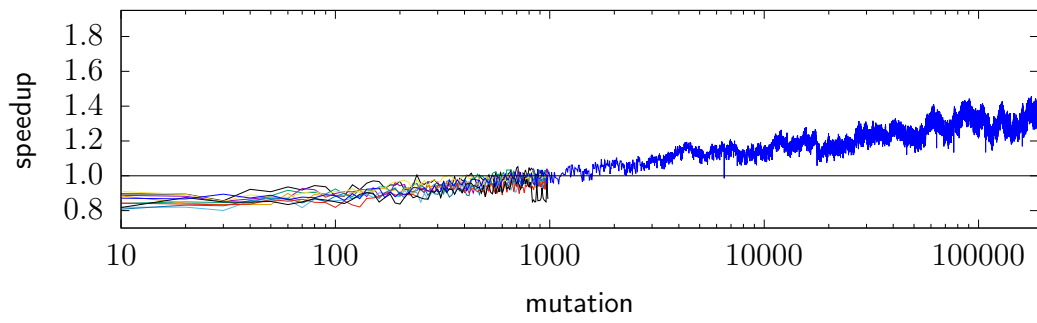
(F) P-224-Square on 7950X



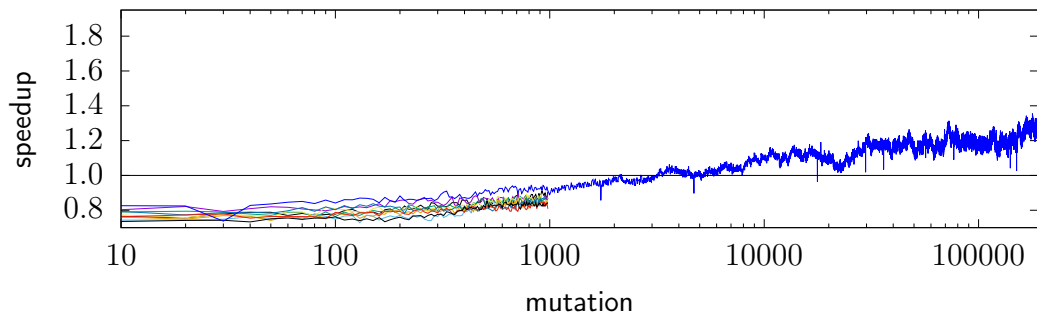
(G) P-256-Multiply on 7950X



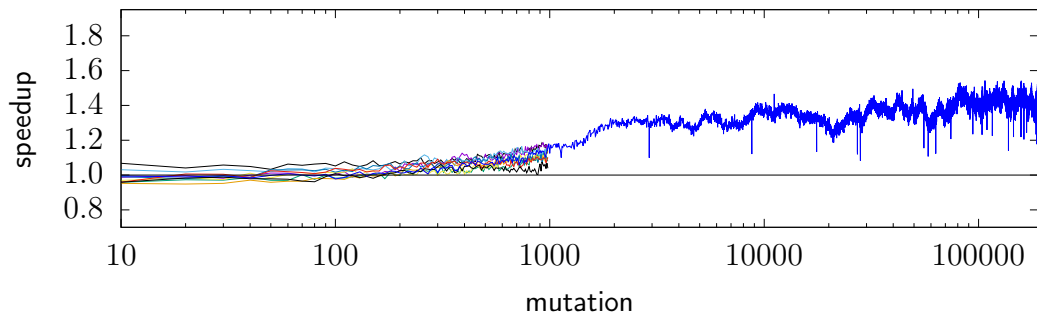
(H) P-256-Square on 7950X



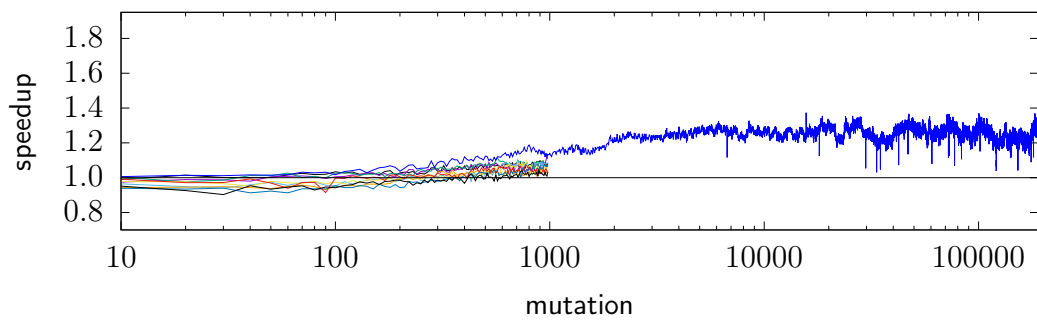
(i) P-384-Multiply on 7950X



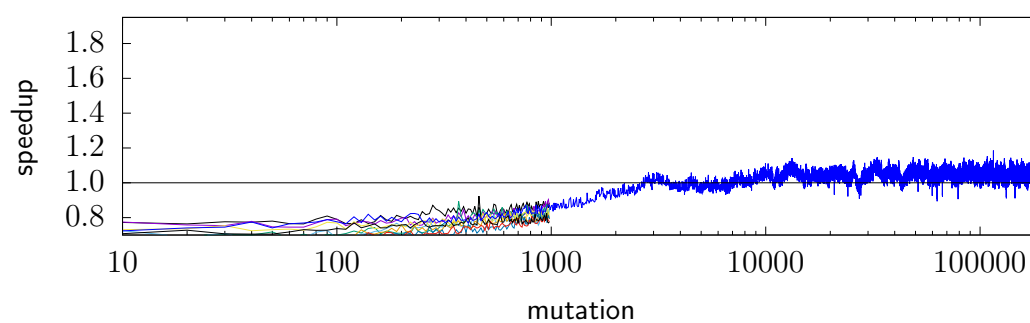
(j) P-384-Square on 7950X



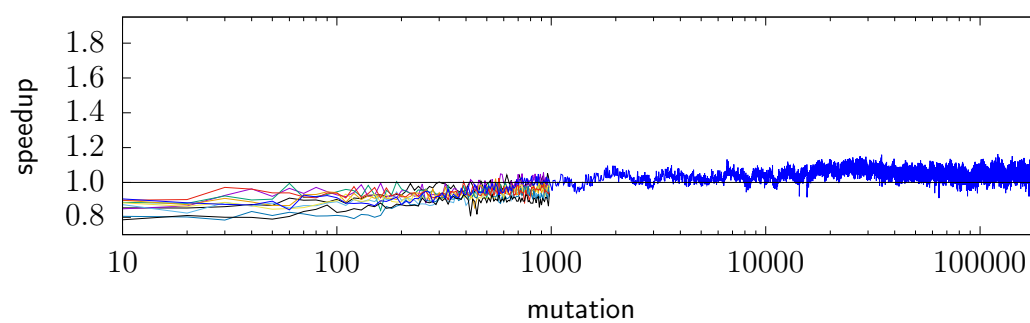
(k) SIKEp434-Multiply on 7950X



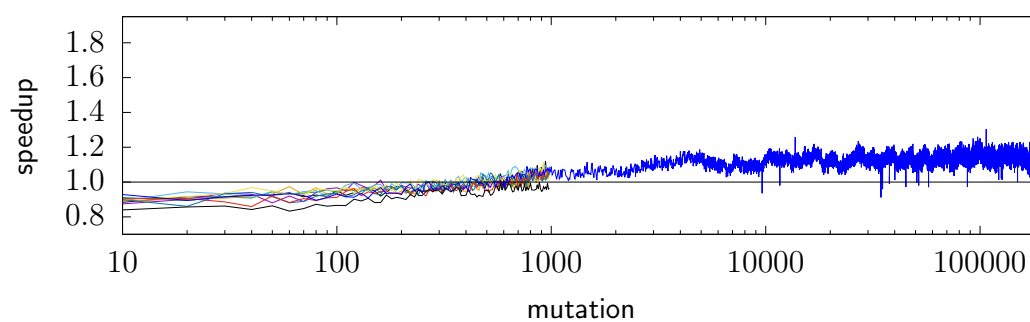
(l) SIKEp434-Square on 7950X



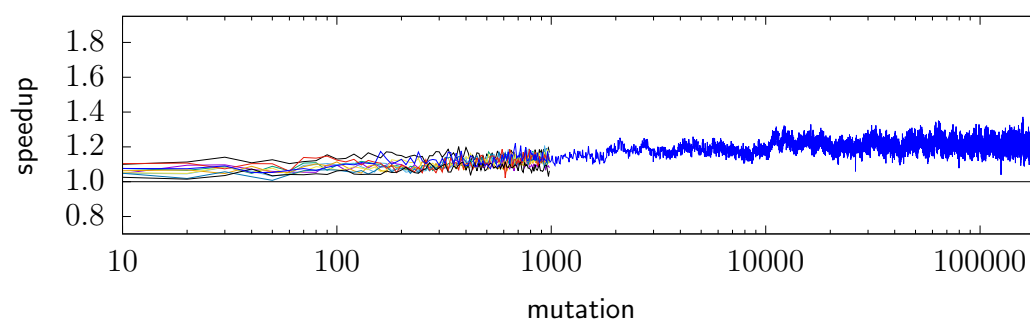
(M) Curve448-Multiply on 7950X



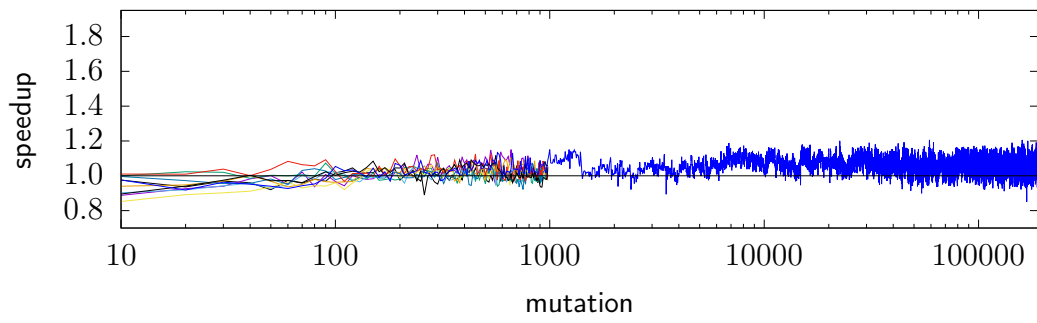
(N) Curve448-Square on 7950X



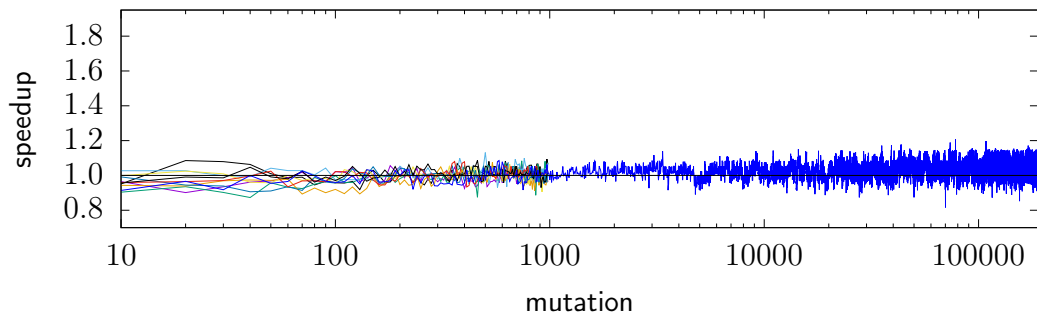
(O) P-521-Multiply on 7950X



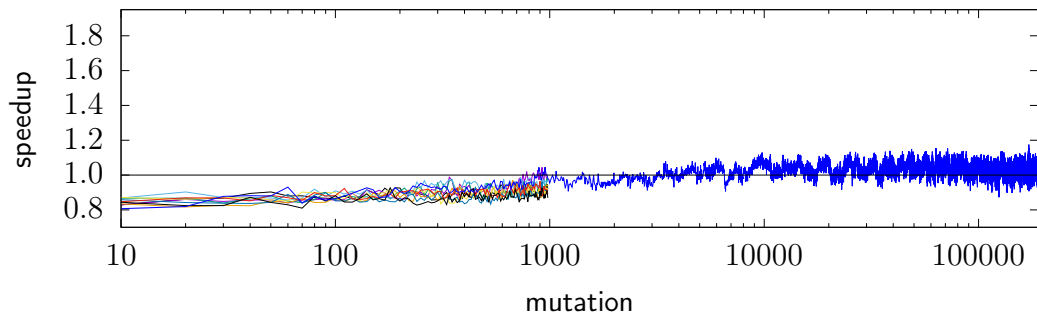
(P) P-521-Square on 7950X



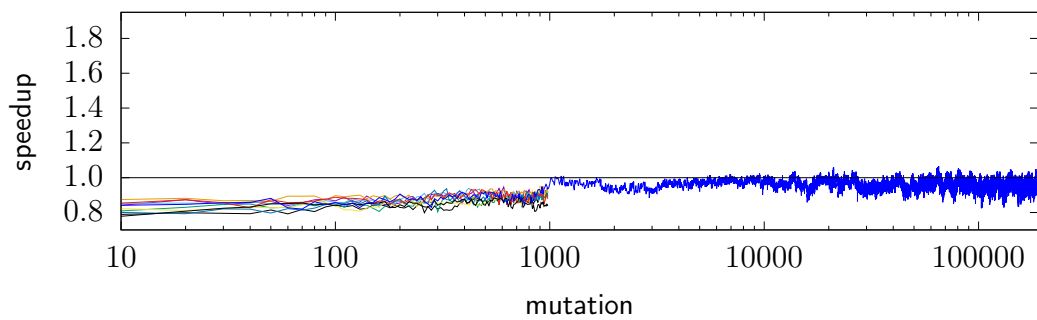
(q) Poly1305-Multiply on 7950X



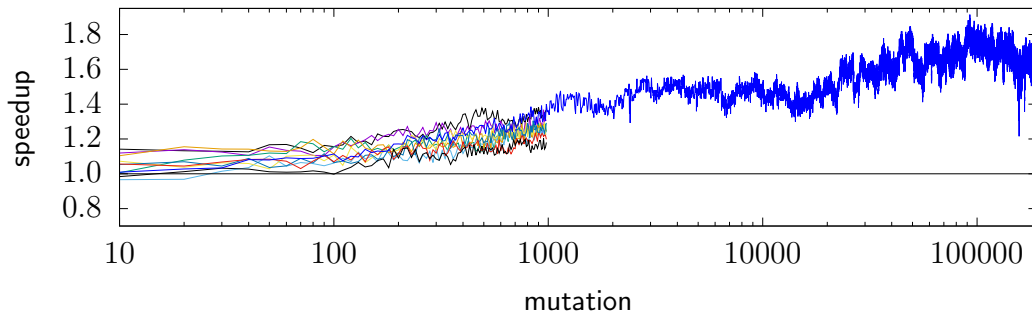
(r) Poly1305-Square on 7950X



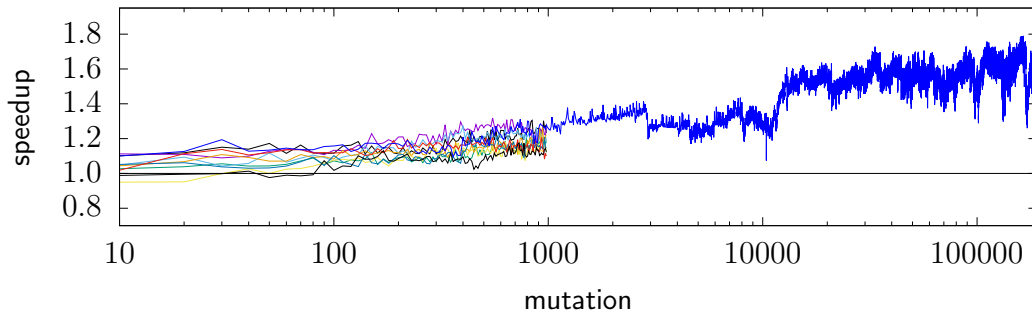
(s) secp256k1-Dettman-Multiply on 7950X



(t) secp256k1-Dettman-Square on 7950X



(u) secp256k1-Multiply on 7950X



(v) secp256k1-Square on 7950X

FIGURE B.4. All 22 functions are optimised on the 7950X microarchitecture, and one data point on the x-axis is generated every ten mutations. The y-axis shows the relative speedup against Clang with `-O3 -march=native -mtune=native`. Some optimisation runs show a relatively clear hill-climbing curve despite rather noisy measurements, some plateau early, and some drift in the configuration space.

B.2 Cross Validation

This section contains the detailed optimisation results in one table per function, i.e. one table per optimised function. Each table shows one function that is optimised on one platform and run on another. The value in the cell represents the cycle count ratio between those two implementations relative to the natively CRYPTOPT-optimised implementations. The higher the value, the faster the natively optimised implementation. Cells with a value greater than 1 are marked in blue, and cells with a value between 0 and 1 are marked in orange. The faster an implementation is, the more saturated the cell is (full saturation if the value x of a cell is $0.9 \geq x \geq 1.1$).

The tables show (visually by many dark blue cells) that for Curve25519, P-224, P-256, Poly1305, and secp256k1, CRYPTOPT can find implementations, which are very microarchitecture specific and are generally faster than off-the-shelf compilers. It is similar for P-521 but not as extreme as for the others. For other functions, such as P-384 and SIKEp434, CryptOpt can generate code that significantly outperforms the off-the-shelf compilers, and the implementations seem to not overfit the microarchitecture. This is shown by smaller numbers in the 10-by-10 matrix but still higher numbers in the rows “Clang” and “GCC”. For Curve448, especially for “multiply” on the AMD machines, the CRYPTOPT-generated code barely matches the performance of GCC compiled code. For 1900X, i9 12G and i9 13G, CRYPTOPT generated code is still slower.

The tables have been generated by running CRYPTOPT with only one set of hyperparameters¹ for all functions on all machines with the intent to give a general overview of CRYPTOPT’s general capabilities. As shown by the optimisation graphs in [Appendix B.1](#), the performance improvement varies. Further research is needed to evaluate the reasons for the sub-par performance of CRYPTOPT on those particular microarchitectures for those particular functions.

¹For both Curve25519 fields as well as the Bitcoin Core (LLVM based) and secp256k1-Dettman, CRYPTOPT is restricted to emit code that does not read from heap memory as soon as it has written to it because of potential memory aliasing (cf. memory-constraints switch in [Section 3.2](#)). This requirement comes from the need to benchmark with SUPERCOP. It restricts CRYPTOPT slightly in the space of fewer permutations, but reducing the search space can be explored.

TABLE B.1. Optimisation matrix for Curve25519.

Curve25519												
	run on	1900X	5800X	5950X	7950X	i7 6G	i7 10G	i9 10G	i7 11G	i9 12G	i9 13G	G.M.
MULTIPLY	opt on											
	1900X		1.08	1.08	1.05	1.09	1.09	1.05	1.06	1.17	1.17	1.08
	5800X	1.08		1.01	1.02	1.09	1.09	1.06	1.08	1.13	1.13	1.07
	5950X	1.08	0.99		1.01	1.13	1.13	1.10	1.15	1.24	1.24	1.10
	7950X	1.09	1.05	1.07		1.08	1.08	1.05	1.01	1.16	1.16	1.07
	i7 6G	1.10	1.15	1.16	1.11		1.00	0.97	1.04	1.16	1.16	1.08
	i7 10G	1.08	1.08	1.08	1.07	1.00		0.97	1.02	1.11	1.11	1.05
	i9 10G	1.09	1.11	1.11	1.06	1.03	1.03		1.01	1.13	1.13	1.07
	i7 11G	1.11	1.08	1.08	1.07	1.07	1.07	1.03		1.10	1.09	1.07
	i9 12G	1.09	1.07	1.08	1.07	1.07	1.07	1.03	1.03		0.99	1.05
	i9 13G	1.10	1.12	1.13	1.09	1.04	1.03	1.00	0.99	1.01		1.05
	Clang	1.25	1.30	1.28	1.24	1.32	1.32	1.28	1.27	1.24	1.21	1.27
	GCC	1.06	1.21	1.22	1.28	1.16	1.16	1.12	1.12	1.16	1.15	1.16
	Final	1.06	1.22	1.22	1.24	1.16	1.16	1.16	1.13	1.16	1.16	1.17
SQUARE	1900X		1.03	1.08	1.05	1.08	1.08	1.08	1.13	1.15	1.16	1.08
	5800X	1.13		1.01	1.10	1.08	1.08	1.09	1.11	1.10	1.10	1.08
	5950X	1.06	0.96		1.05	1.09	1.09	1.10	1.15	1.17	1.17	1.08
	7950X	1.05	0.97	1.01		1.06	1.06	1.07	1.08	1.20	1.20	1.07
	i7 6G	1.12	1.09	1.14	1.16		1.00	1.00	1.05	1.39	1.40	1.13
	i7 10G	1.13	1.04	1.08	1.15	1.01		1.00	1.04	1.04	1.04	1.05
	i9 10G	1.10	1.09	1.14	1.18	0.99	0.99		1.07	1.10	1.11	1.07
	i7 11G	1.12	1.04	1.08	1.19	1.05	1.05	1.06		1.38	1.40	1.13
	i9 12G	1.07	1.00	1.03	1.05	1.00	1.00	1.01	1.04		1.00	1.02
	i9 13G	1.07	0.96	0.99	1.01	0.99	0.99	1.00	0.99	1.00		1.00
	Clang	1.24	1.19	1.24	1.26	1.27	1.27	1.27	1.24	1.22	1.32	1.25
	GCC	1.19	1.19	1.25	1.23	1.13	1.12	1.13	1.15	1.19	1.19	1.18
	Final	1.19	1.24	1.25	1.23	1.14	1.14	1.13	1.16	1.19	1.19	1.19

TABLE B.2. Optimisation matrix for Curve25519-Solinas.

Curve25519-Solinas												
	run on	1900X	5800X	5950X	7950X	i7 6G	i7 10G	i9 10G	i7 11G	i9 12G	i9 13G	G.M.
MULTIPLY	opt on											
	1900X		1.15	1.15	1.15	1.09	1.11	1.14	1.18	1.10	1.13	1.12
	5800X	1.05		1.02	1.01	1.04	1.07	1.09	1.13	1.06	1.09	1.06
	5950X	1.03	0.99		1.05	1.04	1.07	1.09	1.09	1.10	1.12	1.06
	7950X	1.07	1.06	1.06		1.04	1.07	1.09	1.09	1.07	1.10	1.06
	i7 6G	1.06	1.14	1.15	1.17		1.02	1.05	1.04	1.02	1.04	1.07
	i7 10G	1.09	1.08	1.10	1.12	0.98		1.02	1.02	1.04	1.06	1.05
	i9 10G	1.07	1.10	1.11	1.10	0.95	0.98		1.02	1.02	1.04	1.04
	i7 11G	1.09	1.11	1.12	1.15	1.02	1.04	1.07		1.01	1.03	1.06
	i9 12G	1.06	1.13	1.14	1.13	1.00	1.02	1.05	1.05		1.02	1.06
	i9 13G	1.09	1.13	1.14	1.12	1.00	1.02	1.04	1.04	0.98		1.05
	Clang	1.66	1.79	1.81	1.81	1.46	1.50	1.53	1.73	1.62	1.67	1.65
	GCC	2.36	2.35	2.38	2.37	2.70	2.76	2.82	2.70	2.77	2.84	2.60
	Final	1.66	1.81	1.81	1.81	1.53	1.53	1.53	1.73	1.65	1.67	1.67
SQUARE	1900X		1.04	1.11	1.11	1.03	1.06	1.09	1.08	1.08	1.08	1.07
	5800X	1.07		1.06	1.07	1.03	1.06	1.09	1.04	1.03	1.02	1.05
	5950X	1.06	0.94		1.02	1.01	1.03	1.07	1.08	1.05	1.05	1.03
	7950X	1.07	0.96	1.02		1.00	1.03	1.06	1.04	1.07	1.06	1.03
	i7 6G	1.08	1.05	1.11	1.09		1.03	1.06	1.04	1.06	1.06	1.06
	i7 10G	1.08	1.04	1.10	1.07	0.97		1.03	1.03	1.07	1.07	1.05
	i9 10G	1.06	1.03	1.09	1.10	0.95	0.97		1.00	1.04	1.03	1.03
	i7 11G	1.05	1.03	1.09	1.07	1.00	1.03	1.06		1.08	1.07	1.05
	i9 12G	1.05	1.00	1.06	1.03	0.97	1.00	1.03	1.03		0.99	1.02
	i9 13G	1.06	1.07	1.13	1.06	0.99	1.02	1.05	1.07	1.00		1.04
	Clang	1.36	1.47	1.57	1.59	1.34	1.38	1.41	1.54	1.46	1.42	1.45
	GCC	2.74	2.82	2.99	2.87	2.78	2.84	2.92	2.77	2.63	2.64	2.80
	Final	1.36	1.56	1.57	1.59	1.42	1.42	1.41	1.54	1.46	1.42	1.47

TABLE B.3. Optimisation matrix for P-224.

P-224												
run on		1900X	5800X	5950X	7950X	i7 6G	i7 10G	i9 10G	i7 11G	i9 12G	i9 13G	G.M.
MULTIPLY	opt on											
	1900X		1.03	1.07	1.07	1.03	0.99	1.08	1.09	1.00	1.01	1.04
	5800X	1.16		1.04	1.08	1.15	1.10	1.20	1.22	1.19	1.21	1.13
	5950X	1.08	0.96		1.00	1.10	1.06	1.15	1.14	1.09	1.10	1.07
	7950X	1.11	1.00	1.04		1.07	1.02	1.11	1.19	1.07	1.08	1.07
	i7 6G	1.08	1.05	1.08	1.07		0.96	1.04	1.14	1.02	1.03	1.05
	i7 10G	1.12	1.03	1.08	1.08	1.04		1.09	1.11	0.99	1.00	1.05
	i9 10G	1.07	1.01	1.06	1.05	0.96	0.92		1.08	1.02	1.03	1.02
	i7 11G	1.08	1.01	1.06	1.04	1.01	0.97	1.06		0.98	0.98	1.02
	i9 12G	1.10	1.03	1.08	1.06	1.06	1.01	1.10	1.11		1.01	1.06
	i9 13G	1.08	1.03	1.07	1.07	1.05	1.00	1.09	1.10	0.99		1.05
	Clang	1.65	1.48	1.54	1.47	1.64	1.56	1.71	1.74	1.69	1.69	1.62
	GCC	2.51	2.11	2.20	2.13	2.95	2.82	3.07	2.94	3.03	3.10	2.66
	Final	1.65	1.54	1.54	1.47	1.71	1.70	1.71	1.74	1.74	1.72	1.65
SQUARE	1900X		1.08	1.08	1.06	0.99	1.00	1.02	1.02	1.01	0.93	1.02
	5800X	1.08		1.00	0.98	1.02	1.03	1.05	1.05	1.05	0.96	1.02
	5950X	1.09	1.00		1.00	1.05	1.06	1.08	1.08	1.11	1.02	1.05
	7950X	1.11	1.07	1.07		1.06	1.08	1.09	1.10	1.06	0.97	1.06
	i7 6G	1.06	1.08	1.07	1.03		1.02	1.03	1.00	1.01	0.93	1.02
	i7 10G	1.07	1.09	1.09	1.05	0.99		1.01	1.05	1.04	0.95	1.03
	i9 10G	1.10	1.06	1.05	1.02	0.97	0.99		1.06	1.02	0.93	1.02
	i7 11G	1.10	1.11	1.12	1.09	1.06	1.07	1.09		1.04	0.95	1.06
	i9 12G	1.10	1.10	1.10	1.06	1.03	1.05	1.09	1.02		0.92	1.04
	i9 13G	1.15	1.16	1.16	1.13	1.12	1.13	1.15	1.12	1.09		1.12
	Clang	1.42	1.45	1.45	1.34	1.47	1.49	1.51	1.46	1.55	1.36	1.45
	GCC	2.46	2.22	2.22	2.09	2.90	2.91	2.97	2.72	3.21	2.92	2.63
	Final	1.42	1.45	1.45	1.37	1.51	1.51	1.51	1.46	1.55	1.49	1.47

TABLE B.4. Optimisation matrix for P-256.

P-256												
run on		1900X	5800X	5950X	7950X	i7 6G	i7 10G	i9 10G	i7 11G	i9 12G	i9 13G	G.M.
MULTIPLY	opt on											
	1900X		1.03	1.03	1.09	1.02	1.05	1.07	1.08	1.01	0.99	1.04
	5800X	1.11		1.02	1.09	1.06	1.10	1.12	1.17	1.05	1.03	1.07
	5950X	1.10	1.00		1.09	1.10	1.13	1.15	1.12	1.09	1.07	1.08
	7950X	1.01	0.97	0.97		1.00	1.03	1.05	1.06	1.02	1.00	1.01
	i7 6G	1.10	1.07	1.08	1.12		1.03	1.05	1.14	1.06	1.05	1.07
	i7 10G	1.04	1.05	1.03	1.08	0.97		1.02	1.10	1.03	1.02	1.03
	i9 10G	1.01	1.01	1.01	1.04	0.95	0.98		1.03	0.99	0.97	1.00
	i7 11G	1.04	1.04	1.04	1.09	0.98	1.02	1.04		1.00	0.99	1.02
	i9 12G	1.10	1.09	1.09	1.13	1.06	1.10	1.12	1.08		0.99	1.07
	i9 13G	1.11	1.12	1.12	1.16	1.06	1.10	1.12	1.13	1.02		1.09
	Clang	1.63	1.75	1.76	1.69	1.70	1.76	1.79	1.88	1.77	1.76	1.75
	GCC	2.53	2.22	2.23	2.28	2.96	3.06	3.11	2.95	2.92	2.89	2.69
	Final	1.63	1.81	1.81	1.69	1.79	1.79	1.79	1.88	1.79	1.81	1.78
SQUARE	1900X		1.05	1.07	1.05	1.02	1.05	1.06	0.99	1.06	1.03	1.04
	5800X	1.12		1.03	1.03	1.06	1.09	1.10	1.03	1.10	1.07	1.06
	5950X	1.09	0.97		1.00	1.03	1.07	1.07	0.99	1.11	1.08	1.04
	7950X	1.11	1.00	1.01		1.01	1.05	1.05	1.04	1.05	1.03	1.04
	i7 6G	1.14	1.12	1.14	1.10		1.03	1.04	1.00	1.10	1.06	1.07
	i7 10G	1.09	1.01	1.03	1.04	0.97		1.00	1.00	1.04	1.02	1.02
	i9 10G	1.09	1.09	1.11	1.11	0.96	1.00		0.96	1.06	1.03	1.04
	i7 11G	1.14	1.14	1.17	1.15	1.06	1.10	1.10		1.11	1.09	1.11
	i9 12G	1.09	1.06	1.08	1.07	1.02	1.05	1.06	1.00		0.97	1.04
	i9 13G	1.06	1.05	1.07	1.09	1.02	1.05	1.05	0.96	1.03		1.04
	Clang	1.65	1.70	1.74	1.60	1.64	1.69	1.70	1.61	1.82	1.75	1.69
	GCC	2.58	2.26	2.31	2.20	2.90	2.99	3.00	2.65	2.95	2.76	2.64
	Final	1.65	1.75	1.74	1.60	1.70	1.69	1.70	1.68	1.82	1.80	1.71

TABLE B.5. Optimisation matrix for P-384.

P-384												
	run on	1900X	5800X	5950X	7950X	i7 6G	i7 10G	i9 10G	i7 11G	i9 12G	i9 13G	G.M.
MULTIPLY	opt on											
	1900X		1.31	1.26	1.25	1.40	1.39	1.48	1.35	0.99	0.91	1.22
	5800X	0.85		0.96	0.97	1.09	1.12	1.15	1.13	0.82	0.76	0.98
	5950X	0.86	1.04		1.00	1.11	1.14	1.17	1.12	0.78	0.72	0.98
	7950X	0.86	1.05	1.01		1.13	1.16	1.19	1.13	0.81	0.75	1.00
	i7 6G	0.82	1.07	1.02	1.04		1.03	1.06	1.08	0.76	0.71	0.95
	i7 10G	0.81	1.00	0.96	1.00	0.97		1.03	1.01	0.76	0.70	0.92
	i9 10G	0.78	1.04	0.99	1.00	0.95	0.97		1.02	0.74	0.69	0.91
	i7 11G	0.81	0.99	0.95	0.97	0.98	1.01	1.04		0.73	0.67	0.91
	i9 12G	1.06	1.37	1.30	1.27	1.46	1.51	1.55	1.39		0.92	1.26
	i9 13G	1.07	1.37	1.32	1.32	1.40	1.43	1.47	1.45	1.08		1.28
	Clang	1.15	1.65	1.59	1.40	1.71	1.76	1.81	1.75	1.35	1.15	1.51
	GCC	2.00	2.22	2.12	2.17	3.27	3.33	3.45	3.18	2.42	2.24	2.58
	Final	1.48	1.66	1.66	1.45	1.81	1.81	1.81	1.75	1.86	1.70	1.69
SQUARE	1900X		1.31	1.29	1.29	1.44	1.43	1.43	1.39	0.90	0.91	1.22
	5800X	0.88		0.99	1.02	1.18	1.17	1.17	1.18	0.78	0.78	1.00
	5950X	0.86	1.01		1.02	1.16	1.16	1.16	1.20	0.80	0.81	1.01
	7950X	0.86	1.02	1.01		1.18	1.17	1.17	1.16	0.74	0.75	0.99
	i7 6G	0.79	1.02	1.01	1.04		1.00	1.00	1.10	0.71	0.71	0.93
	i7 10G	0.78	1.02	1.01	1.03	1.01		1.00	1.09	0.74	0.75	0.93
	i9 10G	0.79	1.02	1.01	1.02	1.00	1.00		1.10	0.73	0.73	0.93
	i7 11G	0.80	1.00	0.99	1.00	1.01	1.00	1.00		0.67	0.68	0.90
	i9 12G	1.04	1.32	1.30	1.32	1.46	1.45	1.46	1.44		1.01	1.27
	i9 13G	1.02	1.32	1.30	1.31	1.55	1.44	1.54	1.53	0.99		1.28
	Clang	1.12	1.52	1.50	1.33	1.65	1.63	1.64	1.72	1.10	1.09	1.41
	GCC	2.10	2.17	2.14	2.14	3.51	3.48	3.49	3.55	2.16	2.17	2.61
	Final	1.44	1.52	1.52	1.33	1.65	1.63	1.65	1.72	1.63	1.60	1.56

TABLE B.7. Optimisation matrix for Curve448.

	run on	1900X	5800X	5950X	7950X	i7 6G	i7 10G	i9 10G	i7 11G	i9 12G	i9 13G	G.M.
MULTIPLY	opt on											
	1900X		1.17	1.18	1.16	1.18	1.17	1.12	1.13	1.06	0.92	1.10
	5800X	0.96		1.00	0.99	1.14	1.11	1.07	1.04	1.05	0.90	1.02
	5950X	0.97	0.99		1.00	1.13	1.12	1.08	1.04	1.02	0.87	1.02
	7950X	0.96	1.04	1.04		1.13	1.12	1.08	1.02	1.03	0.88	1.03
	i7 6G	0.94	1.00	1.01	0.98		0.99	0.95	0.97	0.94	0.80	0.96
	i7 10G	0.91	1.00	1.01	0.99	1.01		0.96	0.95	0.94	0.81	0.96
	i9 10G	0.96	1.02	1.02	1.01	1.05	1.04		0.98	0.96	0.82	0.98
	i7 11G	0.98	1.07	1.08	1.05	1.10	1.09	1.05		0.99	0.84	1.02
	i9 12G	1.03	1.16	1.17	1.16	1.19	1.18	1.14	1.12		0.85	1.10
	i9 13G	1.14	1.32	1.33	1.31	1.30	1.29	1.24	1.42	1.17		1.25
	Clang	1.06	1.10	1.10	1.09	1.43	1.42	1.36	1.30	1.18	1.12	1.21
GCC	0.89	0.99	0.99	0.99	1.13	1.13	1.08	1.05	0.99	0.84	1.00	
	Final	0.98	1.00	0.99	1.01	1.13	1.13	1.13	1.11	1.05	1.05	1.06
SQUARE	1900X		1.01	1.01	1.00	1.07	1.09	1.06	1.08	1.07	1.00	1.04
	5800X	1.07		1.01	1.04	1.10	1.12	1.09	1.06	1.12	1.05	1.06
	5950X	1.06	1.00		1.02	1.12	1.14	1.11	1.04	1.08	1.01	1.06
	7950X	1.04	0.99	1.00		1.09	1.10	1.07	1.05	1.08	1.01	1.04
	i7 6G	1.07	1.06	1.07	1.07		1.01	0.99	1.00	1.13	1.05	1.05
	i7 10G	1.05	1.05	1.06	1.03	0.99		0.99	1.02	1.16	1.08	1.04
	i9 10G	1.04	1.00	1.00	1.01	1.01	1.02		1.01	1.08	1.01	1.02
	i7 11G	1.07	1.03	1.03	1.05	1.05	1.06	1.04		1.06	0.99	1.04
	i9 12G	1.06	1.00	1.01	1.01	1.06	1.07	1.05	1.02		0.93	1.02
	i9 13G	1.08	1.07	1.08	1.06	1.14	1.16	1.13	1.10	1.07		1.09
	Clang	1.05	1.08	1.08	1.10	1.12	1.13	1.11	1.12	1.12	1.03	1.09
	GCC	1.04	1.07	1.08	1.06	1.06	1.07	1.05	1.06	1.04	0.97	1.05
	Final	1.04	1.08	1.08	1.06	1.07	1.07	1.06	1.06	1.04	1.05	1.06

TABLE B.8. Optimisation matrix for P-521.

P-521												
run on		1900X	5800X	5950X	7950X	i7 6G	i7 10G	i9 10G	i7 11G	i9 12G	i9 13G	G.M.
MULTIPLY	opt on											
	1900X		1.13	1.13	1.07	1.19	1.15	1.15	1.23	1.04	1.10	1.12
	5800X	1.00		1.02	0.96	1.12	1.10	1.11	1.09	1.05	1.09	1.05
	5950X	0.99	0.98		0.95	1.08	1.07	1.08	1.05	1.03	1.09	1.03
	7950X	1.01	1.03	1.05		1.16	1.17	1.16	1.15	1.06	1.11	1.09
	i7 6G	0.96	1.01	1.02	0.98		0.99	0.99	1.01	1.02	1.06	1.00
	i7 10G	0.95	1.03	1.04	0.99	1.01		1.01	1.03	1.03	1.09	1.02
	i9 10G	0.94	1.01	1.02	0.98	1.01	1.00		0.98	0.98	1.03	0.99
	i7 11G	0.97	1.02	1.04	0.97	1.05	1.03	1.04		0.95	0.98	1.00
	i9 12G	1.04	1.15	1.17	1.08	1.18	1.17	1.17	1.31		1.03	1.12
	i9 13G	1.06	1.15	1.16	1.09	1.24	1.23	1.23	1.33	0.97		1.14
	Clang	1.14	1.25	1.26	1.18	1.52	1.50	1.51	1.47	1.39	1.37	1.35
	GCC	1.02	1.16	1.17	1.11	0.98	0.97	0.98	1.00	0.96	0.99	1.03
	Final	1.09	1.18	1.17	1.17	0.98	0.98	0.98	1.01	1.01	1.01	1.06
SQUARE	1900X		1.00	1.01	1.07	1.11	1.09	1.14	1.06	1.17	1.13	1.08
	5800X	1.03		1.02	1.05	1.15	1.13	1.18	1.05	1.20	1.16	1.10
	5950X	1.03	0.98		1.03	1.11	1.09	1.14	1.01	1.19	1.15	1.07
	7950X	1.04	1.02	1.03		1.12	1.10	1.15	1.05	1.15	1.12	1.08
	i7 6G	0.96	1.00	1.01	1.06		0.98	1.02	0.99	1.20	1.21	1.04
	i7 10G	0.97	0.97	1.02	1.02	1.02		1.04	0.94	1.18	1.18	1.03
	i9 10G	0.99	1.01	1.01	1.05	0.98	0.96		1.01	1.23	1.20	1.04
	i7 11G	1.03	1.03	1.04	1.04	1.12	1.10	1.14		1.17	1.14	1.08
	i9 12G	1.04	1.03	1.04	1.06	1.11	1.09	1.14	1.02		0.97	1.05
	i9 13G	1.03	1.04	1.06	1.07	1.13	1.11	1.16	1.01	1.03		1.06
	Clang	1.09	1.24	1.26	1.32	1.61	1.56	1.63	1.66	1.35	1.30	1.39
	GCC	1.07	1.12	1.14	1.17	1.03	1.01	1.05	0.99	1.21	1.18	1.09
	Final	1.11	1.16	1.14	1.17	1.05	1.05	1.05	1.06	1.21	1.21	1.12

TABLE B.9. Optimisation matrix for Poly1305.

Poly1305												
	run on	1900X	5800X	5950X	7950X	i7 6G	i7 10G	i9 10G	i7 11G	i9 12G	i9 13G	G.M.
MULTIPLY	opt on											
	1900X		1.16	1.16	1.13	1.09	1.09	1.09	1.10	1.15	1.14	1.11
	5800X	1.11		1.00	0.99	1.08	1.08	1.08	1.07	1.09	1.07	1.06
	5950X	1.11	1.00		1.00	1.11	1.12	1.12	1.13	1.19	1.19	1.10
	7950X	1.06	1.05	1.05		1.08	1.09	1.09	1.10	1.13	1.11	1.08
	i7 6G	1.15	1.13	1.13	1.09		1.00	1.00	1.04	1.12	1.10	1.08
	i7 10G	1.13	1.15	1.17	1.11	0.99		0.99	1.03	1.33	1.36	1.12
	i9 10G	1.14	1.14	1.14	1.15	1.00	1.01		1.04	1.41	1.42	1.14
	i7 11G	1.12	1.09	1.09	1.07	1.01	1.02	1.01		1.32	1.32	1.10
	i9 12G	1.11	1.05	1.05	1.05	1.00	1.01	1.01	1.01		0.99	1.03
i9 13G	1.13	1.05	1.05	1.07	1.01	1.02	1.01	1.02	1.01		1.04	
	Clang	1.14	1.13	1.13	1.09	1.15	1.15	1.14	1.14	1.23	1.21	1.15
	GCC	1.13	1.18	1.18	1.13	1.26	1.27	1.26	1.21	1.32	1.31	1.22
	Final	1.13	1.13	1.13	1.10	1.15	1.15	1.15	1.14	1.23	1.23	1.15
SQUARE	1900X		1.05	1.06	1.00	1.09	1.15	1.14	1.11	1.11	1.11	1.08
	5800X	1.07		1.00	1.01	1.05	1.10	1.09	1.04	1.05	1.05	1.04
	5950X	1.13	1.00		1.00	1.05	1.10	1.09	1.04	1.08	1.08	1.06
	7950X	1.18	1.06	1.06		1.08	1.13	1.13	1.05	1.11	1.11	1.09
	i7 6G	1.12	1.17	1.17	1.17		1.05	1.05	1.04	1.33	1.39	1.14
	i7 10G	1.14	1.12	1.13	1.11	0.95		0.99	1.02	1.39	1.41	1.12
	i9 10G	1.11	1.17	1.17	1.15	0.95	1.00		1.04	1.33	1.35	1.12
	i7 11G	1.16	1.06	1.06	1.10	1.00	1.05	1.04		1.41	1.42	1.12
	i9 12G	1.08	1.08	1.08	1.03	1.00	1.05	1.04	1.02		0.99	1.04
	i9 13G	1.10	1.06	1.06	1.06	1.00	1.05	1.05	1.02	1.00		1.04
	Clang	1.06	1.11	1.11	1.08	1.09	1.14	1.13	1.10	1.20	1.20	1.12
	GCC	1.14	1.28	1.28	1.16	1.28	1.34	1.33	1.28	1.29	1.28	1.26
	Final	1.06	1.11	1.11	1.09	1.14	1.14	1.14	1.10	1.20	1.21	1.13

TABLE B.10. Optimisation matrix for secp256k1-Dettman.

secp256k1-Dettman												
	run on	1900X	5800X	5950X	7950X	i7 6G	i7 10G	i9 10G	i7 11G	i9 12G	i9 13G	G.M.
MULTIPLY	opt on											
	1900X		1.00	1.02	1.04	1.04	1.05	1.05	1.04	1.31	1.29	1.08
	5800X	1.09		1.02	1.04	1.08	1.10	1.10	1.07	1.29	1.26	1.10
	5950X	1.06	0.95		0.99	1.02	1.06	1.03	1.03	1.17	1.15	1.04
	7950X	1.06	1.02	1.04		1.03	1.05	1.05	1.04	1.23	1.20	1.07
	i7 6G	1.06	1.07	1.09	1.08		1.02	1.01	1.06	1.14	1.12	1.06
	i7 10G	1.06	1.04	1.07	1.06	0.98		1.00	1.04	1.23	1.20	1.07
	i9 10G	1.06	1.05	1.07	1.07	0.98	1.00		1.07	1.26	1.24	1.08
	i7 11G	1.08	1.06	1.08	1.04	1.00	1.02	1.02		1.21	1.19	1.07
	i9 12G	1.06	1.02	1.04	1.05	0.99	1.01	1.01	1.04		0.99	1.02
	i9 13G	1.05	1.07	1.09	1.07	1.02	1.04	1.04	1.02	1.02		1.04
	Clang	0.98	1.07	1.10	1.09	1.15	1.16	1.17	1.11	1.19	1.12	1.11
	GCC	1.22	1.38	1.41	1.38	1.20	1.22	1.23	1.15	1.25	1.22	1.26
Final	0.98	1.12	1.10	1.10	1.17	1.16	1.17	1.11	1.19	1.13	1.12	
SQUARE	1900X		1.04	1.09	1.09	1.10	1.09	1.10	1.10	1.20	1.15	1.10
	5800X	1.10		1.04	1.07	1.11	1.10	1.12	1.21	1.19	1.14	1.11
	5950X	1.11	0.96		1.05	1.10	1.09	1.11	1.11	1.11	1.07	1.07
	7950X	1.08	0.99	1.03		1.07	1.06	1.07	1.10	1.12	1.07	1.06
	i7 6G	1.11	1.05	1.10	1.14		0.99	1.00	1.05	1.18	1.13	1.07
	i7 10G	1.10	1.05	1.09	1.12	1.01		1.01	1.04	1.22	1.16	1.08
	i9 10G	1.09	0.99	1.03	1.07	1.00	0.99		1.03	1.20	1.15	1.05
	i7 11G	1.07	1.04	1.07	1.08	1.07	1.06	1.07		1.27	1.21	1.09
	i9 12G	1.08	1.02	1.07	1.06	1.06	1.04	1.05	1.04		0.96	1.04
	i9 13G	1.10	1.09	1.13	1.12	1.08	1.07	1.08	1.11	1.04		1.08
	Clang	0.95	1.00	1.05	1.04	1.11	1.09	1.11	1.16	1.13	1.10	1.07
	GCC	1.11	1.12	1.17	1.24	1.21	1.20	1.21	1.20	1.10	1.06	1.16
	Final	0.95	1.04	1.05	1.04	1.11	1.11	1.11	1.16	1.10	1.10	1.08

TABLE B.11. Optimisation matrix for secp256k1.

secp256k1												
	run on	1900X	5800X	5950X	7950X	i7 6G	i7 10G	i9 10G	i7 11G	i9 12G	i9 13G	G.M.
MULTIPLY	opt on											
	1900X		1.04	1.04	1.05	0.98	1.06	1.08	1.06	1.08	1.04	1.04
	5800X	1.08		0.98	1.05	0.98	1.06	1.08	1.09	1.09	1.05	1.04
	5950X	1.08	1.02		1.07	0.99	1.08	1.09	1.10	1.11	1.06	1.06
	7950X	1.04	1.00	0.98		0.96	1.05	1.06	1.08	1.09	1.04	1.03
	i7 6G	1.17	1.09	1.07	1.14		1.09	1.10	1.14	1.11	1.07	1.10
	i7 10G	1.04	1.00	0.98	1.04	0.92		1.01	1.06	1.10	1.05	1.02
	i9 10G	1.08	0.99	0.98	1.04	0.91	0.99		1.01	1.05	1.01	1.00
	i7 11G	1.05	1.04	1.02	1.11	0.95	1.03	1.05		1.05	1.00	1.03
	i9 12G	1.07	1.02	1.01	1.11	0.97	1.06	1.07	1.03		0.96	1.03
	i9 13G	1.11	1.11	1.09	1.14	1.00	1.09	1.11	1.05	1.04		1.07
	Clang	1.87	1.93	1.88	1.84	1.76	1.91	1.94	1.97	1.90	1.81	1.88
	GCC	2.52	2.27	2.23	2.33	2.81	3.05	3.09	2.87	3.19	3.01	2.71
	Final	1.87	1.94	1.93	1.84	1.95	1.94	1.94	1.97	1.90	1.90	1.92
SQUARE	1900X		1.04	1.11	1.05	1.03	1.04	1.04	1.04	1.03	1.00	1.04
	5800X	1.08		1.05	1.08	1.06	1.07	1.07	1.16	1.09	1.06	1.07
	5950X	1.06	0.95		1.00	1.04	1.05	1.05	1.06	1.08	1.05	1.03
	7950X	1.09	1.02	1.07		1.05	1.06	1.06	1.10	1.09	1.06	1.06
	i7 6G	1.09	1.08	1.15	1.08		1.01	1.01	1.11	1.09	1.06	1.07
	i7 10G	1.11	1.06	1.12	1.09	0.99		1.01	1.09	1.08	1.05	1.06
	i9 10G	1.05	1.05	1.11	1.07	0.99	1.00		1.08	1.03	1.00	1.04
	i7 11G	1.07	1.03	1.09	1.07	1.01	1.02	1.03		1.04	1.01	1.04
	i9 12G	1.10	1.13	1.19	1.15	1.04	1.05	1.06	1.04		0.97	1.07
	i9 13G	1.10	1.07	1.13	1.10	1.04	1.05	1.06	1.08	1.03		1.07
	Clang	1.72	1.84	1.94	1.71	1.81	1.82	1.83	1.83	1.82	1.80	1.81
	GCC	2.46	2.23	2.34	2.25	2.88	2.90	2.91	2.74	3.10	3.04	2.67
	Final	1.72	1.94	1.94	1.71	1.83	1.82	1.83	1.83	1.82	1.85	1.83

TABLE B.12. Optimisation matrix for Bitcoin Core (LLVM).

Bitcoin Core (LLVM)												
	run on	1900X	5800X	5950X	7950X	i7 6G	i7 10G	i9 10G	i7 11G	i9 12G	i9 13G	G.M.
MULTIPLY	opt on											
	1900X		1.08	1.12	1.08	1.03	1.04	1.05	1.10	1.12	1.14	1.08
	5800X	1.06		0.98	0.97	1.03	1.05	1.05	1.07	1.16	1.18	1.05
	5950X	1.07	1.02		1.00	1.05	1.06	1.06	1.07	1.07	1.08	1.05
	7950X	1.09	1.05	1.03		1.00	1.02	1.02	1.03	1.19	1.22	1.06
	i7 6G	1.04	1.05	1.03	1.05		1.01	1.02	1.05	1.21	1.24	1.07
	i7 10G	1.04	1.08	1.07	1.08	0.99		1.00	1.05	1.14	1.17	1.06
	i9 10G	1.05	1.06	1.04	1.03	0.98	1.00		1.04	1.09	1.11	1.04
	i7 11G	1.06	1.13	1.11	1.07	1.06	1.07	1.07		1.08	1.10	1.08
	i9 12G	1.05	1.07	1.05	0.99	1.01	1.03	1.03	1.03		1.02	1.03
	i9 13G	1.04	1.05	1.03	1.02	1.01	1.01	1.02	1.01	0.98		1.02
	Clang	0.98	1.06	1.04	1.09	1.08	1.09	1.10	1.06	1.13	1.17	1.08
GCC	1.23	1.34	1.32	1.21	1.15	1.17	1.18	1.10	1.22	1.24	1.21	
Final	0.98	1.06	1.06	1.12	1.10	1.09	1.10	1.06	1.15	1.17	1.09	
SQUARE	1900X		1.09	1.11	1.14	1.11	1.11	1.14	1.15	1.21	1.14	1.12
	5800X	1.08		1.02	1.05	1.10	1.09	1.12	1.13	1.13	1.08	1.08
	5950X	1.10	0.98		1.04	1.08	1.08	1.10	1.14	1.15	1.09	1.07
	7950X	1.09	0.97	0.99		1.08	1.08	1.10	1.10	1.14	1.08	1.06
	i7 6G	1.10	1.08	1.10	1.11		1.00	1.02	1.05	1.11	1.06	1.06
	i7 10G	1.12	1.03	1.06	1.11	1.00		1.02	1.07	1.09	1.03	1.05
	i9 10G	1.11	1.09	1.10	1.17	0.98	0.98		1.03	1.06	1.00	1.05
	i7 11G	1.14	1.08	1.10	1.11	1.07	1.07	1.09		1.18	1.12	1.10
	i9 12G	1.11	1.05	1.07	1.10	1.05	1.05	1.08	1.03		0.95	1.05
	i9 13G	1.13	1.08	1.10	1.12	1.09	1.09	1.12	1.05	1.06		1.08
	Clang	0.96	1.00	1.01	1.05	1.10	1.09	1.12	1.09	1.07	1.01	1.05
	GCC	1.40	1.48	1.51	1.41	1.21	1.22	1.24	1.20	1.14	1.08	1.28
Final	0.96	1.03	1.02	1.05	1.12	1.11	1.12	1.09	1.07	1.07	1.06	

B.3 Scalar Multiplication

This section is a more detailed version of Table 8.13, which shows cycles for all machines individually. As Section 8.8 explained, the reported cycles are the cycles for four scalar multiplications: two base point and two variable point multiplications. Additionally, SUPERCOP uses multiple compiler settings and reports the best-performing one. Table B.13 shows those compiler setting combinations. SUPERCOP is used in the version 20221005.

The fastest implementation per section and per platform is highlighted in (**bold text**) and notes a (1.00x). The last column, G.M., is the geometric mean of the cycles across all platforms (cf. Table 8.13); the ratio is then recalculated on that G.M. as well. The language in which the core operations are implemented is written in the column “Lang”. Note: “asm” means x86-64 assembly; “-v” indicates the use of vector instructions. For Curve25519, “-51” and “-64” indicate whether the representation for the field elements is unsaturated or saturated.

TABLE B.13. SUPERCOP compiler settings.

No	Compiler	Options
1	gcc	<code>-march=native -mtune=native -O3</code> <code>-fomit-frame-pointer -fwrapv -fPIC -fPIE</code>
2	gcc	<code>-march=native -mtune=native -Os</code> <code>-fomit-frame-pointer -fwrapv -fPIC -fPIE</code>
3	gcc	<code>-march=native -mtune=native -O2</code> <code>-fomit-frame-pointer -fwrapv -fPIC -fPIE</code>
4	gcc	<code>-march=native -mtune=native -O</code> <code>-fomit-frame-pointer -fwrapv -fPIC -fPIE</code>
5	clang	<code>-march=native -O3 -fomit-frame-pointer -fwrapv</code> <code>-Qunused-arguments -fPIC -fPIE</code>
6	clang	<code>-march=native -Os -fomit-frame-pointer -fwrapv</code> <code>-Qunused-arguments -fPIC -fPIE</code>
7	clang	<code>-march=native -O2 -fomit-frame-pointer -fwrapv</code> <code>-Qunused-arguments -fPIC -fPIE</code>
8	clang	<code>-march=native -O -fomit-frame-pointer -fwrapv</code> <code>-Qunused-arguments -fPIC -fPIE</code>
9	clang	<code>-mcpu=native -O3 -fomit-frame-pointer -fwrapv</code> <code>-Qunused-arguments -fPIC -fPIE</code>

TABLE B.14. Scalar Multiplication cost (in cycles) for different implementations on different machines.

Field	Implementation	Lang.	1900X	5800X	5950X	7950X	i7 6G	i7 10G	i9 10G	i7 11G	i9 12G	i9 13G	G.M.	
Curve25519	sandy2x	asm-v	602k (1.05x)	427k (1.00x)	427k (1.00x)	422k (1.00x)	521k (1.13x)	521k (1.13x)	521k (1.13x)	492k (1.11x)	590k (1.24x)	480k (1.23x)	496k (1.07x)	
	amd64-64	asm	623k (1.09x)	574k (1.35x)	572k (1.34x)	522k (1.24x)	569k (1.23x)	570k (1.24x)	572k (1.24x)	534k (1.20x)	556k (1.17x)	452k (1.16x)	553k (1.19x)	
	amd64-51	asm	775k (1.36x)	587k (1.38x)	584k (1.37x)	562k (1.33x)	567k (1.23x)	566k (1.23x)	567k (1.23x)	491k (1.11x)	520k (1.09x)	425k (1.09x)	558k (1.20x)	
	donna	asm-v	1089k (1.91x)	972k (2.28x)	969k (2.27x)	960k (2.28x)	1014k (2.20x)	1016k (2.20x)	1016k (2.20x)	964k (2.17x)	1073k (2.26x)	880k (2.26x)	994k (2.14x)	
	donna-c64	C	798k (1.40x)	603k (1.41x)	605k (1.42x)	587k (1.39x)	588k (1.28x)	589k (1.28x)	590k (1.28x)	541k (1.22x)	552k (1.16x)	450k (1.15x)	585k (1.26x)	
	fe-51 off-the-shelf	C	665k (1.16x)	547k (1.28x)	546k (1.28x)	549k (1.30x)	540k (1.17x)	540k (1.17x)	541k (1.17x)	490k (1.11x)	511k (1.07x)	417k (1.07x)	531k (1.14x)	
	fe-51 off-the-shelf	asm	664k (1.16x)	547k (1.28x)	544k (1.28x)	549k (1.30x)	540k (1.17x)	539k (1.17x)	540k (1.17x)	490k (1.10x)	509k (1.07x)	416k (1.06x)	531k (1.14x)	
	fe-51+CRYPTOPT	asm	700k (1.23x)	520k (1.22x)	521k (1.22x)	509k (1.21x)	543k (1.18x)	549k (1.19x)	548k (1.19x)	509k (1.15x)	501k (1.05x)	409k (1.05x)	527k (1.13x)	
	fe-64 off-the-shelf	asm	571k (1.00x)	449k (1.05x)	450k (1.05x)	435k (1.03x)	461k (1.00x)	462k (1.00x)	462k (1.00x)	464k (1.05x)	507k (1.07x)	413k (1.06x)	466k (1.00x)	
	fe-64+CRYPTOPT	asm	588k (1.03x)	452k (1.06x)	448k (1.05x)	435k (1.03x)	486k (1.05x)	479k (1.04x)	473k (1.02x)	443k (1.00x)	476k (1.00x)	390k (1.00x)	465k (1.00x)	
	fe-51+Best	asm	723k (1.27x)	527k (1.24x)	529k (1.24x)	524k (1.24x)	545k (1.18x)	546k (1.18x)	546k (1.18x)	497k (1.12x)	503k (1.06x)	410k (1.05x)	530k (1.14x)	
	fe-64+Best	asm	600k (1.05x)	449k (1.05x)	447k (1.05x)	461k (1.09x)	489k (1.06x)	489k (1.06x)	490k (1.06x)	450k (1.01x)	495k (1.04x)	402k (1.03x)	475k (1.02x)	
secp256k1	libsecp256k1	off-the-shelf	asm	719k (1.08x)	571k (1.13x)	567k (1.12x)	560k (1.13x)	568k (1.08x)	565k (1.09x)	564k (1.07x)	539k (1.12x)	537k (1.09x)	439k (1.09x)	559k (1.10x)
	off-the-shelf	C	671k (1.01x)	538k (1.07x)	538k (1.07x)	530k (1.07x)	562k (1.07x)	562k (1.08x)	561k (1.06x)	504k (1.05x)	538k (1.10x)	436k (1.09x)	541k (1.06x)	
	+CRYPTOPT(Opt)	asm	700k (1.05x)	529k (1.05x)	528k (1.05x)	510k (1.03x)	547k (1.04x)	548k (1.05x)	547k (1.04x)	511k (1.06x)	522k (1.06x)	428k (1.07x)	533k (1.05x)	
	+CRYPTOPT(Fiat)	asm	667k (1.00x)	514k (1.02x)	507k (1.01x)	495k (1.00x)	529k (1.01x)	521k (1.00x)	530k (1.01x)	480k (1.00x)	491k (1.00x)	401k (1.00x)	510k (1.00x)	
	+Best(Opt)	asm	709k (1.06x)	533k (1.06x)	533k (1.06x)	521k (1.05x)	557k (1.06x)	558k (1.07x)	554k (1.05x)	515k (1.07x)	543k (1.11x)	440k (1.10x)	543k (1.06x)	
	+Best(Fiat)	asm	678k (1.02x)	504k (1.00x)	504k (1.00x)	503k (1.02x)	525k (1.00x)	526k (1.01x)	527k (1.00x)	481k (1.00x)	494k (1.01x)	405k (1.01x)	511k (1.00x)	

Acronyms

- AES** Advanced Encryption Standard. 20
- CPU** Central Processing Unit. 11–15, 19, 40, 43, 51, 69–72, 101, 102
- CSE** Common Subexpression Elimination. 18, 134, 137
- DSA** Digital Signature Algorithm. 21
- DSL** Domain-Specific Language. 20
- ECC** Elliptic Curve Cryptography. 6, 22, 24, 25, 124, 130, 140
- ECDLP** elliptic-curve discrete-logarithm problem. 21–23
- ECDSA** elliptic-curve digital signature algorithm. 23
- GI** Genetic Improvement. xv, 3, 28
- IR** Intermediate Representation. 15, 17–19, 87, 132
- ISA** Instruction Set Architecture. 5, 11, 12, 53
- ISE** Instruction Set Extension. 12, 13, 19, 108, 109
- MAC** Message Authentication Code. 20
- OS** Operating System. 14, 79, 108
- RLS** Random Local Search. 7, 8, 29, 61, 81, 140
- SA** Simulated Annealing. 81, 94, 96, 97
- SIMD** Single Instruction Multiple Data. 101
- SSA** Static Single Assignment. xiii, 32, 35, 44, 45, 131

Bibliography

- [0xA23a] 0xADE1A1DE. *AssemblyLine v1.3.2*. 2023. URL: <https://github.com/0xADE1A1DE/AssemblyLine>.
- [0xA23b] 0xADE1A1DE. *CryptOpt*. 2023. URL: <https://github.com/0xADE1A1DE/CryptOpt>.
- [0xA23c] 0xADE1A1DE. *MeasureSuite v2.2.2*. 2023. URL: <https://github.com/0xADE1A1DE/MeasureSuite>.
- [AD11] Anne Auger and Benjamin Doerr. *Theory of Randomized Search Heuristics: Foundations and Recent Developments*. Vol. 1. Series on Theoretical Computer Science. World Scientific, 2011. DOI: [10.1142/7438](https://doi.org/10.1142/7438).
- [Adv05] Advanced Micro Devices. *AMD64 Architecture Programmer’s Manual Volume 3: General-Purpose and System Instructions*. Revision 3.10. 2005. URL: <https://community.amd.com/sdtp67534/attachments/sdtp67534/processors-discussions/29160/1/AMD64-3.pdf>.
- [Alm+17] José Bacelar Almeida, Manuel Barbosa, Gilles Barthe, Arthur Blot, Benjamin Grégoire, Vincent Laporte, Tiago Oliveira, Hugo Pacheco, Benedikt Schmidt and Pierre-Yves Strub. ‘Jasmin: High-Assurance and High-Speed Cryptography’. In: *CCS*. ACM, 2017, pp. 1807–1823. DOI: [10.1145/3133956.3134078](https://doi.org/10.1145/3133956.3134078).
- [Alm+19] José Bacelar Almeida, Cécile Baritel-Ruet, Manuel Barbosa, Gilles Barthe, François Dupressoir, Benjamin Grégoire, Vincent Laporte, Tiago Oliveira, Alley Stoughton and Pierre-Yves Strub. ‘Machine-Checked Proofs for Cryptographic Standards: Indifferentiability of Sponge and Secure High-Assurance Implementations of SHA-3’. In: *CCS*. ACM, 2019, pp. 1607–1622. DOI: [10.1145/3319535.3363211](https://doi.org/10.1145/3319535.3363211).
- [Alm+20a] José Bacelar Almeida, Manuel Barbosa, Gilles Barthe, Benjamin Grégoire, Adrien Koutsos, Vincent Laporte, Tiago Oliveira and Pierre-Yves Strub. ‘The Last Mile: High-Assurance and High-Speed Cryptographic Implementations’. In: *IEEE Symposium on Security*

- and Privacy*. IEEE, 2020, pp. 965–982. DOI: [10.1109/sp40000.2020.00028](https://doi.org/10.1109/sp40000.2020.00028).
- [Alm+20b] José Bacelar Almeida, Manuel Barbosa, Gilles Barthe, Vincent Laporte and Tiago Oliveira. ‘Certified Compilation for Cryptography: Extended x86 Instructions and Constant-Time Verification’. In: *INDOCRYPT*. 2020, pp. 107–127. DOI: [10.1007/978-3-030-65277-7_6](https://doi.org/10.1007/978-3-030-65277-7_6).
- [ALS19] Ali Ansari, Pejman Lotfi-Kamran and Hamid Sarbazi-Azad. ‘Code Layout Optimization for Near-Ideal Instruction Cache’. In: *IEEE Computer Architecture Letters* 18.2 (2019), pp. 124–127. DOI: [10.1109/LCA.2019.2924429](https://doi.org/10.1109/LCA.2019.2924429).
- [Ang+22] Alexandros Angelou, Antonios Dadaliaris, Michael Dossis and Georgios Dimitriou. ‘Branchless Code Generation for Modern Processor Architectures’. In: *Proceedings of the 25th Pan-Hellenic Conference on Informatics*. PCI ’21. Volos, Greece: Association for Computing Machinery, 2022, pp. 300–305. ISBN: 9781450395557. DOI: [10.1145/3503823.3503879](https://doi.org/10.1145/3503823.3503879).
- [AR19] Andreas Abel and Jan Reineke. ‘uops.info: Characterizing Latency, Throughput, and Port Usage of Instructions on Intel Microarchitectures’. In: *ASPLOS*. ASPLOS ’19. Providence, RI, USA: ACM, 2019, pp. 673–686. ISBN: 978-1-4503-6240-5. DOI: [10.1145/3297858.3304062](https://doi.org/10.1145/3297858.3304062).
- [AR20] Andreas Abel and Jan Reineke. ‘nanoBench: A Low-Overhead Tool for Running Microbenchmarks on x86 Systems’. In: *2020 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. IEEE, 2020. DOI: [10.1109/ispass48437.2020.00014](https://doi.org/10.1109/ispass48437.2020.00014).
- [AR22] Andreas Abel and Jan Reineke. ‘uiCA: Accurate Throughput Prediction of Basic Blocks on Recent Intel Microarchitectures’. In: *ICS ’22: 2022 International Conference on Supercomputing, Virtual Event, USA, June 27-30, 2022*. Ed. by Lawrence Rauchwerger, Kirk Cameron, Dimitrios S. Nikolopoulos and Dionisios Pnevmatikatos. ICS ’22. ACM, 2022, pp. 1–12. DOI: [10.1145/3524059.3532396](https://doi.org/10.1145/3524059.3532396).
- [Ass04] Motor Industry Software Reliability Association. *MISRA-C: 2004: Guidelines for the use of the C Language in Critical Systems*. MIRA, 2004.

- [ASU86] Alfred V. Aho, Ravi Sethi and Jeffrey D. Ullman. *Compilers: Principles, Techniques, and Tools*. Addison-Wesley series in computer science / World student series edition. Addison-Wesley, 1986.
- [AW03] Alaa R Alameldeen and David A Wood. ‘Variability in architectural simulations of multi-threaded workloads’. In: *HPCA*. 2003, pp. 7–18. DOI: [10.1109/hpca.2003.1183520](https://doi.org/10.1109/hpca.2003.1183520).
- [Aza+19] Reza Azarderakhsh, Matthew Campagna, Craig Costello, Luca De Feo, Basil Hess, Amir Jalali, Brian Koziel, Brian LaMacchia, Patrick Longa, Michael Naehrig, Geovandro Pereira, Joost Renes, Vladimir Soukharev and David Urbanik. *Supersingular Isogeny Key Encapsulation – Submission to the NIST Post-Quantum Standardization Project, round 2*. 2019. URL: <https://sike.org>.
- [BA06] Sorav Bansal and Alex Aiken. ‘Automatic generation of peephole superoptimizers’. In: *ASPLOS*. 2006, pp. 394–403. DOI: [10.1145/1168918.1168906](https://doi.org/10.1145/1168918.1168906).
- [Bal+12] Bharathan Balaji, John McCullough, Rajesh K. Gupta and Yuvraj Agarwal. ‘Accurate Characterization of the Variability in Power Consumption in Modern Mobile Processors’. In: *HotPower*. USENIX Association, 2012.
- [BAW20] Mahmoud A. Bokhari, Brad Alexander and Markus Wagner. ‘Towards rigorous validation of energy optimisation experiments’. In: *GECCO*. 2020, pp. 1232–1240. DOI: [10.1145/3377930.3390245](https://doi.org/10.1145/3377930.3390245).
- [BCL14] Daniel J. Bernstein, Chitchanok Chuengsatiansup and Tanja Lange. ‘Curve41417: Karatsuba Revisited’. In: *CHES*. 2014, pp. 316–334. DOI: [10.1007/978-3-662-44709-3_18](https://doi.org/10.1007/978-3-662-44709-3_18).
- [BCS13] Daniel J. Bernstein, Tung Chou and Peter Schwabe. ‘McBits: Fast Constant-Time Code-Based Cryptography’. In: *CHES*. Vol. 8086. 2013, pp. 250–272. DOI: [10.1007/978-3-642-40349-1_15](https://doi.org/10.1007/978-3-642-40349-1_15).
- [Ber+14] Daniel J. Bernstein, Chitchanok Chuengsatiansup, Tanja Lange and Peter Schwabe. ‘Kummer Strikes Back: New DH Speed Records’. In: *ASIACRYPT*. 2014, pp. 317–337. DOI: [10.1007/978-3-662-45611-8_17](https://doi.org/10.1007/978-3-662-45611-8_17).
- [Ber05a] Daniel J. Bernstein. ‘Have any challenges for qasm?’ In: *FSE*. Workshop. 2005. URL: <https://cr.yp.to/talks.html%5C#2005.02.21-2>.

- [Ber05b] Daniel J. Bernstein. ‘The Poly1305-AES Message-Authentication Code’. In: *FSE*. 2005, pp. 32–49. DOI: [10.1007/11502760_3](https://doi.org/10.1007/11502760_3).
- [Ber06] Daniel J. Bernstein. ‘Curve25519: New Diffie-Hellman Speed Records’. In: *Public Key Cryptography*. Vol. 3958. Lecture Notes in Computer Science. Springer, 2006, pp. 207–228. DOI: [10.1007/11745853_14](https://doi.org/10.1007/11745853_14).
- [Ber07] Daniel J. Bernstein. *Writing high-speed software*. <https://cr.yp.to/qhasm.html>. 2007.
- [Beu+23] Ward Beullens, Ming-Shing Chen, Shih-Hao Hung, Matthias J. Kannwischer, Bo-Yuan Peng, Cheng-Jhih Shih and Bo-Yin Yang. ‘Oil and Vinegar: Modern Parameters and Implementations’. In: *IACR Trans. Cryptogr. Hardw. Embed. Syst.* 2023.3 (2023), pp. 321–365. DOI: [10.46586/tches.v2023.i3.321-365](https://doi.org/10.46586/tches.v2023.i3.321-365).
- [BL22] Daniel J. Bernstein and Tanja Lange. *eBACS: ECRYPT benchmarking of cryptographic systems*. 2022. URL: <http://bench.cr.yp.to/supercop/supercop-20221005.tar.xz>.
- [BLB10] Robert Brummayer, Florian Lonsing and Armin Biere. ‘Automated Testing and Debugging of SAT and QBF Solvers’. In: *SAT*. Vol. 6175. Lecture Notes in Computer Science. Springer, 2010, pp. 44–57. DOI: [10.1007/978-3-642-14186-7_6](https://doi.org/10.1007/978-3-642-14186-7_6).
- [BLS01] Dan Boneh, Ben Lynn and Hovav Shacham. ‘Short Signatures from the Weil Pairing’. In: *ASIACRYPT*. Vol. 2248. Lecture Notes in Computer Science. Springer, 2001, pp. 514–532. DOI: [10.1007/s00145-004-0314-9](https://doi.org/10.1007/s00145-004-0314-9).
- [BM58] G. E. P. Box and Mervin E. Muller. ‘A Note on the Generation of Random Normal Deviates’. In: *The Annals of Mathematical Statistics* 29.2 (1958), pp. 610–611. DOI: [10.1214/aoms/1177706645](https://doi.org/10.1214/aoms/1177706645).
- [Bok+19] Mahmoud A Bokhari, Lujun Weng, Markus Wagner and Bradley Alexander. ‘Mind the gap - a distributed framework for enabling energy optimisation on modern smart-phones in the presence of noise, drift, and statistical insignificance’. In: *IEEE CEC*. 2019, pp. 1330–1337. DOI: [10.1109/CEC.2019.8790246](https://doi.org/10.1109/CEC.2019.8790246).
- [Bor23] BoringSSL. *BoringSSL*. 2023. URL: <https://boringssl.googlesource.com/boringssl>.

- [Bos+16] Joppe W. Bos, Craig Costello, Hüseyin Hisil and Kristin E. Lauter. ‘Fast Cryptography in Genus 2’. In: *J. Cryptol.* 29.1 (2016), pp. 28–60. DOI: [10.1007/s00145-014-9188-7](https://doi.org/10.1007/s00145-014-9188-7).
- [Bos+20] Jay Bosamiya, Sydney Gibson, Yao Li, Bryan Parno and Chris Hawblitzel. ‘Verified Transformations and Hoare Logic: Beautiful Proofs for Ugly Assembly Language’. In: *VSTTE*. 2020, pp. 106–123. DOI: [10.1007/978-3-030-63618-0_7](https://doi.org/10.1007/978-3-030-63618-0_7).
- [Bou+06] Florent Bouchez, Alain Darte, Christophe Guillon and Fabrice Rastello. ‘Register Allocation: What Does the NP-Completeness Proof of Chaitin et al. Really Prove? Or Revisiting Register Allocation: Why and How’. In: *LCPC*. Vol. 4382. Lecture Notes in Computer Science. Springer, 2006, pp. 283–298. DOI: [10.1007/978-3-540-72521-3_21](https://doi.org/10.1007/978-3-540-72521-3_21).
- [BS12] Daniel J. Bernstein and Peter Schwabe. ‘NEON Crypto’. In: *CHES*. Vol. 7428. Lecture Notes in Computer Science. Springer, 2012, pp. 320–339. DOI: [10.1007/978-3-642-33027-8_19](https://doi.org/10.1007/978-3-642-33027-8_19).
- [BSS15] Prathibha A. Ballal, H. Sarojadevi and Harsha P. S. ‘Compiler Optimization: A Genetic Algorithm Approach’. In: *International Journal of Computer Applications* 112 (2015), pp. 9–13.
- [BT93] Dimitris Bertsimas and John Tsitsiklis. ‘Simulated Annealing’. In: *Statistical Science* 8.1 (1993), pp. 10–15. ISSN: 08834237. (Visited on 08/10/2024).
- [But14] Vitalik Buterin. *Ethereum: A Next-Generation Smart Contract and Decentralized Application Platform*. 2014. URL: https://ethereum.org/669c9e2e2027310b6b3cdce6e1c52962/Ethereum%20%5C_%20Whitepaper%5C_-%5C_Buterin%5C_2014.pdf.
- [Cal+97] Brad Calder, Dirk Grunwald, Michael P. Jones, Donald C. Lindsay, James H. Martin, Michael Mozer and Benjamin G. Zorn. ‘Evidence-Based Static Branch Prediction Using Machine Learning’. In: *ACM Trans. Program. Lang. Syst.* 19.1 (1997), pp. 188–222. DOI: [10.1145/239912.239923](https://doi.org/10.1145/239912.239923).
- [CB13] Charlie Curtsinger and Emery D. Berger. ‘STABILIZER: statistically sound performance evaluation’. In: *ASPLOS*. 2013, pp. 219–228. DOI: [10.1145/2499368.2451141](https://doi.org/10.1145/2499368.2451141).

- [CD23] Wouter Castryck and Thomas Decru. ‘An Efficient Key Recovery Attack on SIDH’. In: *EUROCRYPT (5)*. Vol. 14008. Lecture Notes in Computer Science. Springer, 2023, pp. 423–447. DOI: [10.1007/978-3-031-30589-4_16](https://doi.org/10.1007/978-3-031-30589-4_16).
- [Cer00] Certicom Research. *SEC 2: Recommended elliptic curve domain parameters, version 1.0*. 2000. URL: <http://www.secg.org/SEC2-Ver-1.0.pdf>.
- [Cha82] Gregory J Chaitin. ‘Register allocation & spilling via graph coloring’. In: *ACM Sigplan Notices* 17.6 (1982), pp. 98–101. DOI: [10.1145/800230.806984](https://doi.org/10.1145/800230.806984).
- [Che+14] Yu-Fang Chen, Chang-Hong Hsu, Hsin-Hung Lin, Peter Schwabe, Ming-Hsien Tsai, Bow-Yaw Wang, Bo-Yin Yang and Shang-Yi Yang. ‘Verifying Curve25519 Software’. In: *CCS*. 2014, pp. 299–309. DOI: [10.1145/2660267.2660370](https://doi.org/10.1145/2660267.2660370).
- [Cho15] Tung Chou. ‘Sandy2x: New Curve25519 Speed Records’. In: *SAC*. 2015, pp. 145–160. DOI: [10.1007/978-3-319-31301-6_8](https://doi.org/10.1007/978-3-319-31301-6_8).
- [Chu+21] Chi-Ming Marvin Chung, Vincent Hwang, Matthias J. Kannwischer, Gregor Seiler, Cheng-Jhih Shih and Bo-Yin Yang. ‘NTT Multiplication for NTT-unfriendly Rings New Speed Records for Saber and NTRU on Cortex-M4 and AVX2’. In: *IACR Trans. Cryptogr. Hardw. Embed. Syst.* 2021.2 (2021), pp. 159–188. DOI: [10.46586/tches.v2021.i2.159-188](https://doi.org/10.46586/tches.v2021.i2.159-188).
- [Cla23] Clang. *Clang: a C language family frontend for LLVM*. 2023. URL: <https://clang.llvm.org>.
- [CSS99] Keith D. Cooper, Philip J. Schielke and Devika Subramanian. ‘Optimizing for Reduced Code Space using Genetic Algorithms’. In: *Workshop on Languages, Compilers, and Tools for Embedded Systems*. ACM, 1999, pp. 1–9. DOI: [10.1145/315253.314414](https://doi.org/10.1145/315253.314414).
- [de 13] Joost de Winter. ‘Using the Student’s t-test with extremely small sample sizes’. English. In: *Practical Assessment, Research & Evaluation* 18.10 (2013). ISSN: 1531-7714.
- [DGF22] Sushant Dinesh, Grant Garrett-Grossman and Christopher W. Fletcher. ‘SynthCT: Towards Portable Constant-Time Code’. In: *NDSS*. The Internet Society, 2022. DOI: [10.14722/ndss.2022.24215](https://doi.org/10.14722/ndss.2022.24215).

- [DH76] Whitfield Diffie and Martin E. Hellman. ‘New directions in cryptography’. In: *IEEE Trans. Inf. Theory* 22.6 (1976), pp. 644–654. DOI: [10.1109/TIT.1976.1055638](https://doi.org/10.1109/TIT.1976.1055638).
- [DN19] Benjamin Doerr and Frank Neumann. *Theory of evolutionary computation: Recent developments in discrete optimization*. Springer, 2019. DOI: [10.1007/978-3-030-29414-4](https://doi.org/10.1007/978-3-030-29414-4).
- [DR98] Joan Daemen and Vincent Rijmen. ‘The Block Cipher Rijndael’. In: *Smart Card Research and Applications, This International Conference, CARDIS ’98, Louvain-la-Neuve, Belgium, September 14-16, 1998, Proceedings*. Ed. by Jean-Jacques Quisquater and Bruce Schneier. Vol. 1820. Lecture Notes in Computer Science. Springer, 1998, pp. 277–284. DOI: [10.1007/10721064_26](https://doi.org/10.1007/10721064_26).
- [Dwo+01] Morris Dworkin, Elaine Barker, James Nechvatal, James Foti, Lawrence Bassham, E. Roback and James Dray. *Advanced Encryption Standard (AES)*. en. 2001. DOI: [10.6028/NIST.FIPS.197](https://doi.org/10.6028/NIST.FIPS.197).
- [Eis+22] Simon Eismann, Diego Costa, Lizhi Liao, Cor-Paul Bezemer, Weiyi Shang, André van Hoorn and Samuel Kounev. ‘A case study on the stability of performance tests for serverless applications’. In: *Journal of Systems and Software* 189 (2022), p. 111294. DOI: [10.1016/j.jss.2022.111294](https://doi.org/10.1016/j.jss.2022.111294).
- [Erb+19] Andres Erbsen, Jade Philipoom, Jason Gross, Robert Sloan and Adam Chlipala. ‘Simple High-Level Code for Cryptographic Arithmetic - With Proofs, Without Compromises’. In: *IEEE SP*. 2019, pp. 1202–1219. DOI: [10.1109/sp.2019.00005](https://doi.org/10.1109/sp.2019.00005).
- [Erb23a] Andres Erbsen. *BoringSSL Curve25519 Patch*. 2023. URL: <https://boringssl.googlesource.com/boringssl/+/-/9d4f833eec8205e7ad257fb7e7cb321270d3e3cb>.
- [Erb23b] Andres Erbsen. ‘Foundational Integration Verification of Diverse Software and Hardware Components’. PhD thesis. Massachusetts Institute of Technology, 2023. URL: <https://dspace.mit.edu/bitstream/handle/1721.1/150216/Erbсен-andreser-PhD-EECS-2023-thesis.pdf?sequence=1%20%5C&isAllowed=y>.
- [ES15] A. E. Eiben and James E. Smith. *Introduction to Evolutionary Computing, Second Edition*. Natural Computing Series. Springer, 2015.

- [FH15] Armando Faz-Hernández and Julio César López- Hernández. ‘Fast Implementation of Curve25519 Using AVX2’. In: *LATINCRYPT*. Vol. 9230. Lecture Notes in Computer Science. Springer, 2015, pp. 329–345. DOI: [10.1007/978-3-319-22174-8_18](https://doi.org/10.1007/978-3-319-22174-8_18).
- [FLS14] Armando Faz-Hernández, Patrick Longa and Ana H. Sánchez. ‘Efficient and Secure Algorithms for GLV-Based Scalar Multiplication and Their Implementation on GLV-GLS Curves’. In: *CT-RSA*. Vol. 8366. Lecture Notes in Computer Science. Springer, 2014, pp. 1–27. DOI: [10.1007/978-3-319-04852-9_1](https://doi.org/10.1007/978-3-319-04852-9_1).
- [FM14] Matteo Fischetti and Michele Monaci. ‘Exploiting Erraticism in Search’. In: *Operations Research* 62.1 (2014), pp. 114–122. DOI: [10.1287/opre.2013.1231](https://doi.org/10.1287/opre.2013.1231).
- [Fu+19] Yu-Fu Fu, Jiaxiang Liu, Xiaomu Shi, Ming-Hsien Tsai, Bow-Yaw Wang and Bo-Yin Yang. ‘Signed Cryptographic Program Verification with Typed CryptoLine’. In: *CCS*. ACM, 2019, pp. 1591–1606. DOI: [10.1145/3319535.3354199](https://doi.org/10.1145/3319535.3354199).
- [FY53] R.A. Fisher and F. Yates. *Statistical Tables for Biological, Agricultural, and Medical Research*. Hafner Publishing Company, 1953. URL: <https://books.google.com.au/books?id=Kzu11Ri47ykC>.
- [GCC23] GCC. *GCC, the GNU Compiler Collection*. 2023. URL: <https://gcc.gnu.org>.
- [Ge+18] Qian Ge, Yuval Yarom, David Cock and Gernot Heiser. ‘A survey of microarchitectural timing attacks and countermeasures on contemporary hardware’. In: *J. Cryptogr. Eng.* 8.1 (2018), pp. 1–27. DOI: [10.1007/s13389-016-0141-6](https://doi.org/10.1007/s13389-016-0141-6).
- [GG16] Steven D. Galbraith and Pierrick Gaudry. ‘Recent progress on the elliptic curve discrete logarithm problem’. In: *Designs, Codes and Cryptography* 78.1 (2016), pp. 51–72. DOI: [10.1007/s10623-015-0146-7](https://doi.org/10.1007/s10623-015-0146-7).
- [Gos08] William Sealy Gosset. ‘The Probable Error of a Mean’. In: *Biometrika* (1908). DOI: [10.2307/2331554](https://doi.org/10.2307/2331554).
- [Gue12] Shay Gueron. ‘Efficient software implementations of modular exponentiation’. In: *J. Cryptogr. Eng.* 2.1 (2012), pp. 31–43. DOI: [10.1007/s13389-012-0031-5](https://doi.org/10.1007/s13389-012-0031-5).

- [GW01] Gary William Grewal and Thomas Charles Wilson. ‘Mapping reference code to irregular DSPs within the retargetable, optimizing compiler COGEN(T)’. In: *MICRO*. ACM/IEEE Computer Society, 2001, pp. 192–202. DOI: [10.1109/micro.2001.991118](https://doi.org/10.1109/micro.2001.991118).
- [Ham15] Mike Hamburg. ‘Ed448-Goldilocks, a new elliptic curve’. In: *IACR Cryptology ePrint Archive 2015* (2015), p. 625. URL: <http://eprint.iacr.org/2015/625>.
- [HE13] Rodney E. Hooker and Collin Eddy. *Store-to-load forwarding based on load/store address computation source information comparisons*. US Patent 8533438. 2013.
- [Hel02] M.E. Hellman. ‘An overview of public key cryptography’. In: *IEEE Communications Magazine* 40.5 (2002), pp. 42–49. DOI: [10.1109/MCOM.2002.1006971](https://doi.org/10.1109/MCOM.2002.1006971).
- [HM11] Darrel Hankerson and Alfred Menezes. ‘Elliptic Curve Discrete Logarithm Problem’. In: *Encyclopedia of Cryptography and Security*. Ed. by Henk C. A. van Tilborg and Sushil Jajodia. Boston, MA: Springer US, 2011, pp. 397–400. ISBN: 978-1-4419-5906-5. DOI: [10.1007/978-1-4419-5906-5_246](https://doi.org/10.1007/978-1-4419-5906-5_246).
- [Hol06] G.J. Holzmann. ‘The power of 10: rules for developing safety-critical code’. In: *Computer* 39.6 (2006), pp. 95–99. DOI: [10.1109/MC.2006.212](https://doi.org/10.1109/MC.2006.212).
- [Hwa+22] Vincent Hwang, Jiaxiang Liu, Gregor Seiler, Xiaomu Shi, Ming-Hsien Tsai, Bow-Yaw Wang and Bo-Yin Yang. ‘Verified NTT Multiplications for NISTPQC KEM Lattice Finalists: Kyber, SABER, and NTRU’. In: *IACR Transactions on Cryptographic Hardware and Embedded Systems* 2022.4 (2022), pp. 718–750. DOI: [10.46586/tches.v2022.i4.718-750](https://doi.org/10.46586/tches.v2022.i4.718-750).
- [Int20a] Intel Corporation. *Intel 64 and IA-32 Architectures Software Developer’s Manual - Volume 3B*. 2020. URL: <https://software.intel.com/content/dam/develop/external/us/en/documents-tps/325384-sdm-vol-3abcd.pdf>.
- [Int20b] Intel Corporation. *Intel® 64 and IA-32 Architectures Optimization Reference Manual: Volume 1*. 2020. URL: <https://cdrdv2.intel.com/v1/dl/getContent/671488>.

- [Jan+22] Jan Jancar, Marcel Fourné, Daniel De Almeida Braga, Mohamed Sabt, Peter Schwabe, Gilles Barthe, Pierre-Alain Fouque and Yasemin Acar. "They're not that hard to mitigate": What Cryptographic Library Developers Think About Timing Attacks'. In: *IEEE Symposium on Security and Privacy*. IEEE, 2022, pp. 632–649. DOI: [10.1109/SP46214.2022.9833713](https://doi.org/10.1109/SP46214.2022.9833713).
- [JMV01] Don Johnson, Alfred Menezes and Scott Vanstone. 'The Elliptic Curve Digital Signature Algorithm (ECDSA)'. In: *Int. J. Inf. Secur.* 1.1 (Aug. 2001), pp. 36–63. ISSN: 1615-5262. DOI: [10.1007/s102070100002](https://doi.org/10.1007/s102070100002). URL: <https://doi.org/10.1007/s102070100002>.
- [JNR02] Rajeev Joshi, Greg Nelson and Keith H. Randall. 'Denali: A Goal-directed Superoptimizer'. In: *PLDI*. ACM, 2002, pp. 304–314. DOI: [10.1145/543552.512566](https://doi.org/10.1145/543552.512566).
- [Kac09] Marc Kachelriess. 'Branchless vectorized median filtering'. In: *2009 IEEE Nuclear Science Symposium Conference Record (NSS/MIC)*. 2009, pp. 4099–4105. DOI: [10.1109/NSSMIC.2009.5402362](https://doi.org/10.1109/NSSMIC.2009.5402362).
- [Kah62] Arthur B. Kahn. 'Topological sorting of large networks'. In: *Commun. ACM* 5.11 (1962), pp. 558–562. DOI: [10.1145/368996.369025](https://doi.org/10.1145/368996.369025).
- [Kah66] David Kahn. *The Codebreakers: The Story of Secret Writing*. Weidenfeld and Nicolson, 1966. DOI: [10.2307/1853680](https://doi.org/10.2307/1853680).
- [KJ13] Tomas Kalibera and Richard Jones. 'Rigorous benchmarking in reasonable time'. In: *ACM SIGPLAN Notices* 48.11 (2013), pp. 63–74. DOI: [10.1145/2464157.2464160](https://doi.org/10.1145/2464157.2464160).
- [Kob87] Neal Koblitz. 'Elliptic curve cryptosystems'. In: *Mathematics of Computation* 48 (1987), pp. 203–209. URL: <https://api.semanticscholar.org/CorpusID:14288427>.
- [Koc+19] Paul Kocher, Jann Horn, Anders Fogh, Daniel Genkin, Daniel Gruss, Werner Haas, Mike Hamburg, Moritz Lipp, Stefan Mangard, Thomas Prescher, Michael Schwarz and Yuval Yarom. 'Spectre Attacks: Exploiting Speculative Execution'. In: *IEEE Symposium on Security and Privacy*. 2019. DOI: [10.1109/sp.2019.00002](https://doi.org/10.1109/sp.2019.00002).

- [Kue+23a] Joel Kuepper, Andres Erbsen, Jason Gross, Owen Conoly, Chuyue Sun, Samuel Tian, David Wu, Adam Chlipala, Chitchanok Chuengsatiansup, Daniel Genkin, Markus Wagner and Yuval Yarom. *CryptOpt: Automatic Optimization of Straightline Code*. 2023. DOI: [10.1109/ICSE-Companion58688.2023.00042](https://doi.org/10.1109/ICSE-Companion58688.2023.00042).
- [Kue+23b] Joel Kuepper, Andres Erbsen, Jason Gross, Owen Conoly, Chuyue Sun, Samuel Tian, David Wu, Adam Chlipala, Chitchanok Chuengsatiansup, Daniel Genkin, Markus Wagner and Yuval Yarom. ‘CryptOpt: Verified Compilation with Randomized Program Search for Cryptographic Primitives’. In: *PLDI*. Vol. 7. PLDI. 2023, pp. 1268–1292. DOI: [10.1145/3591272](https://doi.org/10.1145/3591272).
- [Kue+23c] Joel Kuepper, Andres Erbsen, Jason Gross, Owen Conoly, Chuyue Sun, Samuel Tian, David Wu, Adam Chlipala, Chitchanok Chuengsatiansup, Daniel Genkin, Markus Wagner and Yuval Yarom. *CryptOpt: Verified Compilation with Randomized Program Search for Cryptographic Primitives (full version)*. 2023. arXiv: [2211.10665](https://arxiv.org/abs/2211.10665) [cs.CR].
- [Kuk15] Jim Kukunas. *Power and performance: Software analysis and optimization*. Morgan Kaufmann, 2015.
- [Lan10] Adam Langley. *Checking that functions are constant time with Valgrind*. accessed 27.06.2023. 2010. URL: <https://www.imperialviolet.org/2010/04/01/ctgrind.html>.
- [Lan15] William B. Langdon. ‘Genetically Improved Software’. In: *Handbook of Genetic Programming Applications*. Ed. by Amir H. Gandomi, Amir H. Alavi and Conor Ryan. Cham: Springer International Publishing, 2015, pp. 181–220. ISBN: 978-3-319-20883-1. DOI: [10.1007/978-3-319-20883-1_8](https://doi.org/10.1007/978-3-319-20883-1_8). URL: https://doi.org/10.1007/978-3-319-20883-1_8.
- [Lan23a] William B. Langdon. *Genetic Programming Library*. 2023. URL: <http://gpbib.cs.ucl.ac.uk/gp-bibliography.bib>.
- [Lan23b] Adam Langley. *Curve25519-donna*. 2023. URL: <https://github.com/agl/curve25519-donna>.
- [LH15] William B. Langdon and Mark Harman. ‘Optimizing Existing Software With Genetic Programming’. In: *IEEE Transactions on Evolutionary Computation* 19.1 (2015), pp. 118–135. DOI: [10.1109/TEVC.2013.2281544](https://doi.org/10.1109/TEVC.2013.2281544).

- [Li+22] Shuyue Stella Li, Hannah Peeler, Andrew N. Sloss, Kenneth N. Reid and Wolfgang Banzhaf. ‘Genetic improvement in the shackleton framework for optimizing LLVM pass sequences’. In: *GECCO Companion*. ACM, 2022, pp. 1938–1939. DOI: [10.1145/3520304.3534000](https://doi.org/10.1145/3520304.3534000).
- [lib23] libsecp256k1. *libsecp256k1-Optimized C library for EC operations on curve secp256k1*. 2023. URL: <https://github.com/bitcoin-core/secp256k1>.
- [Lin+23] Kaizhan Lin, Weize Wang, Zheng Xu and Chang-An Zhao. *A Faster Software Implementation of SQISign*. Cryptology ePrint Archive, Paper 2023/753. 2023. URL: <https://eprint.iacr.org/2023/753>.
- [Lip+18] Moritz Lipp, Michael Schwarz, Daniel Gruss, Thomas Prescher, Werner Haas, Anders Fogh, Jann Horn, Stefan Mangard, Paul Kocher, Daniel Genkin, Yuval Yarom and Mike Hamburg. ‘Melt-down: Reading Kernel Memory from User Space’. In: *USENIX Security*. 2018. DOI: [10.1145/3357033](https://doi.org/10.1145/3357033).
- [LL00] Kevin M. Lepak and Mikko H. Lipasti. ‘On the value locality of store instructions’. In: *ISCA*. 2000, pp. 182–191. DOI: [10.1145/339647.339678](https://doi.org/10.1145/339647.339678).
- [LL19] Geoff Langdale and Daniel Lemire. ‘Parsing gigabytes of JSON per second’. In: *The VLDB Journal* 28.6 (2019), pp. 941–960. DOI: [10.1007/s00778-019-00578-5](https://doi.org/10.1007/s00778-019-00578-5).
- [LLV23] LLVM. *X86CmovConversion.cpp*. accessed 27.06.2023. 2023. URL: <https://github.com/llvm/llvm-project/blob/0c476111317cb7aaa9a3e9f75e1c35f83122ee26/llvm/lib/Target/X86/X86CmovConversion.cpp>.
- [Lu+18] Hong J. Lu, Michael Matz, Milind Girkar, Jan Hubička, Andreas Jaeger and Mark Mitchell. *System V Application Binary Interface: AMD64 Architecture Processor Supplement (With LP64 and ILP32 Programming Models)*. 2018. URL: <https://raw.githubusercontent.com/wiki/hjl-tools/x86-psABI/x86-64-psABI-1.0.pdf>.
- [LVO17] William B. Langdon, Nadarajen Veerapen and Gabriela Ochoa. ‘Visualising the Search Landscape of the Triangle Program’. In: *EuroGP 2017*. Ed. by Mauro Castelli, James McDermott and

- Lukas Sekanina. Vol. 10196. LNCS. Amsterdam: Springer, 2017, pp. 96–113. DOI: [doi:10.1007/978-3-319-55696-3_7](https://doi.org/10.1007/978-3-319-55696-3_7). URL: http://www.cs.ucl.ac.uk/staff/W.Langdon/ftp/papers/langdon%5C_2017%20%5C_%20EuroGP.pdf.
- [Mas87] Henry Massalin. ‘Superoptimizer - A Look at the Smallest Program’. In: *ASPLOS*. ACM Press, 1987, pp. 122–126. DOI: [10.1145/36206.36194](https://doi.org/10.1145/36206.36194).
- [MI04] Prasad S. Mahajan and Ricki G. Ingalls. ‘Evaluation of Methods Used to Detect Warm-Up Period in Steady State Simulation’. In: *WSC*. IEEE Computer Society, 2004, pp. 663–671. DOI: [10.1109/wsc.2004.1371374](https://doi.org/10.1109/wsc.2004.1371374).
- [Mil85] Victor S. Miller. ‘Use of Elliptic Curves in Cryptography’. In: *CRYPTO*. Vol. 218. Lecture Notes in Computer Science. Springer, 1985, pp. 417–426. DOI: [10.1007/3-540-39799-X_31](https://doi.org/10.1007/3-540-39799-X_31).
- [MIT13] MITRE. *CVE-2014-0160*. Available from MITRE, CVE-ID CVE-2014-0160. 2013. URL: <http://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2014-0160>.
- [MIT14] MITRE. *CVE-2014-1266*. Available from MITRE, CVE-ID CVE-2014-1266. 2014. URL: <http://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2014-1266>.
- [MOV96] Alfred Menezes, Paul C. van Oorschot and Scott A. Vanstone. *Handbook of Applied Cryptography*. CRC Press, 1996. DOI: [10.1201/9780429466335](https://doi.org/10.1201/9780429466335).
- [Moz23] Mozilla. *Network Security Services NSS*. 2023. URL: <https://wiki.mozilla.org/NSS>.
- [Myt+09] Todd Mytkowicz, Amer Diwan, Matthias Hauswirth and Peter F Sweeney. ‘Producing wrong data without doing anything obviously wrong!’ In: *ACM SIGARCH Computer Architecture News* 37.1 (2009), pp. 265–276. DOI: [10.1145/1508284.1508275](https://doi.org/10.1145/1508284.1508275).
- [Nak08] Satoshi Nakamoto. *Bitcoin: A Peer-to-Peer Electronic Cash System*. 2008. URL: <https://bitcoin.org/bitcoin.pdf>.
- [NIS00] NIST. *FIPS PUB 186-2: Digital signature standard*. 2000. URL: <https://csrc.nist.gov/publications/fips/archive/fips186-2/fips186-2.pdf>.

- [NIS15] NIST. *FIPS PUB 202: SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions*. 2015. URL: <https://csrc.nist.gov/pubs/fips/202/final>.
- [NIS22] NIST. ‘Chapter 0: Foreword and postscript’. In: (2022). URL: <https://csrc.nist.gov/csrc/media/Projects/post-quantum-cryptography/documents/round-4/submissions/sike-team-note-insecure.pdf>.
- [NIS94] NIST. *FIPS PUB 186: Digital Signature Standard (DSS)*. 1994. URL: <https://web.archive.org/web/20131213131144/http://www.itl.nist.gov/fipspubs/fip186.htm>.
- [NL18] Yoav Nir and Adam Langley. *ChaCha20 and Poly1305 for IETF Protocols*. RFC 8439. 2018. DOI: [10.17487/RFC8439](https://doi.org/10.17487/RFC8439). URL: <https://www.rfc-editor.org/info/rfc8439>.
- [Ope17] OpenSSL. *OpenSSL ec/curve25519.c: "double" ecdhx25519 performance on 64-bit platforms*. 2017. URL: <https://github.com/openssl/openssl/commit/cfc32a1efb464205885e18e503bcb7051c307008>.
- [Ope23a] OpenSSL. *OpenSSL*. 2023. URL: <https://www.openssl.org/>.
- [Ope23b] OpenSSL. *OpenSSL ec/curve25519.c: Curve25519 prefers 64-bit code for Curve25519*. 2023. URL: <https://github.com/openssl/openssl/blob/c499cbc3239e3ac93fa5acf85cec7ea7df116518/crypto/ec/curve25519.c#L705-L707>.
- [pag32] Linux manual page. *perf_event_open - set up performance monitoring*. accessed 15.09.2023. 2032. URL: https://man7.org/linux/man-pages/man2/perf_event_open.2.html.
- [Pan+19] Maksim Panchenko, Rafael Auler, Bill Nell and Guilherme Ottoni. ‘BOLT: A Practical Binary Optimizer for Data Centers and Beyond’. In: *CGO*. IEEE, 2019, pp. 2–14. DOI: [10.1109/cgo.2019.8661201](https://doi.org/10.1109/cgo.2019.8661201).
- [Pat+00] Sanjay J Patel, Tony Tung, Satarupa Bose and Matthew M Crum. ‘Increasing the size of atomic instruction blocks using control flow assertions’. In: *Proceedings of the 33rd annual ACM/IEEE international symposium on Microarchitecture*. 2000, pp. 303–313. DOI: [10.1109/micro.2000.898080](https://doi.org/10.1109/micro.2000.898080).

- [PB10] Gennady Pekhimenko and Angela Demke Brown. ‘Efficient Program Compilation Through Machine Learning Techniques’. In: *Software Automatic Tuning, From Concepts to State-of-the-Art Results*. Springer, 2010, pp. 335–351. DOI: [10.1007/978-1-4419-6935-4_19](https://doi.org/10.1007/978-1-4419-6935-4_19).
- [Pee+22] Hannah Peeler, Shuyue Stella Li, Andrew N. Sloss, Kenneth N. Reid, Yuan Yuan and Wolfgang Banzhaf. ‘Optimizing LLVM Pass Sequences with Shackleton: A Linear Genetic Programming Framework’. In: *GECCO Companion*. ACM, 2022, pp. 578–581. DOI: [10.1145/3520304.3528945](https://doi.org/10.1145/3520304.3528945).
- [PGB12] Kishore Kumar Pusukuri, Rajiv Gupta and Laxmi N Bhuyan. ‘Thread tranquilizer: Dynamically reducing performance variation’. In: *ACM TACO 8.4* (2012), p. 46. DOI: [10.1145/2086696.2086725](https://doi.org/10.1145/2086696.2086725).
- [Pol+18] Andy Polyakov, Ming-Hsien Tsai, Bow-Yaw Wang and Bo- Yin Yang. ‘Verifying Arithmetic Assembly Programs in Cryptographic Primitives (Invited Talk)’. In: *CONCUR*. Vol. 118. LIPIcs. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2018, 4:1–4:16.
- [PSS98] Amir Pnueli, Michael Siegel and Eli Singerman. ‘Translation Validation’. In: *TACAS*. 1998, pp. 151–166. DOI: [10.1145/2499370.2462183](https://doi.org/10.1145/2499370.2462183).
- [PW34] Raymond E. A. C. Paley and Norbert Wiener. *Fourier Transforms in the Complex Domain*. §37. New York: American Mathematical Society, 1934. DOI: [10.1090/coll/019](https://doi.org/10.1090/coll/019).
- [RBV17] Oscar Reparaz, Josep Balasch and Ingrid Verbauwhede. ‘Dude, is my code constant time?’ In: *DATE*. IEEE, 2017, pp. 1697–1702. DOI: [10.23919/date.2017.7927267](https://doi.org/10.23919/date.2017.7927267).
- [RCA99] Glenn Reinman, Brad Calder and Todd M. Austin. ‘Fetch Directed Instruction Prefetching’. In: *MICRO*. ACM/IEEE Computer Society, 1999, pp. 16–27. DOI: [10.1109/micro.1999.809439](https://doi.org/10.1109/micro.1999.809439).
- [Ren+21] Xida Ren, Logan Moody, Mohammadkazem Taram, Matthew Jordan, Dean M. Tullsen and Ashish Venkat. ‘I See Dead μ ops: Leaking Secrets via Intel/AMD Micro-Op Caches’. In: *ISCA*. IEEE, 2021, pp. 361–374. DOI: [10.1109/ISCA52012.2021.00036](https://doi.org/10.1109/ISCA52012.2021.00036).

- [RLT15] Ashay Rane, Calvin Lin and Mohit Tiwari. ‘Raccoon: Closing Digital Side-Channels through Obfuscated Execution’. In: *USENIX Security*. USENIX Association, 2015, pp. 431–446.
- [RPH04] Ronny Ronen, Alexander Peleg and Nathaniel Hoffman. *System and method for fusing instructions*. US Patent 6675376B2. 2004.
- [RSA78] Ronald L. Rivest, Adi Shamir and Leonard M. Adleman. ‘A Method for Obtaining Digital Signatures and Public-Key Cryptosystems’. In: *Commun. ACM* 21.2 (1978), pp. 120–126. DOI: [10.21236/ada606588](#).
- [Sas+18] Raimondas Sasnauskas, Yang Chen, Peter Collingbourne, Jeroen Ketema, Gratian Lup, Jubi Taneja and John Regehr. *Souper: A Synthesizing Superoptimizer*. 2018. arXiv: [1711.04422 \[cs.PL\]](#).
- [SGA07] Michael Sutton, Adam Greene and Pedram Amini. *Fuzzing: Brute Force Vulnerability Discovery*. Addison-Wesley Professional, 2007. ISBN: 0321446119.
- [Sha+13] Rahul Sharma, Eric Schkufza, Berkeley R. Churchill and Alex Aiken. ‘Data-driven equivalence checking’. In: *OOPSLA*. ACM, 2013, pp. 391–406. DOI: [10.1145/2509136.2509509](#).
- [Sha+15] Rahul Sharma, Eric Schkufza, Berkeley R. Churchill and Alex Aiken. ‘Conditionally correct superoptimization’. In: *OOPSLA*. ACM, 2015, pp. 147–162. DOI: [10.1145/2858965.2814278](#).
- [SL06] Samantika Subramaniam and Gabriel H. Loh. ‘Fire-and-Forget: Load/Store Scheduling with No Store Queue at All’. In: *MICRO*. 2006, pp. 273–284. DOI: [10.1109/micro.2006.26](#).
- [Smi82] Alan Jay Smith. ‘Cache Memories’. In: *ACM Comput. Surv.* 14.3 (1982), pp. 473–530. ISSN: 0360-0300. DOI: [10.1145/356887.356892](#).
- [Smi89] J.E. Smith. ‘Dynamic instruction scheduling and the Astronautics ZS-1’. In: *Computer* 22.7 (1989), pp. 21–35. DOI: [10.1109/2.30730](#).
- [SMK13] Thomas Arthur Leck Sewell, Magnus O. Myreen and Gerwin Klein. ‘Translation validation for a verified OS kernel’. In: *PLDI*. 2013, pp. 471–482. DOI: [10.1145/2499370.2462183](#).
- [Sol99] Jerome A. Solinas. ‘CORR 99-39 Generalized Mersenne Numbers’. In: 1999.

- [Sri+01] Viji Srinivasan, Edward S. Davidson, Gary S. Tyson, Mark J. Charney and Thomas R. Puzak. ‘Branch History Guided Instruction Prefetching’. In: *HPCA*. IEEE Computer Society, 2001, pp. 291–300. DOI: [10.1109/hpca.2001.903271](https://doi.org/10.1109/hpca.2001.903271).
- [SSA13] Eric Schkufza, Rahul Sharma and Alex Aiken. ‘Stochastic superoptimization’. In: *ASPLOS*. ACM, 2013, pp. 305–316. DOI: [10.1145/2490301.2451150](https://doi.org/10.1145/2490301.2451150).
- [SSA14] Eric Schkufza, Rahul Sharma and Alex Aiken. ‘Stochastic optimization of floating-point programs with tunable precision’. In: *PLDI*. ACM, 2014, pp. 53–64. DOI: [10.1145/2666356.2594302](https://doi.org/10.1145/2666356.2594302).
- [Ste+03] Mark Stephenson, Una-May O’Reilly, Martin C. Martin and Saman P. Amarasinghe. ‘Genetic Programming Applied to Compiler Heuristic Optimization’. In: *EuroGP*. 2003, pp. 238–253. DOI: [10.1007/3-540-36599-0_22](https://doi.org/10.1007/3-540-36599-0_22).
- [TL08] Jean-Baptiste Tristan and Xavier Leroy. ‘Formal verification of translation validators: a case study on instruction scheduling optimizations’. In: *POPL*. 2008, pp. 17–27. DOI: [10.1145/1328897.1328444](https://doi.org/10.1145/1328897.1328444).
- [TWY17] Ming-Hsien Tsai, Bow-Yaw Wang and Bo-Yin Yang. ‘Certified Verification of Algebraic Properties on Low-Level Mathematical Constructs in Cryptographic Programs’. In: *CCS*. ACM, 2017, pp. 1973–1987. DOI: [10.1145/3133956.3134076](https://doi.org/10.1145/3133956.3134076).
- [VA87] P J M Van Laarhoven and E H L Aarts. *Simulated annealing: Theory and applications*. en. 1987th ed. Mathematics and Its Applications. Dordrecht, Netherlands: Kluwer Academic, June 1987.
- [Vil+20] Pepe Vila, Pierre Ganty, Marco Guarnieri and Boris Köpf. ‘Cache-Query: learning replacement policies from hardware caches’. In: *PLDI*. 2020, pp. 519–532. DOI: [10.1145/3385412.3386008](https://doi.org/10.1145/3385412.3386008).
- [Von45] John Von Neumann. ‘First Draft of a Report on the EDVAC’. In: (1945). DOI: [10.5479/sil.538961.39088011475779](https://doi.org/10.5479/sil.538961.39088011475779).
- [Wei+10] Westley Weimer, Stephanie Forrest, Claire Le Goues and ThanhVu Nguyen. ‘Automatic Program Repair with Evolutionary Computation’. In: *Commun. ACM* 53.5 (2010), pp. 109–116. ISSN: 0001-0782. DOI: [10.1145/1735223.1735249](https://doi.org/10.1145/1735223.1735249).

- [Wik23] WikiChip. *Skylake (client) - Microarchitectures - Intel*. 2023. URL: [https://en.wikichip.org/wiki/intel/microarchitectures/skylake%5C_%20\(client\)](https://en.wikichip.org/wiki/intel/microarchitectures/skylake%5C_%20(client)).
- [WO18] Zheng Wang and Michael O’Boyle. *Machine Learning in Compiler Optimisation*. 2018. arXiv: [1805.03441](https://arxiv.org/abs/1805.03441) [cs.PL].
- [WP78a] James Wilson and A. Pritsker. ‘A Survey of Research on the Simulation Startup Problem’. In: *Transactions of The Society for Modeling and Simulation International - SIMULATION* 31 (1978), pp. 55–58. DOI: [10.1177/003754977803100204](https://doi.org/10.1177/003754977803100204).
- [WP78b] James Wilson and A. Pritsker. ‘Evaluation of Startup Policies in Simulation Experiments’. In: *Transactions of The Society for Modeling and Simulation International - SIMULATION* 31 (1978), pp. 79–89. DOI: [10.1177/003754977803100303](https://doi.org/10.1177/003754977803100303).
- [Wu+15] Fan Wu, Westley Weimer, Mark Harman, Yue Jia and Jens Krinke. ‘Deep Parameter Optimisation’. In: *Proceedings of the 2015 Annual Conference on Genetic and Evolutionary Computation. GECCO ’15*. Madrid, Spain: Association for Computing Machinery, 2015, pp. 1375–1382. ISBN: 9781450334723. DOI: [10.1145/2739480.2754648](https://doi.org/10.1145/2739480.2754648).
- [WV21] David Warburton and Sander Vinberg. *The 2021 TLS Telemetry Report*. accessed 09.08.2023. 2021. URL: <https://www.f5.com/labs/articles/threat-intelligence/the-2021-tls-telemetry-report>.
- [WWW19] Thomas Weise, Zijun Wu and Markus Wagner. ‘An Improved Generic Bet-and-Run Strategy with Performance Prediction for Stochastic Local Search’. In: *AAAI*. 2019, pp. 2395–2402. DOI: [10.1609/aaai.v33i01.33012395](https://doi.org/10.1609/aaai.v33i01.33012395).
- [Zha+23a] Zhiyuan Zhang, Gilles Barthe, Chitchanok Chuengsatiansup, Peter Schwabe and Yuval Yarom. ‘Ultimate SLH: Taking Speculative Load Hardening to the Next Level’. In: *USENIX Security*. 2023.
- [Zha+23b] Zhiyuan Zhang, Mingtian Tao, Sioli O’Connell, Chitchanok Chuengsatiansup, Daniel Genkin and Yuval Yarom. ‘BunnyHop: Exploiting the Instruction Prefetcher’. In: *USENIX Security*. 2023.

- [Zhu+15] Yuhao Zhu, Daniel Richins, Matthew Halpern and Vijay Janapa Reddi. ‘Microarchitectural Implications of Event-Driven Server-Side Web Applications’. In: *Proceedings of the 48th International Symposium on Microarchitecture*. ACM. ACM, 2015, 762–774. URL: <https://doi.org/10.1145/2830772.2830792>.